

LINUX

ULTIMATE GUIDE

Linux Ultimate Guide

PART I: Introduction to Linux	4
Chapter 1: Overview of Linux	4
Chapter 2: Getting Started with Linux	6
Chapter 3: Linux Environment	9
PART II: Basic Linux Commands	12
Chapter 4: File and Directory Management	12
Cheat Sheet:	13
Chapter 5: File Permissions and Ownership	14
Linux File Permissions Cheat Sheet	15
Chapter 6: Basic Text Processing	17
Linux Text Processing Commands Cheat Sheet	18
Chapter 7: Networking Basics	19
Linux networking commands cheat sheet	20
Chapter 8: System Monitoring and Process Management	22
Linux System Monitoring Commands Cheat Sheet.....	23
PART III: Advanced Linux Commands	25
Chapter 9: Advanced File Operations	25
Cheat Sheet for find, sed, awk, and diff Commands	26
Chapter 10: Advanced Networking	27
Cheat Sheet for SSH, ip, and netstat Commands	28
Chapter 11: System Administration	29
Cheat Sheet for User Management, systemctl, and apt-get Commands	30
Chapter 12: Advanced Scripting	31
Cheat Sheet for Bash Scripting and Bash Debugging Commands	33
PART IV: Shell Scripting	35
Chapter 13: Introduction to Shell Scripting	35
Chapter 14: Getting Started with Shell Scripting	35
Chapter 15: Variables and User Input	37
Shell Script Variables Cheat Sheet	37
Chapter 16: Control Structures	39
Chapter 17: Functions	41
Cheat Sheet: Shell Script Functions	41
Chapter 18: Error Handling	42
Chapter 19: Advanced Scripting Techniques	43

Chapter 20: Automating System Tasks.....	45
Chapter 21: Real-world Examples and Case Studies	45
Cron Job Syntax Cheat Sheet	47
PART V: System Administration	48
Chapter 22: User and Group Management	48
User Management, Group Management, and File Permissions Cheat Sheet	49
Chapter 23: Package Management.....	50
APT, YUM, and SNAP Commands Cheat Sheet	51
Chapter 24: Disk Management	52
Disk Usage, Disk Partitioning, and Mounting Filesystems Commands Cheat Sheet	53
Chapter 25: Network Management	54
Network Configuration, Network Monitoring, and Nmap Commands Cheat Sheet	55
Chapter 26: System Monitoring and Performance Tuning	56
System Monitoring, Htop, and Performance Tuning Commands Cheat Sheet	57
Chapter 27: Security and Access Control	58
SSH Configuration, UFW, and SELinux Commands Cheat Sheet	59
Chapter 28: Case Studies and Real-world Applications	61
PART VI: Advanced Topics	62
Chapter 29: Advanced File System Management	62
RAID and LVM Commands Cheat Sheet	64
Chapter 30: Kernel Tuning and Optimization	65
sysctl Commands Cheat Sheet	66
Chapter 31: Virtualization and Containers	67
Docker Commands Cheat Sheet	69
Chapter 32: High Availability and Load Balancing	70
Chapter 33: Security and Hardening	72
SELinux Commands Cheat Sheet	73
Chapter 34: Troubleshooting and Optimization	74
PART VII: Case Studies and Practical Examples.....	78
Chapter 36: Automating Daily System Tasks	78
Cron Expressions and Mail Command Options Cheat Sheet	78
Chapter 37: Network Administration.....	79
iftop and nmap Command Cheat Sheet	79
Chapter 38: Web Server Administration.....	80
Chapter 39: Database Management	82
Chapter 40: Security and Compliance	84
Chapter 41: Advanced Scripting and Automation	86

PART VIII: Cheat Sheets and Resources	88
Chapter 42: Basic Linux Commands Cheat Sheet	88
Chapter 43: Advanced Linux Commands Cheat Sheet	89
Chapter 44: Shell Scripting Cheat Sheet	89
Chapter 45: Network Administration Cheat Sheet	91
Chapter 46: System Administration Cheat Sheet.....	91
Chapter 47: Security and Compliance Cheat Sheet	93
Chapter 48: Performance Tuning Cheat Sheet.....	93
Chapter 49: Scripting and Automation Tools Cheat Sheet	95
Chapter 50: Additional Resources	96
PART IX: Appendices	97
Chapter 51: Appendix A: Mathematical Foundations	97
Chapter 52: Appendix B: Shell Scripting Reference	98
Chapter 53: Appendix C: Linux Resources	99
Chapter 54: Appendix D: Glossary of Terms	100

PART I: Introduction to Linux

Chapter 1: Overview of Linux

History and Evolution of Linux

- A Brief History from Unix to Linux
 - Unix, developed in the 1960s and 1970s, laid the foundation for modern operating systems.
 - The GNU Project began in the 1980s with the goal of creating a free Unix-like operating system.
 - Linus Torvalds released the first version of the Linux kernel in 1991.
 - Linux evolved through contributions from developers worldwide, becoming a robust and versatile OS.

```
bash

# View the current Linux kernel version
uname -r
```

- Key Milestones in Linux Development
 - 1991: Linux kernel version 0.01 released.
 - 1992: Linux becomes GPL-licensed, attracting more developers.
 - 2003: Release of Fedora, a community-supported distribution by Red Hat.
 - 2012: Introduction of systemd, a system and service manager.

```
bash

# Check for systemd version
systemctl --version
```

- Linux Distributions and Their Purposes
 - Distributions (distros) customize the Linux kernel with different software packages and configurations to serve various needs.

Linux Distributions

- Popular Distributions: Ubuntu, Fedora, CentOS, Debian
 - Ubuntu: User-friendly, ideal for desktops and beginners.
 - Fedora: Cutting-edge features, often used by developers.
 - CentOS : Stability-focused, commonly used in servers.
 - Debian: Stability and freedom, a base for many other distributions.

```
bash

# Check the distribution name and version
lsb_release -a
```

- Differences and Use Cases for Each Distribution
 - o Ubuntu: Great for everyday use, multimedia, and general-purpose computing.
 - o CentOS: Preferred for development environments and testing new features.
 - o Red Hat Enterprise Linux: Suited for enterprise environments requiring long-term support.
 - o Debian: Used in environments where stability and software freedom are prioritized.

```
bash
```

```
# Example of installing a package in different distributions
# Ubuntu/Debian
sudo apt-get install <package_name>

# Fedora
sudo dnf install <package_name>

# CentOS
sudo yum install <package_name>
```

Linux Kernel

- Overview of the Linux Kernel
 - o The core component of the Linux operating system, managing hardware resources and system processes.
- Key Components and Functions of the Kernel
 - o Process Management: Handles process creation, scheduling, and termination.
 - o Memory Management : Manages system memory, including allocation and deallocation.
 - o File Systems : Provides access to various file systems like ext4, xfs, and btrfs.
 - o Device Drivers Interfaces with hardware components like disks, networks, and graphics cards.

```
bash
```

```
# Display the kernel ring buffer
dmesg

# View loaded kernel modules
lsmod
```

- How the Kernel Interacts with Hardware and Software
 - o The kernel acts as a bridge between applications and hardware, handling input/output operations, resource allocation, and security enforcement.

```
bash
```

```
# Display hardware information
lshw

# Check CPU information
lscpu
```

Chapter 2: Getting Started with Linux

Introduction

This chapter covers the essentials of getting started with Linux, including installing and configuring Linux on various platforms, understanding the Linux directory structure, and basic navigation and file management commands.

Installing Linux

- Installing Ubuntu on Windows Using WSL

1. Enable WSL

- Open PowerShell as Administrator.
- Run the following command:

```
powershell
```

```
dism.exe /online /enable-feature /featurename:Microsoft-  
Windows-Subsystem-Linux /all /norestart
```

2. Install a Linux Distribution

- Open Microsoft Store and search for "Ubuntu".
- Select Ubuntu and click "Get" to install it.

3. Set Up Ubuntu

- Launch Ubuntu from the Start menu.
- Follow the prompts to set up a new user account.

```
bash
```

```
# Verify WSL installation  
wsl --list --verbose
```

- Installing Ubuntu from a USB Drive

1. Create a Bootable USB Drive

- Download the Ubuntu ISO from the [official site](#).
- Use a tool like Rufus to create a bootable USB drive.

2. Boot from USB

- Restart your computer and enter the BIOS/UEFI settings.
- Set the USB drive as the first boot device.
- Save and exit the BIOS/UEFI settings.

3. Install Ubuntu

- Follow the on-screen instructions to install Ubuntu.

```
bash
```

```
# Example of using dd command to create a bootable USB on Linux  
sudo dd if=~/.Downloads/ubuntu.iso of=/dev/sdX bs=4M  
sync
```

- Installing Ubuntu on a Virtual Machine
 1. Download and Install VirtualBox
 - Download VirtualBox from the [official site](#).
 - Install VirtualBox following the on-screen instructions.
 2. Create a New Virtual Machine
 - Open VirtualBox and click "New".
 - Follow the prompts to create a new virtual machine, selecting Ubuntu as the OS.
 3. Install Ubuntu
 - Start the virtual machine and follow the on-screen instructions to install Ubuntu.

```
bash
```

```
# Check the version of VirtualBox
vboxmanage --version
```

Understanding the Linux Directory Structure

- Key Directories and Their Purposes
 - /: Root directory.
 - /bin: Essential user binaries.
 - /boot: Boot loader files.
 - /dev: Device files.
 - /etc: Configuration files.
 - /home: User home directories.
 - /lib: Shared libraries.
 - /media: Mount points for removable media.
 - /mnt: Temporary mount points.
 - /opt: Optional software packages.
 - /proc: Process and kernel information.
 - /root: Root user's home directory.
 - /run: Application state files.
 - /sbin: System binaries.
 - /srv: Data for services.
 - /tmp: Temporary files.
 - /usr: User binaries and applications.
 - /var: Variable files like logs.

```
bash
```

```
# List root directory contents
ls /
```


Basic Navigation and File Management Commands

- Navigation Commands
 - pwd: Print working directory.
 - ls: List directory contents.
 - cd: Change directory.

```
bash
```

```
# Example of navigation commands
```

```
pwd
```

```
ls -l
```

```
cd /home/user
```

- File Management Commands
 - cp: Copy files or directories.
 - mv : Move or rename files or directories.
 - rm : Remove files or directories.
 - mkdir : Create a directory.

```
bash
```

```
# Example of file management commands
```

```
cp file1.txt file2.txt
```

```
mv file1.txt /home/user/documents/
```

```
rm file2.txt
```

```
mkdir new_directory
```

Chapter 3: Linux Environment

Introduction

This chapter explores the Linux environment, focusing on the shell as the primary interface for interacting with the system. We'll discuss different shell types, shell features, scripting basics, and how to write and execute simple scripts.

The Linux Shell

Introduction to Different Shells: Bash, Zsh, Fish

- **Bash (Bourne Again Shell)**
 - Default shell for many Linux distributions.
 - Combines features of the original Bourne shell (sh) with additional improvements.
 - Known for its wide adoption and extensive scripting capabilities.

```
bash
```

```
# Check current shell
echo $SHELL
```

- Example of Bash prompt customization:

```
bash
```

```
# Customize Bash prompt
PS1="\u@\h:\w$ "
```

- **Zsh (Z Shell)**
 - An extended Bourne shell with many improvements.
 - Features include powerful scripting language, enhanced completion, spell checking, and themes.
 - Highly customizable with frameworks like Oh My Zsh.

```
bash
```

```
# Install and switch to Zsh
sudo apt install zsh
chsh -s $(which zsh)
```

- Example of using Oh My Zsh:

```
bash
```

```
# Install Oh My Zsh
sh -c "$(curl -fsSL
https://raw.githubusercontent.com/ohmyzsh/ohmyzsh/master/tools/install.sh)"
```

- Fish (Friendly Interactive Shell)
 - User-friendly shell with helpful command suggestions and intuitive syntax.
 - Focuses on user experience and ease of use.
 - Includes features like autosuggestions and syntax highlighting out of the box.

```
bash
```

```
# Install Fish shell
sudo apt install fish
```

- Example of Fish shell usage:

```
fish
```

```
# Run Fish shell
fish
```

Shell Features and Scripting Basics

- Common Features of Shells
 - Command history : Allows you to recall and edit previous commands.
 - Command completion : Automatically completes command names and file paths.
 - Environment variables : Variables that affect the way processes run on a system.

```
bash # View command history in Bash
history
bash
```

```
# Enable command completion in Bash
bind 'set show-all-if-ambiguous on'
```

- Scripting Basics
 - Shebang The #! line at the beginning of a script specifying the interpreter.

```
bash
```

```
# Example shebang line for a Bash script
#!/bin/bash
```

- Variables: Store data for use in scripts.

```
bash
```

```
# Declare a variable in Bash
MY_VAR="Hello, World!"
echo $MY_VAR
```

- Control Structures: Conditional statements and loops.

```
bash
```

```
# If statement in Bash
```

```

if [ "$MY_VAR" == "Hello, World!" ]; then
    echo "The variable is set to 'Hello, World!'"
fi
bash

# For loop in Bash
for i in {1..5}; do
    echo "Number $i"
done

```

Writing and Executing Simple Scripts

- **Creating a Simple Script**
 - o Create a new file with a `sh` extension.

```

bash

# Create a script file
nano my_script.sh

```

- o Add the shebang and some commands.

```

bash

#!/bin/bash
echo "This is a simple script."
MY_VAR="Hello, World!"
echo $MY_VAR

```

- **Making the Script Executable**
 - o Change the file permissions to make it executable.

```

bash

# Make the script executable
chmod +x my_script.sh

```

- **Executing the Script**
 - o Run the script from the command line.

```

bash

# Execute the script
./my_script.sh

```

PART II: Basic Linux Commands

Chapter 4: File and Directory Management

- Navigating the Filesystem

- o Commands: `ls`, `cd`, `pwd`

```
bash

# List files and directories
ls -l
# Change directory
cd /home/user
# Print working directory
pwd
```

- o Explanation: `ls` lists files and directories, `cd` changes the current directory, `pwd` prints the current directory path.

- Managing Files

- o Commands: `touch`, `cp`, `mv`, `rm`

```
bash

# Create a new file
touch newfile.txt
# Copy a file
cp newfile.txt copyfile.txt
# Move or rename a file
mv copyfile.txt renamedfile.txt
# Remove a file
rm renamedfile.txt
```

- o Explanation: `touch` creates a new file, `cp` copies a file, `mv` moves or renames a file, `rm` removes a file.

- Managing Directories

- o Commands: `mkdir`, `rmdir`, `rm -r`

```
bash

# Create a new directory
mkdir newdir
# Remove an empty directory
rmdir newdir
# Remove a directory and its contents
rm -r newdir
```

- o Explanation: `mkdir` creates a new directory, `rmdir` removes an empty directory, `rm -r` removes a directory and its contents.

Cheat Sheet:

Command	Description	Example
ls	List directory contents	ls ls -l ls -a
cd	Change directory	cd /path/to/directory cd .. cd ~
pwd	Print working directory	pwd
tree	Display directory tree structure	tree
locate	Find files by name	locate filename
which	Locate a command	which ls
man	Display manual page for a command	man ls
clear	Clear the terminal screen	clear
touch	Create a new empty file	touch newfile.txt
cp	Copy files or directories	cp source_file destination_file cp -r source_directory destination_directory
mv	Move or rename files or directories	mv old_name new_name mv /path/to/source /path/to/destination
rm	Remove files or directories	rm -r directory_name rm file.txt
cat	Concatenate and display file contents	cat file.txt
more	View file contents one screen at a time	more file.txt
less	View file contents with backward movement	less file.txt
head	Display the first lines of a file	head -n 10 file.txt
tail	Display the last lines of a file	tail -n 10 file.txt tail -f logfile.txt
chmod	Change file permissions	chmod 755 script.sh
chown	Change file owner and group	chown user:group file.txt ln -s source_file symbolic_link
ln	Create links between files	
mkdir	Create a new directory	mkdir new_directory
rmdir	Remove an empty directory	rmdir empty_directory
rm -r	Remove a directory and its contents	rm -r directory_to_remove
find	Search for files in a directory hierarchy	find /path -name filename
du	Estimate file space usage	du -sh /path/to/directory
df	Display free disk space	df -h

Chapter 5: File Permissions and Ownership

- Understanding File Permissions

- Explanation of `rwx` (read, write, execute) permissions for user, group, and others.
- Example: `-rwxr-xr--`
- Commands: `ls -l`

```
bash
```

```
# List files with detailed information
ls -l
```

- Changing File Permissions

- Commands: `chmod`

```
bash
```

```
# Change permissions using symbolic mode
chmod u+x myfile.txt
chmod g-w myfile.txt
# Change permissions using numeric mode
chmod 755 myfile.txt
```

- Explanation: `chmod` changes file permissions. Symbolic mode (e.g., `u+x`) adds execute permission for the user. Numeric mode (e.g., `755`) sets permissions directly.

- Changing File Ownership

- Commands: `chown`, `chgrp`

```
bash
```

```
# Change file owner
chown newuser myfile.txt
# Change file group
chgrp newgroup myfile.txt
```

- Explanation: `chown` changes the owner of a file, `chgrp` changes the group of a file.

Linux File Permissions Cheat Sheet

Comman	Description	Example
ls -l	List files with permissions	<code>ls -l</code>
chmod	change file permissions	<code>chmod 755 file.txt</code>
chown	change file owner	<code>chown user file.txt</code>
chgrp	change file group	<code>chgrp group file.txt</code>
umask	Set default file permissions	<code>umask 022</code>

Understanding File Permissions

Symbol	Meaning
r	Read permission
w	Write permission
x	Execute permission
-	No permission

File Permission Representation

Symbol	Description	Example
-	Regular file	<code>-rw-r--r--</code>
d	Directory	<code>drwxr-xr-x</code>
l	Symbolic link	<code>lrwxrwxrwx</code>

Numerical Representation of Permissions

Numbe	Permission	Binary
0	No permission	000
1	Execute only	001
2	Write only	010
3	Write and execute	011
4	Read only	100
5	Read and execute	101
6	Read and write	110
7	Read, write, and execute	111

Changing Permissions with `chmod`

Command	Description	Example
<code>chmod 755 file</code>	Owner can read, write, and execute; others can read and execute	<code>chmod 755 file.txt</code>
<code>chmod u+x file</code>	Add execute permission for the owner	<code>chmod u+x script.sh</code>
<code>chmod g-w file</code>	Remove write permission for the group	<code>chmod g-w file.txt</code>
<code>chmod o+r file</code>	Add read permission for others	<code>chmod o+r file.txt</code>
<code>chmod a-r file</code>	Remove read permission for all	<code>chmod a-r secret.txt</code>
<code>chmod -R 755 dir</code>	Change permissions recursively	<code>chmod -R 755 /path/to/directory</code>

Changing Ownership with `chown`

Command	Description	Example
<code>chown user file</code>	Change owner of a file	<code>chown alice file.txt</code>
<code>chown user:group file</code>	Change owner and group of a file	<code>chown alice:developers file.txt</code>
<code>chown -R user dir</code>	Change ownership recursively	<code>chown -R bob /path/to/directory</code>

Changing Group Ownership with `chgrp`

Command	Description	Example
<code>chgrp group file</code>	Change group ownership of a file	<code>chgrp developers file.txt</code>
<code>chgrp -R group dir</code>	Change group ownership recursively	<code>chgrp -R staff /path/to/directory</code>

Setting Default Permissions with `umask`

Command	Description	Example
<code>umask 022</code>	Set default permissions for new files and directories	<code>umask 022</code>
<code>umask</code>	Display current umask setting	<code>umask</code>

Chapter 6: Basic Text Processing

- Viewing File Contents

- Commands: `cat`, `less`, `more`

```
bash

# View file contents
cat myfile.txt
# View file contents one page at a time
less myfile.txt
more myfile.txt
```

Explanation: `cat` displays file contents, `less` and `more` allow paginated viewing.

- Searching Within Files

- Commands: `grep`

```
bash

# Search for a pattern in a file
grep "search_term" myfile.txt
```

- Explanation: `grep` searches for a specific pattern in a file.

- Manipulating Text Files

- Commands: `head`, `tail`, `cut`, `sort`

```
bash

# View the first 10 lines of a file
head myfile.txt
# View the last 10 lines of a file
tail myfile.txt
# Extract specific columns from a file
cut -d"," -f1,3 myfile.csv
# Sort the contents of a file
sort myfile.txt
```

- Explanation: `head` and `tail` view the beginning and end of a file, `cut` extracts columns, `sort` sorts file contents.

Linux Text Processing Commands Cheat Sheet

Comman	Description	Example
d	Concatenate and display file content	<code>cat file.txt</code>
cat	Concatenate and display file content in	<code>tac file.txt</code>
tac	reverse	
nl	Number lines in a file View file content	<code>nl file.txt</code>
more less	interactively View file content	<code>more file.txt</code>
head	interactively with more	<code>less file.txt</code>
tail cut	options	<code>head -n 10 file.txt tail -</code>
paste	Display the first few lines of a file Display	<code>n 10 file.txt cut -d',' -</code>
sort uniq	the last few lines of a file Remove	<code>f1,3 file.csv paste</code>
wc	sections from each line of files Merge	<code>file1.txt file2.txt sort</code>
grep	lines of files Sort lines of text files Report	<code>file.txt uniq file.txt wc</code>
sed	or omit repeated lines Print newline,	<code>-l file.txt</code>
awk	word, and byte counts for	
tr	each file	<code>grep "pattern" file.txt</code>
fmt pr	Print lines matching a pattern Stream	<code>sed 's/old/new/g'</code>
fold	editor for filtering and transforming	<code>file.txt</code>
join	text	<code>awk '{print \$1, \$3}'</code>
comm diff	Pattern scanning and processing	<code>file.txt</code>
col	language Translate or delete characters	<code>`echo "text"</code>
column	Format text to a specified width Convert	<code>fmt -w 80 file.txt</code>
split	text files for printing Wrap each input	<code>pr -h "Header" file.txt</code>
csplit	line to fit in specified	<code>fold -w 80 file.txt</code>
expand	width	
unexpand	Join lines of two files on a common field	<code>join file1.txt file2.txt</code>
strings	Compare two sorted files line by line	<code>comm file1.txt file2.txt</code>
tee xargs	Compare files line by line Filter reverse	<code>diff file1.txt file2.txt</code>
	line feeds Columnate input Split a file	<code>col -b file.txt column -</code>
	into pieces Split a file into context-	<code>t file.txt</code>
	determined pieces Convert tabs to	<code>split -l 1000 file.txt</code>
	spaces Convert spaces to tabs Print the	<code>csplit file.txt</code>
	printable character sequences in	<code>'/pattern/' '{*}'</code>
	files	<code>expand file.txt</code>
	Read from standard input and write to	<code>unexpand file.txt</code>
	standard output and files	<code>strings file.bin</code>
	Build and execute command lines from	
	standard input	<code>`ls -l</code>
		<code>`find . -type f -name "*.txt"</code>

Chapter 7: Networking Basics

- Checking Network Configuration

- Commands: `ifconfig`, `ip`

```
bash

# Display network configuration
ifconfig
ip addr show
```

- Explanation: `ifconfig` and `ip` show network interface configuration.

- Testing Network Connectivity

- Commands: `ping`, `traceroute`

```
bash

# Ping a host
ping google.com
# Trace the route to a host
traceroute google.com
```

- Explanation: `ping` tests connectivity to a host, `traceroute` shows the route taken to a host.

- Transferring Files

- Commands: `scp`, `rsync`

```
bash

# Securely copy a file to a remote host
scp myfile.txt user@remote_host:/path/to/destination
# Synchronize files between local and remote hosts
rsync -avz myfile.txt user@remote_host:/path/to/destination
```

- Explanation: `scp` securely copies files, `rsync` synchronizes files between hosts.

Linux networking commands cheat sheet

Command	Description	Example
ping	Send ICMP ECHO_REQUEST to network hosts	ping google.com
ifconfig	Configure network interfaces	ifconfig eth0
ip	Show/manipulate routing, devices, policy routing, and tunnels	ip addr show
traceroute	Print the route packets take to the network host	traceroute google.com
netstat	Print network connections, routing tables, interface statistics, masquerade connections, and multicast memberships	netstat -tuln
ss	Another utility to investigate sockets	ss -tuln
nslookup	Query Internet name servers interactively	nslookup google.com
dig	DNS lookup	dig google.com
route	Show/manipulate the IP routing table	route -n
hostname	Show or set the system's hostname	hostname
whois	Perform WHOIS lookups	whois google.com
curl	Transfer data from or to a server	curl http://example.com
wget	Non-interactive network downloader	wget http://example.com/file.zip
ftp	File Transfer Protocol	ftp ftp.example.com
scp	Secure copy (remote file copy program)	scp file.txt user@remote:/path/to/destination
rsync	Remote file and directory synchronization	rsync -avz file.txt user@remote:/path/to/destination
nmap	Network exploration tool and security/port scanner	nmap -sP 192.168.1.0/24
tcpdump	Command-line packet analyzer	tcpdump -i eth0
ethtool	Display or change Ethernet device settings	ethtool eth0
nmcli	Command-line client for NetworkManager	nmcli device status
iwconfig	Configure wireless network interfaces	iwconfig wlan0

wpa_supplicant	Wi-Fi Protected Access client and IEEE 802.1X supplicant	wpa_supplicant -B -i wlan0 -c /etc/wpa_supplicant.conf
ip a	Show all network interfaces and their addresses	ip a
ip link	Show/manage network devices	ip link show
ip route	Show/manipulate the routing table	ip route show
netcat (nc)	Utility for reading from and writing to network connections using TCP or UDP	nc -zv 192.168.1.1 22
arp	Manipulate the system ARP cache	arp -a
mtr	Network diagnostic tool combining ping and traceroute	mtr google.com
iptables	Administration tool for IPv4 packet filtering and NAT	iptables -L

Chapter 8: System Monitoring and Process Management

- Monitoring System Performance

- o Commands: `top`, `htop`, `vmstat`

```
bash

# Display system performance statistics
top
htop
vmstat
```

- o Explanation: `top` and `htop` show real-time system performance, `vmstat` provides system performance statistics.

- Managing Processes

- o Commands: `ps`, `kill`, `pkill`

```
bash

# Display currently running processes
ps aux
# Kill a process by PID
kill 1234
# Kill processes by name
pkill process_name
```

- o Explanation: `ps` lists processes, `kill` terminates a process by its PID, `pkill` terminates processes by name.

- Scheduling Tasks

- o Commands: `crontab` at

```
bash

# Edit crontab file to schedule tasks
crontab -e
# Schedule a one-time task
echo "mycommand" | at 10:00
```

- o Explanation: `crontab` schedules recurring tasks, `at` schedules one-time tasks.

Linux System Monitoring Commands Cheat Sheet

Command	Description	Example
top	Display real-time system information	top htop vmstat
htop	Enhanced version of top	
vmstat	Report virtual memory statistics	5 iostat free -h
iostat	Report CPU and I/O statistics	
df	Display memory usage	df -h du -sh
du	Report file system disk space usage	/home/user
ps	Estimate file space usage	ps aux
pstree	Report a snapshot of current processes	ps tree
netstat	Network statistics	netstat -tuln
ss	Another utility to investigate sockets	ss -tuln
iftop	Display bandwidth usage on an interface	iftop
nload	Collect, report, or save system activity	nload
sar	Show how long the system has been running	sar -u 5
uptime	Versatile resource statistics	uptime
dstat	Cross-platform system monitoring tool	dstat
glances	Report CPU usage	glances
mpstat	Report statistics per PID	mpstat
pidstat	Display I/O usage by processes	pidstat
iotop	Disk usage analyzer with an ncurses interface	iotop
ncdu	Advanced system and process monitor	ncdu
atop	List open files	atop
lsof	Execute a program periodically, showing output fullscreen	lsof
watch	Collection of performance monitoring tools	watch -n 1 df -h
sysstat		sysstat

Additional Elements

Case Studies

- Case Study: Automating File Backup
 - Write a script to automate daily file backups using `rsync`

```
bash

#!/bin/bash
# Daily backup script
rsync -avz /source/directory /backup/directory
```

Diagrams

- Linux Filesystem Hierarchy
 - Search: "Linux filesystem hierarchy diagram"
- Network Configuration Diagram
 - Search: "Linux network configuration diagram"
- Process Management Flowchart
 - Search: "Linux process management flowchart"

Summary

This expanded section on Basic Linux Commands provides comprehensive coverage of essential Linux operations, from file and directory management to system monitoring and process management. Each chapter includes practical examples, cheat sheets, and relevant diagrams to facilitate learning and application.

PART III: Advanced Linux Commands

Chapter 9: Advanced File Operations

- Finding Files and Directories

- o Commands: `find`, `locate`

```
bash
```

```
# Find files by name
find / -name "myfile.txt"
# Find files by type
find / -type d -name "mydir"
# Locate files using the locate database
locate myfile.txt
```

- o Explanation: `find` searches for files and directories based on various criteria, `locate` searches for files using a prebuilt database.

- Manipulating File Contents

- o Commands: `sed`, `awk`

```
bash
```

```
# Replace text in a file using sed
sed 's/oldtext/newtext/g' myfile.txt
# Extract and manipulate text using awk
awk '{print $1, $3}' myfile.txt
```

- o Explanation: `sed` performs text replacements in files, `awk` is a powerful text processing tool for extracting and manipulating text.

- Comparing Files

- o Commands: `diff`, `cmp`

```
bash
```

```
# Compare two files
diff file1.txt file2.txt
# Compare binary files
cmp file1.bin file2.bin
```

- o Explanation: `diff` shows differences between text files, `cmp` compares binary files byte by byte.

Cheat Sheet for `find`, `sed`, `awk`, and `diff` Commands

Command	Description	Example
<code>find</code>	Search for files in a directory hierarchy	<code>find /home/user -name "*.txt"</code>
<code>find /path -type f</code>	Find all files	<code>find /home/user -type f</code>
<code>find /path -type d</code>	Find all directories	<code>find /home/user -type d</code>
<code>find /path -size +100M</code>	Find files larger than 100MB	<code>find /home/user -size +100M</code>
<code>find /path -mtime -7</code>	Find files modified in the last 7 days	<code>find /home/user -mtime -7</code>
<code>find /path -exec command {} \;</code>	Execute a command on each file found	<code>find /home/user -name "*.log" -exec rm {} \;</code>
<code>find /path -perm 644</code>	Find files with specific permissions	<code>find /home/user -perm 644</code>
<code>sed 's/old/new/g' file</code>	Replace all occurrences of 'old' with 'new' in a file	<code>sed 's/foo/bar/g' file.txt</code>
<code>sed -n '1,5p' file</code>	Print lines 1 to 5 of a file	<code>sed -n '1,5p' file.txt</code>
<code>sed 'd' file</code>	Delete lines matching the pattern	<code>sed '/pattern/d' file.txt</code>
<code>awk '{print \$1}' file</code>	Print the first column of a file	<code>awk '{print \$1}' file.txt</code>
<code>awk '/pattern/ {print \$0}' file</code>	Print lines matching the pattern	<code>awk '/foo/ {print \$0}' file.txt</code>
<code>awk 'BEGIN {FS=":"} {print \$1}' file</code>	Set field separator and print first column	<code>awk 'BEGIN {FS=":"} {print \$1}' /etc/passwd</code>
<code>diff file1 file2</code>	Compare two files line by line	<code>diff file1.txt file2.txt</code>
<code>diff -u file1 file2</code>	Unified diff format	<code>diff -u file1.txt file2.txt</code>
<code>diff -r dir1 dir2</code>	Compare directories recursively	<code>diff -r dir1 dir2</code>
<code>cmp file1 file2</code>	Compare two files byte by byte	<code>cmp file1.txt file2.txt</code>
<code>comm file1 file2</code>	Compare two sorted files line by line	<code>comm file1.txt file2.txt</code>
<code>sdiff file1 file2</code>	Side-by-side comparison of two files	<code>sdiff file1.txt file2.txt</code>

Chapter 10: Advanced Networking

- Network Configuration

- o Commands: `ifconfig`, `ip`

```
bash

# Display network interfaces and configuration
ifconfig
ip addr show
# Assign an IP address to an interface
sudo ip addr add 192.168.1.100/24 dev eth0
```

- o Explanation: `ifconfig` and `ip` commands are used for configuring network interfaces.

- Network Troubleshooting

- o Commands: `netstat`, `ss`

```
bash

# Display network connections and routing tables
netstat -tuln
# Display socket statistics
ss -tuln
```

- o Explanation: `netstat` and `ss` provide information about network connections and listening sockets.

- Secure Shell (SSH)

- o Commands: `ssh`, `scp`

```
bash

# Connect to a remote host
ssh user@remote_host
# Copy files securely between hosts
scp user@remote_host:/path/to/remote/file
/path/to/local/destination
```

- o Explanation: `ssh` allows secure access to remote systems, `scp` securely copies files between systems.

Cheat Sheet for `SSH`, `ip`, and `netstat` Commands

Command	Description	Example
<code>ssh user@host</code>	Connect to a remote host	<code>ssh user@192.168.1.100</code>
<code>ssh -i /path/to/key user@host</code>	Connect using a specific key	<code>ssh -i ~/.ssh/id_rsa user@192.168.1.100</code>
<code>ssh -p port user@host</code>	Connect using a specific port	<code>ssh -p 2222 user@192.168.1.100</code>
<code>ssh -L local_port:host:remote_port user@host</code>	Local port forwarding	<code>ssh -L 8080:localhost:80 user@192.168.1.100</code>
<code>ssh -R remote_port:host:local_port user@host</code>	Remote port forwarding	<code>ssh -R 8080:localhost:80 user@192.168.1.100</code>
<code>ssh-copy-id user@host</code>	Copy your public key to a remote host	<code>ssh-copy-id user@192.168.1.100</code>
<code>ip addr show</code>	Display all IP addresses and interfaces	<code>ip addr show</code>
<code>ip link set dev eth0 up</code>	Bring an interface up	<code>ip link set dev eth0 up</code>
<code>ip link set dev eth0 down</code>	Bring an interface down	<code>ip link set dev eth0 down</code>
<code>ip route show</code>	Show the routing table	<code>ip route show</code>
<code>ip route add default via gateway</code>	Add a default gateway	<code>ip route add default via 192.168.1.1</code>
<code>ip link show</code>	Display all network interfaces	<code>ip link show</code>
<code>ip neigh show</code>	Display the ARP table	<code>ip neigh show</code>
<code>ip addr add IP/dev</code>	Add an IP address to an interface	<code>ip addr add 192.168.1.2/24 dev eth0</code>
<code>netstat -tuln</code>	Show listening ports	<code>netstat -tuln</code>
<code>netstat -r</code>	Display the routing table	<code>netstat -r</code>
<code>netstat -i</code>	Show network interfaces	<code>netstat -i</code>
<code>netstat -s</code>	Display network statistics	<code>netstat -s</code>
<code>netstat -anp</code>	Show all connections and listening ports	<code>netstat -anp</code>
<code>netstat -rn</code>	Display the kernel routing table in numeric form	<code>netstat -rn</code>
<code>netstat -g</code>	Display multicast group memberships	<code>netstat -g</code>

Chapter 11: System Administration

- Managing Users and Groups

- o Commands: `useradd`, `usermod`, `groupadd`

```
bash
```

```
# Add a new user
sudo useradd newuser
# Modify an existing user
sudo usermod -aG sudo newuser
# Add a new group
sudo groupadd newgroup
```

- o Explanation: `useradd` creates a new user, `usermod` modifies existing user accounts, `groupadd` creates a new group.

- Managing Services and Daemons

- o Commands: `systemctl`, `service`

```
bash
```

```
# Start a service
sudo systemctl start nginx
# Stop a service
sudo systemctl stop nginx
# Check status of a service
sudo systemctl status nginx
```

- o Explanation: `systemctl` and `service` commands manage system services and daemons.

- Package Management

- o Commands: `apt-get`, `yum`

```
bash
```

```
# Update package lists
sudo apt-get update
# Install a package
sudo apt-get install vim
# Remove a package
sudo apt-get remove vim
```

- o Explanation: `apt-get` and `yum` are package management tools for installing, updating, and removing software packages.

Cheat Sheet for User Management, systemctl, and apt-get Commands

Command	Description	Example
<code>useradd username</code>	Add a new user	<code>useradd alice userdel</code>
<code>userdel username</code>	Delete a user	<code>alice usermod -aG sudo</code>
<code>usermod -aG group username</code>	Add a user to a group	<code>alice</code>
<code>passwd username</code>	Change a user's password	<code>passwd alice</code>
<code>chage -l username</code>	List a user's password aging information	<code>chage -l alice</code>
<code>groupadd groupname</code>	Add a new group	<code>groupadd developers</code>
<code>groupdel groupname</code>	Delete a group	<code>groupdel developers</code>
<code>id username</code>	Display user ID and group ID information	<code>id alice</code>
<code>systemctl start service</code>	Start a service	<code>systemctl start apache2</code>
<code>systemctl stop service</code>	Stop a service	<code>systemctl stop apache2</code>
<code>systemctl restart service</code>	Restart a service	<code>systemctl restart apache2</code>
<code>systemctl status service</code>	Check the status of a service	<code>systemctl status apache2</code>
<code>systemctl enable service</code>	Enable a service to start on boot	<code>systemctl enable apache2</code>
<code>systemctl disable service</code>	Disable a service from starting on boot	<code>systemctl disable apache2</code>
<code>systemctl is-active service</code>	Check if a service is active	<code>systemctl is-active apache2</code>
<code>systemctl is-enabled service</code>	Check if a service is enabled	<code>systemctl is-enabled apache2</code>
<code>systemctl list-units --type=service</code>	List all active services	<code>systemctl list-units --type=service</code>
<code>apt-get update</code>	Update the package list	<code>apt-get update</code>
<code>apt-get upgrade</code>	Upgrade all installed packages	<code>apt-get upgrade</code>
<code>apt-get install package</code>	Install a package	<code>apt-get install vim</code>
<code>apt-get remove package</code>	Remove a package	<code>apt-get remove vim</code>
<code>apt-get autoremove</code>	Remove unnecessary packages	<code>apt-get autoremove</code>
<code>apt-get clean</code>	Clean up the local repository of retrieved package files	<code>apt-get clean</code>
<code>apt-cache search package</code>	Search for a package in the repository	<code>apt-cache search vim</code>
<code>apt-cache show package</code>	Display package details	<code>apt-cache show vim</code>

Chapter 12: Advanced Scripting

- Bash Scripting

- Example Script: Backup Script

```
bash

#!/bin/bash
# Backup script
# Define backup source and destination
SRC="/home/user/data"
DEST="/backup/data"

# Perform backup
rsync -avz $SRC $DEST
```

- Explanation: This script uses `rsync` to perform a backup of user data.

- Using Variables and Functions

- Example Script: Using Variables

```
bash

#!/bin/bash
# Script to demonstrate variables
NAME="John Doe"
echo "Hello, $NAME"
```

- Explanation: This script demonstrates the use of variables in a bash script.

- Control Structures and Loops

- Example Script: Loop Example

```
bash

#!/bin/bash
# Loop example
for i in {1..5}
do
    echo "Iteration $i"
done
```

- Explanation: This script uses a `for` loop to iterate and print numbers from 1 to 5.

- Debugging Scripts

- o Commands: `bash -x`, `set -x`

```
bash
```

```
# Run script with debugging enabled
```

```
bash -x myscript.sh
```

```
# Enable debugging within a script
```

```
set -x
```

```
# Your script code here
```

```
set +x
```

- o Explanation: `bash -x` runs a script in debug mode, `set -x` enables debugging within a script, and `set +x` disables it.

Additional Elements

Case Studies

- Case Study: Automating Server Maintenance

- o Write a script to automate server updates and reboots.

```
bash
```

```
#!/bin/bash
```

```
# Server maintenance script
```

```
# Update package lists
```

```
sudo apt-get update
```

```
# Upgrade packages
```

```
sudo apt-get upgrade -y
```

```
# Reboot the server
```

```
sudo reboot
```

Diagrams

- Advanced Networking Diagram

- o Search: "Linux advanced networking diagram"

- User and Group Management Workflow

- o Search: "Linux user management workflow"

- Service Management Flowchart

- o Search: "systemctl flowchart"

Cheat Sheet for Bash Scripting and Bash Debugging Commands

Command	Description	Example
<code>#!/bin/bash</code>	Shebang to specify the script's interpreter	<code>#!/bin/bash</code>
<code>echo "message"</code>	Print a message to the terminal	<code>echo "Hello, World!"</code>
<code>variable=value</code>	Define a variable	<code>name="Alice"</code>
<code>echo \$variable</code>	Access a variable	<code>echo \$name</code>
<code>read variable</code>	Read user input into a variable	<code>read user_input</code>
<code>if [condition]; then ... fi</code>	If statement	<code>if [\$age -ge 18]; then echo "Adult"; fi</code> <code>for i in {1..5}; do echo \$i; done</code>
<code>for var in list; do ... done</code>	For loop	<code>while [\$count -le 5]; do echo \$count; count=\$((count+1)); done</code>
<code>while [condition]; do ... done</code>	While loop	<code>case \$1 in start) echo "Starting";; stop) echo "Stopping";; esac</code>
<code>case variable in ... esac</code>	Case statement	<code>function greet { echo "Hello, \$1"; }</code> <code>if [\$? -eq 0]; then echo "Success"; fi</code>
<code>function name { ... }</code>	Define a function	<code>exit</code>
<code>\$?</code>	Exit status of the last command	<code>today=\$(date)</code>
<code>\$(command)</code>	Command substitution	
<code>\$(<file)</code>	Read a file into a variable	<code>content=\$(<file.txt)</code>
<code>> file</code>	Redirect output to a file	<code>echo "Hello" > output.txt</code>
<code>>> file</code>	Append output to a file	<code>echo "World" >> output.txt</code>
<code>2> file</code>	Redirect error output to a file	<code>ls /nonexistent 2> error.txt</code>
<code>&></code>	Redirect both stdout and stderr	<code>command &> output.txt</code>
<code>set -e</code>	Exit immediately if a command exits with a non-zero status	<code>set -e</code>
<code>set -u</code>	Treat unset variables as an error when substituting	<code>set -u</code>
<code>set -x</code>	Print commands and their arguments as they are executed	<code>set -x</code>
<code>set -v</code>	Print shell input lines as they are read	<code>set -v</code>
<code>trap 'commands' SIGNAL</code>	Trap signals and execute commands	<code>trap 'echo "Interrupted!"' INT</code>

<code>PS4='+ \$BASH_SOURCE:\$LINENO: '</code>	Customize the debugging prompt	<code>PS4='+ \$BASH_SOURCE:\$LINENO: '</code>
<code>bash -x script.sh</code>	Execute a script in debug mode	<code>bash -x myscript.sh</code>
<code>bash -n script.sh</code>	Check a script for syntax errors without executing it	<code>bash -n myscript.sh</code>
<code>exec 3>&1 4>&2</code>	Save file descriptors 1 and 2	<code>exec 3>&1 4>&2</code>
<code>exec 1>output.log 2>error.log</code>	Redirect stdout and stderr to files	<code>exec 1>output.log 2>error.log</code>
<code>exec 1>&3 2>&4</code>	Restore file descriptors 1 and 2	<code>exec 1>&3 2>&4</code>
<code>select var in list; do ... done</code>	Select loop for creating simple menus	<code>select opt in Start Stop Exit; do echo \$opt; done</code>

Summary

This expanded section on Advanced Linux Commands covers a wide range of advanced topics, from sophisticated file operations and networking to system administration and advanced scripting. Each chapter includes detailed explanations, practical examples, cheat sheets, and diagrams to enhance understanding and application.

PART IV: Shell Scripting

Chapter 13: Introduction to Shell Scripting

- What is Shell Scripting?
 - Overview: Shell scripting is a way to automate tasks in a Unix/Linux environment by writing scripts in a shell language like bash.
- Benefits of Shell Scripting
 - Benefits include task automation, time savings, repeatability, and error reduction.

Chapter 14: Getting Started with Shell Scripting

- Writing Your First Script
 - Example Script: Hello World

```
bash

#!/bin/bash
echo "Hello, World!"
```
 - Explanation: This is the simplest shell script that prints "Hello, World!" to the terminal.
- Making a Script Executable
 - Command:

```
chmod
```

```
bash

chmod +x script.sh
```
 - Explanation: `chmod +x` makes the script executable.

Basic Shell Commands Cheat Sheet and Basic Shell Script Structure

Command	Example	Description
Basic Shell Commands		
List Directory Contents	<code>ls -l</code> <code>cd /path/to/directory</code>	Lists files and directories with detailed information.
Directory Change		Changes the current working directory.
Print Working Directory	<code>pwd</code>	Displays the current directory path.
Copy File	<code>cp source.txt destination.txt</code>	Copies a file from source to destination.
Move/Rename File	<code>mv oldname.txt newname.txt</code>	Moves or renames a file.
Remove File	<code>rm file.txt</code>	Deletes a specified file.
Remove Directory	<code>rm -r directory</code>	Deletes a directory and its contents.
Create Directory	<code>mkdir new_directory</code>	Creates a new directory.
Display File Contents	<code>cat file.txt</code> <code>grep "pattern" file.txt</code>	Displays the contents of a file. Searches for a pattern in a file.
Search in Files		
Basic Shell Script Structure		
Shebang Line	<code>#!/bin/bash</code>	Specifies the script interpreter.
Comment	<code># This is a comment</code>	Adds comments to the script.
Variable Assignment	<code>VAR=value</code> <code>echo "Hello, World!"</code>	Assigns a value to a variable.
Echo Statement	<code>read VAR</code>	Prints text to the terminal.
Read User Input	<code>if [condition]; then</code> <code>commands; fi</code>	Reads input from the user.
If Statement	<code>for i in {1..5}; do</code> <code>commands; done</code>	Executes commands based on a condition.
For Loop	<code>while [condition]; do</code> <code>commands; done</code>	Iterates over a sequence of values.
While Loop	<code>function name { commands; }</code>	Repeats commands while a condition is true.
Function Definition	<code>name</code>	Defines a reusable function.
Function Call	<code>exit 0</code>	Calls a defined function.
Exit Script		Exits the script with a status code.

Chapter 15: Variables and User Input

- Using Variables

- Example Script: Simple Variable

```
bash          #!/bin/bash
NAME="John"   echo  "Hello,
$NAME"
```

- Explanation: Variables store data that can be used later in the script.

- Reading User Input

- Example Script: Read User Input

```
bash

#!/bin/bash
echo "Enter your name:"
read NAME
echo "Hello, $NAME"
```

- Explanation: `read` allows you to take input from the user.

Shell Script Variables Cheat Sheet

Command	Description	Example
variable=value	Define a variable	name="Alice"
echo \$variable	Access a variable	echo \$name
\${variable}	Access a variable	echo \${name}
readonly variable	Make a variable read-only	readonly PI=3.14
unset variable	Remove a variable	unset name
variable=\$(command)	Command substitution	today=\$(date)
variable=\$(<file)	Command substitution	content=\$(<file.txt)
var1=value1; var2=value2	Read file into variable	first="John"; last="Doe"
	Multiple variable assignment in one line	
\${#variable}	Length of a variable	echo \${#name}
\${variable:offset:length}	Substring extraction	echo \${name:0:3}

<code>\${variable/pattern/replacement}</code>	String replacement	<code>echo \${name/Al/Bob}</code>
<code>export variable</code>	Export a variable to the environment	<code>export PATH=\$PATH:/new/path</code>
<code>\$variable</code>	Access a variable	<code>echo \$name</code>
<code>\$variable/default</code>	Use default value if variable is unset	<code>echo \${name:-"Default"}</code>
<code>\${variable:=default}</code>	Set variable to default if unset	<code>echo \${name:="Default"}</code>
<code>\${variable:+replacement}</code>	Use replacement if variable is set	<code>echo \${name:+ "Hello, \$name"}</code>
<code>\${variable:?error_message}</code>	Display error message if variable is unset	<code>echo \${name:? "Name is not set"}</code>
<code>`var=\$(echo \$variable</code>	Positional parameters (arguments)	Transform variable content to uppercase
<code>\$1, \$2, ...</code>	All arguments	<code>echo "First arg: \$1"</code>
<code>\$@</code>	Number of arguments	<code>echo "All args: \$@"</code>
<code>\$#</code>	Exit status of last command	<code>echo "Number of args: \$#"</code>
<code>\$?</code>	Process ID of the script	<code>echo "Last command exit status: \$?"</code>
<code>\$\$</code>	PID of the last background job	<code>echo "Script PID: \$\$"</code>
<code>\$!</code>	Last argument of the previous command	<code>sleep 10 & echo "Last background job PID: \$!"</code>
<code>\$_</code>		<code>echo "Last argument: \$_"</code>

Chapter 16: Control Structures

- Conditional Statements

- o Example Script: If-Else Statement

```
bash
#!/bin/bash
if [ $1 -gt 100 ]
then

    echo "The number is greater than 100"
else
    echo "The number is less than or equal to 100"
fi
```

- o Explanation: if statements allow you to execute code based on conditions.

- Loops

- o Example Script: For Loop

```
bash

#!/bin/bash
for i in {1..5}
do
    echo "Iteration $i"
done
```

- o Example Script: While Loop

```
bash

#!/bin/bash
COUNTER=0
while [ $COUNTER -lt 5 ]
do

    echo "Counter: $COUNTER"
    let COUNTER=COUNTER+1
done
```

- o Explanation: Loops allow you to repeat code multiple times.

Shell Script Conditional Statements Cheat Sheet

Command	Example	Description
if [condition]; then ... fi	<code>if [-f "file.txt"]; then echo "File exists"; fi</code>	Checks if a file exists and prints a message.
if [condition]; then ... else ... fi	<code>if [-d "dir"]; then echo "Directory exists"; else echo "Directory does not exist"; fi</code>	Checks if a directory exists and prints appropriate messages.
if [condition1]; then ... elif [condition2]; then ... fi	<code>if [\$var -eq 1]; then echo "Var is 1"; elif [\$var -eq 2]; then echo "Var is 2"; fi</code>	Checks multiple conditions and executes corresponding commands.
case	<code>case \$var in 1) echo "Var is 1";; 2) echo "Var is 2";; esac</code>	Simplifies checking a variable against multiple values.
&&	<code>[-f "file.txt"] && echo "File exists"</code>	Executes the command after && only if the condition is true.
`		

Shell Script Loops Cheat Sheet

Command	Example	Description
for loop	<code>for i in {1..5}; do echo "Welcome \$i"; done</code>	Iterates over a range of numbers and prints a message.
for loop (list)	<code>for i in apple banana cherry; do echo "Fruit: \$i"; done</code>	Iterates over a list of items and prints each one.
while loop	<code>while [\$count -le 5]; do echo "Count: \$count"; count=\$((count + 1)); done</code>	Executes commands as long as a condition is true.
until loop	<code>until [\$count -gt 5]; do echo "Count: \$count"; count=\$((count + 1)); done</code>	Executes commands until a condition is true.
break	<code>for i in {1..5}; do if [\$i -eq 3]; then break; fi; echo "Welcome \$i"; done</code>	Exits the loop when the condition is met.
continue	<code>for i in {1..5}; do if [\$i -eq 3]; then continue; fi; echo "Welcome \$i"; done</code>	Skips the current iteration and continues with the next.

Chapter 17: Functions

- Creating and Using Functions

- Example Script: Basic Function

```
bash
#!/bin/bash
greet() {

    echo "Hello, $1"
}
greet "World"
```

- Explanation: Functions encapsulate code for reuse and better organization.

Cheat Sheet: Shell Script Functions

Shell Script Functions Cheat Sheet

Command	Example	Description
Defining a function	<pre>my_function() { echo "Hello, World!"; }</pre>	Defines a function named <code>my_function</code> that prints "Hello, World!".
Calling a function	<pre>my_function</pre>	Calls the previously defined function <code>my_function</code> .
Function with parameters	<pre>greet() { echo "Hello, \$1!"; }</pre>	Defines a function <code>greet</code> that takes a parameter and prints "Hello, [parameter]".
Calling a function with parameters	<pre>greet "Alice"</pre>	Calls the <code>greet</code> function with "Alice" as the parameter.
Function returning a value	<pre>sum() { return \$((\$1 + \$2)); }</pre>	Defines a function <code>sum</code> that adds two numbers and returns the result.
Capturing a function's return value	<pre>sum 2 3; result=\$?; echo \$result</pre>	Calls the <code>sum</code> function with 2 and 3 as parameters, captures the return value, and prints it.
Local variables in a function	<pre>my_func() { local var="I'm local"; echo \$var; }</pre>	Defines a function <code>my_func</code> with a local variable <code>var</code> .
Recursive function	<pre>factorial() { if [\$1 -le 1]; then echo 1; else echo \$((\$1 * \$(factorial \$((\$1 - 1)))); fi }</pre>	Defines a recursive function <code>factorial</code> that computes the factorial of a number.
Function with a return statement	<pre>is_even() { if [\$((\$1 % 2)) -eq 0]; then return 0; else return 1; fi }</pre>	Defines a function <code>is_even</code> that checks if a number is even and returns 0 if true, 1 if false.

Chapter 18: Error Handling

- Basic Error Handling

- o Example Script: Error Handling with Exit Status

```
bash
#!/bin/bash
cp /nonexistent/file /tmp/
if [ $? -ne 0 ]; then

    echo "Error occurred while copying file"
fi
```

- o Explanation: `$?` checks the exit status of the last command.

Shell Script Error Handling Cheat Sheet

Command	Example	Description
Exit on error	<code>set -e</code>	Terminates the script if any command returns a non-zero status.
Trap errors	<code>trap 'echo "Error at line \$LINENO"' ERR</code>	Runs a command (e.g., printing an error message) when any command returns a non-zero status.
Try-catch-like error handling	<code>`command` if ! command; then</code>	
Using if for error handling	<code>echo "Command failed"; fi</code>	Executes a command and, if it fails, prints an error message.
Error handling in functions	<code>`my_function() { command` `command</code>	
Custom error messages	<code>command; echo \$?</code>	
Exit status of the last command	<code>command1 && command2</code>	Executes a command and prints the exit status of the last command.
Using && for conditional execution		Executes <code>command2</code> only if <code>command1</code> succeeds.
Using <code>`</code>		<code>`</code> for conditional execution

Chapter 19: Advanced Scripting Techniques

- Working with Arrays

- Example Script: Array Example

```
bash
#!/bin/bash
FRUITS=("Apple" "Banana" "Cherry")
for FRUIT in "${FRUITS[@]}"
do

    echo "Fruit: $FRUIT"
done
```

- Explanation: Arrays store multiple values in a single variable.

- String Manipulation

- Example Script: String Operations

```
bash

#!/bin/bash
STR="Hello World"
echo "Length: ${#STR}"
echo "Substring: ${STR:0:5}"
```

- Explanation: String manipulation techniques to extract and modify substrings.

Shell Script Arrays Cheat Sheet

Command	Example	Description
Declare an array	<code>array=(val1 val2 val3)</code>	Declares an array with values <code>val1</code> , <code>val2</code> , <code>val3</code> .
Access array element	<code>\${array[0]}</code>	Accesses the first element of the array.
All elements	<code>\${array[@]}</code>	Accesses all elements of the array.
Length of array	<code>\${#array[@]}</code>	Returns the number of elements in the array.
Add element	<code>array+=(val4)</code>	Adds <code>val4</code> to the array.
Iterate over array	<code>for i in "\${array[@]"; do echo \$i; done</code>	Iterates over and prints each element in the array.
Remove element	<code>unset array[1]</code>	Removes the second element from the array.
Replace element	<code>array[1]=newval</code>	Replaces the second element with <code>newval</code> .

Shell Script String Operations Cheat Sheet

Command	Example	Description
String length	<code>\${#string}</code>	Returns the length of the string.
Substring extraction	<code>\${string:pos:len}</code>	Extracts a substring from string starting at pos with length len.
Substring replacement	<code>\${string/old/new}</code>	Replaces the first occurrence of old with new in string.
Global substring replacement	<code>\${string//old/new}</code> <code>[[\$string == *"substring"*]]</code>	Replaces all occurrences of old with new in string.
Check if string contains substring		Checks if string contains substring.
Concatenate strings	<code>newstring="\${string1}\${string2}"</code>	Concatenates string1 and string2.
Convert to uppercase	<code>\${string^^}</code>	Converts all characters in string to uppercase.
Convert to lowercase	<code>\${string,,}</code>	Converts all characters in string to lowercase.
Trim leading whitespace	<code>\${string# }</code>	Removes leading whitespace from string.
Trim trailing whitespace	<code>\${string% }</code>	Removes trailing whitespace from string.

Chapter 20: Automating System Tasks

- Scheduled Tasks with Cron

- Example Script: Cron Job

```
bash

# Run backup script at 2am every day
0 2 * * * /path/to/backup.sh
```

- Explanation: Cron jobs automate the execution of scripts at specified times.

- Automating Backups

- Example Script: Backup Script

```
bash

#!/bin/bash
tar -czf /backup/home_$(date +%F).tar.gz /home/user
```

- Explanation: Automates the backup of user home directory.

Chapter 21: Real-world Examples and Case Studies

- Case Study: Server Maintenance Script

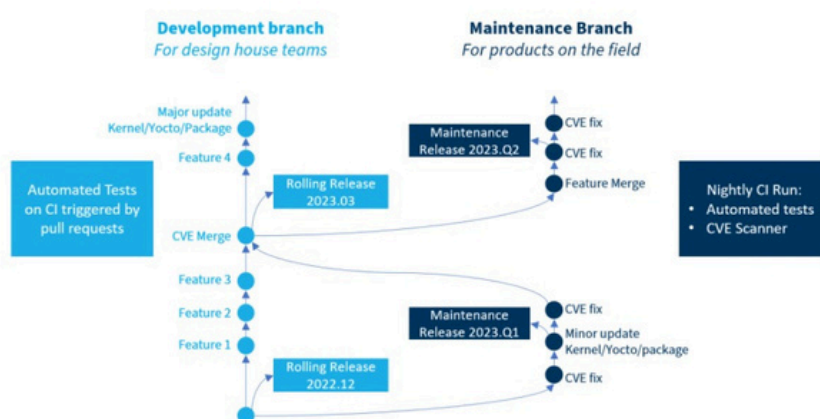
- Example Script: Server Update and Reboot

```
bash

#!/bin/bash
sudo apt-get update && sudo apt-get upgrade -y
sudo reboot
```

- Explanation: Script to update and reboot a server.

Maintenance Process



Server maintenance workflow diagram

- Case Study: Log File Management

- o Example Script: Log Rotation

```
bash
#!/bin/bash
LOG_FILE="/var/log/myapp.log"
mv $LOG_FILE $LOG_FILE.$(date +%F)
touch $LOG_FILE
```

- o Explanation: Rotates log files to prevent them from growing indefinitely.

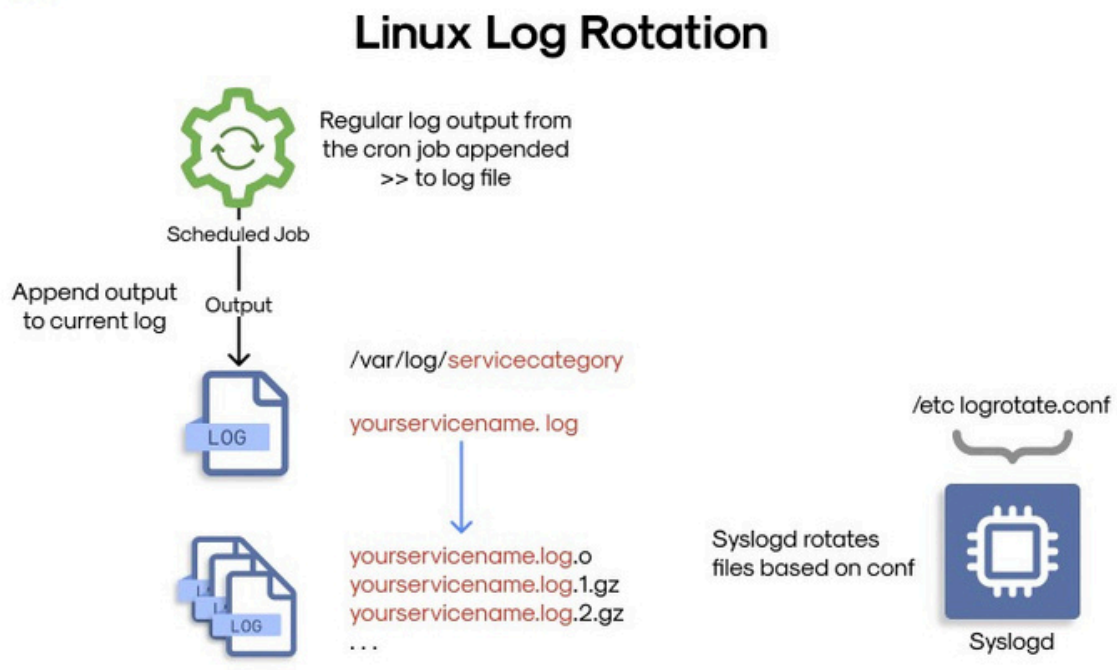


Diagram: Log Rotation Workflow

Cron Job Syntax Cheat Sheet

Command	Example	Description
Schedule a job	<code>* * * * * /path/to/script.sh</code>	Executes script.sh every minute.
Minute field	<code>30 * * * * /path/to/script.sh</code>	Executes script.sh at minute 30 of every hour.
Hour field	<code>0 8 * * * /path/to/script.sh</code>	Executes script.sh at 08:00 AM every day.
Day of month field	<code>0 8 1 * * /path/to/script.sh</code>	Executes script.sh at 08:00 AM on the first day of every month.
Month field	<code>0 8 1 1 * /path/to/script.sh</code>	Executes script.sh at 08:00 AM on January 1st.
Day of week field	<code>0 8 * * 1 /path/to/script.sh</code>	Executes script.sh at 08:00 AM every Monday.
List all cron jobs	<code>crontab -l</code> <code>crontab -e</code> <code>crontab</code>	Lists all the cron jobs for the current user.
Edit cron jobs	<code>-r crontab -u username -e * *</code>	Opens the cron job editor for the current user.
Remove all cron jobs	<code>* * * * * /path/to/script.sh</code> <code>> /path/to/logfile 2>&1</code>	Removes all cron jobs for the current user.
Specific user cron jobs		Edits the cron jobs for username.
Redirect output to file		Executes script.sh every minute and logs output to logfile.
Environment variable	<code>SHELL=/bin/bash\n* * * * *</code> <code>/path/to/script.sh</code>	Uses /bin/bash shell to execute the cron job.
Schedule multiple commands	<code>* * * * * command1 &&</code> <code>command2</code>	Executes command1 and then command2 every minute.
Use ranges	<code>0-30/5 * * * *</code> <code>/path/to/script.sh</code>	Executes script.sh every 5 minutes between minutes 0 and 30.
Use step values	<code>*/10 * * * *</code> <code>/path/to/script.sh</code>	Executes script.sh every 10 minutes.
Use specific months	<code>0 0 1 1,6,12 *</code> <code>/path/to/script.sh</code>	Executes script.sh at midnight on the 1st of January, June, and December.
Use specific days of week	<code>0 0 * * MON,WED,FRI</code> <code>/path/to/script.sh</code>	Executes script.sh at midnight on Monday, Wednesday, and Friday.

Summary

This expanded section on Shell Scripting covers fundamental to advanced topics with practical examples and scripts. Each chapter includes detailed explanations, commands, cheat sheets, diagrams, and real-world case studies to ensure a thorough understanding and application of shell scripting.

PART V: System Administration

Chapter 22: User and Group Management

- Creating and Managing Users

- o Example Script: Create a New User

```
bash

#!/bin/bash
sudo useradd -m -s /bin/bash newuser
sudo passwd newuser
```

- o Explanation: useradd creates a new user with a home directory and bash shell.

- Creating and Managing Groups

- o Example Script: Create a New Group

```
bash

#!/bin/bash
sudo groupadd newgroup
sudo usermod -aG newgroup newuser
```

- o Explanation: groupadd creates a new group and usermod -aG adds the user to the group.

- Managing Permissions

- o Example Script: Change File Permissions

```
bash

#!/bin/bash
sudo chmod 755 /path/to/file
sudo chown user:group /path/to/file
```

- o Explanation: chmod changes file permissions and chown changes file ownership.

User Management, Group Management, and File Permissions Cheat Sheet

Command	Example	Description
User Management		
Add user	<code>sudo adduser username</code>	Adds a new user with <code>username</code> .
Delete user	<code>sudo deluser username</code>	Deletes the user with <code>username</code> .
Change user password	<code>sudo passwd username</code>	Changes the password for <code>username</code> .
Switch user	<code>su - username</code>	Switches to the user <code>username</code> .
List all users	<code>cut -d: -f1 /etc/passwd</code>	Lists all users on the system.
Group Management		
Add group	<code>sudo addgroup groupname</code>	Adds a new group with <code>groupname</code> .
Delete group	<code>sudo delgroup groupname</code>	Deletes the group with <code>groupname</code> .
Add user to group	<code>sudo usermod -aG groupname username</code>	Adds <code>username</code> to <code>groupname</code> .
Remove user from group	<code>sudo gpasswd -d username groupname</code>	Removes <code>username</code> from <code>groupname</code> .
List all groups	<code>cut -d: -f1 /etc/group</code>	Lists all groups on the system.
File Permissions		
Change file permissions		Sets the permissions of <code>filename</code> to 755 (rwxr-xr-x).
Change file owner	<code>sudo chown username filename</code>	Changes the owner of <code>filename</code> to <code>username</code> .
Change file group	<code>sudo chown :groupname filename</code>	Changes the group of <code>filename</code> to <code>groupname</code> .
View file permissions	<code>ls -l filename</code>	Displays the permissions of <code>filename</code> .
Set read, write, execute for owner	<code>chmod u+rwx filename</code>	Adds read, write, and execute permissions to the owner of <code>filename</code> .
Remove read permission for group	<code>chmod g-r filename</code>	Removes read permission for the group of <code>filename</code> .
Add execute permission for others	<code>chmod o+x filename</code>	Adds execute permission for others for <code>filename</code> .
Set permissions recursively	<code>chmod -R 755 directory</code>	Sets permissions to 755 for all files and directories in <code>directory</code> .
Set permissions using symbolic notation	<code>chmod u=rw,go=r filename</code>	Sets read/write for owner, read-only for group and others for <code>filename</code> .

Chapter 23: Package Management

- Managing Packages with apt

- o Example Script: Install and Update Packages

```
bash
#!/bin/bash
sudo apt-get update
sudo apt-get install -y package_name
sudo apt-get upgrade -y
```

- o Explanation: `apt-get` commands for updating, installing, and upgrading packages.

Managing Packages with yum

- o Example Script: Install and Update Packages

```
bash
#!/bin/bash
sudo yum update
sudo yum install -y package_name
sudo yum upgrade
```

- o Explanation: `yum` commands for updating, installing, and upgrading packages.

- Managing Packages with snap

- o Example Script: Install and Update Packages

```
bash
#!/bin/bash
sudo snap install package_name
sudo snap refresh
```

- o Explanation: `snap` commands for installing and refreshing packages.

APT, YUM, and SNAP Commands Cheat Sheet

Command	Example	Description
APT Commands		
Update package list	<code>sudo apt update</code>	Updates the list of available packages and their versions.
Upgrade installed packages	<code>sudo apt upgrade</code>	Installs the newest versions of all installed packages.
Install a package	<code>sudo apt install packagename</code>	Installs the specified packagename.
Remove a package	<code>sudo apt remove packagename</code>	Removes the specified packagename but leaves configuration files.
Purge a package	<code>sudo apt purge packagename</code>	Completely removes the packagename, including configuration files.
Search for a package	<code>apt search keyword</code>	Searches for packages related to keyword.
Show package details	<code>apt show packagename</code>	Displays information about packagename.
List installed packages	<code>apt list --installed</code>	Lists all installed packages.
YUM Commands		
Update package list	<code>sudo yum check-update</code>	Checks for available updates for all packages.
Upgrade installed packages	<code>sudo yum update</code>	Updates all packages to the latest version.
Install a package	<code>sudo yum install packagename</code>	Installs the specified packagename.
Remove a package	<code>sudo yum remove packagename</code>	Removes the specified packagename.
Search for a package	<code>yum search keyword</code>	Searches for packages related to keyword.
Show package details	<code>yum info packagename</code>	Displays information about packagename.
List installed packages	<code>yum list installed</code>	Lists all installed packages.
Clean cache	<code>sudo yum clean all</code>	Cleans up the local repository cache.
SNAP Commands		
Find a package	<code>snap find packagename</code>	Searches the Snap Store for packagename.
Install a package	<code>sudo snap install packagename</code>	Installs the specified packagename from the Snap Store.
Remove a package	<code>sudo snap remove packagename</code>	Removes the specified packagename.
List installed snaps	<code>snap list</code>	Lists all installed snaps.
Refresh snaps	<code>sudo snap refresh</code>	Updates all installed snaps to the latest version.
Info about a snap	<code>snap info packagename</code>	Displays information about packagename.
Revert to previous version	<code>sudo snap revert packagename</code>	Reverts packagename to the previous version.
List available channels	<code>snap info packagename</code>	Lists the available channels for packagename (e.g., stable, beta).

Chapter 24: Disk Management

- Monitoring Disk Usage

- o Example Script: Check Disk Usage

```
bash

#!/bin/bash
df -h
du -sh /path/to/directory
```

- o Explanation: `df -h` displays disk space usage and `du -sh` displays directory size.

- Managing Disk Partitions

- o Example Script: Create and Format Partitions

```
bash

#!/bin/bash
sudo fdisk /dev/sdX
sudo mkfs.ext4 /dev/sdX1
```

- o Explanation: `fdisk` manages disk partitions and `mkfs.ext4` formats partitions.

- Mounting and Unmounting Filesystems

- o Example Script: Mount a Filesystem

```
bash

#!/bin/bash
sudo mount /dev/sdX1 /mnt
sudo umount /mnt
```

- o Explanation: `mount` attaches a filesystem to the directory tree and `umount` detaches it.

Disk Usage, Disk Partitioning, and Mounting Filesystems Commands Cheat Sheet

Command	Example	Description
Disk Usage Commands		
Check disk usage	<code>df -h</code>	Displays disk space usage in a human-readable format.
Check directory size	<code>du -sh /path/to/directory</code>	Displays the size of a specified directory.
Check disk usage of all files and directories	<code>du -ah /path/to/directory</code>	Displays disk usage of all files and directories in a specified path.
Check top largest files/directories	<code>`du -ah /path/to/directory sort -rh</code>	
Disk Partitioning Commands		
List disk partitions	<code>lsblk</code>	Displays information about all available or the specified block devices.
Start fdisk to partition a disk	<code>sudo fdisk /dev/sdX</code>	Opens fdisk utility to create or modify partitions on /dev/sdX.
Print partition table	<code>sudo fdisk -l /dev/sdX</code>	Prints the partition table of /dev/sdX.
Create a new partition	<code>sudo fdisk /dev/sdX then n</code>	Creates a new partition on /dev/sdX using fdisk.
Format a partition	<code>sudo mkfs.ext4 /dev/sdX1</code>	Formats the partition /dev/sdX1 with ext4 filesystem.
Mounting Filesystems Commands		
Mount a filesystem	<code>sudo mount /dev/sdX1 /mnt/mydrive</code>	Mounts the partition /dev/sdX1 to the directory /mnt/mydrive.
Unmount a filesystem	<code>sudo umount /mnt/mydrive</code>	Unmounts the filesystem mounted at /mnt/mydrive.
List mounted filesystems	<code>mount or df -h sudo mount -a lsblk -f</code>	Displays all mounted filesystems.
Mount all filesystems mentioned in /etc/fstab		Mounts all filesystems specified in /etc/fstab.
Show filesystem types		Lists all block devices with their filesystems. <code>sudo tee -a /etc/fstab`</code>
Add entry to /etc/fstab	<code>`echo '/dev/sdX1 /mnt/mydrive ext4 defaults 0 2'</code>	

Chapter 25: Network Management

- Configuring Network Interfaces

- o Example Script: Configure a Static IP

```
bash

#!/bin/bash
sudo ip addr add 192.168.1.100/24 dev eth0
sudo ip link set eth0 up
```

- o Explanation: `ip addr add` assigns an IP address to an interface and `ip link set` activates it.

- Monitoring Network Traffic

- o Example Script: Monitor Traffic with `iftop`

```
bash

#!/bin/bash
sudo iftop
```

- o Explanation: `iftop` displays bandwidth usage on an interface.

- Using `nmap` for Network Scanning

- o Example Script: Basic `nmap` Scan

```
bash

#!/bin/bash
sudo nmap -sP 192.168.1.0/24
```

- o Explanation: `nmap -sP` performs a ping scan to discover active devices on a network.

Network Configuration, Network Monitoring, and Nmap Commands Cheat Sheet

Command	Example	Description
Network Configuration Commands		
Display network interfaces	<code>ifconfig</code>	Shows the configuration of all network interfaces.
Display network interfaces (ip command)	<code>ip addr show</code>	Shows IP address information of network interfaces.
Bring an interface up	<code>sudo ifconfig eth0 up</code>	Activates the network interface eth0.
Bring an interface down	<code>sudo ifconfig eth0 down</code>	Deactivates the network interface eth0.
Assign IP address to interface	<code>sudo ifconfig eth0 192.168.1.10 netmask 255.255.255.0</code>	Sets the IP address and netmask for eth0.
Add a default gateway	<code>sudo route add default gw 192.168.1.1</code>	Adds a default gateway to the routing table.
Configure DNS servers	<code>sudo tee /etc/resolv.conf` `echo 'nameserver 8.8.8.8'</code>	
Network Monitoring Commands		
Check current network connections	<code>netstat -tuln</code>	Displays all listening TCP and UDP ports. Monitors network traffic by interface in real-time.
Monitor network traffic	<code>iftop</code>	Shows the kernel routing table.
Display routing table	<code>netstat -r</code> or <code>route -n</code>	Monitors network bandwidth usage in real-time.
Monitor network bandwidth	<code>nload</code>	Captures network packets on interface eth0.
Capture network packets	<code>tcpdump -i eth0</code>	Shows detailed statistics of network interfaces.
Display network statistics	<code>ip -s link</code>	Scans the specified IP address
Nmap Commands		
Basic scan of an IP or range	<code>nmap 192.168.1.1</code>	or
Scan specific ports	<code>nmap -p 22,80,443 192.168.1.1</code>	range.
Scan all ports	<code>nmap -p- 192.168.1.1</code>	Scans specified ports on the target.
Detect OS and services	<code>nmap -A 192.168.1.1</code>	Scans all 65535 ports on the target.
Scan multiple targets	<code>nmap 192.168.1.1,2,3</code>	Enables OS detection, version detection, script scanning, and traceroute.
Aggressive scan	<code>nmap -T4 -A -v 192.168.1.1</code>	Scans multiple IP addresses. Performs an aggressive scan
Save scan output	<code>nmap -oN output.txt 192.168.1.1</code>	with verbose output.
		Saves the scan results to a file.

Chapter 26: System Monitoring and Performance Tuning

- Monitoring System Performance

- o Example Script: Monitor with top

```
bash

#!/bin/bash
top
```

- o Explanation: top displays real-time system performance and process information.

- Using htop for Enhanced Monitoring

- o Example Script: Install and Run htop

```
bash

#!/bin/bash
sudo apt-get install -y htop
htop
```

- o Explanation: htop provides an enhanced, interactive system-monitoring interface.

- Tuning System Performance

- o Example Script: Modify sysctl Settings

```
bash
#!/bin/bash
sudo sysctl -w net.ipv4.tcp_syncookies=1
sudo sysctl -p
```

- o Explanation: `sysctl` modifies kernel parameters at runtime.

System Monitoring, Htop, and Performance Tuning Commands Cheat Sheet

Command	Example	Description
System Monitoring Commands		
Display system uptime	<code>uptime</code>	Shows how long the system has been running and the load average.
Display memory usage	<code>free -h</code>	Shows the amount of free and used memory in the system.
Display disk usage	<code>df -h</code>	Shows the amount of disk space used and available on filesystems.
Display CPU information	<code>lscpu</code>	Shows detailed information about the CPU architecture.
Display processes	<code>ps aux</code>	Shows a snapshot of all current processes.
Real-time system monitoring	<code>top</code>	Provides a dynamic view of system processes and resource usage.
Check system logs	<code>dmesg</code>	Displays the system message buffer (kernel logs).
Display hardware information	<code>lshw</code>	Provides detailed information about all hardware components.
Htop Commands		
Start htop	<code>htop</code>	Launches the htop interactive process viewer.
Sort by CPU usage	Press F6 , then select <code>PERCENT_CPU</code>	Sorts processes by CPU usage within htop.
Sort by memory usage	Press F6 , then select <code>PERCENT_MEM</code>	Sorts processes by memory usage within htop.
Kill a process	Press F9 , then select signal	Sends a signal to terminate a selected process within htop.
Search for a process	Press F3 , then type process name	Searches for a specific process within htop.
Filter processes	Press F4 , then type filter string	Filters processes based on the input string within htop.
Performance Tuning Commands		
Check CPU frequency scaling	<code>cpupower frequency-info</code>	Displays current CPU frequency and scaling information.
Set CPU governor	<code>sudo cpupower frequency-set -g performance</code>	Sets the CPU frequency governor to "performance".
Adjust swappiness	<code>sudo sysctl vm.swappiness=10</code>	Sets the kernel parameter for swap space usage.
Adjust cache pressure	<code>sudo sysctl vm.vfs_cache_pressure=50</code>	Sets the kernel parameter for cache reclaiming.
Limit process CPU usage	<code>cpulimit -l 50 -p <PID></code>	Limits a process to a maximum of 50% CPU usage.
Monitor I/O performance	<code>iostat -xz 1</code>	Monitors I/O performance and statistics in real-time.
Analyze system performance	<code>perf stat</code>	Collects and displays performance data for a given command.
Profile CPU usage	<code>perf top</code>	Provides a real-time view of CPU usage and hotspots.

Chapter 27: Security and Access Control

- Configuring SSH

- o Example Script: Configure SSH Server

```
bash
#!/bin/bash
sudo apt-get install -y openssh-server
sudo systemctl enable ssh
sudo systemctl start ssh
```

- o Explanation: Installs and starts the SSH server.

- Firewall Management with ufw

- o Example Script: Basic ufw

- Commands

```
bash #!/bin/bash
sudo ufw allow 22
sudo ufw enable
sudo ufw status
```

- o Explanation: ufw is a user-friendly front-end for managing iptables firewall rules.

- Using SELinux for Enhanced Security

- o Example Script: Enable SELinux

```
bash
#!/bin/bash
sudo setenforce 1
sudo getenforce
```

- o Explanation: `setenforce` enables SELinux in enforcing mode.

SSH Configuration, UFW, and SELinux Commands Cheat Sheet

Command	Example	Description
SSH Configuration Commands		
Generate SSH key pair	<code>ssh-keygen -t rsa -b 4096 -C "your_email@example.com"</code>	Generates a new SSH key pair with RSA 4096-bit encryption.
Copy SSH key to remote host	<code>ssh-copy-id user@remote_host</code>	Copies the SSH public key to the remote host for passwordless login.
Connect to a remote host	<code>ssh user@remote_host</code>	Connects to a remote host using SSH.
SSH config file location	<code>~/.ssh/config</code>	The SSH client configuration file path.
SSH config file entry	<code>Host example\n HostName example.com\n User user\n IdentityFile ~/.ssh/id_rsa</code>	An example entry in the SSH config file to simplify connections.
Disable root login	<code>PermitRootLogin no</code> (in <code>/etc/ssh/sshd_config</code>)	Prevents root user from logging in via SSH.
Change SSH port	<code>Port 2222</code> (in <code>/etc/ssh/sshd_config</code>)	Changes the default SSH port from 22 to 2222.
Restart SSH service	<code>sudo systemctl restart sshd</code>	Restarts the SSH service to apply configuration changes.
UFW Commands		
Enable UFW	<code>sudo ufw enable</code>	Enables the Uncomplicated Firewall (UFW).
Disable UFW	<code>sudo ufw disable</code>	Disables UFW.
Check UFW status	<code>sudo ufw status</code>	Displays the current status of UFW, including rules.
Allow a port	<code>sudo ufw allow 22</code>	Allows traffic on port 22 (SSH).

Deny a port	<code>sudo ufw deny 22</code> <code>sudo ufw allow from 192.168.1.100</code>	Denies traffic on port 22.
Allow specific IP		Allows traffic from a specific IP address.
Deny specific IP	<code>sudo ufw deny from 192.168.1.100</code>	Denies traffic from a specific IP address.
Allow a port range	<code>sudo ufw allow 3000:4000/tcp</code>	Allows traffic on a range of ports (3000-4000) for TCP.
Delete a rule	<code>sudo ufw delete allow 22</code>	Deletes the rule allowing traffic on port 22.
SELinux Commands		
Check SELinux status	<code>sestatus</code>	Displays the current status and mode of SELinux.
Set SELinux mode to permissive	<code>sudo setenforce 0</code>	Sets SELinux to permissive mode temporarily.
Set SELinux mode to enforcing	<code>sudo setenforce 1</code>	Sets SELinux to enforcing mode temporarily.
Make SELinux permissive permanently	<code>sudo sed -i 's/SELINUX=enforcing/SELINUX=permissive/' /etc/selinux/config</code>	Sets SELinux to permissive mode permanently by editing the config file.
Relabel filesystem	<code>sudo touch /.autorelabel && sudo reboot</code>	Forces SELinux to relabel the filesystem on next reboot.
List SELinux policies	<code>sudo semanage policy -l</code>	Lists all loaded SELinux policies.
Add SELinux port context	<code>sudo semanage port -a -t http_port_t -p tcp 8080</code>	Adds a port context for SELinux (e.g., allowing HTTP on port 8080).
Modify SELinux boolean	<code>sudo setsebool -P httpd_can_network_connect on</code>	Changes the state of an SELinux boolean (e.g., allowing HTTPD to connect to the network).

Chapter 28: Case Studies and Real-world Applications

- Case Study: Setting Up a Web Server

- Example Script: Install and Configure Apache

```
bash
#!/bin/bash
sudo apt-get update
sudo apt-get install -y apache2
sudo systemctl enable apache2
sudo systemctl start apache2
```

- Explanation: Steps to install and configure Apache web server.

- Case Study: Implementing a Backup System

- Example Script: Automated Backup Script

```
bash
#!/bin/bash
tar -czf /backup/home_$(date +%F).tar.gz /home/user
```

- Explanation: Script to back up the home directory.

- Case Study: Network Configuration for a Small Office

- Example Script: Configure DHCP Server

```
bash
#!/bin/bash
sudo apt-get install -y isc-dhcp-server
sudo systemctl enable isc-dhcp-server
sudo systemctl start isc-dhcp-server
```

- Explanation: Steps to install and configure a DHCP server.

Summary

This expanded section on System Administration covers essential topics with practical examples and scripts. Each chapter includes detailed explanations, commands, cheat sheets, diagrams, and real-world case studies to ensure a comprehensive understanding and application of system administration tasks.

PART VI: Advanced Topics

Chapter 29: Advanced File System Management

- File System Types and Their Uses
 - Explanation: Overview of different file systems like ext4, XFS, Btrfs, and ZFS.

File System Support									
File System	Create	Grow	Shrink	Move	Copy	Check	Label	UUID	Required Software
btrfs	Available offline and online	Available offline and online	Available offline and online	Available offline and online	Available offline and online	Available offline and online	Available offline and online	Available offline and online	btrfs-progs / btrfs-tools
exfat	Available offline and online	Available offline and online	Available offline and online	Available online only	Available online only	Available offline only	Available offline only	Available offline only	
ext2	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	e2fsprogs
ext3	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	e2fsprogs
ext4	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	e2fsprogs v1.41+
f2fs	Available offline and online	Available offline and online	Available offline and online	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	f2fs-tools
fat16	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	dosfstools, mtools
fat32	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	dosfstools, mtools
hfs	Available offline and online	Available offline and online	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	hfsutils
hfs+	Available offline and online	Available offline and online	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	hfsprogs
jfs	Available offline and online	Available offline and online	Available offline and online	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	jfsutils
linux-swap	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	util-linux
luks	Available offline and online	Available offline and online	Available offline and online	Available offline only	Available offline only	Available offline only	Available offline only	Available offline only	cryptsetup, dmsetup

Legend

This chart shows the actions supported on file systems. Not all actions are available on all file systems, in part due to the nature of file systems and limitations in the required software.

- Available offline and online
- Available online only
- Available offline only
- Not Available

Rescan For Supported Actions Close

File system comparison chart

- Configuring RAID
 - Example Script: Configure RAID 1

```
bash
#!/bin/bash
sudo mdadm --create --verbose /dev/md0 --level=1 --raid-
devices=2 /dev/sdX /dev/sdY
sudo mkfs.ext4 /dev/md0
sudo mount /dev/md0 /mnt
```
 - Explanation: mdadm creates a RAID array, mkfs.ext4 formats it, and mount attaches it to the filesystem.

- LVM (Logical Volume Management)
 - o Example Script: Create and Manage LVM

```
bash

#!/bin/bash
sudo pvcreate /dev/sdX
sudo vgcreate vg0 /dev/sdX
sudo lvcreate -L 10G -n lv0 vg0
sudo mkfs.ext4 /dev/vg0/lv0
sudo mount /dev/vg0/lv0 /mnt
```
 - o Explanation: Commands to create a physical volume, volume group, and logical volume.

RAID and LVM Commands Cheat Sheet

Command	Example	Description
RAID Commands		
Create RAID 0	<code>sudo mdadm --create --verbose /dev/md0 --level=0 --raid-devices=2 /dev/sda1 /dev/sdb1</code>	Creates a RAID 0 array with two devices.
Create RAID 1	<code>sudo mdadm --create --verbose /dev/md0 --level=1 --raid-devices=2 /dev/sda1 /dev/sdb1</code>	Creates a RAID 1 array with two devices.
Create RAID 5	<code>sudo mdadm --create --verbose /dev/md0 --level=5 --raid-devices=3 /dev/sda1 /dev/sdb1 /dev/sdc1</code>	Creates a RAID 5 array with three devices.
Create RAID 6	<code>sudo mdadm --create --verbose /dev/md0 --level=6 --raid-devices=4 /dev/sda1 /dev/sdb1 /dev/sdc1 /dev/sdd1</code>	Creates a RAID 6 array with four devices.
Create RAID 10	<code>sudo mdadm --create --verbose /dev/md0 --level=10 --raid-devices=4 /dev/sda1 /dev/sdb1 /dev/sdc1 /dev/sdd1</code>	Creates a RAID 10 array with four devices.
View RAID status	<code>cat /proc/mdstat</code>	Displays the status of RAID devices. Provides detailed information about a specific RAID array.
Detail RAID array	<code>sudo mdadm --detail /dev/md0</code>	Stops a specific RAID array.
Stop RAID array	<code>sudo mdadm --stop /dev/md0</code>	Removes a specific RAID array.
Remove RAID array	<code>sudo mdadm --remove /dev/md0</code>	Adds a spare device to an existing RAID array.
Add spare to RAID	<code>sudo mdadm --manage /dev/md0 --add /dev/sdc1</code>	Initializes a physical volume for use by LVM.
LVM Commands	<code>sudo pvcreate /dev/sda1</code>	Lists all physical volumes.
Create physical volume (PV)	<code>sudo pvs</code>	Creates a volume group named vg1 with specified physical volumes.
Display physical volumes	<code>sudo vgcreate vg1 /dev/sda1 /dev/sdb1</code>	Lists all volume groups.
Create volume group (VG)	<code>sudo vgs</code>	Creates a logical volume named lv1 of size 10G in volume group vg1.
Display volume groups	<code>sudo lvcreate -L 10G -n lv1 vg1</code>	Lists all logical volumes.
Create logical volume (LV)		Extends the logical volume lv1 by 5G.
Display logical volumes	<code>sudo lvs</code>	Resizes the filesystem on a logical volume after extending it.
Extend logical volume	<code>sudo lvextend -L +5G /dev/vg1/lv1</code>	Reduces the size of logical volume
Resize filesystem	<code>sudo resize2fs /dev/vg1/lv1</code>	lv1 by 5G (requires unmounting and filesystem shrinkage first).
Reduce logical volume	<code>sudo lvreduce -L -5G /dev/vg1/lv1</code>	Removes the specified logical volume.
Remove logical volume	<code>sudo lvremove /dev/vg1/lv1</code>	Removes the specified volume group.
Remove volume group	<code>sudo vgremove vg1</code>	Removes the specified physical volume from LVM control.
Remove physical volume	<code>sudo pvremove /dev/sda1</code>	

Chapter 30: Kernel Tuning and Optimization

- Kernel Parameters and sysctl

- Example Script: Tune Kernel Parameters

```
bash
```

```
#!/bin/bash
```

```
sudo sysctl -w vm.swappiness=10
```

```
sudo sysctl -w net.ipv4.tcp_rmem='4096 87380 6291456'
```

```
sudo sysctl -p
```

- Explanation: Adjust kernel parameters for memory management and TCP performance.

- Building and Patching the Kernel

- Example Script: Compile and Install a Custom Kernel

```
bash #!/bin/bash
```

```
wget https://cdn.kernel.org/pub/linux/kernel/v5.x/linux-5.10.tar.xz
```

```
tar -xf linux-5.10.tar.xz
```

```
cd linux-5.10
```

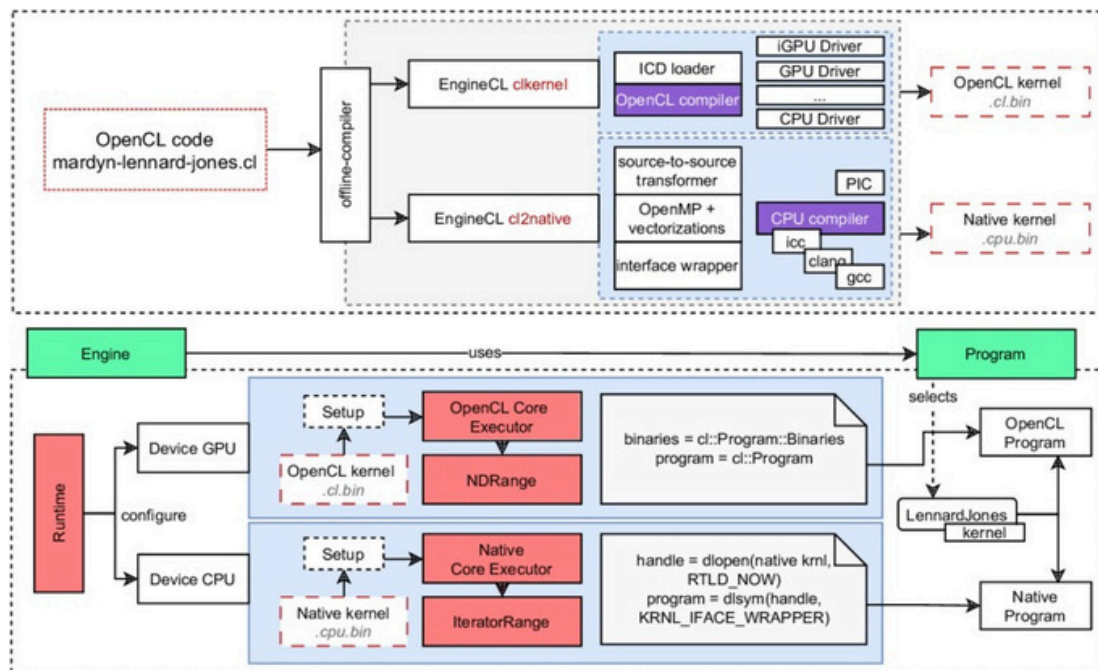
```
make menuconfig
```

```
make -j$(nproc)
```

```
sudo make modules_install
```

```
sudo make install
```

- Explanation: Steps to download, configure, compile, and install a custom kernel.



Kernel compilation workflow diagram

sysctl Commands Cheat Sheet

Command	Example	Description
List all kernel parameters	<code>sysctl -a</code> <code>sysctl net.ipv4.ip_forward</code>	Displays all the current kernel parameters and their values.
Get value of a specific parameter		Retrieves the current value of the <code>net.ipv4.ip_forward</code> parameter.
Set value of a specific parameter	<code>sudo sysctl -w net.ipv4.ip_forward=1</code>	Sets the <code>net.ipv4.ip_forward</code> parameter to 1.
Load settings from a configuration file	<code>sudo sysctl -p /path/to/file</code>	Loads and applies settings from a specified configuration file.
Persist kernel parameters	<code>`echo "net.ipv4.ip_forward = 1"</code>	<code>sudo tee -a /etc/sysctl.conf`</code>
Reload sysctl configuration	<code>sudo sysctl -p</code>	Reloads the configuration from <code>/etc/sysctl.conf</code> .
Apply a single parameter from the command line	<code>sudo sysctl -w kernel.hostname="newhostname"</code>	Changes the hostname to <code>newhostname</code> .
Apply settings from a custom file	<code>sudo sysctl --system</code>	Applies all settings from all <code>*.conf</code> files in <code>/etc/sysctl.d</code> , <code>/usr/lib/sysctl.d</code> , and <code>/run/sysctl.d</code> .
Delete a specific parameter	<code>sudo sysctl -w net.ipv4.ip_forward=""</code> <code>sysctl -n net.ipv4.ip_forward</code>	Deletes the <code>net.ipv4.ip_forward</code> parameter.
Check if a parameter is present		Checks if the <code>net.ipv4.ip_forward</code> parameter is present and prints its value.

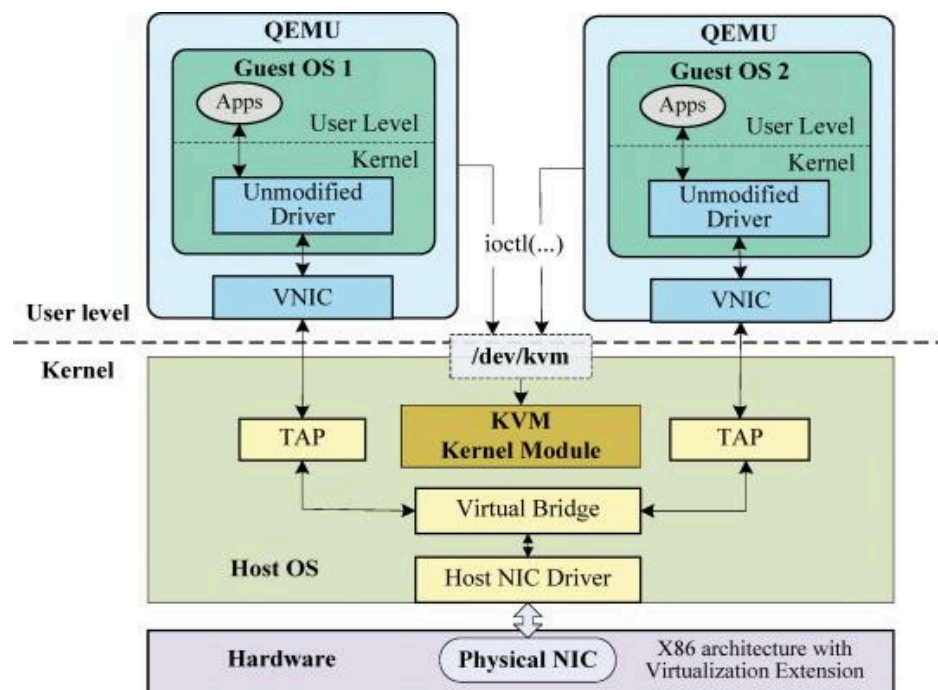
Chapter 31: Virtualization and Containers

- Introduction to Virtualization with KVM

- Example Script: Install and Configure KVM

```
bash
#!/bin/bash
sudo apt-get install -y qemu-kvm libvirt-daemon-system libvirt-
clients bridge-utils
sudo systemctl enable libvirtd
sudo systemctl start libvirtd
sudo virsh list --all
```

- Explanation: Commands to install and start KVM for virtualization.



"KVM virtualization architecture diagram"

- Using Docker for Containerization

- Example Script: Basic Docker Commands

```
bash #!/bin/bash
sudo apt-get install -y docker.io
sudo systemctl start docker
sudo docker run hello-world
sudo docker ps -a
```

- o Explanation: Install Docker, run a test container, and list containers.
- **Advanced Docker Usage**
 - o **Example Script: Docker Compose**

```
bash
#!/bin/bash
sudo apt-get install -y docker-compose
cat > docker-compose.yml <<EOF
version: '3'
services:

  web:
    image: nginx
    ports:
      - "80:80"
EOF
sudo docker-compose up
```
 - o Explanation: Use Docker Compose to manage multi-container applications.

Docker Commands Cheat Sheet

Command	Example	Description
List all Docker containers	<code>docker ps</code>	Lists all running containers. Lists all
List all containers (including stopped)	<code>docker ps -a</code>	containers, both running and stopped. Starts a stopped
Start a container	<code>docker start <container_id></code>	container. Stops a running
Stop a container	<code>docker stop <container_id></code>	container. Removes a stopped
Remove a container	<code>docker rm <container_id></code>	container. Lists all images on the
List all Docker images	<code>docker images</code>	local machine. Downloads an image
Pull an image from Docker Hub	<code>docker pull <image_name></code>	from Docker Hub. Runs a container from
Run a container from an image	<code>docker run <image_name></code>	the specified image. Runs a container and
Run a container with an interactive terminal	<code>docker run -it <image_name> /bin/bash</code>	provides an interactive terminal. Builds an image from a
Build an image from a Dockerfile	<code>docker build -t <image_name> .</code>	Dockerfile in the current directory. Deletes an image from
Remove an image	<code>docker rmi <image_name></code>	the local machine. Displays the logs from
Show logs from a container	<code>docker logs <container_id></code>	a running container. Runs a command in an
Execute a command in a running container	<code>docker exec -it <container_id> <command></code>	existing container. Displays resource
List container resource usage statistics	<code>docker stats</code>	usage statistics for running
Show detailed information about Docker objects	<code>docker inspect <container_id or image_name></code>	containers. Provides detailed
Copy files from container to host	<code>docker cp <container_id>:/path/to/file /host/path</code>	information about a container or image. Copies a file from a
Remove all stopped containers	<code>docker container prune</code>	container to the host machine. Removes all stopped
Remove all unused images	<code>docker image prune</code>	containers. Deletes all unused
Network management	<code>docker network ls</code>	images to free up space. Lists all Docker
Volume management	<code>docker volume ls</code>	networks. Lists all Docker
Create a new volume	<code>docker volume create <volume_name></code>	volumes. Creates a new Docker
Attach a volume to a container	<code>docker run -v <volume_name>:/path/in/container <image_name></code>	volume. Attaches a volume to a
		container at the specified path.

Chapter 32: High Availability and Load Balancing

- Setting Up a Load Balancer with HAProxy
 - Example Script: Install and Configure HAProxy

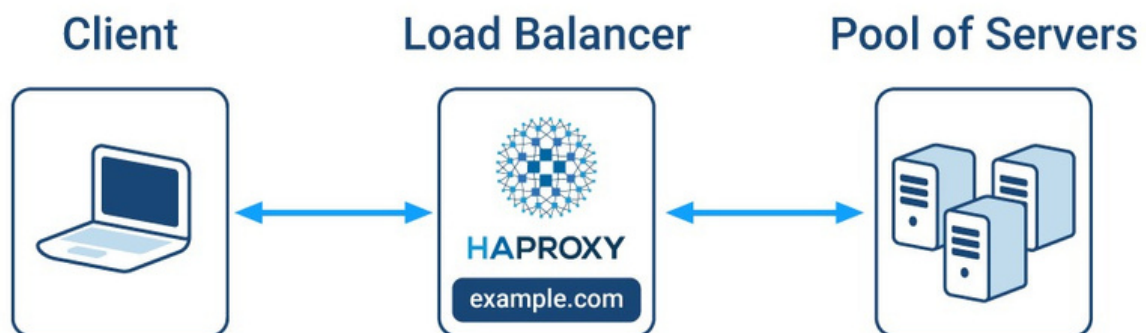
```
bash

#!/bin/bash
sudo apt-get install -y haproxy
sudo tee /etc/haproxy/haproxy.cfg <<EOF
frontend http_front

    bind *:80
    default_backend http_back

backend http_back
    balance roundrobin
    server server1 192.168.1.2:80 check
    server server2 192.168.1.3:80 check
EOF
sudo systemctl restart haproxy
```

- Explanation: Configure HAProxy for load balancing HTTP traffic.



HAProxy load balancing diagram

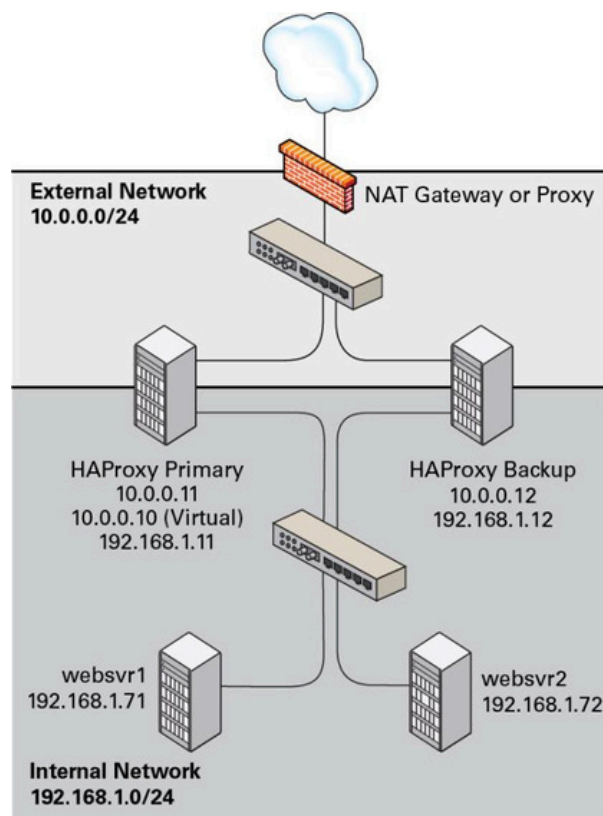
- **Configuring High Availability with Keepalived**

- **Example Script: Set Up Keepalived**

```
bash
#!/bin/bash
sudo apt-get install -y keepalived
sudo tee /etc/keepalived/keepalived.conf <<EOF
vrrp_instance VI_1 {
    state MASTER
    interface eth0
    virtual_router_id 51
    priority 101
    advert_int 1
    authentication {

        auth_type    PASS
        auth_pass    1234
    }
    virtual_ipaddress {
        192.168.1.100
    }
}
EOF
sudo systemctl restart keepalived
```

- **Explanation: Configure Keepalived for high availability with VRRP.**



Chapter 33: Security and Hardening

- Implementing SELinux Policies

- o Example Script: Enable and Configure SELinux

```
bash
#!/bin/bash
sudo setenforce 1
sudo semanage fcontext -a -t httpd_sys_content_t
'/var/www/html(/.*)?'
sudo restorecon -R /var/www/html
```

- o Explanation: Enable SELinux and set security contexts for web directories.

- Using AppArmor for Application Security

- o Example Script: Configure AppArmor Profiles

```
bash
#!/bin/bash
sudo apt-get install -y apparmor apparmor-utils
sudo aa-enforce /etc/apparmor.d/usr.sbin.nginx
sudo aa-status
```

- o Explanation: Install and enforce AppArmor profiles for application security.

- Advanced Firewall Configuration with iptables

- o Example Script: Configure iptables Rules

```
bash #!/bin/bash
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -j DROP
sudo iptables-save | sudo tee /etc/iptables/rules.v4
```

- o Explanation: Create and save iptables rules for allowing SSH and HTTP traffic while dropping all others.

SELinux Commands Cheat Sheet

Command	Example	Description
SELinux Commands		
Check SELinux status	<code>sestatus</code>	Displays the current status of SELinux.
Enable SELinux	<code>setenforce 1</code>	Sets SELinux to enforcing mode.
Disable SELinux	<code>setenforce 0</code>	Sets SELinux to permissive mode.
Change SELinux mode permanently	<code>/etc/selinux/config</code>	Edits the SELinux configuration file to set the mode permanently.
List SELinux booleans	<code>getsebool -a</code>	Displays all SELinux booleans and their states.
Change SELinux boolean	<code>setsebool <boolean_name> on</code>	Changes the state of an SELinux boolean.
List SELinux contexts for files	<code>ls -Z</code>	Lists SELinux contexts for files and directories.
Change SELinux context for a file	<code>chcon -t <type> <file></code>	Changes the SELinux context type for a file.
Restore default SELinux context	<code>restorecon -Rv <directory></code>	Restores the default SELinux context for files and directories recursively.

Chapter 34: Troubleshooting and Optimization

- System Log Analysis

- o Example Script: Analyze Logs with journalctl

```
bash
#!/bin/bash
sudo journalctl -u apache2
sudo journalctl --since "2023-01-01" --until "2023-01-31"
```

- o Explanation: Use journalctl to view logs for specific services and time ranges.

- Performance Monitoring with Performance Co-Pilot (PCP)

- o Example Script: Install and Use PCP

```
bash
#!/bin/bash
sudo apt-get install -y pcp
sudo systemctl start pmcd
sudo pmstat
```

- o Explanation: Install PCP and use pmstat to monitor system performance.

- Memory and CPU Optimization

- o Example Script: Optimize Memory Usage

```
bash

#!/bin/bash
sudo sysctl -w vm.dirty_ratio=10
sudo sysctl -w vm.dirty_background_ratio=5
sudo sysctl -p
```

- o Explanation: Adjust kernel parameters to optimize memory usage.

Command	Example	Description
journalctl Commands		
View all logs	journalctl journalctl -u <unit_name>	Displays all logs from the systemd journal.
View logs for a specific unit		Displays logs for a specific systemd unit (e.g., journalctl -u nginx).
View logs by priority	journalctl -p <priority>	Displays logs of a specified priority (e.g., journalctl -p err).
View logs within a time range	journalctl --since "2021-01-01" --until "2021-01-02"	Displays logs within a specific time range.
Follow live logs	journalctl -f	Continuously displays new log entries (similar to tail -f).
View kernel logs	journalctl -k	Displays only kernel log messages.
View logs for the current boot	journalctl -b	Displays logs from the current boot.
View logs for a specific boot	journalctl -b -1	Displays logs from the previous boot (use -2, -3, etc. for earlier boots).
Clear all journal logs	journalctl --vacuum-time=1s	Deletes all journal logs older than 1 second.
View logs for a specific user	journalctl _UID=<user_id>	Displays logs for a specific user by UID.

Chapter 35: Real-life Applications and Case Studies

- Case Study: Automating System Maintenance

- o Example Script: Automated Maintenance Script

```
bash
#!/bin/bash
sudo apt-get update
sudo apt-get upgrade -y
sudo apt-get autoremove -y
sudo reboot
```

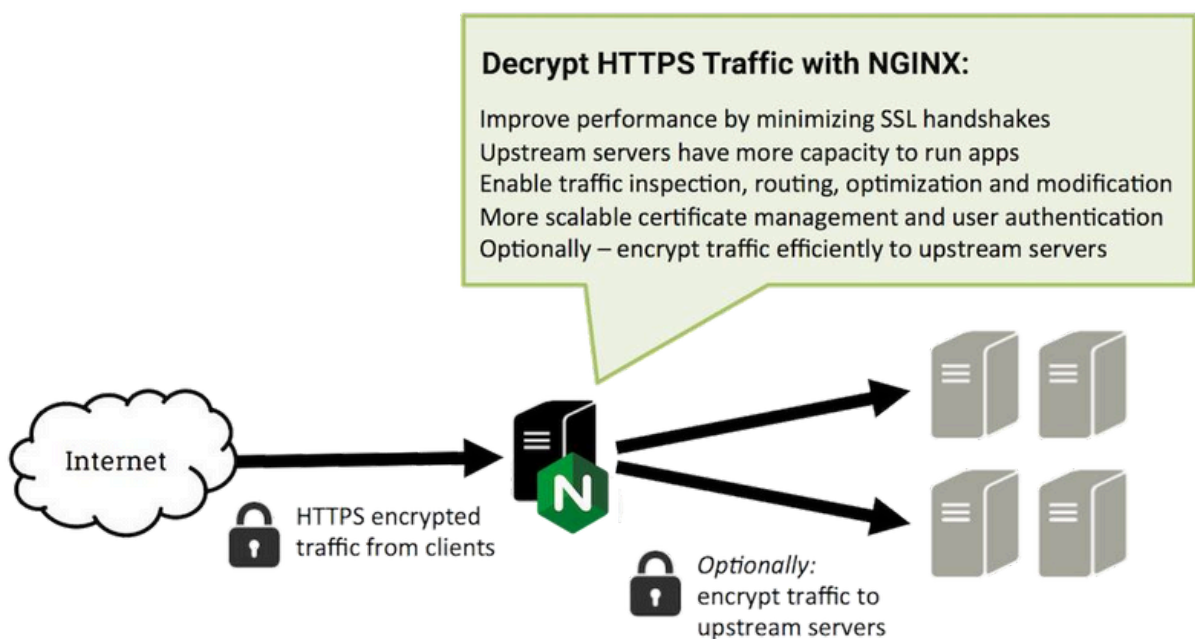
- o Explanation: Script to automate system updates and maintenance.

- Case Study: Setting Up a Secure Web Server

- o Example Script: Install and Secure Nginx with Let's Encrypt

```
bash
#!/bin/bash
sudo apt-get install -y nginx
sudo systemctl start nginx
sudo apt-get install -y certbot python3-certbot-nginx
sudo certbot --nginx -d yourdomain.com
sudo systemctl reload nginx
```

- o Explanation: Steps to install and secure Nginx with SSL/TLS certificates from Let's Encrypt.



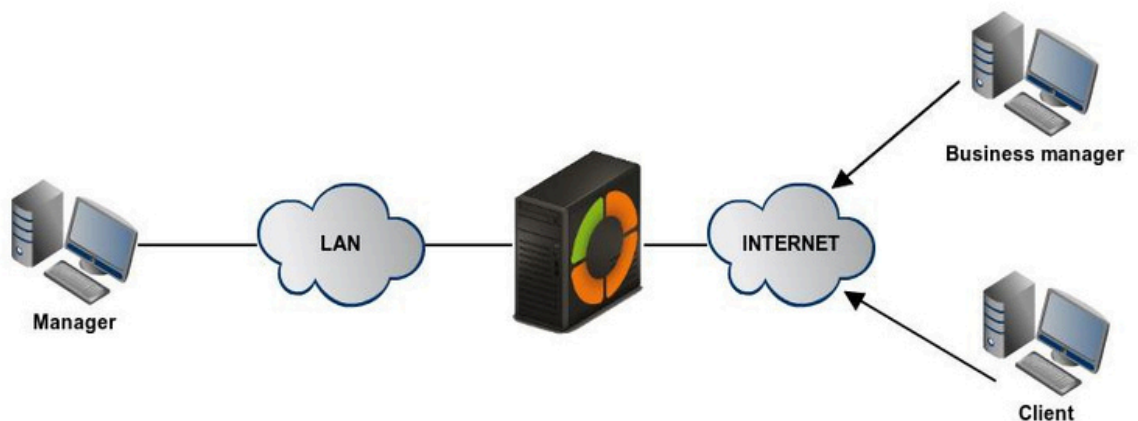
Secure connection using HTTPS on nginx

- Case Study: Configuring a VPN Server

- o Example Script: Install and Configure OpenVPN

```
bash
#!/bin/bash
sudo apt-get install -y openvpn easy-rsa
make-cadir ~/openvpn-ca
cd ~/openvpn-ca
source vars
./build-ca
./build-key-server server
./build-dh
openvpn --genkey --secret keys/ta.key
cd keys
sudo cp ca.crt server.crt server.key dh2048.pem ta.key
/etc/openvpn
sudo cp /usr/share/doc/openvpn/examples/sample-config-files/server.conf.gz /etc/openvpn
cd /etc/openvpn
sudo gzip -d server.conf.gz
sudo systemctl start openvpn@server
sudo systemctl enable openvpn@server
```

- o Explanation: Steps to install and configure an OpenVPN server for secure remote access.



VPN server configuration diagram

Summary

This expanded section on Advanced Topics delves into sophisticated aspects of Linux administration, providing practical scripts, explanations, commands, cheat sheets, diagrams, and case studies. Each chapter equips readers with the necessary knowledge and tools to handle complex system administration tasks effectively.

PART VII: Case Studies and Practical Examples

Chapter 36: Automating Daily System Tasks

- Case Study: Scheduled Backups with Cron

- Example Script: Automate Backups Using Cron

```
bash

#!/bin/bash
crontab -l > mycron
echo "0 2 * * * /usr/bin/rsync -a /home/user/
/backup/home/user/" >> mycron
crontab mycron
rm mycron
```

- Explanation: This script uses crontab to schedule a daily backup at 2 AM using rsync.
 - Diagram: Cron Job Workflow
 - Search: "cron job workflow diagram"

- Example: System Health Monitoring with Scripts

- Example Script: Health Check Script

```
bash

#!/bin/bash
df -h | mail -s "Disk Usage Report" user@example.com
free -m | mail -s "Memory Usage Report" user@example.com
```

- Explanation: This script checks disk and memory usage and emails the report to the system admin.

Cron Expressions and Mail Command Options Cheat Sheet

Command/Expression	Example	Description
Cron Expressions		
Minute Hour Day of Month Month Day	0 0 1 * *	At minute 0
of Week Every 5 minutes Every hour	* /5 * * * *	At the top of the hour (0)
at the 30th minute Daily at midnight	30 * * * *	On the first day of the month
Every Sunday at midnight On the 1st	0 0 * * * 0	Every month
of every month at midnight Every	0 * * 0 0 0	Every day of the week
weekday at midnight On January 1st	1 * * 0 0 *	Runs every 5 minutes
at midnight Last day of the month at	* 1-5 0 0 1	Runs every hour at 30 minutes past the hour
midnight	1 * 0 0 L *	Runs daily at midnight
	*	Runs every Sunday at midnight
		Runs at midnight on the 1st of every month
		Runs every weekday at midnight
		Runs at midnight on January 1st
		Runs at midnight on the last day of the month

Chapter 37: Network Administration

- Case Study: Network Performance Monitoring

- o Example Script: Monitor Network Bandwidth with **iftop**

```
bash

#!/bin/bash
sudo apt-get install -y iftop
sudo iftop -n -B -i eth0
```

- o Explanation: This script installs iftop and monitors network bandwidth on interface eth0.

- Example: Network Troubleshooting with **nmap**

- o Example Script: Scan Network for Open Ports

```
bash
#!/bin/bash
sudo apt-get install -y nmap
nmap -sP 192.168.1.0/24
```

- o Explanation: This script installs nmap and scans the local network for active hosts and open ports.

iftop and nmap Command Cheat Sheet

Command	Example	Description
iftop Commands		
Start iftop	<code>iftop</code>	Starts the iftop network bandwidth monitoring tool.
Specify interface	<code>iftop -i eth0</code>	Monitors the eth0 network interface.
Show port numbers	<code>iftop -P</code>	Displays port numbers as well as hostnames/IP addresses.
Display in bytes	<code>iftop -B</code>	Displays bandwidth usage in bytes instead of bits.
Disable DNS resolution	<code>iftop -n</code>	Prevents iftop from resolving hostnames, showing IP addresses only.
Set screen update rate	<code>iftop -t</code>	Sets the screen update rate to "text" mode.
Set limit for display	<code>iftop -L 50</code>	Limits the display to the top 50 lines.
Set custom filter	<code>iftop -f "port 80"</code>	Filters the display to show only traffic on port 80.

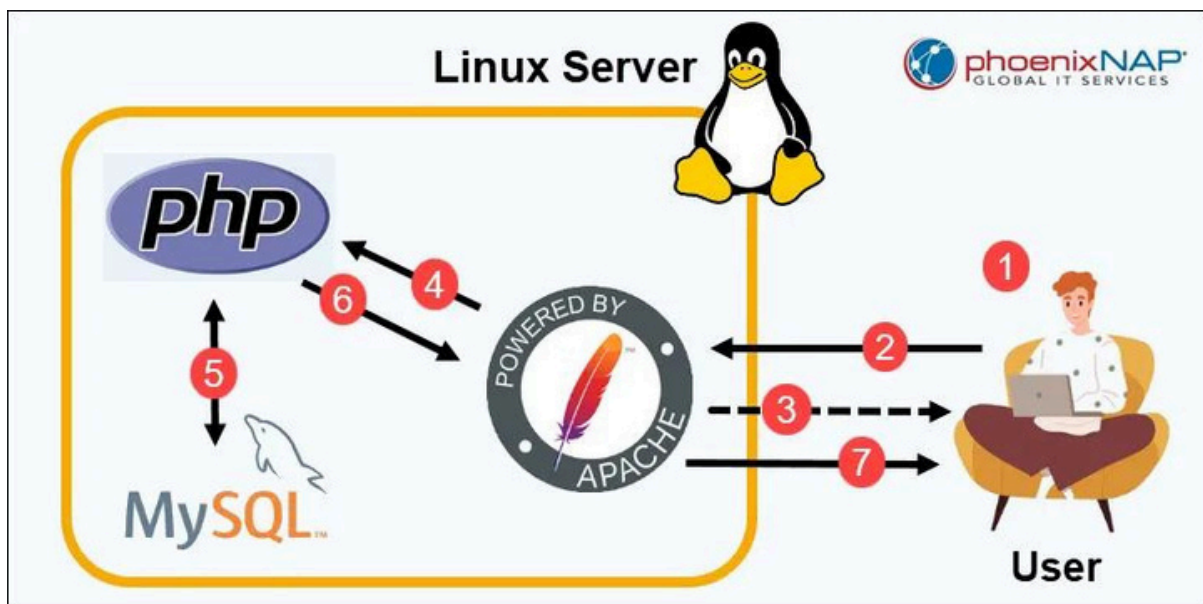
Chapter 38: Web Server Administration

- Case Study: Setting Up a Secure LAMP Stack
 - Example Script: Install and Configure LAMP Stack

```
bash
```

```
#!/bin/bash
sudo apt-get install -y apache2 mysql-server php libapache2-
mod-php php-mysql
sudo systemctl start apache2
sudo mysql_secure_installation
```

- Explanation: This script installs Apache, MySQL, and PHP, and secures the MySQL installation.



LAMP stack architecture diagram

- Example: Configuring HTTPS with Let's Encrypt
 - Example Script: Secure Apache with Let's Encrypt

```
bash
```

```
#!/bin/bash
sudo apt-get install -y certbot python-certbot-apache
sudo certbot --apache -d example.com
sudo systemctl reload apache2
```

- Explanation: This script uses Certbot to obtain and install a Let's Encrypt SSL certificate for Apache.

LAMP Stack Commands Cheat Sheet and Certbot Commands Cheat Sheet

Command	Example	Description
LAMP Stack Commands		
Install Apache	<code>sudo apt-get install apache2</code> <code>sudo apt-get install mysql-server</code>	Installs the Apache web server.
Install MySQL		Installs the MySQL database server.
Install PHP	<code>sudo apt-get install php</code> <code>libapache2-mod-php php-mysql</code>	Installs PHP and its modules.
Restart Apache	<code>sudo systemctl restart apache2</code>	Restarts the Apache service.
Secure MySQL Installation	<code>sudo mysql_secure_installation</code>	Runs the security script for MySQL.
Enable Apache mod_rewrite	<code>sudo a2enmod rewrite</code>	Enables the mod_rewrite module in Apache.
Check Apache Status	<code>sudo systemctl status apache2</code>	Checks the status of the Apache service.
Check MySQL Status	<code>sudo systemctl status mysql</code>	Checks the status of the MySQL service.
Test PHP	<code>php -r 'echo "Hello, World!";'</code>	Runs a simple PHP command to test installation.
Certbot Commands		
Install Certbot	<code>sudo apt-get install certbot</code> <code>python3-certbot-apache</code>	Installs Certbot for Apache.
Obtain SSL Certificate	<code>sudo certbot --apache</code>	Obtains and installs an SSL certificate for Apache.
Renew SSL Certificate	<code>sudo certbot renew</code> <code>sudo certbot renew --dry-run</code>	Renews the SSL certificates.
Test Renewal Process		Tests the renewal process without making changes.
List Certificates	<code>sudo certbot certificates</code>	Lists all managed certificates.
Delete Certificate	<code>sudo certbot delete --cert-name example.com</code>	Deletes a specific certificate.
Configure Automatic Renewal	<code>sudo crontab -e</code>	Edits crontab to schedule automatic renewals.

Chapter 39: Database Management

- Case Study: Optimizing MySQL Performance

- Example Script: MySQL Performance Tuning

```
bash
```

```
#!/bin/bash
```

```
sudo mysql -e "SET GLOBAL innodb_buffer_pool_size = 2G;"
```

```
sudo mysql -e "SET GLOBAL query_cache_size = 256M;"
```

- Explanation: This script tunes MySQL performance by adjusting buffer pool and query cache size.
 - Diagram: MySQL Performance Tuning
 - Search: "MySQL performance tuning diagram"

- Example: Backing Up and Restoring MySQL Databases

- Example Script: MySQL Backup and Restore

```
bash
```

```
#!/bin/bash
```

```
mysqldump -u root -p database_name > backup.sql
```

```
mysql -u root -p database_name < backup.sql
```

- Explanation: This script demonstrates how to back up and restore a MySQL database using mysqldump.

MySQL Optimization Commands Cheat Sheet and mysqldump Commands Cheat Sheet

Command	Example	Description
MySQL Optimization Commands		
Check Slow Queries	<code>SHOW VARIABLES LIKE 'slow_query%';</code>	Displays slow query settings.
Enable Slow Query Log	<code>SET GLOBAL slow_query_log = 'ON';</code>	Enables the slow query log.
Optimize Table	<code>OPTIMIZE TABLE table_name;</code>	Optimizes the specified table.
Analyze Table	<code>ANALYZE TABLE table_name;</code>	Analyzes the specified table for optimization.
Check Table	<code>CHECK TABLE table_name;</code>	Checks the specified table for errors.
Repair Table	<code>REPAIR TABLE table_name;</code>	Repairs the specified table if needed.
mysqldump Commands		
Backup Database	<code>mysqldump -u root -p database_name > backup.sql</code>	Backs up a database to a SQL file.
Backup All Databases	<code>mysqldump -u root -p --all-databases > all_databases.sql</code>	Backs up all databases to a SQL file.
Backup with Compression	<code>`mysqldump -u root -p database_name mysql -u root -p database_name < backup.sql`</code>	<code>gzip > backup.sql.gz`</code>
Restore Database	<code>mysqldump -u root -p --single-transaction database_name > backup.sql</code>	Restores a database from a SQL file.
Skip Lock Tables	<code>mysqldump -u root -p --ignore-database=database_name > backup.sql</code>	Backs up a database without locking tables.
Exclude Databases		Excludes specific databases from the backup.

Chapter 40: Security and Compliance

- Case Study: Implementing SELinux for Security
 - Example Script: Enable and Configure SELinux

```
bash

#!/bin/bash
sudo setenforce 1
sudo semanage fcontext -a -t httpd_sys_content_t
'/var/www/html(/.*)?'
sudo restorecon -R /var/www/html
```

- Explanation: This script enables SELinux and configures security contexts for a web directory.

- Example: Hardening SSH with Best Practices
 - Example Script: Secure SSH Configuration

```
bash

#!/bin/bash
sudo sed -i 's/#PermitRootLogin yes/PermitRootLogin no/'
/etc/ssh/sshd_config
sudo sed -i 's/#PasswordAuthentication
yes/PasswordAuthentication no/' /etc/ssh/sshd_config
sudo systemctl restart sshd
```

- Explanation: This script hardens SSH by disabling root login and password authentication.

SELinux Commands Cheat Sheet and SSH Configuration Cheat Sheet

Command	Example	Description
SELinux Commands		
Check SELinux Status	<code>sestatus</code>	Displays the current status of SELinux.
Enable SELinux	<code>setenforce 1</code>	Enables SELinux enforcement mode.
Disable SELinux	<code>setenforce 0</code>	Disables SELinux enforcement mode.
Configure SELinux Mode	<code>vi /etc/selinux/config</code>	Edits the SELinux configuration file.
List SELinux Contexts	<code>ls -Z</code>	Lists the SELinux context of files and directories.
Change SELinux Context	<code>chcon -R -t httpd_sys_content_t /var/www/html</code>	Changes the SELinux context of a directory.
SSH Configuration		
Commands		
Generate SSH Key	<code>ssh-keygen -t rsa -b 4096 -C "your_email@example.com"</code>	Generates a new SSH key pair.
Copy SSH Key to Server	<code>ssh-copy-id user@server</code>	Copies the SSH key to a remote server.
Connect with SSH	<code>ssh user@server</code>	Connects to a remote server using SSH.
Edit SSH Config	<code>vi ~/.ssh/config</code>	Edits the SSH client configuration file.
Disable Root Login	<code>PermitRootLogin no</code>	Disables root login in the SSH server configuration.
Change SSH Port	<code>Port 2222</code>	Changes the default SSH port in the SSH server configuration.

Chapter 41: Advanced Scripting and Automation

- Case Study: Automating System Updates
 - Example Script: Update System Packages

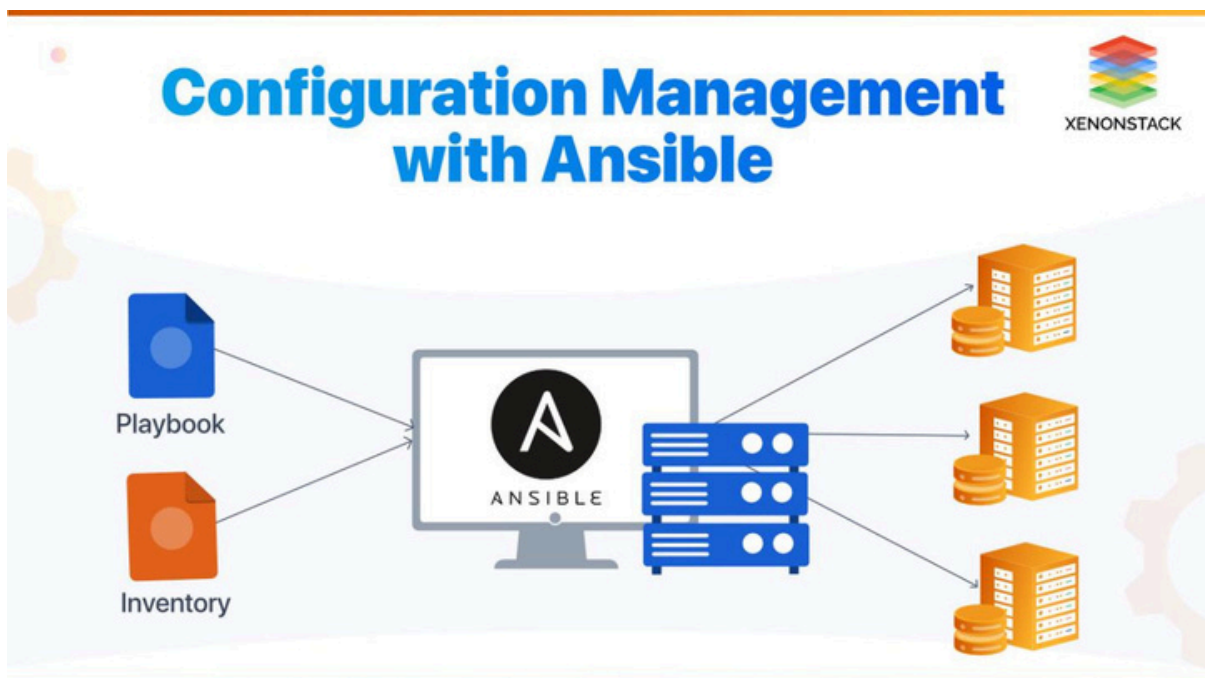
```
bash

#!/bin/bash
sudo apt-get update
sudo apt-get upgrade -y
sudo apt-get autoremove -y
```
 - Explanation: This script automates the update and upgrade of system packages.
- Example: Using **ansible** for Configuration Management
 - Example Script: Basic Ansible Playbook

```
yaml

---
- hosts: all

tasks:
  - name: Install Nginx
    apt:
      name: nginx
      state: present
```
 - Explanation: This playbook installs Nginx on all hosts in the inventory.



Ansible configuration management diagram

apt-get Commands Cheat Sheet

Command	Example	Description
Update Package List	<code>sudo apt-get update</code>	Updates the list of available packages.
Upgrade Packages	<code>sudo apt-get upgrade</code>	Upgrades all installed packages to the latest versions.
Install Package	<code>sudo apt-get install package_name</code>	Installs a specified package.
Remove Package	<code>sudo apt-get remove package_name</code>	Removes a specified package.
Purge Package	<code>sudo apt-get purge package_name</code>	Removes a package and its configuration files.
Search for Package	<code>apt-cache search package_name</code>	Searches for a package by name or keyword.
Show Package Info	<code>apt-cache show package_name</code>	Displays detailed information about a package.
Clean Cache	<code>sudo apt-get clean</code>	Clears out the local repository of retrieved package files.
Autoremove	<code>sudo apt-get autoremove</code>	Removes unnecessary packages automatically installed.
Check for Broken Dependencies	<code>sudo apt-get check</code>	Checks for broken dependencies in the package database.

Summary

This expanded section on Case Studies and Practical Examples provides comprehensive scripts, explanations, commands, cheat sheets, and diagrams for various real-world scenarios in Linux administration. Each chapter equips readers with practical knowledge and tools to automate tasks, manage networks, administer web servers, handle databases, ensure security, and utilize advanced scripting techniques effectively.

PART VIII: Cheat Sheets and Resources

Chapter 42: Basic Linux Commands Cheat Sheet

- Cheat Sheet Commonly Used Linux Commands
 - Explanation: A quick reference guide for basic Linux commands such as `cd`, `cp`, `mv`, `rm`, `chmod`, and `chown`.
 - Example Commands:

```
bash
```

```
ls -la
```

```
cd /path/to/directory
```

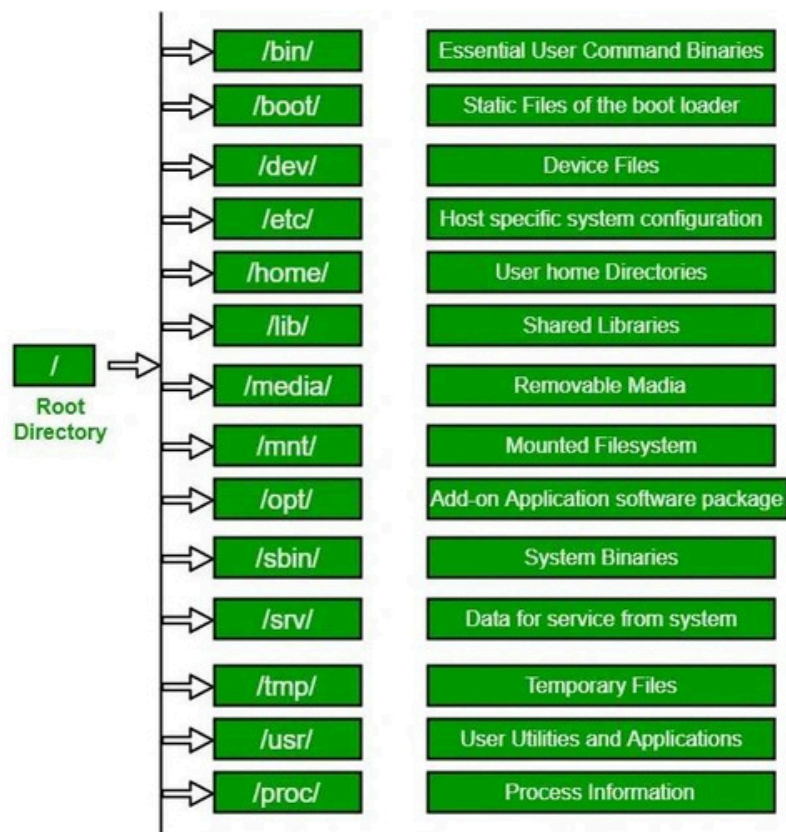
```
cp file1.txt file2.txt
```

```
mv file1.txt /path/to/destination/
```

```
rm file1.txt
```

```
chmod 755 script.sh
```

```
chown user:group file1.txt
```



Linux file system structure diagram

Chapter 43: Advanced Linux Commands Cheat Sheet

- **Cheat Sheet Advanced Linux Commands**
 - **Explanation:** A quick reference guide for advanced Linux commands such as `grep`, `awk`, `sed`, `find`, `xargs`, `tar`, and `curl`.
 - **Example Commands:**

```
bash
grep "pattern" file.txt
awk '{print $1}' file.txt
sed 's/old/new/g' file.txt
find /path/to/search -name "*.txt"
xargs -n 1 echo < file.txt
tar -cvf archive.tar /path/to/directory
curl -O http://example.com/file.txt
```

Chapter 44: Shell Scripting Cheat Sheet

- **Cheat Sheet Essential Shell Scripting Commands**
 - **Explanation:** A reference guide for shell scripting commands and syntax, including variables, loops, conditionals, and functions.
 - **Example Script:**

```
bash

#!/bin/bash
echo "Hello, World!"
for i in {1..5}; do
    echo "Iteration $i"
done
if [ -f "file.txt" ]; then
    echo "File exists."
fi
function greet {
    echo "Hello, $1!"
}
greet "Alice"
```

grep, awk, sed Cheat Sheet

Command	Example	Description
grep Commands		
Search for Pattern	<code>grep "pattern" file.txt</code>	Searches for a pattern in a file.
Recursive Search	<code>grep -r "pattern" /path</code>	Recursively searches for a pattern in a directory.
Case-Insensitive Search	<code>grep -i "pattern" file.txt</code>	Searches for a pattern in a file, case-insensitively.
Show Line Numbers	<code>grep -n "pattern" file.txt</code>	Shows line numbers with matched lines.
awk Commands		
Print Column	<code>awk '{print \$1}' file.txt</code>	Prints the first column of a file.
Pattern Matching	<code>awk '/pattern/{print}' file.txt</code>	Prints lines matching a pattern.
Field Separator	<code>awk -F, '{print \$1}' file.csv</code>	Uses a comma as the field separator.
Arithmetic Operations	<code>awk '{print \$1 + \$2}' file.txt</code>	Performs arithmetic on columns.
sed Commands		
Find and Replace	<code>sed 's/old/new/g' file.txt</code>	Replaces all occurrences of a pattern.
Delete Line	<code>sed '2d' file.txt</code>	Deletes the second line of a file.
Insert Line	<code>sed '2i\n new line' file.txt</code>	Inserts a line before the second line.
Substitute with		
Backup	<code>sed -i.bak 's/old/new/g' file.txt</code>	Replaces a pattern and creates a backup.

Shell Scripting Patterns Cheat Sheet

Command	Example	Description
For Loop	<code>for i in {1..5}; do echo \$i; done</code>	Iterates over a sequence of numbers.
While Loop	<code>while [\$x -le 5]; do echo \$x; x=\$((x+1)); done</code>	Executes a loop while a condition is true.
If-Else Statement	<code>if [\$x -eq 5]; then echo "Equal"; else echo "Not equal"; fi</code>	Executes code based on a condition.
Case Statement	<code>case \$x in 1) echo "One";; 2) echo "Two";; esac</code>	Executes code based on a matching pattern.
Function Definition	<code>function name { echo "Hello, \$1"; }</code>	Defines a reusable function.
Function Call	<code>name "World"</code>	Calls a defined function with arguments.

Chapter 45: Network Administration Cheat Sheet

- **Cheat Sheet Network Commands and Tools**
 - **Explanation:** A reference guide for network-related commands and tools such as ping, traceroute, netstat, nmap, and ssh.
 - **Example Commands:**

```
bash
ping -c 4 example.com
traceroute example.com
netstat -tuln
nmap -sP 192.168.1.0/24
ssh user@example.com
```

Chapter 46: System Administration Cheat Sheet

- **Cheat Sheet System Administration Commands**
 - **Explanation:** A reference guide for system administration commands and tools such as systemctl, journalctl, top, htop, df, du, ps, and kill.
 - **Example Commands:**

```
bash

systemctl start/stop/restart service
journalctl -u service
top
htop
df -h
du -sh /path/to/directory
ps aux
kill -9 PID
```

nmap SSH Cheat Sheet

Command	Example	Description
nmap Commands		
Basic Scan	<code>nmap 192.168.1.1</code>	Scans the specified IP address.
Scan Subnet	<code>nmap 192.168.1.0/24</code>	Scans all IP addresses in a subnet.
Service Version Detection	<code>nmap -sV 192.168.1.1</code>	Detects versions of services running on open ports.
OS Detection	<code>nmap -O 192.168.1.1</code>	Attempts to determine the operating system of the target.
SSH Commands		
Generate SSH Key	<code>ssh-keygen -t rsa -b 4096 -C "your_email@example.com"</code>	Generates a new SSH key pair.
Copy SSH Key to Server	<code>ssh-copy-id user@server</code>	Copies the SSH key to a remote server.
Connect with SSH	<code>ssh user@server</code>	Connects to a remote server using SSH.
Edit SSH Config	<code>vi ~/.ssh/config</code>	Edits the SSH client configuration file.
Disable Root Login	<code>PermitRootLogin no</code>	Disables root login in the SSH server configuration.

8) systemctl Journalctl Cheat Sheet

Command	Example	Description
systemctl Commands		
Start Service Stop	<code>sudo systemctl start service_name</code>	Starts a specified service.
Service Restart	<code>sudo systemctl stop service_name</code>	Stops a specified service.
Service Enable	<code>sudo systemctl restart service_name</code>	Restarts a specified service.
Service Disable	<code>sudo systemctl enable service_name</code>	Enables a service to start at boot.
Service	<code>sudo systemctl disable service_name</code>	Disables a service from starting at boot.
journalctl	<code>journalctl</code>	Displays system logs.
Commands	<code>journalctl -r</code>	Displays logs in reverse order.
View Logs View Recent Logs View Boot Logs	<code>journalctl -b</code>	Displays logs from the current boot.
View Logs for a Service	<code>journalctl -u service_name</code>	Displays logs for a specific service.
Follow Logs	<code>journalctl -f</code>	Continuously displays new log entries.

Chapter 47: Security and Compliance Cheat Sheet

- Cheat Sheet Security Commands and Best Practices
 - Explanation: A reference guide for security-related commands and best practices such as using iptables, firewallld, fail2ban, selinux, and configuring SSH.

- Example Commands:

```
bash

iptables -L
firewall-cmd --list-all
fail2ban-client status
sudo setenforce 1
sudo vi /etc/ssh/sshd_config
```

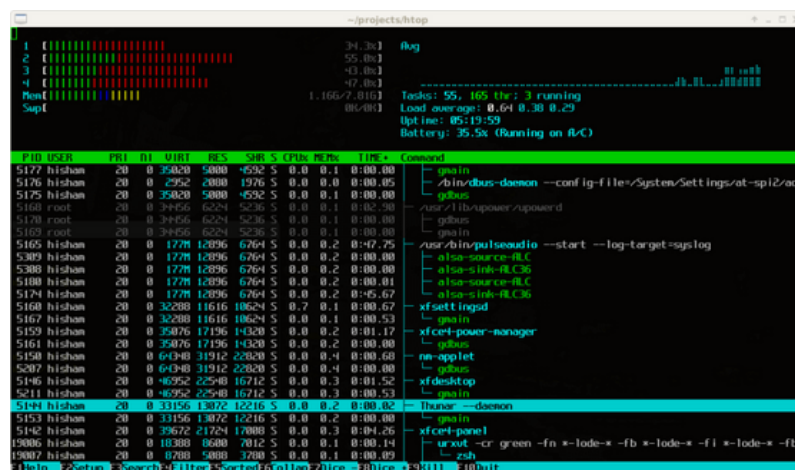
Chapter 48: Performance Tuning Cheat Sheet

- Cheat Sheet Performance Tuning Commands and Tools
 - Explanation: A reference guide for performance tuning commands and tools such as top, htop, iotop, vmstat, iostat, and sar.

- Example Commands:

```
bash

top
htop
iotop
vmstat 1
iostat -x 1
sar -u 1
```



Running htop on a linux machine

iptables firewalld fail2ban Cheat Sheet

Command	Example	Description
iptables Commands		
List Rules	<code>sudo iptables -L</code>	Lists all current iptables rules.
Add Rule	<code>sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT</code>	Allows incoming SSH connections.
Delete Rule	<code>sudo iptables -D INPUT -p tcp --dport 22 -j ACCEPT</code>	Deletes a specific rule.
Save Rules	<code>sudo iptables-save > /etc/iptables/rules.v4</code>	Saves current rules to a file.
firewalld Commands		
Start firewalld	<code>sudo systemctl start firewalld</code>	Starts the firewalld service.
Stop firewalld	<code>sudo systemctl stop firewalld</code>	Stops the firewalld service.
Enable firewalld	<code>sudo systemctl enable firewalld</code>	Enables firewalld to start at boot.
List Zones	<code>sudo firewall-cmd --get-zones</code>	Lists available zones.
Add Service to Zone	<code>sudo firewall-cmd --zone=public --add-service=http --permanent</code>	Adds a service to a zone.
fail2ban Commands		
Start fail2ban	<code>sudo systemctl start fail2ban</code>	Starts the fail2ban service.
Stop fail2ban	<code>sudo systemctl stop fail2ban</code>	Stops the fail2ban service.
Enable fail2ban	<code>sudo systemctl enable fail2ban</code>	Enables fail2ban to start at boot.
Status of fail2ban	<code>sudo fail2ban-client status</code>	Shows the status of fail2ban.
View Jail Status	<code>sudo fail2ban-client status jail_name</code>	Shows the status of a specific jail.

top htop iotop Cheat Sheet

Command	Example	Description
top Commands		
Start top	<code>top</code>	Starts the top command-line process viewer.
Sort by Memory Usage	<code>top -o %MEM</code>	Sorts processes by memory usage.
Kill a Process	<code>k</code> then PID	Kills a process by PID.
htop Commands		
Start htop	<code>htop</code>	Starts the htop command-line process viewer.
Tree View	<code>F5</code>	Toggles tree view for processes.
Search for Process	<code>/</code> then name	Searches for a process by name.
iotop Commands		
Start iotop	<code>sudo iotop</code>	Starts the iotop I/O process viewer.
Only Active Processes	<code>sudo iotop -o</code>	Shows only processes with I/O activity.
Sort by Disk Read	<code>sudo iotop -d</code>	Sorts processes by disk read activity.

Chapter 49: Scripting and Automation Tools Cheat Sheet

- Cheat Sheet Automation Tools and Commands
 - Explanation: A reference guide for scripting and automation tools such as
 - cron, anacron, at, and systemd timers.
- Example Commands:**
- ```
bash crontab -e anacron -d at now + 5 minutes systemctl list-timers
```

### cron anacron at Cheat Sheet

| Command                 | Example                              | Description                               |
|-------------------------|--------------------------------------|-------------------------------------------|
| <b>cron Commands</b>    |                                      |                                           |
| Edit crontab            | <code>crontab -e</code>              | Opens the crontab file for editing.       |
| List crontab            | <code>crontab -l</code>              | Lists the current user's crontab entries. |
| Remove crontab          | <code>crontab -r</code>              | Removes the current user's crontab.       |
| <b>anacron Commands</b> |                                      |                                           |
| Edit anacrontab         | <code>sudo vi /etc/anacrontab</code> | Edits the anacrontab file.                |
| Run anacron             | <code>sudo anacron -s</code>         | Runs anacron jobs.                        |
| List anacron jobs       | <code>anacron -L</code>              | Lists scheduled anacron jobs.             |
| <b>at Commands</b>      |                                      |                                           |
| Schedule Job            | <code>`echo "command" at 5pm`</code> |                                           |
| List Jobs               | <code>atq</code>                     | Lists all scheduled at jobs.              |
| Remove Job              | <code>atrm job_number</code>         | Removes a scheduled at job by job number. |

## Chapter 50: Additional Resources

- Resources Books, Websites, and Communities
  - Explanation: A curated list of additional resources for further learning and support, including books, websites, forums, and online communities.
  - Example Resources:
    - Books:
      - "The Linux Command Line" by William Shotts
      - "Linux Pocket Guide" by Daniel J. Barrett
    - Websites:
      - [Linux Documentation Project](#)
      - [Stack Overflow](#)
    - Communities:
      - [Reddit: r/linux](#)
      - [LinuxQuestions.org](#)

## Summary

This expanded section on Cheat Sheets and Resources provides detailed, quick reference guides for a variety of Linux commands and tools, complete with scripts, explanations, commands, cheat sheets, and diagrams for better understanding. Each chapter equips readers with the essential commands and tools they need to efficiently manage and troubleshoot Linux systems.

## PART IX: Appendices

### Chapter 51: Appendix A: Mathematical Foundations

- Subsections:

1. Basic Arithmetic and Algebra

- Explanation: Fundamental concepts of arithmetic and algebra relevant to computing and scripting.
- Example:

```
bash

Simple arithmetic operations in shell

echo $((2 + 3)) # Addition
echo $((5 - 2)) # Subtraction
echo $((3 * 4)) # Multiplication
echo $((10 / 2)) # Division
```

2. Probability and Statistics

- Explanation: Basic probability and statistical concepts used in performance analysis and system monitoring.
- Example:

```
bash

Generating random numbers
echo $RANDOM
echo $((RANDOM % 100)) # Random number between 0 and 99
```

3. Matrices and Vectors

- Explanation: Usage of matrices and vectors in computing, particularly for tasks like graphics rendering or machine learning.
- Example:

```
bash

Creating a simple matrix using arrays
matrix=([0,0]=1 [0,1]=2 [1,0]=3 [1,1]=4)
echo ${matrix[0,0]} # Accessing matrix element
```

## Chapter 52: Appendix B: Shell Scripting Reference

- Subsections:

1. Built-in Shell Variables

- Explanation: Common built-in shell variables and their uses.
- Example:

```
bash

echo "Script name: $0"
echo "Number of arguments: $#"
```

```
echo "All arguments: $@"
```

2. Special Characters and Operators

- Explanation: Special characters and operators in shell scripting and their functions.
- Example:

```
bash

echo "Current directory: $(pwd)"
echo "Home directory: $HOME"
```

```
echo "Current user: $USER"
```

3. Regular Expressions

- Explanation: Use of regular expressions in shell scripting for pattern matching.
- Example:

```
bash

grep '^abc' file.txt
grep 'xyz$' file.txt # Lines starting with 'abc'
 # Lines ending with 'xyz'
```

### Shell Variables Cheat Sheet and Shell Special Characters Cheat Sheet

| Command                  | Example    | Description                                   |
|--------------------------|------------|-----------------------------------------------|
| Shell Variables          |            |                                               |
| Define Variable          | VAR=value  | Defines a new variable.                       |
| Access Variable          | echo \$VAR | Accesses the value of a variable.             |
| Unset Variable           | unset VAR  | Unsets a variable.                            |
| Shell Special Characters |            |                                               |
| Newline                  | \n \t \    | Represents a newline character.               |
| Tab                      | command1   | Represents a tab character.                   |
| Escape                   | command2   | Escapes special characters.                   |
| Pipe                     |            | Pipes the output of one command into another. |

## Chapter 53: Appendix C: Linux Resources

- Subsections:
  1. Books and Guides
    - Explanation: Recommended books and guides for further reading on Linux and shell scripting.
    - Example:
      - Books:
        - "The Linux Command Line" by William Shotts
        - "Linux Pocket Guide" by Daniel J. Barrett
  2. Online Tutorials and Documentation
    - Explanation: Useful websites and online resources for learning and troubleshooting.
    - Example:
      - Websites:
        - [Linux Documentation](#)
        - [Project Stack Overflow](#)
  3. Communities and Forums
    - Explanation: Online communities and forums where users can ask questions and share knowledge.
    - Example:
      - Communities:
        - [Reddit: r/linux](#)
        - [LinuxQuestions.org](#)
  4. Linux Distributions
    - Explanation: Overview of popular Linux distributions and their specific uses.
    - Example:
      - Distributions:
        - Ubuntu
        - Fedora
        - CentOS

## Chapter 54: Appendix D: Glossary of Terms

- Subsections:
  1. Linux Terminology
    - Explanation: Definitions of common Linux terms and jargon.
    - Example:
      - Kernel: The core component of a Linux operating system, managing hardware and system processes.
      - Shell: The interface that allows users to interact with the operating system via commands.
  2. Scripting Terminology
    - Explanation: Definitions of common scripting terms and jargon.
    - Example:
      - Script: A file containing a series of commands that are executed by the shell.
      - Variable: A storage location identified by a name that holds data.
  3. Networking Terms
    - Explanation: Definitions of common networking terms used in Linux networking.
    - Example:
      - IP Address: A unique string of numbers separated by periods that identifies each computer using the Internet Protocol to communicate over a network.
      - Port: A communication endpoint used by applications to exchange data.
  4. System Administration Terms
    - Explanation: Definitions of common system administration terms.
    - Example:
      - Daemon: A background process that runs continuously to handle periodic service requests.
      - Cron: A time-based job scheduler in Unix-like operating systems.

## Summary

This expanded section on Appendices provides detailed reference material to supplement the core content of the book. Each chapter is designed to give readers quick access to essential information, scripts, explanations, commands, cheat sheets, and diagrams, helping them to deepen their understanding of Linux and shell scripting.

## GitLab CI/CD and Shell Scripting Cheatsheet

| Command/Variable            | Action                                          | Example                                                               |
|-----------------------------|-------------------------------------------------|-----------------------------------------------------------------------|
| <b>stages</b>               | Defines the stages of the pipeline              | <code>stages: - build - test - deploy</code>                          |
| <b>script</b>               | Defines the script to execute in a job          | <code>script: - echo "Hello, World!"</code>                           |
| <b>variables</b>            | Defines environment variables for a job         | <code>variables: DEPLOY_PROXY_PASSWORD: "your_secret_password"</code> |
| <b>\$CI_COMMIT_REF_NAME</b> | GitLab predefined variable for the branch name  | <code>echo "Branch: \$CI_COMMIT_REF_NAME"</code>                      |
| <b>\$CI_JOB_ID</b>          | GitLab predefined variable for the job ID       | <code>echo "Job ID: \$CI_JOB_ID"</code>                               |
| <b>before_script</b>        | Script to execute before the main script        | <code>before_script: - apt-get update</code>                          |
| <b>after_script</b>         | Script to execute after the main script         | <code>after_script: - echo "Job finished"</code>                      |
| <b>cache</b>                | Defines files to cache between jobs             | <code>cache: paths: - node_modules/</code>                            |
| <b>artifacts</b>            | Defines files to keep after a job is done       | <code>artifacts: paths: - dist/</code>                                |
| <b>only</b>                 | Defines conditions to limit when jobs are run   | <code>only: - master</code>                                           |
| <b>except</b>               | Defines conditions to exclude when jobs are run | <code>except: - tags</code>                                           |
| <b>rules</b>                | More advanced rules for job execution           | <code>rules: - if: '\$CI_COMMIT_REF_NAME == "master"'</code>          |



|                                |                                                                                               |                                                |
|--------------------------------|-----------------------------------------------------------------------------------------------|------------------------------------------------|
| <b>tags</b>                    | Tags to match jobs to specific runners                                                        | tags: - docker                                 |
| <b>allow_failure</b>           | Allows jobs to fail without failing the pipeline                                              | allow_failure: true                            |
| <b>dependencies</b>            | Specifies jobs that a job depends on Defines when to run the job (on_success, always, manual) | dependencies: - build                          |
| <b>when</b>                    | Defines how many times to retry a job on failure                                              | when: manual                                   |
| <b>retry</b>                   | Defines environment for a job                                                                 | retry: 2                                       |
| <b>environment</b>             | Environment variable defined in .gitlab-ci.yml                                                | environment: production                        |
| <b>\$DEPLOY_PROXY_PASSWORD</b> | Checks if a variable is defined and not null                                                  | proxy_password="\$DEPLOY_PROXY_PASSWORD"       |
| <b>if [ -n "\$VAR" ]</b>       | Checks if a variable is not defined or is null                                                | if [ -n "\$VAR" ]; then echo "Defined"; fi     |
| <b>if [ -z "\$VAR" ]</b>       | Prints text to the console                                                                    | if [ -z "\$VAR" ]; then echo "Not defined"; fi |
| <b>echo</b>                    | Shebang to specify the script interpreter                                                     | echo "Hello, World!"                           |
| <b>#!/bin/bash</b>             | Makes a script executable                                                                     | #!/bin/bash (at the top of the script)         |
| <b>chmod +x script.sh</b>      | Executes a script                                                                             | chmod +x deploy_script.sh                      |
| <b>./script.sh</b>             |                                                                                               | ./deploy_script.sh                             |

|                         |                                          |                                     |
|-------------------------|------------------------------------------|-------------------------------------|
| <b>apt-get update</b>   | Updates the package list                 | apt-get update                      |
| <b>apt-get install</b>  | Installs a package                       | apt-get install -y git              |
| <b>export VAR=value</b> | Sets an environment variable in a script | export MY_VAR="value"               |
| <b>curl -O</b>          | Downloads a file from a URL              | curl -O http://example.com/file.txt |
| <b>tar -xzf</b>         | Extracts a tar.gz file                   | tar -xzf archive.tar.gz             |
| <b>zip -r</b>           | Creates a zip archive                    | zip -r archive.zip folder/          |
| <b>scp</b>              | Copies files over SSH                    | scp file.txt user@host:/path/       |
| <b>ssh</b>              | Connects to a remote host over SSH       | ssh user@host                       |
| <b>nmap</b>             | Scans network for hosts and open ports   | nmap -sP 192.168.0.0/24             |
| <b>ping</b>             | Checks connectivity to a host            | ping google.com                     |
| <b>grep</b>             | Searches for patterns in text            | grep "pattern" file.txt             |
| <b>sed</b>              | Stream editor for modifying files        | sed -i 's/old/new/g' file.txt       |
| <b>awk</b>              | Text processing and data extraction      | awk '{print \$1}' file.txt          |
| <b>ps aux</b>           | Lists running processes                  | ps aux                              |
| <b>kill</b>             | Kills a process by PID                   | kill 1234                           |
| <b>crontab -e</b>       | Edits the cron jobs                      | crontab -e                          |
| <b>crontab -l</b>       | Lists the cron jobs                      | crontab -l                          |

|                      |                                                      |                                                           |
|----------------------|------------------------------------------------------|-----------------------------------------------------------|
| <b>df -h</b>         | Displays disk space usage                            | df -h                                                     |
| <b>du -sh</b>        | Displays the size of a directory                     | du -sh /path/to/directory                                 |
| <b>top</b>           | Displays running processes and system resource usage | top                                                       |
| <b>htop</b>          | Interactive process viewer                           | htop                                                      |
| <b>free -h</b>       | Displays memory usage                                | free -h                                                   |
| <b>ifconfig</b>      | Displays network configuration                       | ifconfig                                                  |
| <b>netstat -tuln</b> | Displays active network connections                  | netstat -tuln                                             |
| <b>iptables</b>      | Configures the IP packet filter rules                | iptables -L                                               |
| <b>chmod</b>         | Changes file permissions                             | chmod 755 script.sh                                       |
| <b>chown</b>         | Changes file owner and group                         | chown user:group file.txt                                 |
| <b>ln -s</b>         | Creates a symbolic link                              | ln -s /path/to/file /path/to/link                         |
| <b>find</b>          | Searches for files and directories                   | find /path -name "file.txt"                               |
| <b>alias</b>         | Creates a shortcut for a command                     | alias ll="ls -la"                                         |
| <b>.bashrc</b>       | Bash configuration file                              | ~/.bashrc (edit to add aliases and environment variables) |
| <b>.zshrc</b>        | Zsh configuration file                               | ~/.zshrc (edit to add aliases and environment variables)  |
| <b>vim</b>           | Text editor                                          | vim file.txt                                              |

|                |                                                                   |                         |
|----------------|-------------------------------------------------------------------|-------------------------|
| <b>nano</b>    | Text editor                                                       | nano file.txt           |
| <b>less</b>    | Views files one screen at a time                                  | less file.txt           |
| <b>tail -f</b> | Displays the end of a file and updates as it grows                | tail -f /var/log/syslog |
| <b>head</b>    | Displays the beginning of a file                                  | head -n 10 file.txt     |
| <b>cat</b>     | Concatenates and displays file contents                           | cat file.txt            |
| <b>tee</b>     | Reads from standard input and writes to standard output and files | `echo "Hello"           |

THE ULTIMATE STEP BY STEP  
**GUIDE TO QUICKLY**  
AND EASILY LEARNING LINUX



# LINUX

TED DAWSON

# LINUX

## *The Ultimate Step by Step Guide to Quickly and Easily Learning Linux*

*TED DAWSON*

© Copyright 2015 by WE CANT BE BEAT LLC

[Chapter One: LINUX HISTORY](#)

[Chapter Two: LINUX DISTRIBUTION \(DISTRO\)](#)

## **[Introduction](#)**

### **[GUIDE TO CHOOSING DISTRIBUTION](#)**

**[Linux Mint](#)**

**[Ubuntu](#)**

**[Debian GNU/Linux](#)**

**[Mageia](#)**

**[Fedora](#)**

**[openSUSE](#)**

**[Arch Linux](#)**

**[CentOS](#)**

**[PCLinuxOS](#)**

**[Slackware Linux](#)**

**[FreeBSD](#)**

[Chapter Three: LICENSING](#)

## **[COMMUNITY](#)**

### **[DEVELOPMENT](#)**

[Chapter Four: INSTALLING DEBIAN 8](#)

## **[What is Debian](#)**

### **[Customizing your System](#)**

**[Terminal](#)**

**[Install sudo](#)**

**[Set up the network](#)**

**[Setup your hosts file](#)**

**[Log in via SSH!](#)**

**[Installing the basics](#)**

**[Installing MySQL](#)**

**[Setting up MySQL](#)**

[Chapter Five: INSTALLING CENTOS 7](#)

## **[Installation of CenOS7](#)**

**[Step 1:](#)** [Download the ISO Image](#)

**[Step 2:](#)** [Make a bootable Drive](#)

**[Step 3:](#)** [Begin Installation](#)

**[Step 4:](#)** [Select Language and Keyboard](#)

**[Step 5:](#)** [Change the Installation Destination](#)

**[Step 6:](#)** [Select the Partitioning Scheme](#)

**[Step 7:](#)** [Create a Swap Space](#)

**[Step 8:](#)** [Create a Mountpoint](#)

**[Step 9:](#)** [Accept Changes](#)

**[Step 10:](#)** [Set Date and Time](#)

**Step 11: Begin Installation**

**Step 12: Set Up Root Password**

**Step 13: Create a User Account**

**Step 14: Complete Installation**

**Change and Set Hostname Command**

**Method #1: hostnamectl**

**How do I see the host names?**

**How do I delete a particular host name?**

**How do I change host name remotely?**

**Method #2: nmtui**

**Method #3: nmcli**

**To view the host name using nmcli:**

**To set the host name using nmcli:**

**Chapter Six: LINUX AND UNIXMAN COMMAND**

**Syntax**

**Description**

**General Options**

**Main Modes of Operation**

**Finding Manual Pages**

**Controlling Formatted Output**

**Section Numbers**

**Exit Status**

**Environment**

**Files**

**Examples**

**Chapter Seven: LINUX DIRECTORY COMMAND**

**sample outputs**

**List only files in Unix**

**Task: Create aliases to save time**

**Chapter Eight: WORKING WITH FILES**

**UNIX File Names**

**Looking at the Contents of Files**

**Cat Command**

**More Command**

**Head Command**

**Tail Command**

**Copying, Erasing, Renaming**

**Copying Files**

**Erasing Files**

**Renaming a File**

**Using the Command Line**

**Standard Input and Standard Output**



## **Redirection**

## **Using Pipes and Filters**

## **Some Additional File Handling Commands**

## **Word Count**

## **Comparing the Contents of Two Files: the cmp and diff Commands**

Chapter Nine: NAVIGATION AND FILE MANAGEMENT

## **Prerequisites and Goals**

## **Navigation and Exploration**

### **Finding where you are with the “pwd” command**

### **Looking at the Contents of Directories with “ls”**

### **Moving Around the Filesystem with “cd”**

## **Viewing Files**

## **File and Directory Manipulation**

### **Create a File with “touch”**

### **Create a Directory with “mkdir”**

### **Moving and Renaming Files and Directories with “mv”**

### **Copying Files and Directories with “cp”**

### **Removing Files and Directories with “rm” and “rmdir”**

## **Editing Files**

Chapter Ten: UNIX SHELL SCRIPTING

## **Shell Scripting Introduction**

Chapter Eleven: SHELL BASIC OPERATOR

## **Arithmetic Operators**

## **Relational Operators:**

## **Boolean Operators**

## **String Operators**

## **File Test Operators**

### **Run The .Sh File Shell Script In Linux / Unix**

## **.sh As Root User**

## **chmod Command: Run Shell Script In Linux**

Chapter Twelve: SHELL EMBEDDING AND OPTIONS

## **Shell installing**

### **Backticks**

## **Backticks or single quotes**

## **Shell Alternatives**

## **Practice: Shell Installing**

Chapter Thirteen: SHELL HISTORY SEARCH COMMAND

## **Emacs Line-Edit Mode Command History Searching**

## **fc command**

## **Delete command history**

## **FILE NAME GLOBBING WITH \*, ?, [ ]**

Chapter fourteen: UNIX - SHELL INPUT/OUTPUT REDIRECTIONS

## Chapter Fifteen: UNIX SHELL FUNCTION

### **Creating Functions**

#### **Example**

#### **Pass Parameters to a Function**

#### **Returning Values from Functions**

#### **Example**

#### **Nested Functions**

#### **Function Call from Prompt**

#### **Unix - Pipes and Filters**

#### **The grep Command**

#### **The sort Command**

#### **The pg and more Commands**

## Chapter Sixteen: UNIX USEFUL COMMAND

### **Files and Directories**

#### **Manipulating data**

#### **Messages between Users**

## Chapter Seventeen: REGULAR EXPRESSION

### **Invoking sed**

#### **The sed General Syntax**

#### **Deleting All Lines with sed**

#### **The sed Addresses**

#### **The sed Address Ranges**

#### **The Substitution Command**

#### **Substitution Flags**

#### **Using an Alternative String Separator**

#### **Replacing with Empty Space**

#### **Address Substitution**

#### **The Matching Command**

#### **Using Regular Expression**

#### **Using Multiple sed Commands**

#### **Back References**

## Chapter Eighteen: FILE SYSTEM BASICS

### **Directory Structure**

#### **The df Command**

#### **The du Command**

#### **Mounting the File System**

#### **Unmounting the File System**

#### **User and Group Quotas**

## Chapter Nineteen: UNIX-USER ADMINISTRATION

### **Managing Users and Groups**

#### **Modify a Group**

#### **Delete a Group:**

**Create an Account**

**Modify an Account**

**Delete an Account**

Chapter Twenty: SYSTEM PERFORMANCE

**Performance Components**

**Unix - System Logging**

**The /etc/syslog.conf file**

**Logging Actions**

**Log Rotation**

**Important Log Locations**

Chapter Twenty-one: UNIX SIGNALS AND TRAPS

LIST of Signals

**Default Actions**

**Sending Signals**

**Trapping Signals**

Cleaning Up Temporary Files

**Ignoring Signals**

**Resetting Traps**

**ABOUT THE AUTHOR**

LINKS AND RESOURCES

**DEDICATION**

**SPECIAL THANKS**

# Chapter One: LINUX HISTORY

The history of Linux began in 1991 with the commencement of a personal project by Finnish student Linus Torvalds to create a new free operating system kernel. Since then, the resulting Linux kernel has been marked by constant growth throughout its history. Since the initial release of its source code in 1991, it has grown from a small number of C files under a license prohibiting commercial distribution to the 3.18 version in 2015 with more than 18 million lines of source code under the GNU General Public License (GPL). He wrote the program specifically for the hardware he was using and independent of an operating system because he wanted to use the functions of his new PC with an Intel 80386 processor. Development was done on MINIX using the GNU C compiler. The GNU C Compiler is still the main choice for compiling Linux today. The code can be built with other compilers, such as the Intel C Compiler. As Torvalds wrote in his book 'Just for Fun,' he eventually ended up writing an operating system kernel. On 25 August 1991 (age 21), he announced this new system in a Usenet posting to the newsgroup "comp.os.minix."

Torvalds first published the Linux kernel under its own license, which had a restriction on commercial activity. The software used with the kernel was developed as part of the GNU project licensed under the GNU GPL, a free software license. The first release of the Linux kernel, Linux 0.01, included a binary of GNU's Bash shell. In the "Notes for Linux release 0.01", Torvalds lists the GNU software that is required to run Linux: Sadly, a kernel by itself gets you nowhere. To get a working system, you need a shell, compilers, a library, etc. These are separate parts and may be under stricter (or even looser) copyright. Most of the tools used with Linux are GNU software and are under the GNU copyright. These tools are not in the distribution - ask me (or GNU) for more info. In 1992, he suggested releasing the kernel under the GNU GPL. He first announced this decision in the release notes of version 0.12. In the middle of December 1992, he published version 0.99 using the GNU GPL. Linux and GNU developers worked to integrate GNU components with Linux to make a fully functional free operating system. Torvalds has stated, "Making Linux GPL'd was definitely the best thing I ever did." Torvalds initially used the designation "Linux" only for the Linux kernel. The kernel was, however, frequently used together with other software, especially

that of the GNU project. This quickly became the most popular adoption of GNU software. In June 1994 in GNU's Bulletin, Linux was referred to as a "free UNIX clone," and the Debian project began calling its product Debian GNU/Linux. In May 1996, Richard Stallman published the editor Emacs 19.31, in which the type of system was renamed from Linux to Linux. This spelling was intended to refer specifically to the combination of GNU and Linux, but this was soon abandoned in favor of "GNU/Linux." This name garnered varying reactions. The GNU and Debian projects use the name, although most people simply use the term "Linux" to refer to the combination.

The largest part of the work on Linux is performed by the community: the thousands of programmers around the world that use Linux to send their suggested improvements to the maintainers. Various companies have also helped not only with the development of the kernels but also with the writing the body of auxiliary software, which is distributed with Linux. As of February 2015, over 80% of Linux kernel developers are paid. It is released both by organized projects such as Debian and by projects connected directly with companies such as Fedora and openSUSE. The members of these respective projects meet at various conferences and fairs, in order to exchange ideas. One of the largest of these fairs is the LinuxTag in Germany (currently in Berlin), where about 10,000 people assemble annually, in order to discuss Linux and the projects associated with it. The Open Source Development Lab (OSDL) was created in the year 2000, and is an independent nonprofit organization, which pursues the goal of optimizing Linux for employment in data centers and in the carrier range. It served as sponsored working premises for Linus Torvalds and also for Andrew Morton (until the middle of 2006 when Morton transferred to Google). Torvalds worked full-time on behalf of OSDL, developing the Linux kernels.

## Chronology

- 1991:** The Linux kernel is publicly announced on 25 August by 21-year-old Finnish student Linus Benedict Torvalds.
- 1992:** The Linux kernel is re-licensed under the GNU GPL. The first Linux distributions are created.
- 1993:** Over 100 developers work on the Linux kernel. With their assistance, the

kernel is adapted to the GNU environment, which creates a large spectrum of application types for Linux. The oldest currently (as of 2015) existing Linux distribution, Slackware, is released for the first time. Later that same year, the Debian project is established. Today it is the largest distribution community.

- 1994:** Torvalds judges all components of the kernel to be fully matured: he releases version 1.0 of Linux. The XFree86 project contributes a graphical user interface (GUI). Commercial Linux distribution makers Red Hat and SUSE publish version 1.0 of their Linux distributions.
- 1995:** Linux is ported to the DEC Alpha and the Sun SPARC systems. Over the following years, it is ported to an ever-greater number of platforms.
- 1996:** Version 2.0 of the Linux kernel is released. The kernel can now serve several processors at the same time using symmetric multi-processing (SMP), and thereby becomes a serious alternative for many companies.
- 1998:** Many major companies such as IBM, Compaq, and Oracle announce their support for Linux. The Cathedral and the Bazaar were first published as an essay (later as a book), resulting in Netscape publicly releasing the source code to its Netscape Communicator web browser suite. Netscape's actions and crediting of the essay brings Linux's open source development model to the attention of the popular technical press. In addition, a group of programmers began developing the graphical user interface KDE.
- 1999:** A group of developers began work on the graphical environment GNOME, destined to become a free replacement for KDE, which at the time depended on the then proprietary, Qt GUI toolkit. During this year, IBM announced an extensive project for the support of Linux.
- 2000:** Dell announces that it is now the No. 2 provider of Linux-based systems worldwide and the first major manufacturer to offer Linux across its full product line.
- 2002:** The media reported, "Microsoft killed Dell Linux."
- 2004:** The XFree86 team splits up and joins with the existing X standards body to form the X.Org Foundation, which results in a substantially faster development of the X server for Linux.

- 2005:** The project openSUSE begins free distribution from Novell's community. In addition, the project OpenOffice.org introduces version 2.0 which then started supporting OASIS OpenDocument standards.
- 2006:** Oracle releases its own distribution of Red Hat Enterprise Linux. Novell and Microsoft announce cooperation for better interoperability and mutual patent protection.
- 2007:** Dell starts distributing laptops with Ubuntu pre-installed.
- 2009:** Red Hats market capitalization equals Suns, interpreted as a symbolic moment for the "Linux-based economy."
- 2011:** Version 3.0 of the Linux kernel is released.
- 2012:** The aggregate Linux server market revenue exceeds that of the rest of the UNIX market.
- 2013:** Google's Linux-based Android claims 75% of the Smartphone market share, in terms of the number of phones shipped.
- 2014:** Ubuntu claims 22,000,000 users.
- 2015:** Version 4.0 of the Linux kernel is released.

# Chapter Two: LINUX DISTRIBUTION (DISTRO)

## Introduction

The bewildering choice and the ever increasing number of Linux distributions can be confusing for those who are new to Linux. This is why this book was created. It lists 10 Linux distributions (plus an honorable mention of FreeBSD, by far the most popular of all of the BSDs), it is considered to be the most widely-used by Linux users around the world. There are no figures to back up this claim, and there are many other distributions that might suit your particular purpose better, but as a general rule, all of these are popular and have very active forums or mailing lists where you can ask questions if you get stuck. Ubuntu, Linux Mint, and PCLinuxOS are considered the easiest for new users who want to get productive in Linux as soon as possible without having to master all of its complexities. On the other end of the spectrum, Slackware Linux, Arch Linux, and FreeBSD are more advanced distributions that require a deeper understanding, before they can be used effectively. openSUSE, Fedora, Debian GNU/Linux and Mageia can be classified as good “middle-road” distributions. CentOS is an enterprise distribution, suitable for those who prefer stability, reliability, and long-term support to cutting-edge features and software.

## GUIDE TO CHOOSING DISTRIBUTION

### Linux Mint

Linux Mint, a distribution based on Ubuntu, was first launched in 2006 by Clement Lefebvre, a French-born IT specialist living in Ireland. Originally maintaining a Linux web site dedicated to providing help, tips and documentation to new Linux users, the author saw the potential of developing a Linux distribution that would address the many usability drawbacks associated with the generally more technical, mainstream products. Since its beginnings, the developers have been adding a variety of graphical “mint” tools for enhanced usability; this includes mintDesktop - a utility for configuring the desktop environment, mintMenu - a new and elegant menu structure for easier navigation, mintInstall - an easy-to-use software installer, and mintUpdate - a software



updater. Mint's reputation for ease of use has been further enhanced by the inclusion of proprietary and patent-encumbered multimedia codecs that are often absent from larger distributions due to potential legal threats. Perhaps one of the best features of Linux Mint is the fact that the developers listen to the users and are always fast in implementing good suggestions. While Linux Mint is available as a free download, the project generates revenue from donations, advertising, and professional support services.

**Pros:** Superb collection of “minty” tools developed in-house, hundreds of user-friendly enhancements, the inclusion of multimedia codecs, open to users' suggestions.

**Cons:** The alternative “community” editions do not always include the latest features; the project does not issue security advisories.

## Ubuntu

The launch of Ubuntu was first announced in September 2004. Although a relative newcomer to the Linux distribution scene, the project took off like no other, with its mailing lists soon filled with discussions by eager users and enthusiastic developers. In the years that followed, Ubuntu grew to become the most popular desktop Linux distribution and has contributed greatly toward developing an easy-to-use and free desktop operating system that can compete well with any of the proprietary ones available on the market. On the technical side of things, Ubuntu is based on Debian “Sid” (unstable branch), but with some prominent packages, such as GNOME, Firefox, and LibreOffice, updated to their latest versions. It uses a custom user interface called “Unity.” It has a predictable, 6-month release schedule, with an occasional Long Term Support (LTS) release that supports security updates for 5 years, depending on the edition (non-LTS release are supported for 9 months). Other special features of Ubuntu include an installable live DVD, creative artwork, desktop themes, migration assistant for Windows users, support for the latest technologies, such as 3D desktop effects, easy installation of proprietary device drivers for ATI and NVIDIA graphics cards, wireless networking, and on-demand support for non-free or patent-encumbered media codecs.

**Pros:** Fixed release cycle and support period; long-term support (LTS) variants with 5 years of security updates; novice-friendly; a wealth of documentation, both official and user-contributed.

**Cons:** Lacks compatibility with Debian; frequent major changes tend to drive some users away, the Unity user interface has been criticized as being more suitable for mobile devices rather than desktop computers; non-LTS releases come with only 9 months of security support.

## Debian GNU/Linux

Debian GNU/Linux was first announced in 1993. Its founder, Ian Murdock, envisaged the creation of a completely non-commercial product developed by hundreds of volunteer developers. With skeptics far outnumbering optimists at the time, it seemed destined to disintegrate and collapse, but the reality was very different. Debian not only survived, it thrived and, in less than a decade, it became the largest Linux distribution and possibly the largest collaborative software product ever created! The following numbers can illustrate the success of Debian GNU/Linux. Over 1,000 volunteer developers develop it, its software repositories contain close to 50,000 binary packages (compiled for 8 processor architectures) and is responsible for inspiring over 120 Debian-based distributions and live CDs. These figures are unmatched by any other Linux-based operating system. The actual development of Debian takes place in three main branches (or four if one includes the bleeding-edge “experimental” branch) of increasing levels of stability: “unstable” (also known as “sid”), “testing” and “stable.” However, this lengthy and complex development style has some drawbacks: the stable releases of Debian are not particularly up-to-date, and they age rapidly, especially since new stable releases are only published once every 1 - 3 years. Users who prefer the latest packages and technologies are forced to use the potentially buggy Debian testing distributions or unstable branches. The highly democratic structures of Debian have led to controversial decisions and have led to infighting among the developers. This has contributed to stagnation and reluctance to make radical decisions that would take the project forward.

**Pros:** Very stable; remarkable quality control; includes over 30,000 software packages; supports more processor architectures than any other Linux distribution.

**Cons:** Conservative - due to its support for many processor architectures, newer technologies are not always included; slow release cycle (one stable release every 1 - 3 years); discussions on developer mailing lists and blogs can be uncultured at times.

## Mageia

Mageia may be the newest distribution on this list, but its roots go back to July 1998 when Gaël Duval launched Mandrake Linux. At the time, it was just a branch of Red Hat Linux with KDE as the default desktop, better hardware detection, and some user-friendly features, but it gained instant popularity due to positive reviews in the media. Mandrake was later turned into a commercial enterprise and renamed to Mandriva (to avoid some trademark-related hassles and to celebrate its merger with Brazil's Conectiva) before almost going bankrupt in 2010. A Russian venture capital firm eventually saved it, but this came at a cost when the new management decided to lay off most of the established Mandriva developers in the company's Paris headquarters. Upon finding themselves out of work, they decided to form Mageia, a community project that is a logical continuation of Mandrake and Mandriva, perhaps more so than Mandriva itself. Mageia is primarily a desktop distribution. Its best-loved features are cutting-edge software, a superb system administration suite (Mageia Control Centre), the ability to attract a large number of volunteer contributors, and extensive internationalization support. It features one of the easiest, but more powerful system installers on its installation DVD, while it also releases a set of live images with both KDE or GNOME desktops and comprehensive language support, with the ability to install it onto a hard disk directly from the live desktop session. The distribution's well-established package management features, with powerful command-line options and a graphical software management module, allow easy access to thousands of software packages. The unique Mageia Control Center continues to improve with each release, offering a powerful tool for configuring just about any aspect of their computer without ever reaching for the terminal.

**Pros:** Beginner-friendly; excellent central configuration utility; very good out-of-the-box support for dozens of languages; installable live media.

**Cons:** Lacks reputation and mindshare following its split from Mandriva, some concern over the developers' ability to maintain the distribution long-term on a volunteer basis.

## Fedora

Although Fedora was formally only unveiled in September 2004, its origins date back to 1995 when it was launched by two Linux visionaries — Bob Young and Marc Ewing — under the name of Red Hat Linux. The company's first product,

Red Hat Linux 1.0 “Mother’s Day,” was released the same year and was quickly followed by several bug-fix updates. In 1997, Red Hat introduced its revolutionary RPM package management system with dependency resolution and other advanced features which greatly contributed to the distribution’s rapid rise in popularity and its overtaking of Slackware Linux as the most widely-used Linux distribution in the world. In later years, Red Hat standardized on a regular, 6-month release schedule. In 2003, just after the release of Red Hat Linux 9, the company introduced some radical changes to its product line-up. It retained the Red Hat trademark for its commercial products, notably Red Hat Enterprise Linux, and introduced Fedora Core (later renamed to Fedora), a Red Hat sponsored, but community-oriented distribution designed for the “Linux hobbyist”. After the initial criticism of the changes, the Linux community accepted the “new” distribution as the logical continuation of Red Hat Linux. A few quality releases was all it took for Fedora to regain its former status as one of the best-loved operating systems on the market. At the same time, Red Hat quickly became the biggest and most profitable Linux company in the world, with an innovative product line-up, excellent customer support, and other popular initiatives, such as its Red Hat Certified Engineer (RHCE) certification program. Although Fedora’s direction is still largely controlled by Red Hat, Inc. and the product is sometimes seen — rightly or wrongly — as a test bed for Red Hat Enterprise Linux, there is no denying that Fedora is one of the most innovative distributions available today. Its contributions to the Linux kernel, glibc and GCC are well-known and its more recent integration of SELinux functionality, virtualization technologies, system service manager, cutting-edge journaled file systems, and other enterprise-level features are much appreciated by the company’s customers. On a negative side, Fedora still lacks a clear desktop-oriented strategy that would make the product easier to use for those beyond the “Linux hobbyist” target.

**Pros:** Highly innovative; outstanding security features; a large number of supported packages; strict adherence to the free software philosophy; availability of live CDs featuring many popular desktop environments.

**Cons:** Fedora’s priorities tend to lean towards enterprise features, rather than desktop usability; some bleeding edge features, such as early switch to KDE 4 and GNOME 3, occasionally alienate desktop users.

**openSUSE**

The beginnings of openSUSE date back to 1992 when four German Linux enthusiasts — Roland Dyroff, Thomas Fehr, Hubert Mantel and Burchard Steinbild — launched the project under the name of SUSE (Software und System Entwicklung) Linux. In the early days, the young company sold sets of floppy disks containing a German edition of Slackware Linux, but it wasn't long before SUSE Linux became an independent distribution with the launch of version 4.2 in May 1996. In the following years, the developers adopted the RPM package management format and introduced YaST, an easy-to-use graphical system administration tool. Frequent releases, excellent printed documentation, and easy availability of SUSE Linux in stores across Europe and North America resulted in growing popularity of the distribution. SUSE Linux was acquired by Novell, Inc. in late 2003, and then fell into the hands of Attachmate in November 2010. Major changes in the development, licensing and availability of SUSE Linux followed shortly after the first acquisition - YaST was released under the GPL, the ISO images were freely distributed from public download servers, and, most significantly, the development of the distribution was opened to public participation for the first time. Since the launch of the openSUSE project and the release of version 10.0 in October 2005, the distribution became completely free in both senses of the word. The openSUSE code has become a base system for Novell's commercial products, first named as Novell Linux, but later renamed to SUSE Linux Enterprise Desktop and SUSE Linux Enterprise Server. Today, openSUSE has a large following of satisfied users. The principal reason for openSUSE receiving high marks from its users are pleasant and polished desktop environments (KDE and GNOME), an excellent system administration utility (YaST), and, for those who buy the boxed edition, some of the best-printed documentation available. However, the infamous deal between Novell and Microsoft, which apparently concedes to Microsoft's argument that it has intellectual property rights over Linux, has resulted in condemnation by many Linux personalities and has prompted some users to switch distributions. Although Novell has downplayed the deal, and Microsoft has yet to exercise any rights, this issue remains a thorn in the side of the otherwise very community-friendly Linux Company.

**Pros:** Comprehensive and intuitive configuration tool; large repository of software packages, excellent web site infrastructure, and printed documentation.

**Cons:** Novell's patent deal with Microsoft in November 2006 seemingly



legitimized Microsoft's intellectual property claims over Linux; its resource-heavy desktop set-up and graphical utilities are sometimes seen as "bloated and slow."

## **Arch Linux**

The KISS (Keep It Simple, Stupid) philosophy of Arch Linux was devised around the year 2002 by Judd Vinet, a Canadian computer science graduate who launched the distribution the same year. For several years it lived as a marginal project designed for intermediate and advanced Linux users and only shot to stardom when it began promoting itself as a "rolling-release" distribution that only needs to be installed once and is then kept up-to-date thanks to its powerful package manager and an always fresh software repository. As a result, Arch Linux "releases" are few and far between and are now limited to a basic installation DVD that is issued only when considerable changes in the base system warrant a fresh install. Besides featuring the much-loved "rolling-release" update mechanism, Arch Linux is also renowned for its fast and powerful package manager called "Pacman", the ability to install software packages from source code, easy creation of binary packages thanks to its AUR infrastructure, and the ever increasing software repository of well-tested packages. Its highly regarded documentation, complemented by the excellent Arch Linux Handbook makes it possible for even less experienced Linux users to install and customize the distribution. The powerful tools available at the user's disposal mean that the distro is infinitely customizable to the minutest detail and that no two installations can possibly be the same. On the negative side, any rolling-release update mechanism has its dangers: a human mistake can creep in, a library or dependency goes missing, a new version of an application already in the repository has a yet-to-be-reported critical bug... It is not unheard of to end up with an unbootable system following a Pacman upgrade. As such, Arch Linux is the kind of distribution that requires its users to be alert and to have enough knowledge to fix any such problems. In addition, the infrequent install media releases mean that it is sometimes no longer possible to use the old media to install the distribution due to important system changes or lack of hardware support in the older Linux kernel.

**Pros:** Excellent software management infrastructure; unparalleled customization and tweaking options; superb online documentation.

**Cons:** Occasional instability and risk of breakdown.

## **CentOS**

Launched in late 2003, CentOS is a community project with the goals of rebuilding the source code for Red Hat Enterprise Linux (RHEL) into an installable Linux distribution and to provide timely security updates for all included software packages. To put in more bluntly, CentOS is an RHEL clone. The only technical difference between the two distributions is branding - CentOS replaced all Red Hat trademarks and logos with its own. Nevertheless, the relations between Red Hat and CentOS remain amicable, and many CentOS developers are in active contact with, or even employed directly by, Red Hat. CentOS is often seen as a reliable server distribution. It comes with the same well-tested and stable Linux kernel and set of software packages that form the basis of its parent, Red Hat Enterprise Linux. Despite being a community project run by volunteers, it has gained a reputation for being a solid, free alternative to more costly server products on the market, especially among experienced Linux system administrators. CentOS is also suitable as an enterprise desktop solution, specifically where stability, reliability and long-term support are preferred over latest software and new features, like RHEL, CentOS includes approximately 7-10 years of security updates. Despite its advantages, CentOS might not be the best solution in all deployment scenarios. Those users who prefer a distribution with the latest Linux technologies and newest software packages should look elsewhere. Major CentOS versions, which follow RHEL versioning, are only released every 2 - 3 years, while “point” releases (e.g. 5.1) tend to arrive in 6 - 9 month intervals. The point releases do not usually contain any major features (although they do sometimes include support for more recent hardware), and only a handful of software packages may get updated to newer versions. The Linux kernel, the base system, and most application versions remain unchanged, but occasionally a newer version of an important software package (e.g. LibreOffice or Firefox) may be provided on an experimental basis. As a side project, CentOS also builds updated packages for the users of its distributions, but the repositories containing them are not enabled by default as they may break upstream compatibility.

**Pros:** Extremely well tested, stable and reliable; free to download and use; comes with 7+ years of free security updates.

**Cons:** Lacks latest Linux technologies; occasionally the project fails to live up its.

## **PCLinuxOS**

Bill “Texstar” Reynolds first introduced PCLinuxOS in 2003. Prior to

creating his own distribution, Texstar was already a well-known developer in the Mandrake Linux community of users for building up-to-date RPM packages for the popular distribution and providing them as a free download. In 2003, he decided to build a new distribution, initially based on Mandrake Linux, but with several significant usability improvements. The goals? It should be beginner-friendly, have out-of-the box support for proprietary kernel modules, browser plug-in and media codecs, and should function as a live CD with a simple and intuitive graphical installer. Several years and development releases later, PCLinuxOS is rapidly approaching its intended state. In terms of usability, the project offers out-of-the-box support for many technologies that most Windows-to-Linux migrants would expect from a new operating system. On the software side of things, PCLinuxOS is a KDE-oriented distribution, with a customized and always up-to-date version of the popular desktop environment. Its growing software repository contains other desktops and offers a great variety of desktop packages for many common tasks. For system configuration, PCLinuxOS has retained much of Mandriva's excellent Control Centre but has replaced its package management system with APT and Synaptic, a graphical package management front-end. On the negative side, PCLinuxOS lacks any form of roadmap or release goals. Despite growing community involvement, most development and decision-making remains in the hands of Texstar, who tends to be conservative when judging the stability of a release. As a result, the development process of PCLinuxOS is often arduous. For example, despite frequent calls for a 64-bit edition, the developers held off producing a 64-bit build until fairly recently. Furthermore, the project does not provide any security advisories, relying instead on the users' willingness to keep their system up-to-date via the included package management tools.

**Pros:** Out-of-the-box support for graphics drivers, browser plug-ins, and media codecs; rolling-release update mechanism; up-to-date software.

**Cons:** no out-of-the-box support for non-English languages; lacks release planning and security advisories.

## Slackware Linux

Slackware Linux, created by Patrick Volkerding in 1992, is the oldest surviving Linux distribution. Separated from the now-discontinued SLS project, Slackware 1.0 came on 24 floppy disks and was built on top of Linux kernel



version 0.99pl11-alpha. It quickly became the most popular Linux distribution; with some estimates putting its market share as much as 80% of all Linux installations in 1995. Its popularity decreased dramatically with the arrival of Red Hat Linux and other, user-friendlier distributions, but Slackware Linux still remains a much-appreciated operating system among the more technically oriented system administrators and desktop users. Slackware Linux is a highly technical, clean distribution, with only a limited number of custom utilities. It uses a simple, text-based system installer and a comparatively primitive package management system that does not resolve software dependencies. As a result, Slackware is considered one of the cleanest and least buggy distributions available today - the lack of Slackware-specific enhancements reduces the likelihood of new bugs being introduced into the system. The entire system configuration is completed by editing text files. There is a saying in the Linux community that if you learn Red Hat, you'll know Red Hat, but if you learn Slackware, you'll know Linux. This is particularly true today when many other Linux distributions keep developing heavily customized products to meet the needs of less technical Linux users. While this philosophy of simplicity has its fans, the fact is that in today's world, Slackware Linux is increasingly becoming a "core system" upon which new, custom solutions are built, rather than a complete distribution with a wide variety of supported software. The only exception is the server market, where Slackware remains popular, though even here, the distribution's complex upgrade procedure and lack of officially supported automated tools for security updates make it increasingly uncompetitive. Slackware's conservative attitude towards the system's base components means that it requires much manual post-installation work before it can be turned into a modern desktop system.

**Pros:** Considered highly stable, clean and largely bug-free, strong adherence to UNIX principles.

**Cons:** Limited number of officially supported applications; conservative in terms of base package selection; complex upgrade procedure.

## FreeBSD

FreeBSD, an indirect descendant of AT&T UNIX via the Berkeley Software Distribution (BSD), has a long and turbulent history dating back to 1993. Unlike Linux distributions, which are defined as integrated software solutions consisting of the Linux kernel and thousands of software applications, FreeBSD is a tightly

integrated operating system built from a BSD kernel and the so-called “userland” (therefore usable even without extra applications). This distinction is largely lost once installed on the average computer system - like many Linux distributions, a large collection of easily installed, (mostly) open source applications are available for extending the FreeBSD core, but these are usually provided by third-party contributors and aren't strictly part of FreeBSD. FreeBSD has developed a reputation for being a fast, high-performance and extremely stable operating system, especially suitable for web serving and similar tasks. Many large web search engines and organizations with mission-critical computing infrastructures have deployed and used FreeBSD on their computer systems for years. Compared to Linux, FreeBSD is distributed under a much less restrictive license, which allows virtually unrestricted use and modification of the source code for any purpose. Even Apple's Mac OS X is known to have been derived from FreeBSD. Besides the core operating system, the project also provides over 24,000 software applications in binary and source code forms for easy installation on top of the core FreeBSD. While FreeBSD can certainly be used as a desktop operating system, although it does not compare well to more popular Linux distributions. The text-mode system installer offers little in terms of hardware detection or system configuration, leaving much of the dirty work to the user in a post-installation setup. In terms of support for modern hardware, FreeBSD generally lags behind Linux, especially in supporting cutting-edge desktop and laptop gadgets, such as wireless network cards or digital cameras. Those users seeking to exploit the speed and stability of FreeBSD on a desktop or workstation should consider one of the available desktop FreeBSD projects, rather than FreeBSD itself.

**Pros:** Fast and stable; availability of over 24,000 software applications (or “ports”) for installation; very good documentation.

**Cons:** Tends to lag behind Linux in terms of support for new and exotic hardware, limited availability of commercial applications; lacks graphical configuration tools.

# Chapter Three: LICENSING

Code is contributed to the Linux kernel under a number of licenses, but all code must be compatible with version 2 of the GNU (GPLv2), which is the license covering the kernel distribution as a whole. In practice, that means that all code contributions are covered either by GPLv2 (with, optionally, language allowing distribution under later versions of the GPL) or the three-clause BSD license. Any contributions, which are not covered by a compatible license, will not be accepted into the kernel. Copyright assignments are not required (or requested) for code contributed to the kernel. All code merged into the mainline kernel retains its original ownership; as a result, the kernel now has thousands of owners. One implication of this ownership structure is that any attempt to change the licensing of the kernel is doomed to almost certain failure. There are few practical scenarios where the agreement of all copyright holders could be obtained (or their code removed from the kernel). Therefore, there is no prospect of a migration to version 3 of the GPL in the foreseeable future. It is imperative that all code contributed to the kernel be legitimately free software. For that reason, code from anonymous (or pseudonymous) contributors will not be accepted.

All contributors are required to “sign off” on their code, stating that the code can be distributed with the kernel under the GPL. Code which its owner, has not licensed as free software, or which risks creating copyright-related problems for the kernel (such as code which derives from reverse-engineering efforts lacking proper safeguards) cannot be contributed. Questions about copyright-related issues are common on Linux development mailing lists. Such questions will normally receive no shortage of answers, but one should bear in mind that the people answering those questions are not lawyers and cannot provide legal advice. If you have legal questions relating to Linux source code, there is no substitute for talking with a lawyer who understands this field. Relying on answers obtained on technical mailing lists is a risky affair.

## COMMUNITY

Linux communities come in two basic forms: developer and user.

One of the most compelling features of Linux is that it is accessible to developers; anybody with the requisite skills can improve Linux and influence the direction of

its development. Proprietary products cannot offer this kind of openness, which is a characteristic of the free software process. Developer communities can volunteer to maintain and support whole distributions, such as the Debian or Gentoo Projects. Novell and Red Hat also support community-driven versions of their products, openSUSE and Fedora, respectively. The improvements to these community distros are then incorporated into the commercial server and desktop products. The Linux kernel itself is primarily supported by its developer community and is one of the largest and most active free software projects in existence. A typical three-month kernel development cycle can involve over 1000 developers working for more than 100 different companies (or for no company at all).

With the growth of Linux has come an increase in the number of developers (and companies) wishing to participate in its development. Hardware vendors want to ensure that Linux supports their products well, making their products attractive to Linux users. Embedded systems vendors, who use Linux as a component in an integrated product, want Linux to be as capable and well suited to the task at hand as possible. Distributors and other software vendors who base their products on Linux have a clear interest in the capabilities, performance, and reliability of the Linux kernel. Other developer communities focus on different applications and environments that run on Linux, such as Firefox, OpenOffice, GNOME, and KDE. End users, too, can make valuable contributions to the development of Linux. With online communities such as Linux.com, LinuxQuestions, and the many and varied communities hosted by distributions and applications, the Linux user base is often a very vocal, but usually positive advocate and guide for the Linux operating system. The Linux community is not just a online presence. Local groups known as Linux Users Groups (LUGs) often meet to discuss issues regarding the Linux operating system, and provide other local users with free demonstrations, training, technical support, and install-fests.

## **DEVELOPMENT**

Linux is an operating system comprised of many different development languages. A very large percentage of the distributions' code is written in either the C (52.86%) or C++ (25.56%) languages. All of the rest of the code falls into single-digit percentages, with Java, Perl, and Lisp rounding out the top 5 languages. The Linux kernel itself has an even more dominant C presence, with

over 95 percent of the kernel's code written in that language. However, other languages make up the kernel as well, making it more heterogeneous than other operating systems. The kernel community has evolved its own distinct way of operating which allows it to function smoothly (and produce a high-quality product) in an environment where thousands of lines of code are being changed every day. This means the Linux kernel development process differs greatly from proprietary development methods.

The kernel's development process may come across as strange and intimidating to new developers, but there are good reasons and solid experience behind it. A developer who does not understand the kernel community's ways (or, worse, who tries to flout or circumvent them) will become very frustrated. The development community, while being helpful to those who are trying to learn, has little time for those who will not listen or who does not care about the development process. While many Linux developers, still use text-based tools such as Emacs or Vim to develop their code, Eclipse, Anjuta, and Netbeans all provide more robust integrated development environments.

# Chapter Four: INSTALLING DEBIAN 8

## What is Debian

Debian is an all-volunteer organization dedicated to developing free software and promoting the ideals of the Free Software community. The Debian Project began in 1993, when Ian Murdock issued an open invitation to software developers to contribute to a complete and coherent software distribution based on the relatively new Linux kernel. That relatively small band of dedicated enthusiasts, originally funded by the [Free Software Foundation](#) and influenced by the [GNU](#) philosophy, has grown over the years into an organization of around 1026 *Debian Developers*.

## Debian GNU/Linux

The combination of Debian's philosophy, methodology, GNU tools, the Linux kernel, and other important free software, form a unique software distribution called Debian GNU/Linux. This distribution is made up of a large number of software *packages*. Each package in the distribution contains executables, scripts, documentation, and configuration information, and has a *maintainer* who is primarily responsible for keeping the package up-to-date, tracking bug reports, and communicating with the upstream author(s) of the packaged software. The extremely large user base, combined with a bug tracking system ensures that problems are found and fixed quickly. Debian's attention to detail allows them to produce a high-quality, stable, and scalable distribution. Installations can be easily configured to serve many roles, from stripped-down firewalls to scientific desktop workstations to high-end network servers. Debian is especially popular amongst advanced users because of its technical excellence and its deep commitment to the needs and expectations of the Linux community. Debian also introduced many features to Linux that are now commonplace. For example, Debian was the first Linux distribution to include a package management system for the easy installation and removal of software. It was also the first Linux distribution that could be upgraded without requiring re-installation. Debian continues to be a leader in Linux development. Its development process is an example of just how well the Open Source development model can work — even for very complex tasks such as building and maintaining a



complete operating system.

The feature that most distinguishes Debian from other Linux distributions is its package management system. These tools give the administrator of a Debian system complete control over the packages installed, including the ability to install a single package or automatically update the entire operating system. Individual packages can also be protected from being updated. You can even tell the package management system about software you have compiled yourself and what dependencies it fulfills. To protect your system against “Trojan horses” and other malevolent software, Debian’s servers verify that uploaded packages come from registered Debian maintainers. Debian packagers also take great care to configure their packages in a secure manner. When security problems in shipped packages do appear, fixes are usually available very quickly.

### **Installing Debian 8 as a Virtual Machine.**

First, you will need to download and install VirtualBox, the Oracle software that allows you to run any OS on... any other OS, as a virtual environment. This procedure is a lot safer than a dual boot: no need to fool around with boot sector or disk partitions, all is virtually created. The counterpart is that the two OS’s share your computer resources. To avoid any performance issue, we will choose a lightweight but efficient desktop environment. This book will use the XFCE Debian edition, that you can download using this link below:

<https://www.debian.org/CD/torrent-cd/>

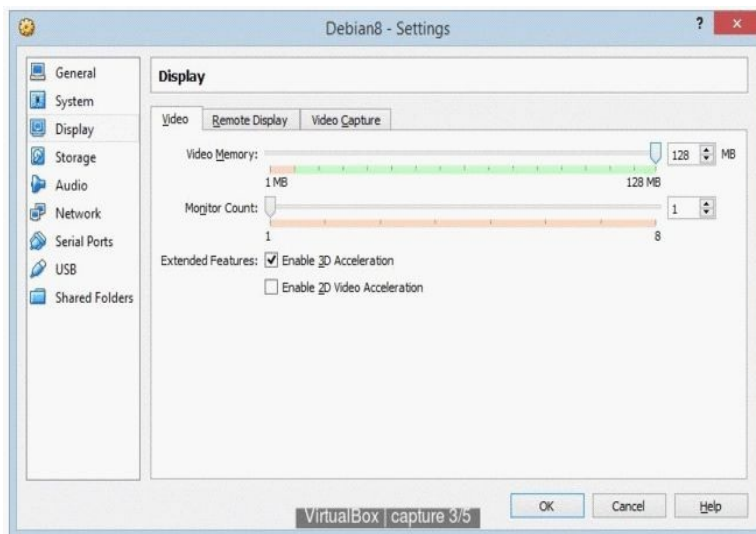
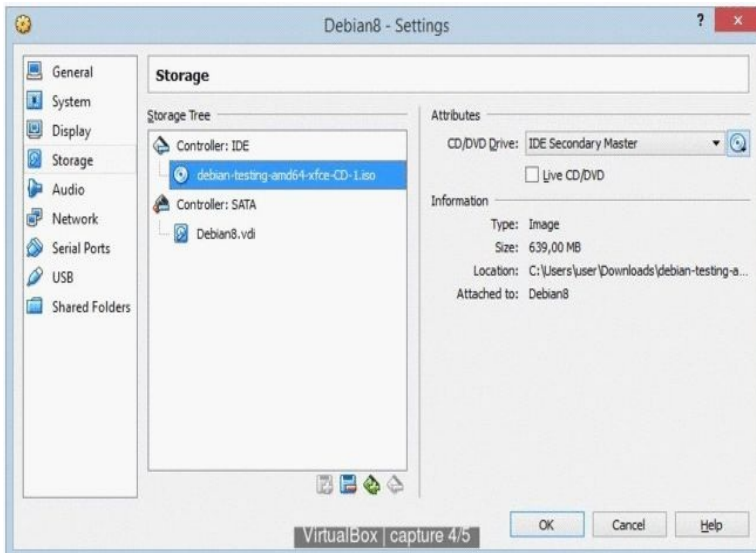
XFCE is not the newest desktop environment, however it is among the most powerful, lightweight, and customizable, while being easy to use with its classic desktop metaphor. Imagine an improved version of Windows XP, a great way to get started with Linux.

### **Create your Virtual System**

Once installed, start Virtual Box and follow these steps. We will begin the VM configuration:

- New > Name, Type (debian), Version 64
- General > Advanced > Activate clipboard and Drag’n’drop
- System > check memory, deactivate Floppy, check the nb of CPU (2)

- Display > max video memory, enable 3D acceleration
- Storage > select your Debian ISO under “Controller IDE”
- Shared Folders > choose which folder to share between the systems. We will get back to this function later.



We are now ready to run the distribution installation.

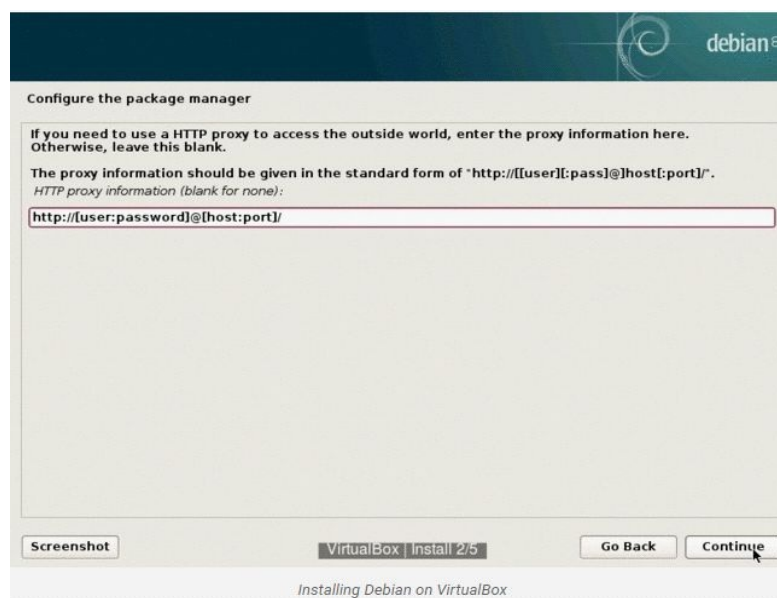
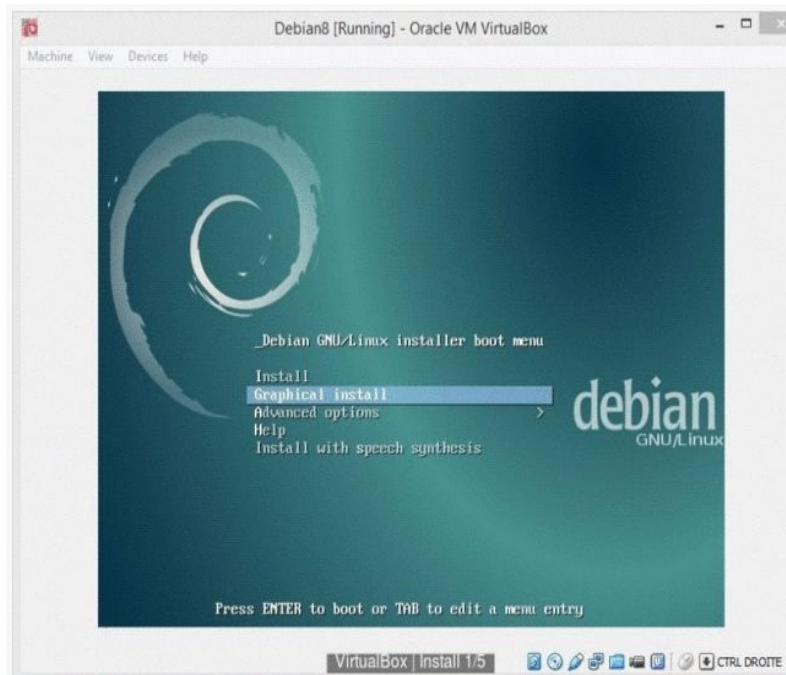
## Run Debian installer

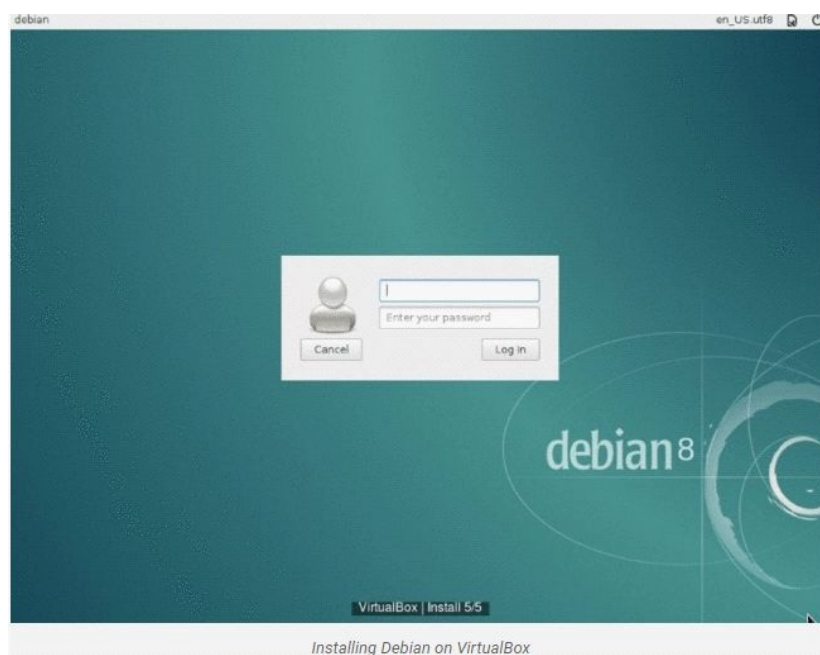
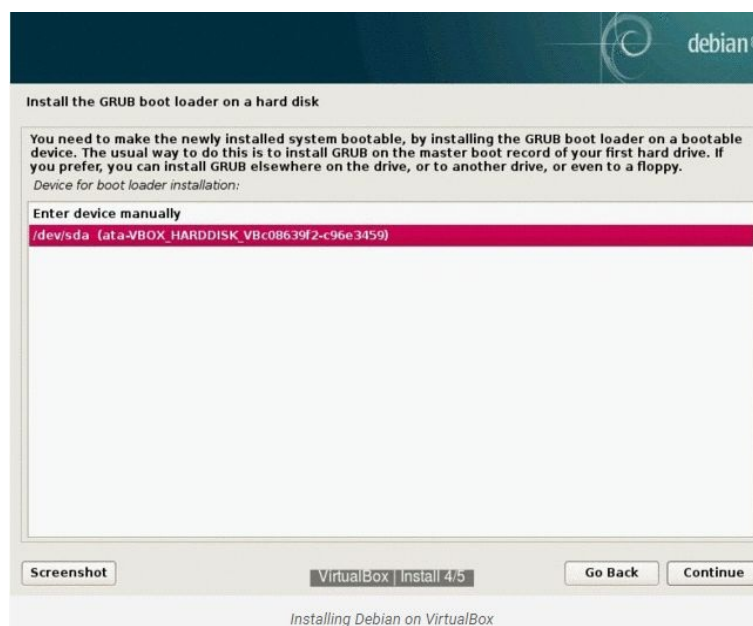
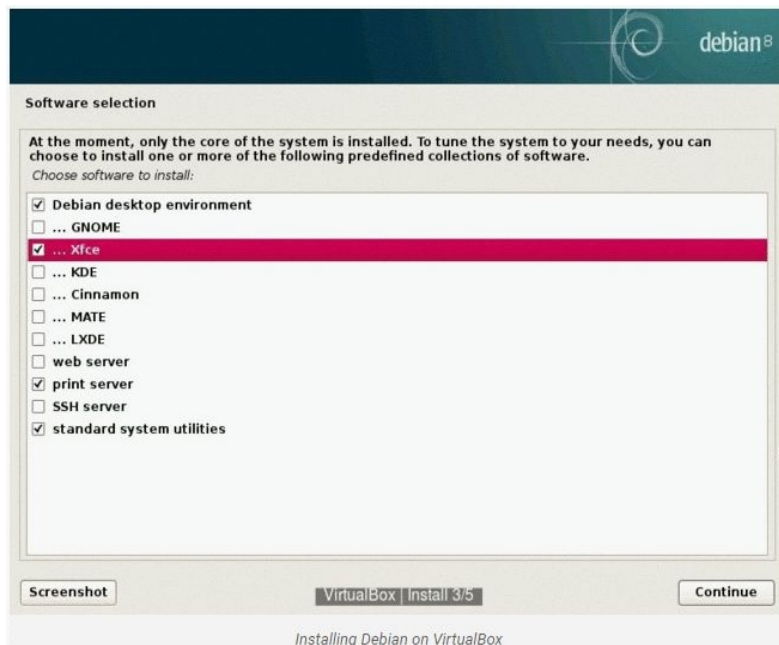
Select your virtual machine and click the **‘Start’** icon. The Debian graphic installer is straightforward: just follow the steps, when asked enter a root password; create your user (name, username, and password). Once you get to the partitioning, select ‘Guided, use entire disk, all in one partition.’ If you wish to modify the size of the swap partition, you may select the manual partitioning, or proceed later with Parted, the partitioning utility tool.

Proxy, part #1: select your network mirror. If you need to configure a proxy, use the following syntax in the required field: `http://user:pass@host:port/`.



Proceed to the installation. Once it has finished, you will have to choose where to install Grub (the boot loader package): because this is a VM install, choose /dev/sda (ata-VBOX\_HARDDISK). This would be the tricky part if you were installing on a dual-boot, system so enjoy the comfort of a VM!

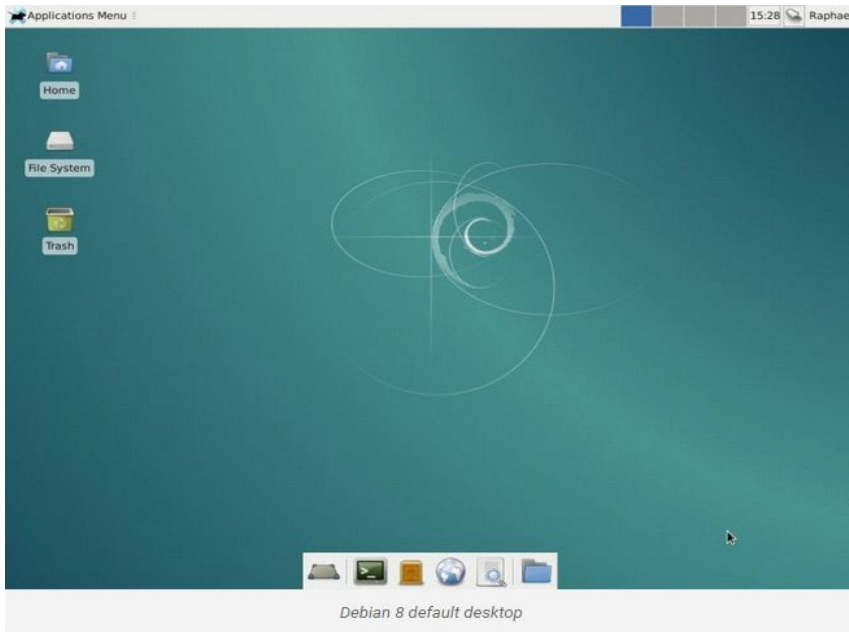




*Now reboot the VM.*

## First Boot and Updates

At startup the Grub menu is displayed, stick with the default entry and wait for your system to initialize. At XFCE first start, you will be asked to choose between two panel setups, select the default config.



Check your time configuration. If incorrect, launch as root a `dpkg-reconfigure tzdata` and select your country.

**Proxy, part #2** (if needed): during install, you entered your proxy address for **apt** connection (repositories connection). It has been written inside the file `/etc/apt/apt.conf`, and can be modified any time. I use **nano** as command line text editor, change at will.

```
[root] $ nano /etc/apt/apt.conf
Acquire::http::Proxy "http://user:password@proxy.address:port/";
```

Now add the proxy setting for the entire system:

```
[root] $ nano /etc/environment
http_proxy=http://user:password@proxy.address:port/
https_proxy=http://user:password@proxy.address:port/
ftp_proxy=http://user:password@proxy.address:port/
```

Congratulations, you are now a confirmed Linux user! Now we can update and install some useful tools, then we will modify the default UI. To use the full capacity of your VM you need to install a complementary VirtualBox extension called **Guest Additions**. To do so, some dependencies are required.

Check your sources, remove the CD entries, add the **contrib** and **non-free** repos:

```
[root] $ nano /etc/apt/sources.list
deb http://ftp.fr.debian.org/debian/ jessie main contrib non-free
deb http://security.debian.org/ jessie/updates main contrib non-free
```

Update your system:

```
[root] $ apt-get update && apt-get dist-upgrade
```

Install the Guest Additions requirements

```
[root] $ apt-get install build-essential module-assistant
[root] $ m-a prepare
```

Insert the Guest Additions CD image (Host + D), go to the root of the mounted drive and execute the program:

```
[root] $ sh ./VBoxLinuxAdditions.run
```

## Customizing your System

... to get something that looks a little more modern.



## Terminal

The terminal is one of the most useful tools on a Linux system. Spend some time making it pleasant to look at: enable your prompt color ink **.bashrc**, add your aliases, and enable colors and auto completion in root bash.

```
$ nano ~/.bashrc
```

uncomment the “force\_colored\_prompt=yes” line

```
[root] $ nano .bashrc
```

```
set a fancy red prompt
```

```
PS1='${debian_chroot:+($debian_chroot)}\[\33[01;31m\]\u@\h\[\33[00m\]:\[\33[01;34m\]\w\[\33[00m\] \$ '
```

```
enable auto-completion
```

```
if
```

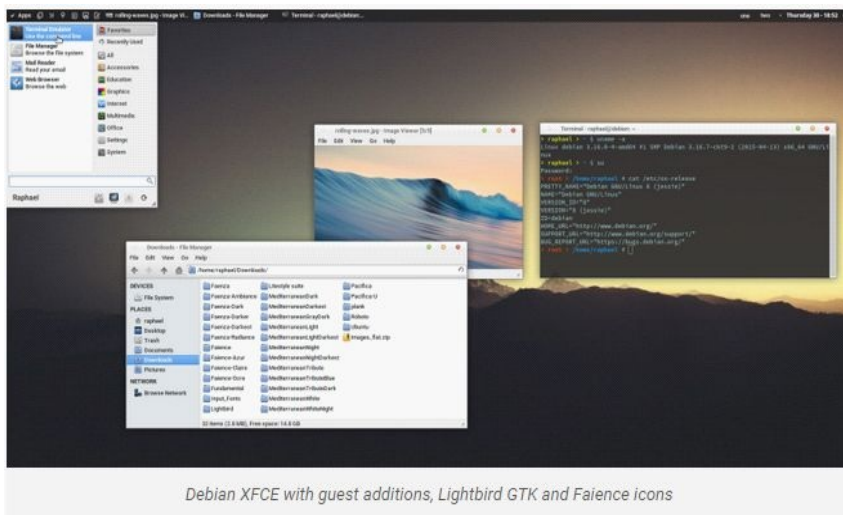
```
[-f /etc/bash_completion] && ! shopt -oq posix;
```

```
then . /etc/bash_completion
```

```
fi
```

## Fonts

Enable **font hinting** in the *Appearance Panel* to get the best font rendering for your system. Settings > Appearance > Fonts > Enable anti-aliasing + Hinting Slight + Sub-pixel order RGB. This should be enough in Jessie, no need to create a fonts.conf file or install the Infinality engine.



## Themes and icons

GTK Themes, windows decoration and iconsets are the most important elements to customizing your desktop. Start with installing some complementary gtk engines that are needed by some themes:

```
[root] $ apt-get install gtk2-engines-murrine gtk2-engines-pixbuf dmz-cursor-theme
```

## Install sudo

Debian doesn't come with sudo out of the box, this is a handy little tool for temporarily giving you root access.



```
$ su
$ apt-get install sudo
$ nano /etc/sudoers
```

Add below **root ALL=(ALL) ALL :**

```
USERNAME ALL=(ALL) NOPASSWD: ALL
```

Replace **USERNAME** with your username. The **NOPASSWD** flag removes the requirement to enter a password every time you use **sudo**. This is *not* a good idea on a production server!

Reboot your machine to make the changes take effect: **\$ reboot**

From now on all operations that require root should be run with the **sudo** prefix, without having to enter the password each time.

## Set up the network

Log in to your new VM using the username and password you chose during installation. We're going to add a network connection to your VM that will allow you too easily SSH into the server.

```
$ sudo nano /etc/network/interfaces
```

Add the following to the end of the file:

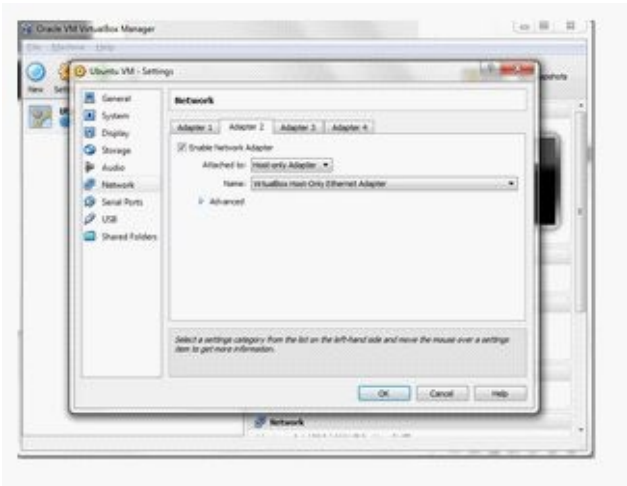
```
auto eth1
iface eth1 inet static
address 192.168.56.101
netmask 255.255.255.0
```

Hit **CTRL + X**, then **Y** and **ENTER** to save changes.

Shutdown the VM.

```
$ sudo shutdown -h now
```

At the main Virtual Box screen, hit the **Settings** button. Then select **Network** from the list on the left, choose *Adapter 2* from the tabs, check 'Enable Network Adapter' and choose 'Host-only Adapter' from the dropdown, then click **OK**.



What we have done is set up your VM to use a static IP address. This is a good idea because it allows us to always access our VM using a single IP address/hostname without having to look it up each time we boot.

By default, Virtual Box utilizes the **192.168.56.1** address in your network, and it assigns IP addresses in the **192.168.56.1xx** range to all your VMs.

By editing **/etc/network/interfaces** we told the OS that it should expect a network resource to be available at that address.

## Setup your hosts file

Now your server is configured lets add the hostname to your hosts file!

Simply add the following entry into your hosts file. This should be done on your host machine – be it Windows or Mac OS X or Linux, not the VM itself.

```
192.168.56.101 debian-vm
```

Keep in mind that for every domain you setup on your VM, you will need to add it to your hosts file.

## Log in via SSH!

Now that you have setup the network adapter in Virtual Box, and added the correct settings to the VM interfaces file, you're ready to actually SSH into your server and begin installing everything! You may be wondering why you need to SSH and not simply use the VM window. The reason I do it this way is that the server does not support copy/paste! There's a lot of typing ahead and having the ability to simply copy/paste into your terminal is going to speed things up quite a bit!

For SSH on Windows, I use [KiTTY](#). It's an SSH client that adds some nifty features to [PuTTY](#). There is also [Poderosa](#). For Mac/Linux just use the terminal! Since

you've already added the correct lines to your hosts file, you can set the address to connect to as **debian-vm** (or whatever you chose during setup). Make sure to actually start the VM from VirtualBox before attempting to login. Just start it, there is no need to login from the VirtualBox server window.

## Installing the basics

First things first, we're going to install the basic necessities, like **make**, **curl**, **wget**, as well as the Apache Server, Mercurial, Git and Subversion:

```
$ sudo apt-get install gcc make wget cron curl libxml2 libxml2-dev libzip-dev \
libbz2-dev curl libcurl4-openssl-dev libcurl3 libcurl3-gnutls libjpeg62 \
libjpeg62-dev libpng12-0 libpng12-dev libmcrypt-dev libmcrypt4 libxslt1-dev \
libxml2-dev apache2 apache2-mpm-prefork apache2-prefork-dev apache2-utils \
apache2.2-common git mercurial subversion libcupsys2 samba samba-common
```

Edit the new Apache2 config file, **\$ sudo nano /etc/apache2/httpd.conf** and add

```
ServerName localhost
```

You now have the Apache server up and running! Just point your browser to **http://debian-vm** and behold the magic.

Now lets enable Apache's **ModRewrite** module:

```
$ sudo a2enmod rewrite
```

## Installing MySQL

We will be installing MySQL 5.1 from repo, as this is the easiest way and works fairly well.

```
$ sudo apt-get install mysql-client-5.1 mysql-server-5.1
```

On the screens asking for a MySQL password, leave it blank and hit Enter. Since this is only for a local server there is no point in setting up a password. Do NOT use a blank password in production environments.

## Setting up MySQL

We need to update the IP address that MySQL will listen to for connections by editing the my.cnf file.

```
$ sudo nano /etc/mysql/my.cnf
```

Do a search for **bind-address** (**CTRL + W**) and change the setting to:

```
bind-address = 127.0.0.1
bind-address = 192.168.56.101
```



Now let's grant root MySQL user all permissions:

```
$ mysql -u root
GRANT ALL ON *.* TO 'root'@'%';
exit;
```

and restart the service:

```
$ sudo service mysql restart
```

# Chapter Five: INSTALLING CENTOS 7

This is a community-supported distribution derived from sources freely provided to the public by Red Hat for Red Hat Enterprise Linux (RHEL). As such, CentOS Linux aims to be functionally and compatible with RHEL. The CentOS Project mainly changes packages to remove upstream vendor branding and artwork. CentOS Linux is free and free to redistribute. Each CentOS version is maintained for up to 10 years (by means of security updates — the duration of the support interval by Red Hat has varied over time with respect to Sources released). A new CentOS version is released approximately every 2 years and each CentOS version is periodically updated (roughly every 6 months) to support newer hardware. This results in a secure, low-maintenance, reliable, predictable and reproducible Linux environment.

## Installation of CenOS7

Finally, the much-awaited CentOS 7 is out. CentOS (Community Enterprise Operating System) are forked from Red Hat Linux, a Linux Distro fine tuned for servers. You will learn how to install Centos 7 in a few easy steps.

### Step 1: Download the ISO Image

To get a copy of CentOS 7 download from the link below:

<http://mirror.centos.org/centos/7/>

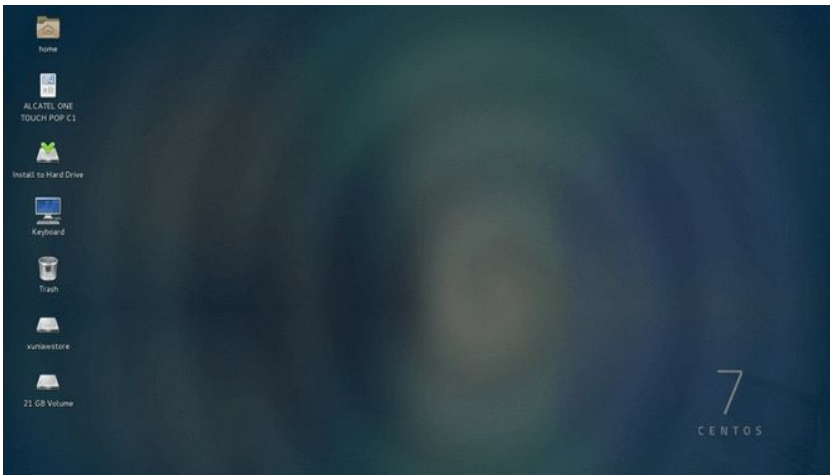
CentOS 7 is only available for 64-bit platforms; currently there is no 32-bit ISO image. This is primarily due to the fact that most production servers are 64-bit.

### Step 2: Make a bootable Drive

After you download the ISO image, make a bootable USB drive using Unetbootin. Alternatively, you can burn a DVD using Brasero or your favorite CD/DVD burning software.

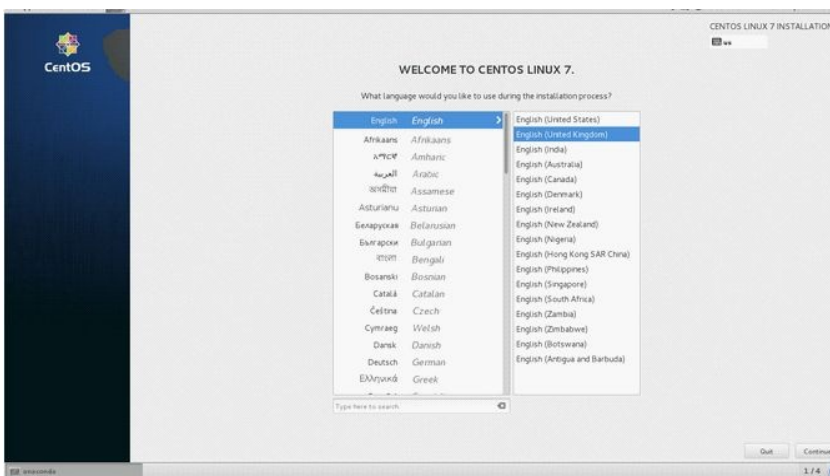
### Step 3: Begin Installation

To begin installation, click on the Install to Hard Drive icon on the desktop.



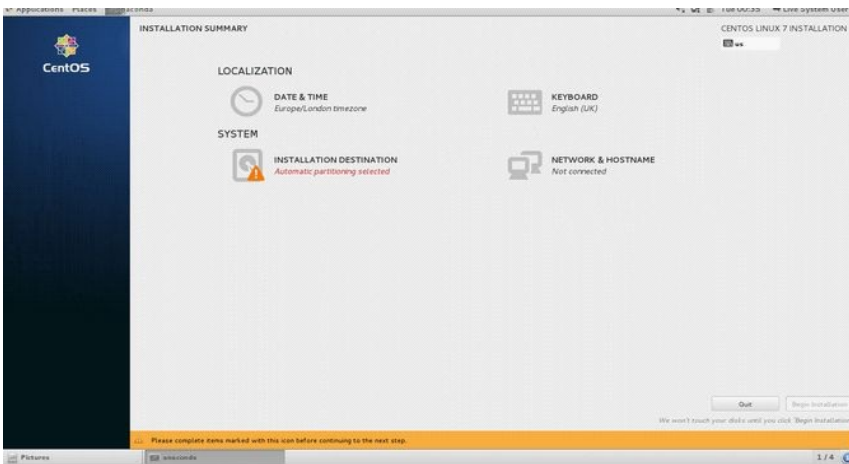
## Step 4: Select Language and Keyboard

Select your preferred language.

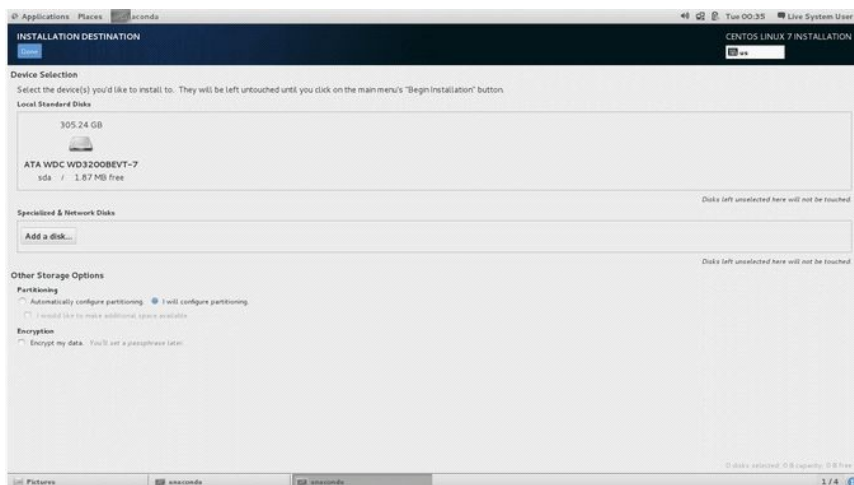


## Step 5: Change the Installation Destination

By default, the Anaconda installer will choose automatic partitioning for your hard disk. Click on the INSTALLATION DESTINATION icon to change to custom partitioning.

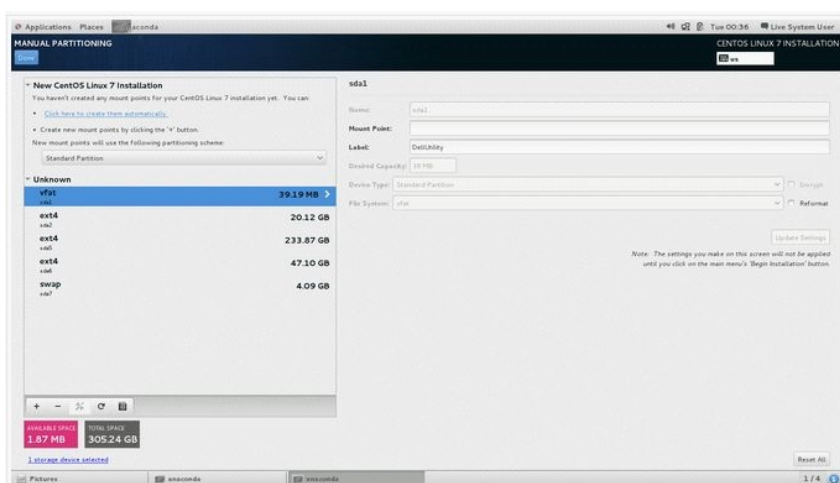


Select the hard drive where you want to install CentOS 7 and under Other Storage Options, choose 'I will configure partitioning' then click Done.



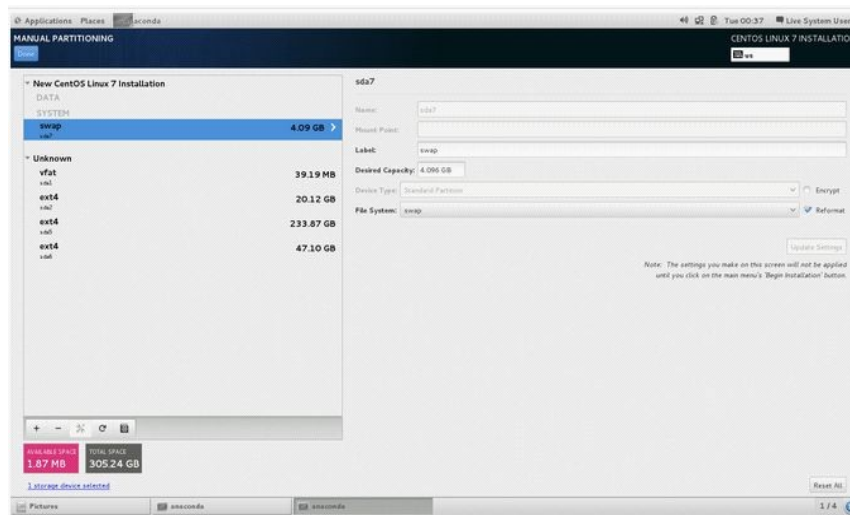
## Step 6: Select the Partitioning Scheme

Next, select the partitioning scheme to use for the mountpoints. In this case, choose Standard Partition.



## Step 7: Create a Swap Space

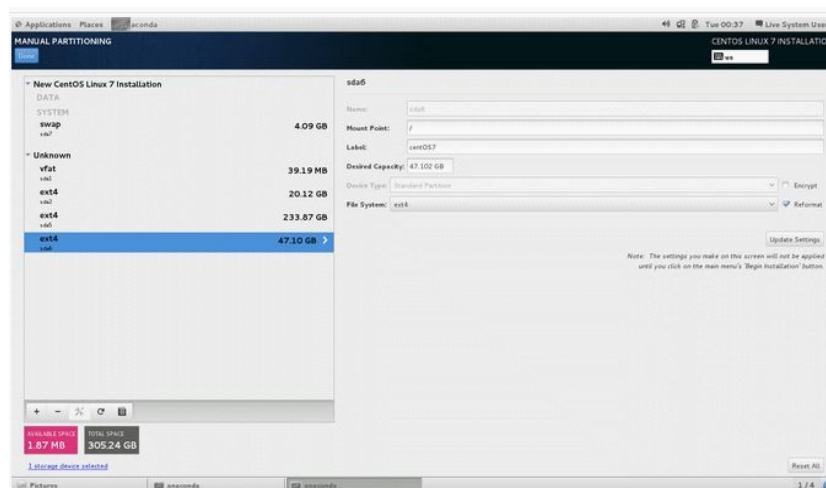
You can create a swap space in one of the partitions and set the desired capacity, which is dependent on how much RAM, the system has. Choose the File System for swap space as swap, and select the Reformat option. You can also name your swap space to whatever name you like but a name like swap is more descriptive.



## Step 8: Create a Mountpoint

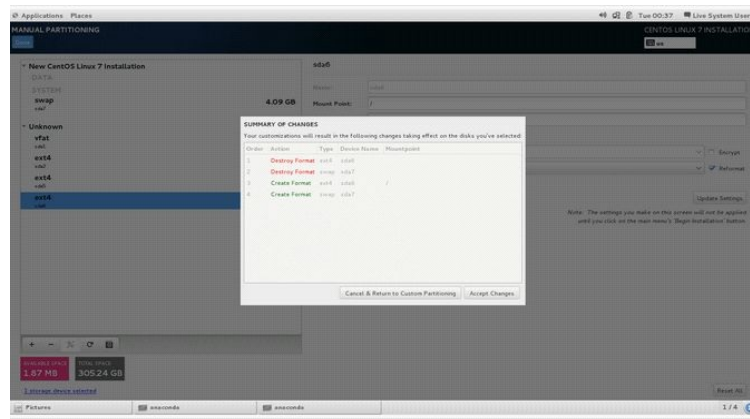
The next step is to create the mountpoint where the root partition will be installed. Depending on your requirements, you might need to put the boot, home and root partitions on different mountpoints. In this instance, we shall only create one mountpoint `/`.

Next, set the **Label** and **Desired Capacity** to whatever you wish. A rule of thumb is to use descriptive names for the label especially if the computer is to be managed by different system administrators. Choose the file system as `ext4` and select the **Reformat** option.



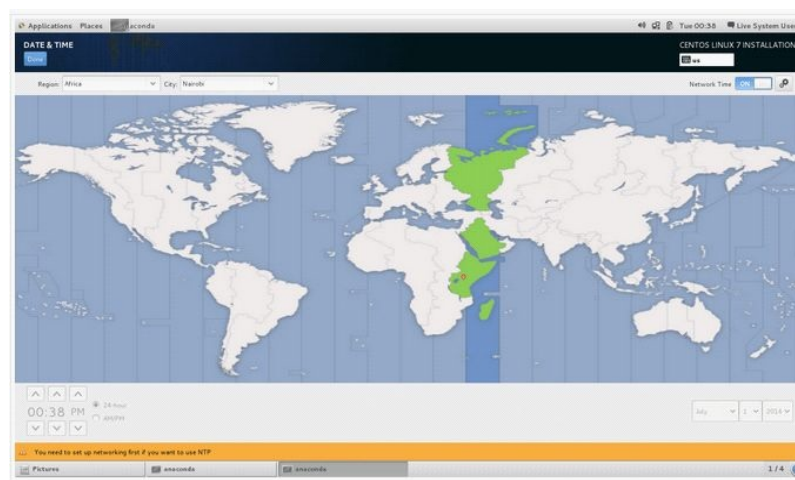
## Step 9: Accept Changes

After Steps 7 and 8 have successfully completed click on **Done** button. A prompt window will appear with a summary of the changes that will take place. If you are satisfied with them click **Accept Changes**.



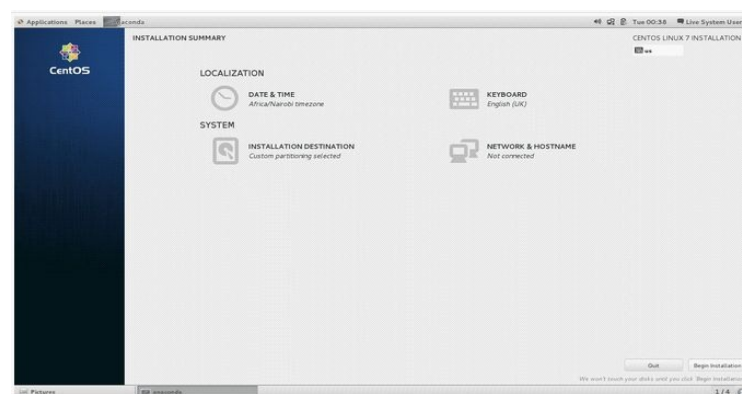
## Step 10: Set Date and Time

Click on the Clock icon under the localization menu and select your time zone from the map of the world, then click Done.



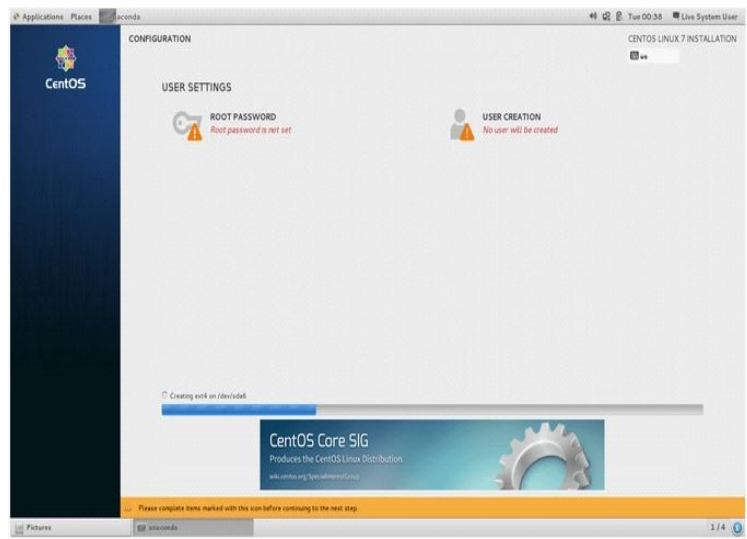
## Step 11: Begin Installation

Click on the Begin Installation button.



Installation will begin immediately and as it proceeds you will need to set up a user account as well as the root password.

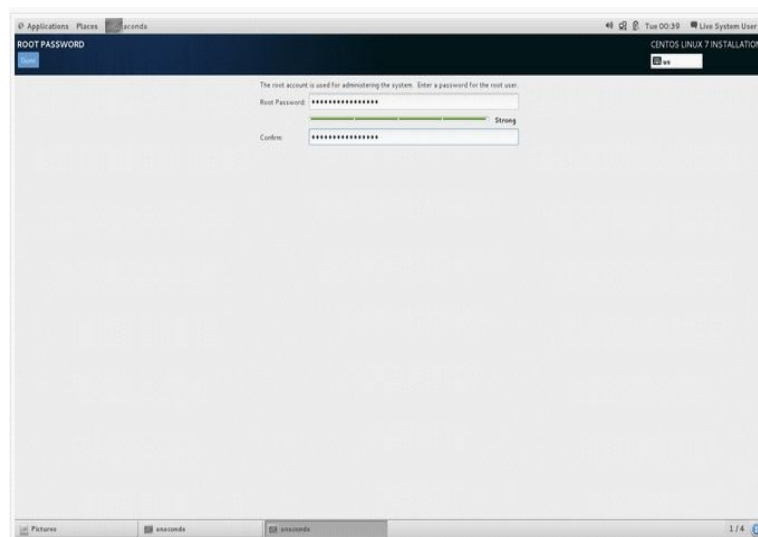




Set

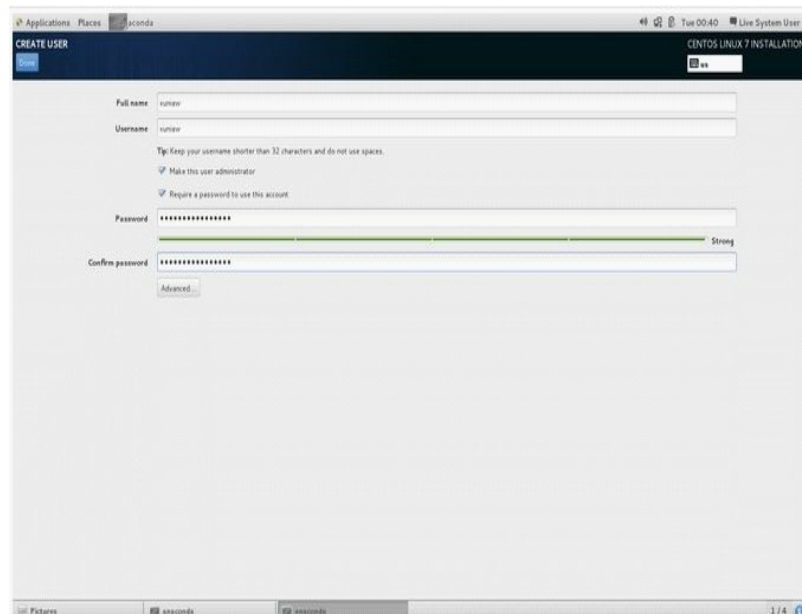
## Step 12: Set Up Root Password

Click on the ROOT PASSWORD option and enter a password and confirmation of the same then click Done.



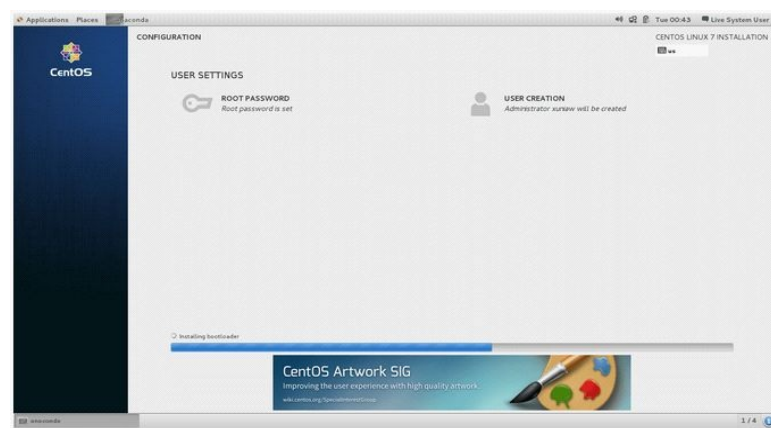
## Step 13: Create a User Account

The next step is to create a user account. Enter the details and if this is an administrator account, check Make this user administrator and Require a password to use this account for security purposes.



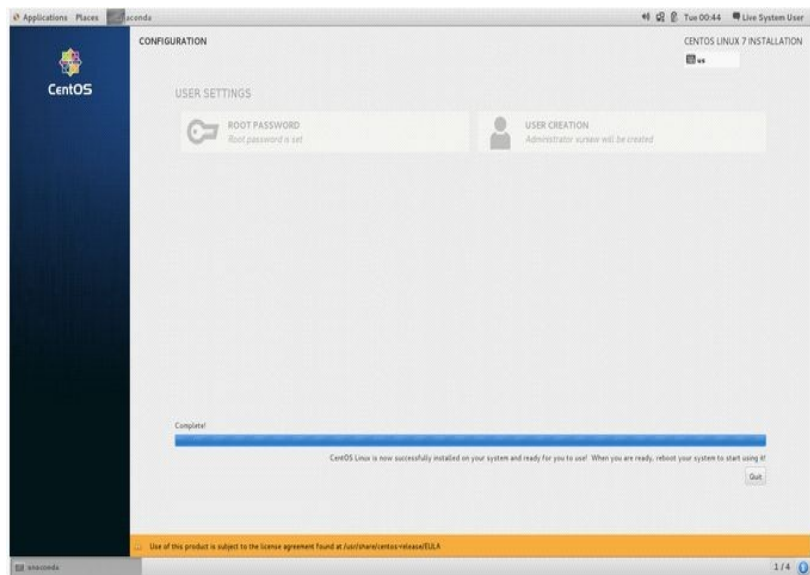
## Step 14: Complete Installation

The installer should finish installing the software and boot loader.



Hopefully, once the install is complete you will get a success message, after which you can click Quit. Now logout from the live system and login to your new installation. Finally, once you login to your CentOS 7 accept the EULA agreement and enjoy!





## Change and Set Hostname Command

On a CentOS Linux 7 server, you can use any one of the following tools to manage hostnames:

- **hostnamectl**: Control the system hostname. This is the recommended method.
- **nmtui**: Control the system hostname using text user interface (TUI).
- **nmcli**: Control the system hostname using CLI part of NetworkManager.

### Method #1: hostnamectl

Let us see how to use the hostnamectl command.

To verify new settings, enter:

```
hostnamectl status
```

### How do I see the host names?

Sample outputs:

Static hostname: server1.cyberciti.biz

Pretty hostname: Senator Padmé Amidala's Laptop

Transient hostname: r2-d2

Icon name: computer

Chassis: n/a

Machine ID: b5470b10ccfd49ed8e4a3b0e953a53c3

Boot ID: f79de79e2dac4670bddfe528e826b61f

Virtualization: oracle

Operating System: CentOS Linux 7 (Core)

CPE OS Name: cpe:/o:centos:centos:7

Kernel: Linux 3.10.0-229.1.2.el7.x86\_64

Architecture: x86\_64

## How do I delete a particular host name?

The syntax is:

```
hostnamectl set-hostname ""
hostnamectl set-hostname "" --static
hostnamectl set-hostname "" --pretty
```

## How do I change host name remotely?

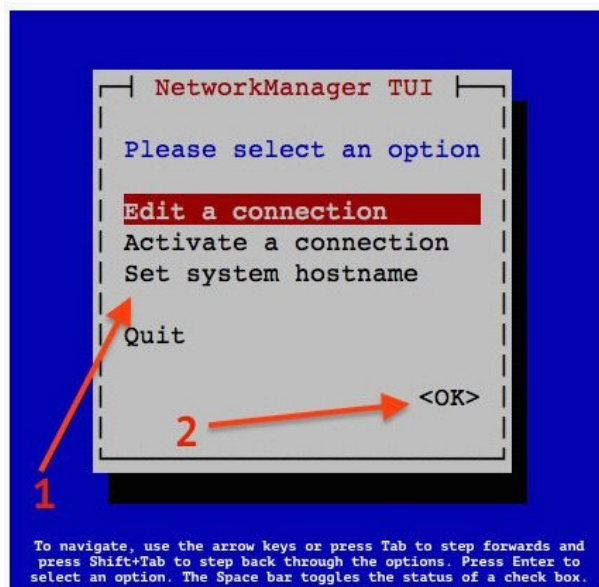
Use any of the following syntax:

```
ssh root@server-ip-here hostnamectl set-hostname server1
OR set server1 as host name on a remote server called 192.168.1.42 using ssh:
hostnamectl set-hostname server1 -H root@192.168.1.42
```

## Method #2: nmtui

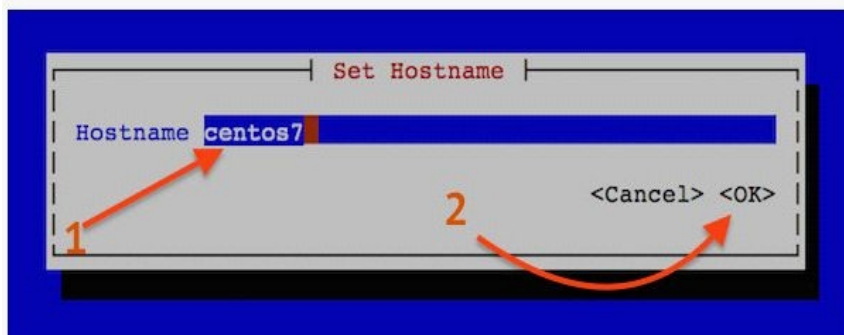
You can set host name using the `nmtui` command, which has text user interface for new users:

```
nmtui
```



Sample outputs:

Use the **Down** arrow key > select the 'Set system hostname' menu option > Click



the OK button:

You will see the confirmation box as follows:



Finally, restart the hostnamed service by typing the following command:

```
systemctl restart systemd-hostnamed
```

To verify the changes, enter:

```
hostnamectl status
```

### Method #3: nmcli

The **nmcli** is a command-line tool for controlling Network Manager and reporting network status.

#### To view the host name using nmcli:

The syntax is:

```
nmcli general hostname
```

To set the host name using nmcli:

The syntax is:

```
nmcli general hostname R2-D2
```

```
nmcli general hostname server42.cyberciti.biz
```

Finally, restart the systemd-hostnamed service:

```
systemctl restart systemd-hostnamed
```

# Chapter Six: LINUX AND UNIXMAN COMMAND

On **Linux** and other **Unix-like operating systems**, *man* is the interface used to view the system's reference manuals.

## Syntax

```
man [-C file] [-d] [-D] [-warnings[=warnings]] [-R encoding] [-L locale]
 [-m system[,...]] [-M path] [-S list] [-e extension] [-i|-I]
 [-regex|-wildcard] [-names-only] [-a] [-u] [-no-subpages] [-P pager]
 [-r prompt] [-7] [-E encoding] [-no-hyphenation] [-no-justification]
 [-p string] [-t] [-T[device]] [-H[browser]] [-X[dpi]] [-Z]
 [[section] page ...] ...
man -k [apropos options] regexp ...
man -K [-w|-W] [-S list] [-i|-I] [-regex] [section] term ...
man -f [whatis options] page ...
man -l [-C file] [-d] [-D] [-warnings[=warnings]] [-R encoding]
 [-L locale] [-P pager] [-r prompt] [-7] [-E encoding] [-p string] [-t]
 [-T[device]] [-H[browser]] [-X[dpi]] [-Z] file ...
man -w|-W [-C file] [-d] [-D] page ...
man -c [-C file] [-d] [-D] page ...
man [-hV]
```

## Description

Man is the system's manual viewer; it can be used to display manual pages, scroll up and down, search for occurrences of specific text, and other useful functions. Each argument given to *man* is normally the name of a program, utility or function. The pages associated with each of these arguments are then found and displayed. A section number, if provided, will direct *man* to look only in that section of the manual. The default action is to search in all of the available sections, following a pre-defined order and to show only the first page found, even if pages exist in several sections.

## General Options

|                           |                                                    |
|---------------------------|----------------------------------------------------|
| <b>-h, --help</b>         | <b>Print a help message and exit.</b>              |
| <b>-V, --version</b>      | Display version information and exit.              |
| <b>-C file, --config-</b> | Use configuration file <i>file</i> rather than the |

**file=***file*

default of `~/.manpath`.

**-d, --debug**

Print debugging information.

**-D, --default**

This option, when used, is normally specified as the first option; it resets **man**'s behaviour to its default. Its use is to reset those options that may have been set in **\$MANOPT**. Any options that follow **-D** will have their usual effect.

**--  
warnings[=warnings]**

Enable warnings from the **groff** text formatter. This may be used to perform sanity checks on the source text of manual pages. *warnings* is a comma-separated list of warning names; if it is not supplied, the default is "**mac**". See the "**Warnings**" node in the **groff info** page for a list of available warning names.

## Main Modes of Operation

**-f, --whatis**

Equivalent to the whatis command; displays a short description from the manual page, if available.

**-k, --apropos**

Equivalent to the apropos command; Search the short manual page descriptions for keywords and display any matches.

**-K, --global-apropos**

Search for text in all manual pages. This is a brute-force search, and is likely to take some time; if you can, you should specify a section to reduce the number of pages that need to be searched. Search terms may be simple strings (the default), or regular expressions if the **--regex** option is used.

**-l, --local-file**

Activate 'local' mode. Format and display local manual files instead of searching through the system's manual collection. Each manual page argument will be interpreted as an nroff source file in the correct format. No cat file is produced. If a dash ('-') is listed as one of the arguments, input will be taken from stdin. When this option is not used, and **man** fails to find the page required, before displaying the error message it attempts to act as if this option was supplied, using the name as a filename and looking for an exact match.

**-w, —where, —location** Don't actually display the manual pages; instead print the location(s) of the source **nroff** files that would be formatted.

**-W, —where-cat, —location-cat** Don't actually display the manual pages, but do print the location(s) of the **cat**files that would be displayed. If **-w** and **-W** are both specified, print both, separated by a space.

**-c, —catman** This option is not for general use and should only be used by the **catman** program.

**-R *encoding*, —recode=*encoding*** Instead of formatting the manual page in the usual way, output its source converted to the specified encoding. If you already know the encoding of the source file, you can also use **manconv** directly. However, this option allows you to convert several manual pages to a single encoding without having to explicitly state the encoding of each, provided that they were already installed in a structure similar to a manual page hierarchy.

## Finding Manual Pages

**-L *locale*, —locale=*locale*** **man** will normally determine your current locale by a call to the C function `setlocale` which checks the values of various environment variables, possibly including `$LC_MESSAGES` and `$LANG`. To temporarily override the determined value, use this option to supply a locale string directly to **man**. Note that it will not take effect until the search for pages actually begins. Output such as the help message will always be displayed in the initially determined locale.

**-m *system*[...], —systems=*system*[, ...]** If this system has access to other operating system's manual pages, they can be accessed using this option. To search for a manual page from (for example) the "NewOS" manual page collection, use the option **-m NewOS**.

The system specified can be a combination of comma delimited operating system names. To include a search of the native operating

system's manual pages, include the system name **man** in the argument string. This option will override the **\$SYSTEM** environment variable.

**-M path, —  
manpath=path**

Specify an alternate manpath to use. This option overrides the **\$MANPATH** environment variable and causes option **-m** to be ignored.

A path specified as a manpath must be the root of a manual page hierarchy structured into sections as described in the **man-db** manual (under “The manual page system”). To view manual pages outside such hierarchies, see the **-l** option.

**-S list, -s list, —  
sections=list**

*list* is a colon- or comma-separated list of ‘order specific’ manual sections to search. This option overrides the **\$MANSECT** environment variable. (The **—s** spelling is for compatibility with System V.)

**-e sub-  
extension, —  
extension=sub-  
extension**

Some systems incorporate large packages of manual pages, such as those that accompany the **Tcl** package, into the main manual page hierarchy. To get around the problem of having two manual pages with the same name such as **exit**, the **Tcl** pages were usually all assigned to section **l** (lowercase L). However, it is now possible to put the pages in the correct section, and to assign a specific “extension” to them, in this case, **exit (3tcl)**. Under normal operation, **man** will display **exit** in preference to **exit (3tcl)**. To negotiate this situation and to avoid having to know which section the page you require resides in, it is now possible to give **man** a sub-extension string indicating which package the page must belong to. Using the above example, supplying the option **-e tcl** to **man** will restrict the search to pages having an extension of **\*tcl**.

**-i, —ignore-case**

Ignore case when searching for manual pages. This is the default.

**-I, —match-case** Search for manual pages case-sensitively.

**—regex** Show all pages with any part of either their names or their descriptions matching each page argument as a regular expression, as with **apropos**. Since there is usually no reasonable way to pick a “best” page when searching for a regular expression, this option implies **-a**.

**—wildcard** Show all pages with any part of either their names or their descriptions matching each page argument using shell-style wildcards, as with **apropos —wildcard**. The page argument must match the entire name or description, or match on word boundaries in the description. Since there is usually no reasonable way to pick a “best” page when searching for a wildcard, this option implies **-a**.

**—names-only** If the **—regex** or **—wildcard** option is used, match only page names, not page descriptions, as with **whatis**. Otherwise, this option has no effect.

**-a, —all** By default, **man** will exit after displaying the most suitable manual page it finds. Using this option forces **man** to display all the manual pages with names that match the search criteria.

**-u, —update** This option causes **man** to perform an inode-level consistency check on its database caches to ensure that they are an accurate representation of the filesystem. It will only have a useful effect if **man** is installed with the **setuidbit** set.

**—no-subpages** By default, **man** will try to interpret pairs of manual page names given on the command line as equivalent to a single manual page name containing a hyphen or an underscore. This supports the common pattern of programs that implement a number of subcommands, allowing them to provide manual pages for each that can be accessed



using similar syntax as would be used to invoke the subcommands themselves. For example, the command:

```
man -aw git diff
```

displays the manual page:

```
/usr/share/man/man1/git-diff.1.gz
```

To disable this behavior, use the **—no-subpages** option.

For example:

```
man -aw —no-subpages git diff
```

Will instead show the manual pages for both **git** and **diff**:

```
/usr/share/man/man1/git.1.gz
```

```
/usr/share/man/man3/Git.3pm.gz
```

```
/usr/share/man/man1/diff.1.gz
```

---

## Controlling Formatted Output

---

**-P *pager*, —  
pager=*pager***

Specify which output pager to use. By default, **man** uses pager **-s**. This option overrides the **\$MANPAGER** environment variable, which in turn overrides the **\$PAGER** environment variable. It is not used in conjunction with **-f** or **-k**.

The value may be a simple command name or a command with arguments, and may use shell quoting (backslashes, single quotes, or double quotes). It may not use pipes to connect multiple commands; if you need that, use a wrapper script, which may take the file to display either as an argument or on standard input.

**-r *prompt*, —  
prompt=*prompt***

If a recent version of less is used as the pager, **man** will attempt to set its prompt and

some sensible options. The default prompt looks like:

```
|Manual page name(sec) line x|
```

where name denotes the manual page name, sec denotes the section it was found under and x the current line number. This is achieved by using the \$LESS environment variable.

Supplying -r with a string will override this default. The string may contain the text \$MAN\_PN which will be expanded to the name of the current manual page and its section name surrounded by “(” and “)”. The string used to produce the default could be expressed as;

```
| \ Manual\ page\ \ $MAN_PN\ ?
|ltline\ %lt?L/%L.:
|byte\ %bB?s/%s..?\ (END):?pB\
| %pB\%..
|(press h for help or q to quit)|
```

It is broken into three lines here for the sake of readability only. For its meaning see the man page for less.

The shell first evaluates the prompt string. All double quotes, back-quotes and backslashes in the prompt must be escaped by a preceding backslash. The prompt string may end in an escaped \$ which may be followed by further options for less. By default, man sets the -ix8 options.

If you want to override man’s prompt string processing completely, use the \$MANLESS environment variable described below.

## **-7, —ascii**

When viewing a pure ASCII manual page on a 7-bit terminal or terminal emulator, some characters may not display correctly when using the latin1 device description with GNU nroff. This option allows pure ASCII man pages to be displayed in ASCII with the latin1 device. It will not translate

any latin1 text. The following table shows the translations performed: some parts of it may only be displayed properly when using GNU nroff's latin1 device. This option is ignored when using options -t, -H, -T, or -Z and may be useless for versions of nroff other than GNU's.

**-E *encoding*, —  
encoding=*encoding***

Generate output for a character encoding other than the default. For backward compatibility, encoding may be an nroff device such as ascii, latin1, or utf8 as well as a true character encoding such as UTF-8.

**—no-  
hyphenation, —nh**

Normally, nroff will automatically hyphenate text at line breaks even in words that do not contain hyphens, if it is necessary to do lay out three words on a line without excessive spacing. This option disables automatic hyphenation, so words will only be hyphenated if they already contain hyphens. If you are writing a man page and simply want to prevent nroff from hyphenating a word at an inappropriate point, do not use this option, but consult the nroff documentation instead; for instance, you can put “\%” inside a word to indicate that it may be hyphenated at that point, or put “\%” at the start of a word to prevent it from being hyphenated.

**—no-  
justification, —nj**

Normally, nroff will automatically justify text to both margins. This option disables full justification, leaving justified only to the left margin, sometimes called “ragged-right” text.

If you are writing a man page and simply want to prevent nroff from justifying certain paragraphs, do not use this option, but consult the nroff documentation; for instance, you can use the “.na”, “.nf”, “.fi”, and “.ad” requests to temporarily disable adjusting and filling.

**-p *string*, —  
preprocessor=*string***

Specify the sequence of preprocessors to run before nroff or troff/groff. Not all installations will have a full set of preprocessors. Some of the preprocessors and the letters used to designate them are: eqn (e), grap (g), pic (p),

tbl (t), vgrind (v), refer (r). This option overrides the \$MANROFFSEQ environment variable. zsoelim is always run as the first preprocessor.

**-t, —troff** Use groff -mandoc to format the man page to standard output. This option is not required in conjunction with -H, -T, or -Z.

**-T[*device*], —troff-device[=*device*]** This option is used to change groff (or possibly troff's) output to be suitable for a device other than the default. It implies -t. Examples include dvi, latin1, ps, utf8, X75 and X100.

**-H[*browser*], —html[=*browser*]** This option will cause groff to produce HTML output, and will display the output in a web browser. The choice of browser is determined by the optional browser argument if one is provided, by the \$BROWSER environment variable, or by a compile-time default if that is unset (usually lynx). This option implies -t, and will only work with GNU troff.

**-X[*dpi*], —gxditview[=*dpi*]** This option displays the output of groff in a graphical window using thegxditview program. The *dpi* (dots per inch) may be 75, 75-12, 100, or 100-12, defaulting to 75; the -12 variants use a 12-point base font. This option implies -T with the X75, X75-12, X100, or X100-12 devices, respectively.

**-Z, —ditroff** groff will run troff and then use an appropriate post-processor to produce output suitable for the chosen device. If groff -mandoc is groff, this option is passed to groff and will suppress the use of a post-processor. It implies -t.

## Section Numbers

The section numbers of the manual are listed below. While reading documentation, if you see a command name followed by a number in parentheses, the number refers to one of these sections. For example, man is the documentation of man found in section number1. Some commands may have documentation in more than one section, so the numbers after the command

name may direct you to the correct section to find a specific type of information.

The section numbers, and the topics they cover, are as follows:

---

| section # | Topic                                                                     |
|-----------|---------------------------------------------------------------------------|
| 1         | Executable programs or shell commands                                     |
| 2         | System calls (functions provided by the kernel)                           |
| 3         | Library calls (functions within program libraries)                        |
| 4         | Special files (usually found in /dev)                                     |
| 5         | File formats and conventions eg /etc/passwd                               |
| 6         | Games                                                                     |
| 7         | Miscellaneous (including macro packages and conventions), e.g. man, groff |
| 8         | System administration commands (usually only for root)                    |
| 9         | Kernel routines [Non standard]                                            |

## Exit Status

When it terminates, **man** will return one of the following exit status:

---

|    |                                                                                                         |
|----|---------------------------------------------------------------------------------------------------------|
| 0  | Returned upon successful program execution.                                                             |
| 1  | Returned if there was a usage, syntax, or configuration file error.                                     |
| 2  | Returned if there was an operational error.                                                             |
| 3  | Returned if a child process returned a non-zero exit status.                                            |
| 16 | Returned if one or more of the pages, files, or keywords searched for did not exist or was not matched. |

---

## Environment

**man** makes use of the following [environment variables](#):

---

**MANPATH**

If **\$MANPATH** is set, its value is used as the path to search for manual pages.

**MANROFFOPT**

The contents of **\$MANROFFOPT** are added to the command line every time **man** invokes the formatter (nroff, troff, or **groff**).

**MANROFFSEQ**

If **\$MANROFFSEQ** is set, its value is used to determine the set of preprocessors to pass each manual page through. The default preprocessor list is system-dependent.

**MANSECT**

If **\$MANSECT** is set, its value is a colon-delimited list of sections and it is used to determine which man sections to search and in what order.

**MANPAGER, PAGER**

If **\$MANPAGER** or **\$PAGER** is set (**\$MANPAGER** is used in preference), its value is used as the name of the program used to display the man page. By default, **pager -s** is used. The value may be a simple command name or a command with arguments, and may use shell quoting (backslashes, single quotes, or double quotes). It may not use pipes to connect multiple commands; if you need that, use a wrapper script, which may take the file to display either as an argument or on standard input.

**MANLESS**

If **\$MANLESS** is set, man will not perform any of its usual processing to set up a prompt string for the less pager. Instead, the value of **\$MANLESS** will be copied verbatim into **\$LESS**. For example, if you want to set the prompt string unconditionally to “my prompt string”, set **\$MANLESS** to ‘**-Pmy prompt string**’.

---

**BROWSER**

If **\$BROWSER** is set, its value is a colon-delimited list of commands, each of which in turn is used to try to start a web browser for **man —html**. In each command, **%s** is replaced by a filename containing the HTML output from **groff**, **%%** is replaced by a single percent sign (%), and **%c** is replaced by a colon (:).

**SYSTEM**

If **\$SYSTEM** is set, it will have the same effect as if it had been specified as the argument to the **-m** option.

**MANOPT**

If **\$MANOPT** is set, it will be parsed prior to **mans** command line and is expected to be in a similar format. As all of the other man specific environment variables can be expressed as command line options, and are thus candidates for being included in **\$MANOPT** it is expected that they will become obsolete. Note: all spaces that should be interpreted as part of an option's argument must be escaped (preceded with a backslash).

**MANWIDTH**

If **\$MANWIDTH** is set, its value is used as the line length for which manual pages should be formatted. If it is not set, manual pages will be formatted with a line length appropriate to the current terminal (using an **ioctl** if available, the value of **\$COLUMNS**, or falling back to 80 characters if neither is available). **cat** pages will only be saved when the default formatting can be used, that is when the terminal line length is between 66 and 80 characters.

**MAN\_KEEP\_FORMATTING**

Normally, when output is not being directed to a terminal (such as to a file or a pipe), formatting characters are discarded to make it easier to read the result without special tools. However,

if **\$MAN\_KEEP\_FORMATTING** is set to any non-empty value, these formatting characters are retained. This may be useful for wrappers around **man** that can interpret formatting characters.

#### **MAN\_KEEP\_STDERR**

Normally, when output is being directed to a terminal (usually a pager), any error output from the command used to produce formatted versions of man pages is discarded to avoid interfering with the pager's display. Programs such as **groff** often produce relatively minor error messages about typographical problems such as poor alignment, which are unsightly and generally confusing when displayed along with the man page. However, you might want to see them anyway, so if **\$MAN\_KEEP\_STDERR** is set to a non-empty value, error output will be displayed as usual.

#### **LANG, LC\_MESSAGES**

Depending on the system and implementation, either or both of **\$LANG** and **\$LC\_MESSAGES** will be interrogated for the current message locale, **man** will display its messages in that locale (if available).

---

## Files

These files are used by **man**:

---

**/etc/manpath.config**

**The man-db configuration file.**

**/usr/share/man**

A global manual page hierarchy.

**/usr/share/man/index.  
(bt|db|dir|pag)**

A traditional global index database



cache.

`/var/cache/man/index.  
(bt|db|dir|pag)`

An FHS compliant  
global index  
database cache.

## Examples

```
man man
```

**View the man page for the man command.**

```
man --nh --nj man
```

# Chapter Seven: LINUX DIRECTORY COMMAND

Linux or UNIX-like systems use the `ls` command to list files and directories. However, `ls` does not have an option to list only directories. You can use combination of `ls` and `grep` to list directory names only. You can use the `find` command too. In this quick tutorial, you will learn how to list only directories in Linux or UNIX.

List all directories in Unix

Type the following command:

```
$ ls -l | grep `^d`
ls -l | egrep `^d`
```

**Try the following `ls` command to list directories in the current directory:**

```
$ ls -d */
```

```
desktop01:~ vivek$ pwd
/Users/vivek
desktop01:~ vivek$ ls -d */
Accounting/ Pictures/ music/
Applications/ Public/ old.data/
Books/ VirtualBox VMs/ perl5/
Desktop/ backups/ r/
Documents/ bin/ securebackups.realdata/
Downloads/ demo/ usb/
Library/ dev/ yuicompressor/
Movies/ google-cloud-sdk/
Music/ isos/
desktop01:~ vivek$ find . -maxdepth 1 -type d
.
./.Trash
./.ansible
./.bashish
./.cache
./.config
./.cpan
./.cups
./.dbus
./.dconf
./.dotnet
./.emacs.d
./.fonts
./.git
./.gnome2
./.gvfs
./.local
./.m2
./.maven2
./.npm
./.npmrc
./.nvm
./.oh-my-zsh
./.p2
./.pki
./.python27
./.python3
./.rvm
./.sbt
./.ssh
./.stack
./.tmux
./.vim
./.vscode
./.wsl
./.wsl2
./.wsl3
./.wsl4
./.wsl5
./.wsl6
./.wsl7
./.wsl8
./.wsl9
./.wsl10
./.wsl11
./.wsl12
./.wsl13
./.wsl14
./.wsl15
./.wsl16
./.wsl17
./.wsl18
./.wsl19
./.wsl20
./.wsl21
./.wsl22
./.wsl23
./.wsl24
./.wsl25
./.wsl26
./.wsl27
./.wsl28
./.wsl29
./.wsl30
./.wsl31
./.wsl32
./.wsl33
./.wsl34
./.wsl35
./.wsl36
./.wsl37
./.wsl38
./.wsl39
./.wsl40
./.wsl41
./.wsl42
./.wsl43
./.wsl44
./.wsl45
./.wsl46
./.wsl47
./.wsl48
./.wsl49
./.wsl50
./.wsl51
./.wsl52
./.wsl53
./.wsl54
./.wsl55
./.wsl56
./.wsl57
./.wsl58
./.wsl59
./.wsl60
./.wsl61
./.wsl62
./.wsl63
./.wsl64
./.wsl65
./.wsl66
./.wsl67
./.wsl68
./.wsl69
./.wsl70
./.wsl71
./.wsl72
./.wsl73
./.wsl74
./.wsl75
./.wsl76
./.wsl77
./.wsl78
./.wsl79
./.wsl80
./.wsl81
./.wsl82
./.wsl83
./.wsl84
./.wsl85
./.wsl86
./.wsl87
./.wsl88
./.wsl89
./.wsl90
./.wsl91
./.wsl92
./.wsl93
./.wsl94
./.wsl95
./.wsl96
./.wsl97
./.wsl98
./.wsl99
./.wsl100
./.wsl101
./.wsl102
./.wsl103
./.wsl104
./.wsl105
./.wsl106
./.wsl107
./.wsl108
./.wsl109
./.wsl110
./.wsl111
./.wsl112
./.wsl113
./.wsl114
./.wsl115
./.wsl116
./.wsl117
./.wsl118
./.wsl119
./.wsl120
./.wsl121
./.wsl122
./.wsl123
./.wsl124
./.wsl125
./.wsl126
./.wsl127
./.wsl128
./.wsl129
./.wsl130
./.wsl131
./.wsl132
./.wsl133
./.wsl134
./.wsl135
./.wsl136
./.wsl137
./.wsl138
./.wsl139
./.wsl140
./.wsl141
./.wsl142
./.wsl143
./.wsl144
./.wsl145
./.wsl146
./.wsl147
./.wsl148
./.wsl149
./.wsl150
./.wsl151
./.wsl152
./.wsl153
./.wsl154
./.wsl155
./.wsl156
./.wsl157
./.wsl158
./.wsl159
./.wsl160
./.wsl161
./.wsl162
./.wsl163
./.wsl164
./.wsl165
./.wsl166
./.wsl167
./.wsl168
./.wsl169
./.wsl170
./.wsl171
./.wsl172
./.wsl173
./.wsl174
./.wsl175
./.wsl176
./.wsl177
./.wsl178
./.wsl179
./.wsl180
./.wsl181
./.wsl182
./.wsl183
./.wsl184
./.wsl185
./.wsl186
./.wsl187
./.wsl188
./.wsl189
./.wsl190
./.wsl191
./.wsl192
./.wsl193
./.wsl194
./.wsl195
./.wsl196
./.wsl197
./.wsl198
./.wsl199
./.wsl200
./.wsl201
./.wsl202
./.wsl203
./.wsl204
./.wsl205
./.wsl206
./.wsl207
./.wsl208
./.wsl209
./.wsl210
./.wsl211
./.wsl212
./.wsl213
./.wsl214
./.wsl215
./.wsl216
./.wsl217
./.wsl218
./.wsl219
./.wsl220
./.wsl221
./.wsl222
./.wsl223
./.wsl224
./.wsl225
./.wsl226
./.wsl227
./.wsl228
./.wsl229
./.wsl230
./.wsl231
./.wsl232
./.wsl233
./.wsl234
./.wsl235
./.wsl236
./.wsl237
./.wsl238
./.wsl239
./.wsl240
./.wsl241
./.wsl242
./.wsl243
./.wsl244
./.wsl245
./.wsl246
./.wsl247
./.wsl248
./.wsl249
./.wsl250
./.wsl251
./.wsl252
./.wsl253
./.wsl254
./.wsl255
./.wsl256
./.wsl257
./.wsl258
./.wsl259
./.wsl260
./.wsl261
./.wsl262
./.wsl263
./.wsl264
./.wsl265
./.wsl266
./.wsl267
./.wsl268
./.wsl269
./.wsl270
./.wsl271
./.wsl272
./.wsl273
./.wsl274
./.wsl275
./.wsl276
./.wsl277
./.wsl278
./.wsl279
./.wsl280
./.wsl281
./.wsl282
./.wsl283
./.wsl284
./.wsl285
./.wsl286
./.wsl287
./.wsl288
./.wsl289
./.wsl290
./.wsl291
./.wsl292
./.wsl293
./.wsl294
./.wsl295
./.wsl296
./.wsl297
./.wsl298
./.wsl299
./.wsl300
./.wsl301
./.wsl302
./.wsl303
./.wsl304
./.wsl305
./.wsl306
./.wsl307
./.wsl308
./.wsl309
./.wsl310
./.wsl311
./.wsl312
./.wsl313
./.wsl314
./.wsl315
./.wsl316
./.wsl317
./.wsl318
./.wsl319
./.wsl320
./.wsl321
./.wsl322
./.wsl323
./.wsl324
./.wsl325
./.wsl326
./.wsl327
./.wsl328
./.wsl329
./.wsl330
./.wsl331
./.wsl332
./.wsl333
./.wsl334
./.wsl335
./.wsl336
./.wsl337
./.wsl338
./.wsl339
./.wsl340
./.wsl341
./.wsl342
./.wsl343
./.wsl344
./.wsl345
./.wsl346
./.wsl347
./.wsl348
./.wsl349
./.wsl350
./.wsl351
./.wsl352
./.wsl353
./.wsl354
./.wsl355
./.wsl356
./.wsl357
./.wsl358
./.wsl359
./.wsl360
./.wsl361
./.wsl362
./.wsl363
./.wsl364
./.wsl365
./.wsl366
./.wsl367
./.wsl368
./.wsl369
./.wsl370
./.wsl371
./.wsl372
./.wsl373
./.wsl374
./.wsl375
./.wsl376
./.wsl377
./.wsl378
./.wsl379
./.wsl380
./.wsl381
./.wsl382
./.wsl383
./.wsl384
./.wsl385
./.wsl386
./.wsl387
./.wsl388
./.wsl389
./.wsl390
./.wsl391
./.wsl392
./.wsl393
./.wsl394
./.wsl395
./.wsl396
./.wsl397
./.wsl398
./.wsl399
./.wsl400
./.wsl401
./.wsl402
./.wsl403
./.wsl404
./.wsl405
./.wsl406
./.wsl407
./.wsl408
./.wsl409
./.wsl410
./.wsl411
./.wsl412
./.wsl413
./.wsl414
./.wsl415
./.wsl416
./.wsl417
./.wsl418
./.wsl419
./.wsl420
./.wsl421
./.wsl422
./.wsl423
./.wsl424
./.wsl425
./.wsl426
./.wsl427
./.wsl428
./.wsl429
./.wsl430
./.wsl431
./.wsl432
./.wsl433
./.wsl434
./.wsl435
./.wsl436
./.wsl437
./.wsl438
./.wsl439
./.wsl440
./.wsl441
./.wsl442
./.wsl443
./.wsl444
./.wsl445
./.wsl446
./.wsl447
./.wsl448
./.wsl449
./.wsl450
./.wsl451
./.wsl452
./.wsl453
./.wsl454
./.wsl455
./.wsl456
./.wsl457
./.wsl458
./.wsl459
./.wsl460
./.wsl461
./.wsl462
./.wsl463
./.wsl464
./.wsl465
./.wsl466
./.wsl467
./.wsl468
./.wsl469
./.wsl470
./.wsl471
./.wsl472
./.wsl473
./.wsl474
./.wsl475
./.wsl476
./.wsl477
./.wsl478
./.wsl479
./.wsl480
./.wsl481
./.wsl482
./.wsl483
./.wsl484
./.wsl485
./.wsl486
./.wsl487
./.wsl488
./.wsl489
./.wsl490
./.wsl491
./.wsl492
./.wsl493
./.wsl494
./.wsl495
./.wsl496
./.wsl497
./.wsl498
./.wsl499
./.wsl500
./.wsl501
./.wsl502
./.wsl503
./.wsl504
./.wsl505
./.wsl506
./.wsl507
./.wsl508
./.wsl509
./.wsl510
./.wsl511
./.wsl512
./.wsl513
./.wsl514
./.wsl515
./.wsl516
./.wsl517
./.wsl518
./.wsl519
./.wsl520
./.wsl521
./.wsl522
./.wsl523
./.wsl524
./.wsl525
./.wsl526
./.wsl527
./.wsl528
./.wsl529
./.wsl530
./.wsl531
./.wsl532
./.wsl533
./.wsl534
./.wsl535
./.wsl536
./.wsl537
./.wsl538
./.wsl539
./.wsl540
./.wsl541
./.wsl542
./.wsl543
./.wsl544
./.wsl545
./.wsl546
./.wsl547
./.wsl548
./.wsl549
./.wsl550
./.wsl551
./.wsl552
./.wsl553
./.wsl554
./.wsl555
./.wsl556
./.wsl557
./.wsl558
./.wsl559
./.wsl560
./.wsl561
./.wsl562
./.wsl563
./.wsl564
./.wsl565
./.wsl566
./.wsl567
./.wsl568
./.wsl569
./.wsl570
./.wsl571
./.wsl572
./.wsl573
./.wsl574
./.wsl575
./.wsl576
./.wsl577
./.wsl578
./.wsl579
./.wsl580
./.wsl581
./.wsl582
./.wsl583
./.wsl584
./.wsl585
./.wsl586
./.wsl587
./.wsl588
./.wsl589
./.wsl590
./.wsl591
./.wsl592
./.wsl593
./.wsl594
./.wsl595
./.wsl596
./.wsl597
./.wsl598
./.wsl599
./.wsl600
./.wsl601
./.wsl602
./.wsl603
./.wsl604
./.wsl605
./.wsl606
./.wsl607
./.wsl608
./.wsl609
./.wsl610
./.wsl611
./.wsl612
./.wsl613
./.wsl614
./.wsl615
./.wsl616
./.wsl617
./.wsl618
./.wsl619
./.wsl620
./.wsl621
./.wsl622
./.wsl623
./.wsl624
./.wsl625
./.wsl626
./.wsl627
./.wsl628
./.wsl629
./.wsl630
./.wsl631
./.wsl632
./.wsl633
./.wsl634
./.wsl635
./.wsl636
./.wsl637
./.wsl638
./.wsl639
./.wsl640
./.wsl641
./.wsl642
./.wsl643
./.wsl644
./.wsl645
./.wsl646
./.wsl647
./.wsl648
./.wsl649
./.wsl650
./.wsl651
./.wsl652
./.wsl653
./.wsl654
./.wsl655
./.wsl656
./.wsl657
./.wsl658
./.wsl659
./.wsl660
./.wsl661
./.wsl662
./.wsl663
./.wsl664
./.wsl665
./.wsl666
./.wsl667
./.wsl668
./.wsl669
./.wsl670
./.wsl671
./.wsl672
./.wsl673
./.wsl674
./.wsl675
./.wsl676
./.wsl677
./.wsl678
./.wsl679
./.wsl680
./.wsl681
./.wsl682
./.wsl683
./.wsl684
./.wsl685
./.wsl686
./.wsl687
./.wsl688
./.wsl689
./.wsl690
./.wsl691
./.wsl692
./.wsl693
./.wsl694
./.wsl695
./.wsl696
./.wsl697
./.wsl698
./.wsl699
./.wsl700
./.wsl701
./.wsl702
./.wsl703
./.wsl704
./.wsl705
./.wsl706
./.wsl707
./.wsl708
./.wsl709
./.wsl710
./.wsl711
./.wsl712
./.wsl713
./.wsl714
./.wsl715
./.wsl716
./.wsl717
./.wsl718
./.wsl719
./.wsl720
./.wsl721
./.wsl722
./.wsl723
./.wsl724
./.wsl725
./.wsl726
./.wsl727
./.wsl728
./.wsl729
./.wsl730
./.wsl731
./.wsl732
./.wsl733
./.wsl734
./.wsl735
./.wsl736
./.wsl737
./.wsl738
./.wsl739
./.wsl740
./.wsl741
./.wsl742
./.wsl743
./.wsl744
./.wsl745
./.wsl746
./.wsl747
./.wsl748
./.wsl749
./.wsl750
./.wsl751
./.wsl752
./.wsl753
./.wsl754
./.wsl755
./.wsl756
./.wsl757
./.wsl758
./.wsl759
./.wsl760
./.wsl761
./.wsl762
./.wsl763
./.wsl764
./.wsl765
./.wsl766
./.wsl767
./.wsl768
./.wsl769
./.wsl770
./.wsl771
./.wsl772
./.wsl773
./.wsl774
./.wsl775
./.wsl776
./.wsl777
./.wsl778
./.wsl779
./.wsl780
./.wsl781
./.wsl782
./.wsl783
./.wsl784
./.wsl785
./.wsl786
./.wsl787
./.wsl788
./.wsl789
./.wsl790
./.wsl791
./.wsl792
./.wsl793
./.wsl794
./.wsl795
./.wsl796
./.wsl797
./.wsl798
./.wsl799
./.wsl800
./.wsl801
./.wsl802
./.wsl803
./.wsl804
./.wsl805
./.wsl806
./.wsl807
./.wsl808
./.wsl809
./.wsl810
./.wsl811
./.wsl812
./.wsl813
./.wsl814
./.wsl815
./.wsl816
./.wsl817
./.wsl818
./.wsl819
./.wsl820
./.wsl821
./.wsl822
./.wsl823
./.wsl824
./.wsl825
./.wsl826
./.wsl827
./.wsl828
./.wsl829
./.wsl830
./.wsl831
./.wsl832
./.wsl833
./.wsl834
./.wsl835
./.wsl836
./.wsl837
./.wsl838
./.wsl839
./.wsl840
./.wsl841
./.wsl842
./.wsl843
./.wsl844
./.wsl845
./.wsl846
./.wsl847
./.wsl848
./.wsl849
./.wsl850
./.wsl851
./.wsl852
./.wsl853
./.wsl854
./.wsl855
./.wsl856
./.wsl857
./.wsl858
./.wsl859
./.wsl860
./.wsl861
./.wsl862
./.wsl863
./.wsl864
./.wsl865
./.wsl866
./.wsl867
./.wsl868
./.wsl869
./.wsl870
./.wsl871
./.wsl872
./.wsl873
./.wsl874
./.wsl875
./.wsl876
./.wsl877
./.wsl878
./.wsl879
./.wsl880
./.wsl881
./.wsl882
./.wsl883
./.wsl884
./.wsl885
./.wsl886
./.wsl887
./.wsl888
./.wsl889
./.wsl890
./.wsl891
./.wsl892
./.wsl893
./.wsl894
./.wsl895
./.wsl896
./.wsl897
./.wsl898
./.wsl899
./.wsl900
./.wsl901
./.wsl902
./.wsl903
./.wsl904
./.wsl905
./.wsl906
./.wsl907
./.wsl908
./.wsl909
./.wsl910
./.wsl911
./.wsl912
./.wsl913
./.wsl914
./.wsl915
./.wsl916
./.wsl917
./.wsl918
./.wsl919
./.wsl920
./.wsl921
./.wsl922
./.wsl923
./.wsl924
./.wsl925
./.wsl926
./.wsl927
./.wsl928
./.wsl929
./.wsl930
./.wsl931
./.wsl932
./.wsl933
./.wsl934
./.wsl935
./.wsl936
./.wsl937
./.wsl938
./.wsl939
./.wsl940
./.wsl941
./.wsl942
./.wsl943
./.wsl944
./.wsl945
./.wsl946
./.wsl947
./.wsl948
./.wsl949
./.wsl950
./.wsl951
./.wsl952
./.wsl953
./.wsl954
./.wsl955
./.wsl956
./.wsl957
./.wsl958
./.wsl959
./.wsl960
./.wsl961
./.wsl962
./.wsl963
./.wsl964
./.wsl965
./.wsl966
./.wsl967
./.wsl968
./.wsl969
./.wsl970
./.wsl971
./.wsl972
./.wsl973
./.wsl974
./.wsl975
./.wsl976
./.wsl977
./.wsl978
./.wsl979
./.wsl980
./.wsl981
./.wsl982
./.wsl983
./.wsl984
./.wsl985
./.wsl986
./.wsl987
./.wsl988
./.wsl989
./.wsl990
./.wsl991
./.wsl992
./.wsl993
./.wsl994
./.wsl995
./.wsl996
./.wsl997
./.wsl998
./.wsl999
./.wsl1000
./.wsl1001
./.wsl1002
./.wsl1003
./.wsl1004
./.wsl1005
./.wsl1006
./.wsl1007
./.wsl1008
./.wsl1009
./.wsl1010
./.wsl1011
./.wsl1012
./.wsl1013
./.wsl1014
./.wsl1015
./.wsl1016
./.wsl1017
./.wsl1018
./.wsl1019
./.wsl1020
./.wsl1021
./.wsl1022
./.wsl1023
./.wsl1024
./.wsl1025
./.wsl1026
./.wsl1027
./.wsl1028
./.wsl1029
./.wsl1030
./.wsl1031
./.wsl1032
./.wsl1033
./.wsl1034
./.wsl1035
./.wsl1036
./.wsl1037
./.wsl1038
./.wsl1039
./.wsl1040
./.wsl1041
./.wsl1042
./.wsl1043
./.wsl1044
./.wsl1045
./.wsl1046
./.wsl1047
./.wsl1048
./.wsl1049
./.wsl1050
./.wsl1051
./.wsl1052
./.wsl1053
./.wsl1054
./.wsl1055
./.wsl1056
./.wsl1057
./.wsl1058
./.wsl1059
./.wsl1060
./.wsl1061
./.wsl1062
./.wsl1063
./.wsl1064
./.wsl1065
./.wsl1066
./.wsl1067
./.wsl1068
./.wsl1069
./.wsl1070
./.wsl1071
./.wsl1072
./.wsl1073
./.wsl1074
./.wsl1075
./.wsl1076
./.wsl1077
./.wsl1078
./.wsl1079
./.wsl1080
./.wsl1081
./.wsl1082
./.wsl1083
./.wsl1084
./.wsl1085
./.wsl1086
./.wsl1087
./.wsl1088
./.wsl1089
./.wsl1090
./.wsl1091
./.wsl1092
./.wsl1093
./.wsl1094
./.wsl1095
./.wsl1096
./.wsl1097
./.wsl1098
./.wsl1099
./.wsl1100
./.wsl1101
./.wsl1102
./.wsl1103
./.wsl1104
./.wsl1105
./.wsl1106
./.wsl1107
./.wsl1108
./.wsl1109
./.wsl1110
./.wsl1111
./.wsl1112
./.wsl1113
./.wsl1114
./.wsl1115
./.wsl1116
./.wsl1117
./.wsl1118
./.wsl1119
./.wsl1120
./.wsl1121
./.wsl1122
./.wsl1123
./.wsl1124
./.wsl1125
./.wsl1126
./.wsl1127
./.wsl1128
./.wsl1129
./.wsl1130
./.wsl1131
./.wsl1132
./.wsl1133
./.wsl1134
./.wsl1135
./.wsl1136
./.wsl1137
./.wsl1138
./.wsl1139
./.wsl1140
./.wsl1141
./.wsl1142
./.wsl1143
./.wsl1144
./.wsl1145
./.wsl1146
./.wsl1147
./.wsl1148
./.wsl1149
./.wsl1150
./.wsl1151
./.wsl1152
./.wsl1153
./.wsl1154
./.wsl1155
./.wsl1156
./.wsl1157
./.wsl1158
./.wsl1159
./.wsl1160
./.wsl1161
./.wsl1162
./.wsl1163
./.wsl1164
./.wsl1165
./.wsl1166
./.wsl1167
./.wsl1168
./.wsl1169
./.wsl1170
./.wsl1171
./.wsl1172
./.wsl1173
./.wsl1174
./.wsl1175
./.wsl1176
./.wsl1177
./.wsl1178
./.wsl1179
./.wsl1180
./.wsl1181
./.wsl1182
./.wsl1183
./.wsl1184
./.wsl1185
./.wsl1186
./.wsl1187
./.wsl1188
./.wsl1189
./.wsl1190
./.wsl1191
./.wsl1192
./.wsl1193
./.wsl1194
./.wsl1195
./.wsl1196
./.wsl1197
./.wsl1198
./.wsl1199
./.wsl1200
./.wsl1201
./.wsl1202
./.wsl1203
./.wsl1204
./.wsl1205
./.wsl1206
./.wsl1207
./.wsl1208
./.wsl1209
./.wsl1210
./.wsl1211
./.wsl1212
./.wsl1213
./.wsl1214
./.wsl1215
./.wsl1216
./.wsl1217
./.wsl1218
./.wsl1219
./.wsl1220
./.wsl1221
./.wsl1222
./.wsl1223
./.wsl1224
./.wsl1225
./.wsl1226
./.wsl1227
./.wsl1228
./.wsl1229
./.wsl1230
./.wsl1231
./.wsl1232
./.wsl1233
./.wsl1234
./.wsl1235
./.wsl1236
./.wsl1237
./.wsl1238
./.wsl1239
./.wsl1240
./.wsl1241
./.wsl1242
./.wsl1243
./.wsl1244
./.wsl1245
./.wsl1246
./.wsl1247
./.wsl1248
./.wsl1249
./.wsl1250
./.wsl1251
./.wsl1252
./.wsl1253
./.wsl1254
./.wsl1255
./.wsl1256
./.wsl1257
./.wsl1258
./.wsl1259
./.wsl1260
./.wsl1261
./.wsl1262
./.wsl1263
./.wsl1264
./.wsl1265
./.wsl1266
./.wsl1267
./.wsl1268
./.wsl1269
./.wsl1270
./.wsl1271
./.wsl1272
./.wsl1273
./.wsl1274
./.wsl1275
./.wsl1276
./.wsl1277
./.wsl1278
./.wsl1279
./.wsl1280
./.wsl1281
./.wsl1282
./.wsl1283
./.wsl1284
./.wsl1285
./.wsl1286
./.wsl1287
./.wsl1288
./.wsl1289
./.wsl1290
./.wsl1291
./.wsl1292
./.wsl1293
./.wsl1294
./.wsl1295
./.wsl1296
./.wsl1297
./.wsl1298
./.wsl1299
./.wsl1300
./.wsl1301
./.wsl1302
./.wsl1303
./.wsl1304
./.wsl1305
./.wsl1306
./.wsl1307
./.wsl1308
./.wsl1309
./.wsl1310
./.wsl1311
./.wsl1312
./.wsl1313
./.wsl1314
./.wsl1315
./.wsl1316
./.wsl1317
./.wsl1318
./.wsl1319
./.wsl1320
./.wsl1321
./.wsl1322
./.wsl1323
./.wsl1324
./.wsl1325
./.wsl1326
./.wsl1327
./.wsl1328
./.wsl1329
./.wsl1330
./.wsl1331
./.wsl1332
./.wsl1333
./.wsl1334
./.wsl1335
./.wsl1336
./.wsl1337
./.wsl1338
./.wsl1339
./.wsl1340
./.wsl1341
./.wsl1342
./.wsl1343
./.wsl1344
./.wsl1345
./.wsl1346
./.wsl1347
./.wsl1348
./.wsl1349
./.wsl1350
./.wsl1351
./.wsl1352
./.wsl1353
./.wsl1354
./.wsl1355
./.wsl1356
./.wsl1357
./.wsl1358
./.wsl1359
./.wsl1360
./.wsl1361
./.wsl1362
./.wsl1363
./.wsl1364
./.wsl1365
./.wsl1366
./.wsl1367
./.wsl1368
./.wsl1369
./.wsl1370
./.wsl1371
./.wsl1372
./.wsl1373
./.wsl1374
./.wsl1375
./.wsl1376
./.wsl1377
./.wsl1378
./.wsl1379
./.wsl1380
./.wsl1381
./.wsl1382
./.wsl1383
./.wsl1384
./.wsl1385
./.wsl1386
./.wsl1387
./.
```

## List only files in Unix

Type the following command:

```
$ ls -l | egrep -v '^d'
$ ls -l | egrep -v '^d'
```

The grep command is used to searches input. It will filter out directory names by matching first character 'd'. To reverse the effect i.e. just to display files you need to pass the -v option. It inverts the sense of matching, to select non-matching lines.

## Task: Create aliases to save time

You can create two aliases to list only directories and files.

```
alias lf="ls -l | egrep -v '^d'"
alias ldir="ls -l | egrep '^d'"
```

Put above two aliases in your bash shell startup file:

```
$ cd
$ vi .bash_profile
```

Append two lines:

```
alias lf="ls -l | egrep -v '^d'"
alias ldir="ls -l | egrep '^d'"
```

Save and close the file.

Now just type lf - to list files and ldir - to list directories only:

```
$ cd /etc
$ lf
```

Sample output:

```
-rw-r--r-- 1 root root 2149 2006-09-04 23:25 adduser.conf
-rw-r--r-- 1 root root 44 2006-09-29 05:11 adjtime
-rw-r--r-- 1 root root 197 2006-09-04 23:48 aliases
-rw--- 1 root root 144 2002-01-18 13:43 at.deny
-rw-r--r-- 1 root root 162 2006-09-22 23:24 aumixrc
-rw-r--r-- 1 root root 28 2006-09-22 23:24 aumixrc1
```

List directory names only:

```
$ cd /etc
$ ldir
```

Sample output:

```
drwxr-xr-x 4 root root 4096 2006-09-22 16:41 alsa
drwxr-xr-x 2 root root 4096 2006-09-20 20:59 alternatives
drwxr-xr-x 6 root root 4096 2006-09-22 16:41 apm
drwxr-xr-x 3 root root 4096 2006-09-07 02:51 apt
drwxr-xr-x 2 root root 4096 2006-09-08 01:46 bash_completion.d
```

**Use find command to list either files or directories**

The find command can be used as follows: to list all directories in /nas, enter:

```
find /nas -type d
```

```
find /nas -type d -ls
```

```
find . -type d -ls
```

Sample output:

|         |   |            |    |      |      |      |     |    |       |              |
|---------|---|------------|----|------|------|------|-----|----|-------|--------------|
| 1070785 | 8 | drwxrwxrwt | 8  | root | root | 4096 | Jul | 5  | 07:12 | .            |
| 1070797 | 8 | drwx--     | 2  | root | root | 4096 | Jul | 4  | 07:22 | ./orbit-root |
| 1070843 | 8 | drwxr-xr-x | 2  | root | root | 4096 | Jun | 16 | 18:55 | ./w          |
| 1070789 | 8 | drwxr-xr-x | 10 | root | root | 4096 | Jun | 17 | 14:54 | ./b          |
| 1071340 | 8 | drwxr-xr-x | 2  | root | root | 4096 | Jun | 16 | 18:55 | ./b/init.d   |
| 1071581 | 8 | drwxr-xr-x | 3  | root | root | 4096 | Jun | 16 | 18:55 | ./b/bind     |
| 1071584 | 8 | drwxr-xr-x | 2  | root | root | 4096 | Jun | 16 | 18:55 | ./b/bind/bak |
| 1071617 | 8 | drwxr-xr-x | 2  | root | root | 4096 | Jun | 16 | 18:55 | ./b/fw       |
| 1071628 | 8 | drwxr-xr-x | 8  | root | root | 4096 | Jun | 16 | 18:55 | ./b/scripts  |

# Chapter Eight: WORKING WITH FILES

This chapter will first describe general characteristics of Unix commands. It will then discuss commands, which are commonly used to create and manipulate files. A summary of some of the most commonly used UNIX commands is presented in Command Comparisons.

## UNIX File Names

It is important to understand the rules for creating UNIX files: UNIX is case sensitive! For example, “fileName” is different from “filename.” It is recommended that you limit names to the alphabetic characters, numbers, underscore (`_`), and dot (`.`). Dots (`.`) used in UNIX filenames are simply characters and not delimiters between filename components; you may include more than one dot in a filename. Including a dot as the first character of a filename makes the file invisible (hidden) to the normal `ls` command; use the `-a` flag of the `ls` command to display hidden files. Although many systems will allow more, the recommended length is 14 characters per file name. Unix shells typically include several important wildcard characters. The asterisk (`*`) is used to match 0 or more character (e.g., `abc*` will match any file beginning with the letters `abc`), the question mark (`?`) is used to match any single character, and the left (`[`) and right (`]`) square brackets are used to enclose a string of characters, any one of which is to match. Execute the following commands and observe the results:

```
ls m*
ls *.f
ls *.?
ls [a-d]*
```

Notes for PC users: Unix uses forward slashes ( `/` ) instead of backslashes ( `\` ) for directories

## Looking at the Contents of Files

You can examine the contents of files using a variety of commands. `cat`, `more`, `pg`, `head`, and `tail` are described here. Of course, you can always use an editor; to use `vi` in “read-only” mode to examine the contents of the file “argtest”, enter:

```
vi -R argtest
```

You can now use the standard `vi` commands to move through the file; however, you will not be able to make any changes to the contents of the file. This option is useful when you simply want to look at a file and want to guarantee that you make no changes while doing so.

Use the `vi ""` command to exit from the file.

## Cat Command

`cat` is a utility used to conCATenate files. Thus, it can be used to join files together, but it is perhaps more commonly used to display the contents of a file on the screen.

Observe the output produced by each of the following commands:

```
cd; cd xmp
cat cars
cat -vet cars
cat -n cars
```

The semicolon (;) in the first line of this example is a command separator which enables entry of more than one command on a line. When the <Return> key is pressed following this line, the command `cd` is issued which changes to your home directory. Then the command “`cd xmp`” is issued to change into the subdirectory “xmp.” Entering this line is equivalent to having entered these commands sequentially on separate lines. These two commands are included in the example to guarantee that you are in the subdirectory containing “cars” and the other example files. You need not enter these commands if you are already in the “xmp” directory created when you copied the example file.

The “`-vet`” option enables display of tab, end-of-line, and other non-printable characters within a file; the “`-n`” option numbers each line as it is displayed.

You can also use the `cat` command to join files together:

```
cat page1
cat page2
cat page1 page2 > document
cat document
```

**Note:** If the file “document” had previously existed, it will be replaced by the contents of files “page1” and “page2.”

**Cautions to using the cat command:** The cat command should only be used with “text” files; it should not be used to display the contents of binary (e.g., compiled C or FORTRAN programs). Unpredictable results may occur, including the termination of your logon session. Use the command “*file \**” to display the characteristics of files within a directory prior to using the cat command with any unknown file. You can use the `od`(enter “`man od`” for details on use of Octal Dump) command to display the contents of non-text files. For example, to display the contents of “a.out” in both hexadecimal and character representation, enter:

```
od -xc a.out
```

**Warning:** `cat` (and other Unix commands) can destroy files if not used correctly. For example, as illustrated in the Sobell book, the `cat` (also `cp` and `mv`) command can overwrite and thus destroy files. Observe the results of the following command:

```
cat letter page1 > letter
```

Typically, UNIX does not return a message when a command executes successfully. Here the UNIX operating system will attempt to complete the requested command by first initializing the file “letter” and then writing the current contents of “letter” (now nothing) and “page1” into this file. Since “letter” has been reinitialized and is also named as a source file, an error diagnostic is generated. Part of the UNIX philosophy is “No news is good news.” Thus, the appearance of a message is a warning that the command was not completed successfully.

Now use the “`cat`” command to individually examine the contents of the files “letter” and “page1.” Observe that the file “letter” does not contain the original contents of the files “letter” and “page1” as was intended.

Use the following command to restore the original file “letter”:

```
cp ~aixstu00/xmp/letter.
```

## More Command

You may type or browse files using the more command. The “`more`” command is useful when examining a large file as it displays the file contents one page at a time, allowing each page to be examined at will. As with the `man` command, you must press the space bar to proceed to the next screen of the file. On many systems, pressing the <b> key will enable you to page backwards in the file. To terminate more at any time, press <q>.

To examine a file with the more command, simply enter:

```
more file_name
```

The `man` command uses the `more` command to display the manual pages; thus, the commands you are familiar with using `man` will also work with `more`.

Not all Unix systems include the `more` command; some implement the `pg` command instead. VTAIX includes both the `more` and `pg` commands. When using the `pg` command, press <Return> to page down through a file instead of using the space bar.

Observe the results of entering the following commands:

```
more argtest
```

```
pg argtest
```

## Head Command

The `head` command is used to display the first few lines of a file. This command can be useful when you wish to look for specific information, in the beginning of the file. For example, enter:

```
head argtest
```

## Tail Command

The `tail` command is used to display the last lines of a file. This command can be useful to monitor the status of a program, which appends output to the end of a file. For example, enter:

```
tail argtest
```

## Copying, Erasing, Renaming

**Warning:** The typical Unix operating system provides no ‘unerase’ or ‘undelete’ command. If you mistakenly delete a file you are dependent upon the backups you or the system administrator have maintained in order to recover the file. You need to be careful when using commands like `copy` and `move`, which may result in overwriting existing files. If you are using the C or Korn Shell, you can create a command [alias](#) , which will prompt you for verification before overwriting files with these commands.

## Copying Files

The `cp` command is used to copy a file or group of files. You have already seen an example application of the `cp` command when you copied the sample files to your `userid`. Now let’s make a copy of one of these files. Recall that you can obtain a

listing of the files in the current directory using the `ls` command. Observe the results from the following commands:

```
ls l*

cp letter letter.2

ls l*
```

**Note:** Unlike many other operating systems, such as PC/DOS, you must specify the target with the copy command; it does not assume the current directory if no “copy-to” target is specified.

## Erasing Files

Unix uses the command `rm` (ReMove) to delete unwanted files. To remove the file “letter.2” which we have just created, enter:

```
rm letter.2
```

Enter the command “`ls l*`” to display a list of all files beginning with the letter “l.” Note that `letter.2` is no longer present in the current directory. The remove command can be used with wildcards in filenames; however, this can be dangerous as you might end up erasing files you had wanted to keep. It is recommended that you use the “`-i`” (interactive) option of `rm` for wildcard deletes — you will then be prompted to respond with a “y” or “Y” for each file you wish to delete.

## Renaming a File

The typical Unix operating system utilities do not include a rename command; however, we can use the `mv` (MoVe) command (see for additional uses of this command) to “move” Working with Directories) a file from one name to another. Observe the results of the following commands:

```
ls [d,l]*

mv letter document

ls [d,l]*

mv document letter

ls [d,l]*
```

**Note:** The first `mv` command overwrites the file “document” which you had created in an earlier exercise by concatenating “page1” and “page2.” No warning is



issued when the `mv` command is used to move a file into the name of an existing file. If you would like to be prompted for confirmation if the `mv` command were to overwrite an existing file, use the “`-i`” (interactive) option of the `mv` command, e.g.:

```
mv -i page1 letter
```

You will now be told that the file “letter” already exists and you will be asked if you wish to proceed with the `mv` command. Answer anything but “`y`” or “`Y`” and the file “letter” will not be overwritten.

## Using the Command Line

The command interpreter (shell) provides the mechanism by which input commands are interpreted and passed to the Unix kernel or other programs for processing. Observe the results of entering the following commands:

```
./filesize
```

```
./hobbit
```

```
./add2
```

```
ls -F
```

Observe that “filesize” is an executable shell script, which displays the size of files. Also note that “./hobbit” and “./add2” generate error diagnostics as there is no command or file with the name “hobbit” and the file “add2” lacks execute permission.

## Standard Input and Standard Output

As you have can see, Unix expects standard input to come from the keyboard, e.g., enter:

```
cat
```

```
my_text
```

```
<Ctrl-D>
```

Standard output is typically displayed on the terminal screen, e.g., enter:

```
cat cars
```

Standard error (a listing of program execution error diagnostics) is typically displayed on the terminal screen, e.g., enter:

```
ls xyzpqrz
```

## Redirection

As illustrated above, many Unix commands read from standard input (typically the keyboard) and write to standard output (typically the terminal screen). The redirection operators enable you to read input from a file (<) or write program output to a file (>). When output is redirected to a file, the program output replaces the original contents of the file if it already exists; to add program output to the end of an existing file, use the append redirection operator (>>).

Observe the results of the following command:

```
./a.out
```

You will be prompted to enter a Fahrenheit temperature. After entering a numeric value, a message will be displayed on the screen informing you of the equivalent Centigrade temperature. In this example, you entered a numeric value as standard input via the keyboard and the output of the program was displayed on the terminal screen.

In the next example, you will read data from a file and have the result displayed on the screen (standard output):

```
cat data.in
./a.out < data.in
```

Now you will read from standard input (keyboard) and write to a file:

```
./a.out > data.two
35
cat data.two
```

Now read from standard input and append the result to the existing file:

```
./a.out < data.in >> data.two
```

As another example of redirection, observe the result of the following two commands:

```
ls -la /etc > temp
more temp
```

Here we have redirected the output of the `ls` command to the file “temp” and then used the `more` command to display the contents of this file a page at a time. In the next section, we will see how the use of pipes could simplify this operation.

## Using Pipes and Filters

A filter is a Unix program, which accepts input from standard input and places its output in standard output. Filters add power to the UNIX system as programs can be written to use the output of another program as input and create output, which can be used by yet another program. A pipe (indicated by the symbol “|” — vertical bar) is used between UNIX commands to indicate that the output from the first is to be used as input by the second. Compare the output from the following two commands:

```
ls -la /etc
```

```
ls -la /etc | more
```

The first command displays all the files in the “/etc” directory in long format. It is difficult to make use of this information since it scrolls rapidly across the screen. In the second line, the results of the `ls` command are piped into the `more` command. We can now examine this information one screen at a time and can even back up to a prior screen of information if we wish to. As you became more familiar with UNIX, you will find that piping output to the `more` command can be very useful in a variety of applications.

The `sort` command can be used to sort the lines in a file in a desired order. Now enter the following commands and observe the results:

```
who
```

```
sort cars
```

```
who | sort
```

The `who` command displays a list of users currently logged onto the system the `sort` command enables us to sort the information. The second command sorts the lines in the file `cars` alphabetically by first field and displays the result in standard output. The third command illustrates how the result of the `who` command can be passed to the `sort` command prior to being displayed. The result is a listing of logged on users in alphabetical order.

The following example uses the “awk” and “sort” commands to select and reorganize the output generated by the “ls” command:

```
ls -l | awk '/:/{print $5,$9}' | sort -nr
```

**Note:** Curly braces do not necessarily display correctly on all output devices. In the above example, there should be a left curly brace in front of the word `print` and a right curly brace following the number `9`.

Observe that the output displays the filesize and filename in decreasing order of size. Here the `ls` command first generates a “long” listing of the files in the current directory, which is piped to the “`awk`” utility, whose output is then piped to the “`sort`” command.

“`awk`” is a powerful utility which processes one or more program lines to find patterns within a file and perform selective actions based on what is found. Slash (/) characters are used as delimiters around the pattern to be matched and the action to be taken is enclosed in curly braces. If no pattern is specified, all lines in the file are processed and if no action is specified, all lines matching the specified pattern are output. Since a colon (:) is used here, all lines containing file information (the time column corresponding to each file contains a colon) are selected and the information contained in the 5th and 9th columns are output to the `sort` command.

**Note:** If the `ls` command on your system does not include a column listing group membership, use `{print $4,$8}` instead of the “`print`” command option of `awk` listed above.

Here the “`sort`” command options “`-nr`” specify that the output from “`awk`” be sorted in reverse numeric order, i.e., from largest to smallest.

The preceding command is somewhat complex and it is easy to make a mistake in entering it. If this were a command you would use frequently, we could include it in a [shell script](#) as in sample file “`filesize`”. To use this shell script, simply enter the command:

```
./filesize
or
sh filesize
```

If you examine the contents of this file with the `cat` or `vi` commands, you will see that it contains nothing more the piping of the `ls` command to `awk` and then piping the output to `sort`.

The `tee` utility is used to send output to both a file and the screen:

```
who | tee who.out | sort

cat who.out
```

Here you should have observed that a list of logged on users was displayed on the screen in alphabetical order and that the file “`who.out`” contained an unsorted

listing of the same userids.

## Some Additional File Handling Commands

### Word Count

The command `wc` displays the number of lines, words, and characters in a file.

To display the number of lines, words, and characters in the file `file_name`, enter:

```
wc file_name
```

### Comparing the Contents of Two Files: the `cmp` and `diff` Commands

The `cmp` and `diff` commands are used to compare files; the “`comp`” command is not used to compare files, but to “compose a message.”

The `cmp` command can be used for both binary and text files. It indicates the location (byte and line) where the first difference between the two files appears.

The `diff` command can be used to compare text files and its output shows the lines which are different in the two files: a less than sign (“<”) appears in front of lines from the first file which differ from those in the second file, a greater than symbol (“>”) precedes lines from the second file. Matching lines are not displayed.

Observe the results of the following commands:

```
cmp page1 page2
```

```
diff page1 page2
```

Lines 1 and 2 of these two files are identical, lines 3 differ by one character, and page one contains a blank line following line three, while page2 does not.

# Chapter Nine: NAVIGATION AND FILE MANAGEMENT

If you do not have much experience working with Linux systems, you may be overwhelmed by the prospect of controlling an operating system from the command line. In this book, we will attempt to get you up to speed with the basics.

## Prerequisites and Goals

In order to follow along with this book, you will need to have access to a Linux server. You will also want to have a basic understanding of how the terminal works and what Linux commands look like. This book covers terminal basics, so you should check it out if you are new to using terminals. All of the material in this book can be accomplished with a regular, non-root (non-administrative) user account. You can learn how to configure this type of user account by following your distribution's initial server setup guide ([Ubuntu 14.04](#), [CentOS 7](#)). When you are ready to begin, connect to your Linux server using SSH and continue below.

## Navigation and Exploration

The most fundamental skills you need to master are navigating the filesystem. We will discuss the tools that allow you to do this in this section.

### Finding where you are with the “pwd” command

When you log onto your server, you are typically dropped into your user accounts **home directory**. A home directory is the directory set aside for your account to store files and create directories. It is the location in the filesystem where you have full dominion. To find out where your home directory is in relationship to the rest of the filesystem, you can use the `pwd` command. This command displays the directory that we are currently in:

```
pwd
```

You should get back some information that looks like this:

```
/home/demo
```

The home directory is named after the user account, so the above example is what the value would be if you were logged into the server with an account called `demo`. This directory is within a directory called `/home`, which is itself within the top-level directory, which is called “root” represented by a single slash `/`.

## Looking at the Contents of Directories with “ls”

Now that you know how to display the directory that you are in, we can show you how to look at the contents of a directory.

Currently, your home directory does contain much, so we will go to another, more populated directory to explore. Type the following command in your terminal to change directory (we will explain the details of moving directories in the next section). Afterward, we will use `pwd` to confirm that we successfully moved:

```
cd /usr/share
pwd
/usr/share
```

Now that we are in a new directory, let us look at what’s inside. To do this, we can use the `ls` command:

```
ls
adduser groff pam-configs
applications grub perl
apport grub-gfxpayload-lists perl5
apps hal pixmaps
apt il8n pkgconfig
aptitude icons polkit-1
apt-xapian-index info popularity-contest
...
```

As you can see, there are *many* items in this directory. We can add some optional flags to the command to modify the default behavior. For instance, to list all of the contents in an extended form, we can use the `-l` flag (for “long” output):

```
ls -l
total 440
drwxr-xr-x 2 root root 4096 Apr 17 2014 adduser
drwxr-xr-x 2 root root 4096 Sep 24 19:11 applications
drwxr-xr-x 6 root root 4096 Oct 9 18:16 apport
drwxr-xr-x 3 root root 4096 Apr 17 2014 apps
drwxr-xr-x 2 root root 4096 Oct 9 18:15 apt
drwxr-xr-x 2 root root 4096 Apr 17 2014 aptitude
drwxr-xr-x 4 root root 4096 Apr 17 2014 apt-xapian-index
drwxr-xr-x 2 root root 4096 Apr 17 2014 awk
...
```

This view gives us plenty of information, most of which looks rather unusual. The first block describes the file type (if the first column is a “d” the item is a directory, if it is a “-“, it is a normal file) and permissions. Each subsequent column, separated by white space, describes the number of hard links, the owner, group owner, item size, last modification time, and the name of the item. We will describe some of these at another time, but for now, just know that you can view this information with the `-l` flag of `ls`.

To get a listing of all files, including *hidden* files and directories, you can add the `-a` flag. Since there are no real hidden files in the `/usr/share` directory, let’s go back to our home directory and try that command. You can get back to the home directory by typing `cd` with no arguments:

```
cd
ls -a
. .. .bash_logout .bashrc .profile
```

As you can see, there are three hidden files in this demonstration, along with `.` and `..`, which are special indicators. You will find that often, configuration files are stored as hidden files.

The dot and double dot entries, are built-in methods of referring to related directories. The single dot indicates the current directory, and the double dot indicates this directory’s parent directory. This will come in handy in the next section.

## Moving Around the Filesystem with “cd”

We have already changed directories twice to demonstrate some properties of `ls`. Let’s take a better look at the command here.

Begin by going back to the `/usr/share` directory by typing this:

```
cd /usr/share
```

This is an example of changing a directory by giving an absolute path. In Linux, every file and directory is under the top-most directory, which is called the “root” directory, but referred to by a single leading slash “/”. An absolute path indicates the location of a directory in relation to this top-level directory. This lets us refer to directories in an unambiguous way from any place in the filesystem. Every absolute path must begin with a slash.

The alternative is to use relative paths. Relative paths refer to directories in relation to the *current* directory. For directories close to the current directory in the hierarchy, this is usually easier and shorter. Any directory within the current



directory can be referenced by name without a leading slash. We can change to the `locale` directory within `/usr/share` from our current location by typing:

```
cd locale
```

We can likewise change multiple directory levels with relative paths by providing the portion of the path that comes after the current directory's path. From here, we can get to the `LC_MESSAGES` directory within the `en` directory by typing:

```
cd en/LC_MESSAGES
```

To move up one directory level, we use the special double dot indicator we talked about earlier. For instance, we are now in the `/usr/share/locale/en/LC_MESSAGES` directory. To move up one level, we can type:

```
cd ..
```

This takes us to the `/usr/share/locale/en` directory.

A shortcut that you saw earlier that will always take you back to your home directory is to use `cd` without providing a directory:

```
cd
```

```
pwd
```

```
/home/demo
```

## Viewing Files

In the last section, we learned how to navigate the filesystem. In this section, we will discuss different ways to view files. In contrast to some operating systems, Linux and other Unix-like operating systems rely on plain text files for vast portions of the system. The main way that we will view files is with the `less` command. This is what we call a “pager,” because it allows us to scroll through pages of a file. While the previous commands immediately executed and returned you to the command line, `less` is an application that will continue to run and occupy the screen until you exit.

We will open the `/etc/services` file, which is a configuration file that contains the systems services information:

```
less /etc/services
```

The file will be opened in `less`, allowing you to see the portion of the document that fits in the terminal window:

```
Network services, Internet style
```

```
#
```

```
Note that it is presently the policy of IANA to assign a single well-known
```

```
port number for both TCP and UDP; hence, officially ports have two entries
even if the protocol doesn't support UDP operations.
#
Updated from http://www.iana.org/assignments/port-numbers and other
sources like http://www.freebsd.org/cgi/cvsweb.cgi/src/etc/services .
New ports will be added on request if they have been officially assigned
by IANA and used in the real-world or are needed by a debian package.
If you need a huge list of used numbers please install the nmap package.

tcpmux 1/tcp # TCP port service multiplexer
echo 7/tcp
...

```

To scroll, you can use the up and down arrow keys on your keyboard. To page down one whole screens-worth of information, you can use either the space bar, the “Page Down” button on your keyboard, or the `CTRL-f` shortcut.

To scroll back up, you can use either the “Page Up” button, or the `CTRL-b` keyboard shortcut. To search for some text in the document, you can type a forward slash “/” followed by the search term. For instance, to search for “mail”, we would type: `/mail` This will search forward through the document and stop at the first result. To get to another result, you can type the lower-case `n` key:

`n`

To move backwards to the previous result, use a capital `N` instead:

`N`

When you wish to exit the `less` program, you can type `q` to quit:

`q`

While we focused on the `less` tool in this section, there are many other ways of viewing a file. The `cat` command displays a file’s contents and returns you to the prompt immediately. The `head` command, by default, shows the first 10 lines of a file. Likewise, the `tail` command shows the last 10 lines. These commands display file contents in a way that is useful for “piping” to other programs. We will discuss this concept in a future guide.

Feel free to see how these commands display the `/etc/services` file differently.

## File and Directory Manipulation

We learned in the last section how to view a file. In this section, we will demonstrate how to create and manipulate files and directories.

### Create a File with “touch”

Many commands and programs can create files. The most basic method of creating a file is with the `touch` command. This will create an empty file using the name and location specified.

First, we should make sure we are in our home directory, since this is a location where we have permission to save files. Then, we can create a file called `file1` by typing:

```
cd
touch file1
```

Now, if we view the files in our directory, we can see our newly created file:

```
ls
file1
```

If we use this command on an existing file, the command simply updates the data our filesystem stores on the time when the file was last accessed and modified.

We can also create multiple files at the same time. We can use absolute paths as well. For instance, if our user account is called `demo`, we could type:

```
touch /home/demo/file2 /home/demo/file3
ls
file1 file2 file3
```

## Create a Directory with “`mkdir`”

Similar to the `touch` command, the `mkdir` command allows us to create empty directories.

For instance, to create a directory within our home directory called `test`, we could type:

```
cd
mkdir test
```

We can make a directory *within* the `test` directory called `example` by typing:

```
mkdir test/example
```

For the above command to work, the `test` directory must already exist. To tell `mkdir` that it should create any directories necessary to construct a given directory path, you can use the `-p` option. This allows you to create nested directories in one step. We can create a directory structure that looks like `some/other/directories` by typing:

```
mkdir -p some/other/directories
```

The command will make the `some` directory first, then it will create the `other` directory in the `some` directory. Finally, it will create the `directories` directory in the `other` directory.

## Moving and Renaming Files and Directories with “mv”

We can move a file to a new location using the `mv` command. For instance, we can move `file1` into the `test` directory by typing:

```
mv file1 test
```

For this command, we list all items that we want to move, with the location to move them to. We can move that file *back* to our home directory by using the special dot reference to refer to our current directory. We should make sure we are in our home directory, and then execute the command:

```
cd
mv test/file1 .
```

This may seem unintuitive at first, but the `mv` command is also used to *rename* files and directories. In essence, moving and renaming are both just adjusting the location and name for an existing item.

So to rename the `test` directory to `testing`, we could type:

```
mv test testing
```

**Note:** It is important to realize that your Linux system will not prevent you from certain destructive actions. If you are renaming a file and choose a name that *already* exists, the previous file will be **overwritten** by the file you are moving. There is no way to recover the previous file if you accidentally overwrite it.

## Copying Files and Directories with “cp”

With the `mv` command, we could move or rename a file or directory, but we could not duplicate it. The `cp` command can make a new copy of an existing item.

For instance, we can copy `file3` to a new file called `file4`:

```
cp file3 file4
```

Unlike the `mv` operation, after which `file3` would no longer exist, we now have both `file3` and `file4`.

**Note:** As with the `mv` command, it is possible to overwrite a file if you are not careful about the filename you are using as the target of the operation. For instance, if `file4` already existed in the above example, its contents would be completely replaced by the contents of `file3`.

In order to copy directories, you must include the `-r` option in the command. This stands for “recursive,” as it copies the directory, plus all of the directory’s contents.

This option is necessary with directories, regardless of whether the directory is empty. For instance, to copy the `some` directory structure to a new structure called `again`, we could type:

```
cp -r some again
```

Unlike with files, with which an existing destination would lead to an overwrite, if the target is an *existing directory*, the file or directory is copied *into* the target:

```
cp file1 again
```

This will create a new copy of `file1` and place it inside of the `again` directory.

## Removing Files and Directories with “rm” and “rmdir”

To delete a file, you can use the `rm` command.

**Note:** Be extremely careful when using any destructive command like `rm`. There is no “undo” command for these actions so it is possible to accidentally destroy important files permanently.

To remove a regular file, just pass it to the `rm` command:

```
cd
```

```
rm file4
```

Likewise, to remove *empty* directories, we can use the `rmdir` command. This will only succeed if there is nothing in the directory in question. For instance, to remove the `example` directory within the `testing` directory, we can type:

```
rmdir testing/example
```

If you wish to remove a *non-empty* directory, you will have to use the `rm` command again. This time, you will have to pass the `-r` option, which removes all of the directory’s contents recursively, plus the directory itself.

For instance, to remove the `again` directory and everything within it, we can type:

```
rm -r again
```

Once again, it is worth reiterating that these are permanent actions. Be entirely sure that the command you typed is the one that you wish to execute.

## Editing Files

Currently, we know how to manipulate files as objects, but we have not learned how to actually edit and add content to them. The `nano` command is one of the simplest command-line Linux text editors, and is a great starting point for beginners. It operates somewhat similarly to the `less` program we discussed earlier, in that it occupies the entire terminal for the duration of its use. The `nano` editor can open existing files, or create a file. If you want to create a new file, you can give it a name when you call the `nano` editor, or later on, when you wish to save your content.

We can open the `file1` file for editing by typing:

```
cd
nano file1
```

The `nano` application will open the file (which is currently blank). The interface looks something like this:

```
GNU nano 2.2.6 File: file1

[Read 0 lines]

^G Get Help ^O WriteOut ^R Read File ^Y Prev Page ^K
Cut Text ^C Cur Pos

^X Exit ^J Justify ^W Where Is ^V Next Page ^U
UnCut Text ^T To Spell
```

Along the top, we have the name of the application and the name of the file we are editing. In the middle, the content of the file, currently blank, is displayed. Along the bottom, we have a number of key combinations that indicate some basic controls for the editor. For each of these, the `^` character means the `CTRL` key.

To get help from within the editor, type:

```
CTRL-G
```

When you are finished browsing the help, type `CTRL-X` to get back to your document.

Type in or modify any text you would like. For this example, we will just type these two sentences:

```
Hello there.
```

```
Here is some text.
```

To save type:

```
CTRL-O
```

This is the letter “o,” not a zero. It will ask you to confirm the name of the file you wish to save to:

```
File Name to Write: file1
```

```
^G Get Help M-D DOS Format M-A Append
```

```
M-B Backup File
```

```
^C Cancel M-M Mac Format M-P Prepend
```

As you can see, the options at the bottom have also changed. These are contextual, meaning they will change depending on what you are trying to do. If file1 is still the file you wish to write to, hit “ENTER.”

If we make additional changes and wish to save the file and exit the program, we will see a similar prompt. Add a new line, and then try to exit the program by typing:

```
CTRL-X
```

If you have not saved after making your modification, you will be asked whether you wish to save the modifications you made:

```
Save modified buffer (ANSWERING “No” WILL DESTROY CHANGES) ?
```

```
Y Yes
```

```
N No ^C Cancel
```

You can type “Y” to save your changes, “N” to discard your changes and exit, or “CTRL-C” to cancel the exit operation. If you choose to save, you will be given the same file prompt that you received before, confirming that you want to save the changes to the same file. Press ENTER to save the file and exit the editor.

You can see the contents of the file you created using either the `cat` command to display the contents, or the `less` command to open the file for viewing. After viewing with `less`, remember that you should hit `q` to get back to the terminal.

```
less file1
```

```
Hello there.
```

Here is some text.

Another line.



# Chapter Ten: UNIX SHELL SCRIPTING

Time is precious. It does not make sense to waste time typing a frequently used sequence of commands at a command prompt, more especially if they are abnormally long or complex. Scripting is a way by which one can alleviate this problem by automating these command sequences in order to make life at the shell easier and more productive. Scripting is all about making the computer, the tool, do the work. By the end of this tutorial you will have a good understanding of the kind of scripting languages available for Unix and how to apply them to your problems. UNIX contains many wonderful and strange commands that can be very useful in the world of scripting, the more tools you know and the better you know them, the more use you will find for them. Most of the Unix commands and many of the built-in commands have `man` pages; `man` pages contain the usage instructions pertaining to the parent tool. They are not always very clear and may require reading several times. In order to access a man page in Unix the following command sequence is applied:

**`man command`**

If a man page exists for the command specified the internal viewer will be invoked and you will be able to read about the various options and usage instructions.

## **Shell Scripting Introduction**

UNIX uses shells to accept commands given by the user; there are quite a few different shells available. The most commonly used shells are SH (Bourne SHell) CSH (C SHell) and KSH (Korn SHell), most of the other shells you encounter will be variants of these shells and will share the same syntax, KSH is based on SH as is BASH (Bourne again shell). TCSH (Extended C SHell) is based on CSH.

The various shells all have built in functions which allow for the creation of shell scripts, that is, the stringing together of shell commands and constructs to automate repetitive tasks in order to make life easier for the user.

With all these different shells available, what shell should we use? This is debatable. For the purpose of this tutorial we will be using SH because it is practically guaranteed to be available on most Unix systems and be supported by the SH based shells. Your default shell may not be SH. Fortunately we do not have to be using a specific shell in order to exploit its features because we can specify

the shell we want to interpret our shell script within the script itself by including the following in the first line.

```
#!/path/to/shell
```

Usually anything following (#) is interpreted as a comment and ignored but if it occurs on the first line with a (!) following it is treated as being special and the filename following the (!) is considered to point to the location of the shell that should interpret the script.

When a script is “executed”, it is being interpreted by an invocation of the shell that is running it. Hence, the shell is said to be running non-interactively, when the shell is used “normally” it is said to be running interactively.

Note: There are many variations of the basic commands and extra information which is too specific to be mentioned in this short tutorial, you should read the man page for your shell to get a more comprehensive idea of the options available to you. This tutorial will concentrate on highlighting the most often used and useful commands and constructs.

## ***Command Redirection and Pipelines***

By default a normal command accepts input from standard input, which we abbreviate to stdin, standard input is the command line in the form of arguments passed to the command. By default a normal command directs its output to standard output, which we abbreviate to stdout, standard output is usually the console display. For some commands this may be the desired action but other times we may wish to get our input for a command from somewhere other than stdin and direct our output to somewhere other than stdout. This is done by redirection:

We use > to redirect stdout to a file, for instance, if we wanted to redirect a directory listing generated by the **ls** we could do the following:

```
ls > file
```

We use < to specify that we want the command immediately before the redirection symbol to get its input from the source specified immediately after the symbol, for instance, we could redirect the input to `grep` (which searches for strings within files) so that it comes from a file like this:

```
grep searchterm < file
```

We use `>>` to append stdout to a file, for example, if we wanted to append the date to the end of a file we would redirect the output from `date` like so:

```
date >> file
```

One can redirect standard error (stderr) to a file by using `2>`, if we wanted to redirect the standard error from `commandA` to a file we would use:

```
commandA 2>
```

Pipes are another form of redirection that are used to chain commands so that powerful composite commands can be constructed, the pipe symbol `|` takes the stdout from the command preceding it and redirects it to the command following it:

```
ls -l | grep searchword | sort -r
```

The example above first requests a long (`-l`) directory listing of the current directory using the `ls` command, the output from this is then piped to `grep` which filters out all the listings containing the searchword and then finally pipes this through to `sort` which then sorts the output in reverse (`-r`), `sort` then passes the output on normally to stdout.

## Variables

When a script starts, all environment variables are turned into shell variables. New variables can be instantiated like this:

```
name=value
```

You must do it exactly like that, with no spaces, the name must only be made up of alphabetic characters, numeric characters and underscores; it cannot begin with a numeric character. You should avoid using keywords like *for* or anything like that, the interpreter will let you use them but doing so can lead to obfuscated code ;)

Variables are referenced like this: `$name`, here is an example:

```
#!/bin/sh
msg1=Hello
msg2=There!
echo $msg1 $msg2
```

This would echo “Hello There!” to the console display, if you want to assign a string to a variable and the string contains spaces you should enclose the string in double quotes (“), the double quotes tell the shell to take the contents literally and ignore keywords, however, a few keywords are still processed. You can still use \$ within a (“) quoted string to include variables:

```
#!/bin/sh
msg1="one"
msg2="$msg1 two"
msg3="$msg2 three"
echo $msg3
```

Would echo “one two three” to the screen. The escape character can also be used within a double quoted section to output special characters, the escape character is “”, it outputs the character immediately following it literally so \ would output \. A special case is when the escape character is followed by a newline; the shell ignores the newline character, which allows the spreading of long commands that must be executed on a single line in reality over multiple lines within the script. The escape character can be used anywhere, except within single quotes.

Surrounding anything with single quotes causes it to be treated as literal text that will be passed on exactly as intended, this can be useful for sending command sequences to other files in order to create new scripts because the text between the single quotes will remain untouched. For example:

```
#!/bin/sh
echo `msg="Hello World!"` > hello
echo `echo $msg` >> hello
chmod 700 hello
./hello
```

This would cause “msg=“Hello World!” to be echoed and redirected to the file `hello`, “echo \$msg” would then be echoed and redirected to the file `hello` but this time appended to the end. The `chmod` line changes the file permissions of `hello` so that we can execute it. The final line executes **hello** causing it output “Hello World.” If we had not used literal quotes we would not have had to use escape characters to ensure that (\$) and (“) were echoed to the file, this makes the code a little clearer.

A variable may be referenced like so \${VARIABLENAME}, this allows one to place

characters immediately preceding the variable like `${VARIABLENAME}aaa` without the shell interpreting `aaa` as being part of the variable name.

## Command Line Arguments

Command line arguments are treated as special variables within the script, the reason I am calling them variables is because they can be changed with the **shift** command. The command line arguments are enumerated in the following manner `$0`, `$1`, `$2`, `$3`, `$4`, `$5`, `$6`, `$7`, `$8` and `$9`. `$0` is special in that it corresponds to the name of the script itself. `$1` is the first argument, `$2` is the second argument and so on. To reference after the ninth argument you must enclose the number in brackets like this `${nn}`. You can use the **shift** command to shift the arguments 1 variable to the left so that `$2` becomes `$1`, `$1` becomes `$0` and so on, `$0` gets scrapped because it has nowhere to go, this can be useful to process all the arguments using a loop, using one variable to reference the first argument and **shifting** until you have exhausted the arguments list.

As well as the command line arguments there are some special built-in variables:

- `$#`  represents the parameter count. Useful for controlling loop constructs that need to process each parameter.
- `$@`  expands to all the parameters separated by spaces. Useful for passing all the parameters to some other function or program.
- `$-`  expands to the flags(options) the shell was invoked with. Useful for controlling program flow based on the flags set.
- `$$`  expands to the process id of the shell innovated to run the script. Useful for creating unique temporary filenames relative to this instantiation of the script.

Note: The command line arguments will be referred to as parameters from now on, this is because SH also allows the definition of functions which can take parameters and when called the `$n`family will be redefined, hence these variables are always parameters, its just that in the case of the parent script the parameters are passed via the command line. One exception is `$0` which is always set to the name of the parent script regardless of whether it is inside a function or not.

## Command Substitution

In the words of the SH manual “*Command substitution allows the output of a*

*command to be substituted in place of the command name itself*". There are two ways this can be done. The first is to enclose the command like this:

```
$(command)
```

The second is to enclose the command in back quotes like this:

```
`command`
```

The command will be executed in a sub-shell environment and the standard output of the shell will replace the command substitution when the command finishes.

## Arithmetic Expansion

Arithmetic expansion is also allowed and comes in the form:

```
$((expression))
```

The value of the expression will replace the substitution. Eg:

```
#!/bin/sh
echo $((1 + 3 + 4))
```

Will echo "8" to stdout

## Control Constructs

The flow of control within SH scripts is done via four main constructs; if... then... elif..., else..., do..., while..., for... and case....

*If.. Then.. Elif.. Else*

This construct takes the following generic form; the parts enclosed within ([) and (]) are optional:

```
if list
then list
[elif list
then list] ...
[else list]
fi
```

When a Unix command exits it exits with what is known as an *exit status*, this indicates to anyone who wants to know the degree of success a command, usually when a command executes without error it terminates with an exit status of zero. An exit status of some other value would indicate that an error has occurred, the details of which are specific to the command. The commands' man pages detail the exit status messages.

A list is defined in the SH as “*a sequence of zero or more commands separated by newlines, semicolons, or ampersands, and optionally terminated by one of these three characters.*” Hence in the generic definition of the *if* above the list will determine which of the execution paths the script takes. For example, there is a command called `test` on UNIX, which evaluates an expression and if it evaluates true will return zero and will return one otherwise, this is how we can test conditions in the *list* part(s) of the *if* construct because `test` is a command.

We do not actually have to type the `test` command directly into the *list* to use it; it can be implied by encasing the test case within `([` and `])` characters, as illustrated by the following (silly) example:

```
#!/bin/sh
if ["$1" = "1"]
then
 echo "The first choice is nice"
elif ["$1" = "2"]
then
 echo "The second choice is just as nice"
elif ["$1" = "3"]
then
 echo "The third choice is excellent"
else
 echo "I see you were wise enough not to choose"
 echo "You win"
fi
```

What this example does is compare the first parameter (command line argument in this case) with the strings “1”, “2” and “3” using **tests**’ (`=`) test which compares two strings for equality, if any of them match it prints out the corresponding message. If none of them match it prints out the final case. OK the example is silly and actually flawed (the user still wins even if they type in (4) or something) but it illustrates how the *if* statement works.

Notice that there are spaces between `(if)` and `([`, `([` and the test and the test and



(`]`), these spaces must be present otherwise the shell will complain. There must also be spaces between the operator and operands of the test otherwise it will not work properly. Notice how it starts with `(if)` and ends with `(fi)`, also, notice how `(then)` is on a separate line to the test above it and that `(else)` does not require a `(then)` statement. You must construct this construct exactly like this for it to work properly.

It is also possible to integrate logical AND and OR into the testing, by using two tests separated by either `&&` or `||` respectively. For example, we could replace the third test case in the example above with:

```
elif ["$1" = "3"] || ["$1" = "4"]
then echo "The third choi..."
```

The script would print out "The third choice is excellent" if the first parameter was either "3" OR "4". To illustrate the use of `&&` we could replace the third test case with:

```
elif ["$1" = "3"] && ["$2" = "4"]
then echo "The third choi..."
```

The script would print out "The third choice is excellent" if and only if the first parameter was "3" AND the second parameter was "4".

`&&` and `||` are both lazily evaluating which means that in the case of `&&`, if the first test fails it won't bother evaluating the second because the list will only be true if they *BOTH* pass and since one has already failed there is no point wasting time evaluating the second. In the case of `||` if the first test passes it won't bother evaluating the second test because we only need *ONE* of the tests to pass for the whole list to pass. See the `test` man page for the list of tests possible (other than the string equality test mentioned here).

### *Do...While*

The *Do...While* takes the following generic form:

```
while list
do list
done
```

In the words of the SH manual *"The two lists are executed repeatedly while the*



*exit status of the first list is zero.*” There is a variation on this that uses `until` in place of `while` which executes *until* the exit status of the first list is zero. Here is an example use of the `while` statement:

```
#!/bin/sh
count=$1 # Initialise count to first
parameter # parameter
while [$count -gt 0] # while count is greater than
10 do
do
 echo $count seconds till supper time!
 count=$(expr $count -1) # decrement count by 1
 sleep 1 # sleep for a second using
the Unix sleep command
done
echo Supper time!!, YEAH!! # were finished
```

If called from the command line with an argument of 4 this script will output

```
4 seconds till supper time!
3 seconds till supper time!
2 seconds till supper time!
1 seconds till supper time!
Supper time!!, YEAH!!
```

You can see that this time we have used the `-gt` of the `test` command implicitly called by `[` and `]`, which stands for greater than. Pay careful attention to the formatting and spacing.

### *For*

The syntax of the `for` command is:

```
for variable in word ...
do list
done
```

The SH manual states “*The words are expanded, and then the list is executed repeatedly with the variable set to each word in turn.*” A word is essentially some other variable that contains a list of values of some sort, the *for* construct assigns each of the values in the word to variable and then variable can be used within the body of the construct, upon completion of the body variable will be assigned the next value in word until there are no more values in word. This example should

make this clearer:

```
#!/bin/sh
fruitlist="Apple Pear Tomato Peach Grape"
for fruit in $fruitlist
do
 if ["$fruit" = "Tomato"] || ["$fruit" = "Peach"]
 then
 echo "I like ${fruit}es"
 else
 echo "I like ${fruit}s"
 fi
done
```

In this example, *fruitlist* is word, *fruit* is variable and the body of the statement outputs how much this person loves various fruits but includes an *if... then... else* statement to deal with the correct addition of letters to describe the plural version of the fruit, notice that the variable *fruit* was expressed like *\${fruit}* because otherwise the shell would have interpreted the preceding letter(s) as being part of the variable and echoed nothing because we have not defined the variables *fruits* and *fruites* When executed this script will output:

```
I like Apples
I like Pears
I like Tomatoes
I like Peachs
I like Grapes
```

Within the *for* construct, *do* and *done* may be replaced by '{' and '}'.

## Case

The *case* construct has the following syntax:

```
case word in
pattern) list ;;
...

esac
```

An example of this should make things clearer:

---

```
#!/bin/sh
case $1
in
1) echo 'First Choice';;
2) echo 'Second Choice';;
*) echo 'Other Choice';;
esac
```

“1”, “2” and “\*” are patterns, word is compared to each pattern and if a match is found the body of the corresponding pattern is executed, we have used “\*” to represent everything, since this is checked last we will still catch “1” and “2” because they are checked first. In our example word is “\$1”, the first parameter, hence if the script is ran with the argument “1” it will output “First Choice”, “2” “Second Choice” and anything else “Other Choice”. In this example we compared against numbers (essentially still a string comparison however) but the pattern can be more complex, see the SH man page for more information.

## Functions

The syntax of an SH function is defined as follows:

```
name () command
```

It is usually laid out like this:

```
name() {
commands
}
```

A function will return with a default exit status of zero, one can return different exit statuses by using the notation *return exit status*. Variables can be defined locally within a function using *local name=value*. The example below shows the use of a user defined increment function:

## Increment Function Example

```
#!/bin/sh

inc() { ❶ # The increment is defined first so we can
use it use it

 echo $(($1 + $2)) # We echo the result of the first parameter
plus the second parameter

}
```

```

We check to see that all the command line
arguments are present
if ["$1" ""] || ["$2" = ""] || ["$3" = ""]
then
 echo USAGE:
 echo " counter startvalue incrementvalue endvalue"
else
 count=$1 # Rename are variables with clearer names
 value=$2
 end=$3
 while [$count -lt $end] # Loop while count is less than end
 do
 echo $count
 count=$(inc $count $value) ❷ # Call increment with count and value as
parameters
 done # so that count is incremented by value
fi

```

```

inc() {
 echo $(($1 + $2))
}

```

The function is defined and opened with `inc() {`, the line `echo $(( $1 + $2 ))` uses the notation for arithmetic expression substitution which is `$((expression))` to enclose the expression, `$1 + $2` which adds the first and second parameters passed to the function together, the `echo` bit at the start echoes them to standard output, we can catch this value by assigning the function call to a variable, as is illustrated by the function call.

```
count=$(inc $count $value)
```

We use command substitution which substitutes the value of a command to substitute the value of the function call whereupon it is assigned to the *count* variable. The command within the command substitution block is `inc $count $value`, the last two values being its parameters. Which are then referenced from within the function using `$1` and `$2`. We could have used the other command substitution notation to call the function if we had wanted:

```
count=`inc $count $value`
```

Here is another example illustrating the scope of variables:

## Variable Scope, Example

```
#!/bin/sh
inc() {
 local value=4
 echo "value is $value within the function\n"
 echo "\b\$1 is $1 within the function"
}

value=5
echo value is $value before the function
echo "\$1 is $1 before the function"
echo
echo -e $(inc $value)
echo
echo value is $value after the function
echo "\$1 is $1 after the function"
```

```
inc() {
 local value=4
 echo "value is $value within the function\n"
 echo "\b\$1 is $1 within the function"
}
```

We assign a local value to the variable *value* of 4. The next three lines construct the output, remember that this is being echoed to a buffer and will be replace the function call with all that was passed to stdout within the function when the function exits. So, the calling code will be replaced with whatever we direct to standard output within the function. The function is called like this:

```
echo -e $(inc $value)
```

We have passed the option `-e` to the **echo** command which causes it to process C-style backslash escape characters, so we can process any backslash escape characters which the string generated by the function call contains. If we just echo the lines we want to be returned by the function it will not pass the newline character onto the buffer even if we explicitly include it with an escape character reference so what we do is actually include the sequence of characters that will

produce a new line within the string so that when it is echoed by the calling code with the `-e` the escape characters will be processed and the newlines will be placed where we want them.

```
echo "value is $value within the function\n"
```

Notice how the newline has been inserted with `\n`, the first two backslashes indicate that we want to echo a backslash because within double quotes a backslash indicates to process the next character literally, we have to do this because we are only between double quotes and not the literal-text single quotes. If we had used single quotes we would have had to echo the bit with the newline in separately from the bit that contains `$value` otherwise `$value` would not be expanded.

```
echo "\b$1 is $1 within the function"
```

This is our second line, and is contained within double quotes so that the variable `$1` will be expanded, `\b` is included so that `\b` will be placed in the echoed line and our calling code processes this as a backspace character. If we do not do that the shell prefixes a space to the second line, the backspace removes this space.

The output from this script called with 2 as the first argument is:

```
value is 5 before the function
```

```
$1 is 2 before the function
```

```
value is 4 within the function
```

```
$1 is 5 within the function
```

```
value is 5 after the function
```

```
$1 is 2 after the function
```

**Tip:** You can use `". DIRECTORY/common.sh"` to import functions from a script called `common.sh` in `DIRECTORY`, a quick example is shown below, first is `test.sh`:

```
#!/bin/sh
```

```
. ./common.sh
```

```
if ["$1" = ""]; then
```

```
 echo USAGE:
```

```
 echo "sh test.sh type"
```

```
 exit
```

```
fi
```

```
if `validtype $1`; then
 echo Valid type
else
 echo Invalid type
fi
```

## Here is common.sh:

```
#!/bin/sh

validtype() {
 if ["$1" = "TYPEA"] ||
 ["$1" = "TYPEB"] ||
 ["$1" = "TYPEEC"] ||
 ["$1" = "TYPED"] ||
 ["$1" = "TYPEEE"];
 then
 exit 0
 else
 exit 1
 fi
}
```

# Chapter Eleven: SHELL BASIC OPERATOR

Each shell supports various operators. This chapter is based on default shell (Bourne) so we are going to cover all the important Bourne Shell operators in this tutorial.

We will discuss the following operators –

- Arithmetic Operators.
- Relational Operators.
- Boolean Operators.
- String Operators.
- File Test Operators.

The Bourne shell did not originally have any mechanism to perform simple arithmetic but it uses external programs, either `awk` or the must simpler program `expr`.

Here is simple example to add two numbers –

```
#!/bin/sh

val=`expr 2 + 2`
echo "Total value : $val"
```

This produces the following result –

```
Total value : 4
```

Note:

- There must be spaces between operators and expressions for example `2+2` is not correct, it should be written as `2 + 2`.
- Complete expressions should be enclosed between “```”, inverted commas.

## Arithmetic Operators

The following are arithmetic operators supported by the Bourne Shell.

Assume variable ‘a’ holds 10 and variable ‘b’ holds 20 then –

Example, using all the arithmetic operators –



```
#!/bin/sh

a=10
b=20
val=`expr $a + $b`
echo "a + b : $val"
```

```
val=`expr $a - $b`
echo "a - b : $val"
```

```
val=`expr $a \* $b`
echo "a * b : $val"
```

```
val=`expr $b / $a`
echo "b / a : $val"
```

```
val=`expr $b % $a`
echo "b % a : $val"
```

```
if [$a == $b]
then
 echo "a is equal to b"
fi
```

```
if [$a != $b]
then
 echo "a is not equal to b"
fi
```

This produces the following result –

```
a + b : 30
a - b : -10
a * b : 200
b / a : 2
b % a : 0
a is not equal to b
```

Note:

- There must be spaces between operators and expressions for example 2+2 is not correct, whereas it should be written as 2 + 2.
- Complete expression should be enclosed between “`, inverted commas.

- You should use \ on the \* symbol for multiplication.
- **The if...then...fi** statement is a decision-making statement, which will be explained in the next chapter.

Operator	Description	Example
+	Addition - Adds values on either side of the operator	`expr \$a + \$b` will give 30
-	Subtraction - Subtracts right hand operand from left hand operand	`expr \$a - \$b` will give -10
*	Multiplication - Multiplies values on either side of the operator	`expr \$a \* \$b` will give 200
/	Division - Divides left hand operand by right hand operand	`expr \$b / \$a` will give 2
%	Modulus - Divides left hand operand by right hand operand and returns remainder	`expr \$b % \$a` will give 0
=	Assignment - Assign right operand in left operand	a=\$b would assign value of b into a
==	Equality - Compares two numbers, if both are same then returns true.	[ \$a == \$b ] would return false.
!=	Not Equality - Compares two numbers, if both are different then returns true.	[ \$a != \$b ] would return true.

It is very important to note that all the conditional expressions should be put inside square braces with one spaces around them, for example [ \$a == \$b ] is correct whereas [\$a==\$b] is incorrect.

All the arithmetical calculations are done using long integers.

## Relational Operators:

Bourne Shell supports the following relational operators, which are specific to numeric values. These operators will not work for string values unless their value is numeric. For example, the following operators check a relation between 10 and 20 as well as in between “10” and “20” but not in between “ten” and “twenty”. Assume variable a holds 10 and variable b holds 20 then –

This example uses all the relational operators –

```
#!/bin/sh
```

```
a=10
```

```
b=20
```

```
if [$a -eq $b]
```

```
then
```

```
 echo "$a -eq $b : a is equal to b"
```

```
else
```

```
 echo "$a -eq $b: a is not equal to b"
```

```
fi
```

```
if [$a -ne $b]
```

```
then
```

```
 echo "$a -ne $b: a is not equal to b"
```

```
else
```

```
 echo "$a -ne $b : a is equal to b"
```

```
fi
```

```
if [$a -gt $b]
```

```
then
```

```
 echo "$a -gt $b: a is greater than b"
```

```
else
```

```
 echo "$a -gt $b: a is not greater than b"
```

```
fi
```

```
if [$a -lt $b]
```

```
then
```

```
 echo "$a -lt $b: a is less than b"
```

```
else
```

```
 echo "$a -lt $b: a is not less than b"
```

```
fi
```

```
if [$a -ge $b]
```

```
then
```

```
 echo "$a -ge $b: a is greater or equal to b"
```

```
else
```

```
 echo "$a -ge $b: a is not greater or equal to b"
```

```
fi
```

```

if [$a -le $b]
then
 echo "$a -le $b: a is less or equal to b"
else
 echo "$a -le $b: a is not less or equal to b"
fi

```

This produces the following result –

```

10 -eq 20: a is not equal to b
10 -ne 20: a is not equal to b
10 -gt 20: a is not greater than b
10 -lt 20: a is less than b
10 -ge 20: a is not greater or equal to b
10 -le 20: a is less or equal to b

```

Note:

- There must be spaces between operators and expressions for example 2+2 is not correct, whereas it should be written as 2 + 2.
- **if...then...else...fi** statement is a decision making statement which will be explained in the next chapter.

Operator	Description	Example
<b>-eq</b>	Checks if the value of two operands are equal, if yes then condition becomes true.	[ \$a -eq \$b ] is not true.
<b>-ne</b>	Checks if the value of two operands are equal, if values are not equal then condition becomes true.	[ \$a -ne \$b ] is true.
<b>-gt</b>	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	[ \$a -gt \$b ] is not true.
<b>-lt</b>	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	[ \$a -lt \$b ] is true.
<b>-ge</b>	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	[ \$a -ge \$b ] is not true.
<b>-le</b>	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes	[ \$a -le \$b ] is true.

true.

---

It is very important to note that all the conditional expressions should be put inside square braces with one space around them, for example [ \$a <= \$b ] is correct whereas [\$a <= \$b] is incorrect.

## Boolean Operators

The Bourne Shell supports the following boolean operators.

Assume variable 'a' holds 10 and variable 'b' holds 20 then –

This example uses all the boolean operators –

```
#!/bin/sh

a=10
b=20

if [$a != $b]
then
 echo "$a != $b : a is not equal to b"
else
 echo "$a != $b: a is equal to b"
fi

if [$a -lt 100 -a $b -gt 15]
then
 echo "$a -lt 100 -a $b -gt 15 : returns true"
else
 echo "$a -lt 100 -a $b -gt 15 : returns false"
fi

if [$a -lt 100 -o $b -gt 100]
then
 echo "$a -lt 100 -o $b -gt 100 : returns true"
else
 echo "$a -lt 100 -o $b -gt 100 : returns false"
fi

if [$a -lt 5 -o $b -gt 100]
then
 echo "$a -lt 100 -o $b -gt 100 : returns true"
```

```
else
 echo "$a -lt 100 -o $b -gt 100 : returns false"
fi
```

This produces the following result –

```
10 != 20 : a is not equal to b
10 -lt 100 -a 20 -gt 15 : returns true
10 -lt 100 -o 20 -gt 100 : returns true
10 -lt 5 -o 20 -gt 100 : returns false
```

Note:

- There must be spaces between operators and expressions for example 2+2 is not correct, whereas it should be written as 2 + 2.

Operator	Description	Example
<b>!</b>	This is a logical negation. This inverts a true condition into false and vice versa.	[ ! false ] is true.
<b>-o</b>	This is a logical OR. If one of the operands were true then condition would be true.	[ \$a -lt 20 -o \$b -gt 100 ] is true.
<b>-a</b>	This is a logical AND. If both the operands were true then condition would be true otherwise, it would be false.	[ \$a -lt 20 -a \$b -gt 100 ] is false.

## String Operators

The Bourne Shell supports the following string operators.

Assume variable ‘a’ holds “abc” and variable ‘b’ holds “efg” then –

This example uses all the string operators –

```
#!/bin/sh

a="abc"
b="efg"

if [$a = $b]
then
 echo "$a = $b : a is equal to b"
else
 echo "$a = $b: a is not equal to b"
```

```

fi

if [$a != $b]
then
 echo "$a != $b : a is not equal to b"
else
 echo "$a != $b: a is equal to b"
fi

if [-z $a]
then
 echo "-z $a : string length is zero"
else
 echo "-z $a : string length is not zero"
fi

if [-n $a]
then
 echo "-n $a : string length is not zero"
else
 echo "-n $a : string length is zero"
fi

if [$a]
then
 echo "$a : string is not empty"
else
 echo "$a : string is empty"
fi

```

This produces the following result –

```

abc = efg: a is not equal to b
abc != efg : a is not equal to b
-z abc : string length is not zero
-n abc : string length is not zero
abc : string is not empty

```

Note:

- There must be spaces between operators and expressions for example 2+2 is not correct, whereas it should be written as 2 + 2.

---

Operator	Description	Example
----------	-------------	---------

---

<b>=</b>	Checks if the value of two operands is equal, if yes then condition becomes true.	[ \$a = \$b ] is not true.
<b>!=</b>	Checks if the value of two operands are equal, if values are not equal then condition becomes true.	[ \$a != \$b ] is true.
<b>-z</b>	Checks if the given string operand size is zero. If it is zero length then it returns true.	[ -z \$a ] is not true.
<b>-n</b>	Checks if the given string operand size is non-zero. If it is non-zero length then it returns true.	[ -z \$a ] is not false.
<b>str</b>	Check if str is not the empty string. If it is empty then it returns false.	[ \$a ] is not false.

## File Test Operators

The following operators test various properties associated with a Unix file.

Assume a variable `file` holds an existing file name “test” whose size is 100 bytes and has read, write and execute permissions –

This example uses all the file test operators –

```
#!/bin/sh

file="/var/www/tutorialspoint/unix/test.sh"

if [-r $file]
then
 echo "File has read access"
else
 echo "File does not have read access"
fi

if [-w $file]
then
 echo "File has write permission"
```



```
else
 echo "File does not have write permission"
fi

if [-x $file]
then
 echo "File has execute permission"
else
 echo "File does not have execute permission"
fi

if [-f $file]
then
 echo "File is an ordinary file"
else
 echo "This is special file"
fi

if [-d $file]
then
 echo "File is a directory"
else
 echo "This is not a directory"
fi

if [-s $file]
then
 echo "File size is zero"
else
 echo "File size is not zero"
fi

if [-e $file]
then
 echo "File exists"
else
 echo "File does not exist"
fi
```

This produces the following result –

```
File has read access
File has write permission
```

File has execute permission

File is an ordinary file

This is not a directory

File size is zero

File exists

Note:

- There must be spaces between operators and expressions for example 2+2 is not correct, whereas it should be written as 2 + 2.

Operator	Description	Example
<b>-b file</b>	Checks if file is a block special file if yes then condition becomes true.	[ -b \$file ] is false.
<b>-c file</b>	Checks if file is a character special file if yes then condition becomes true.	[ -c \$file ] is false.
<b>-d file</b>	Check if file is a directory if yes then condition becomes true.	[ -d \$file ] is not true.
<b>-f file</b>	Check if file is an ordinary file as opposed to a directory or special file if yes then condition becomes true.	[ -f \$file ] is true.
<b>-g file</b>	Checks if file has its set group ID (SGID) bit set if yes then condition becomes true.	[ -g \$file ] is false.
<b>-k file</b>	Checks if file has its sticky bit set if yes then condition becomes true.	[ -k \$file ] is false.
<b>-p file</b>	Checks if file is a named pipe if yes then condition becomes true.	[ -p \$file ] is false.
<b>-t file</b>	Checks if file descriptor is open and associated with a terminal if yes then condition becomes true.	[ -t \$file ] is false.
<b>-u file</b>	Checks if file has its set user id (SUID) bit set if yes then condition becomes true.	[ -u \$file ] is false.
<b>-r file</b>	Checks if file is readable if yes then condition becomes true.	[ -r \$file ] is true.
<b>-w file</b>	Checks if file is writable if yes then condition becomes true.	[ -w \$file ] is true.
<b>-x file</b>	Checks if file is execute if yes then	[ -x \$file ] is

## Run The .Sh File Shell Script In Linux / Unix

After downloading the software, the .sh file is nothing but the shell script to install a given application or to perform other tasks under UNIX like operating systems. The easiest way to run .sh shell script in Linux or UNIX is to type either of the following commands. Open the terminal (your shell prompt) and type the command:

**sh file.sh**

OR

**bash file.sh**

## .sh As Root User

Sometimes you will need to install an application that requires root level

	condition becomes true.	true.
<b>-s file</b>	Checks if file has size greater than 0 if yes then condition becomes true.	[ -s \$file ] is true.
<b>-e file</b>	Checks if file exists. Is true even if file is a directory but exists.	[ -e \$file ] is true.

privileges.

Root access is disabled by default on many Linux and UNIX like systems. Simply use sudo or su as follows:

```
sudo bash filename.sh
```

Type your password. Another option is to use the su command as follows to become superuser:

```
su -
```

Type root user password and finally run your script:

```
bash filename.sh
```

## chmod Command: Run Shell Script In Linux

Another recommended option is to set an executable permission using the chmod command as follows:

```
chmod +x file.sh
```

Now you can run the .sh file.

```
./file.sh
```

# Chapter Twelve: SHELL EMBEDDING AND OPTIONS

This chapter investigates tyke shells, implanted shells and shell alternatives.

## Shell installing

Shells can be installed to work on a command line; the command line can bring forth new procedures containing a fork of the present shell. You can use variables to demonstrate that the new shells are made. The variable `$var1` just exists in the (interim) sub shell.

```
[paul@RHELv4u3 gen]$ reverberation $var1
[paul@RHELv4u3 gen]$ reverberation $(var1=5;echo $var1)
5
[paul@RHELv4u3 gen]$ reverberation $var1
[paul@RHELv4u3 gen]$
```

You can install a shell into an inserted shell; this is called settled implanting of shells.

```
paul@deb503:~$ A=shell
paul@deb503:~$ reverberation CB$A $(B=sub;echo CB$A; reverberation
$(C=sub;echo CB$A))
```

shell subshell subsubshell

## Backticks

Single installation can change your present index and uses backticks instead of dollar section to implant.

```
[paul@RHELv4u3 ~]$ reverberation `cd/and so forth; ls - d * | grep pass`
passwd-passwd.OLD
[paul@RHELv4u3 ~]$
```

The backticks `/()` and the `$()` documentation can be used to implant a shell.

## Backticks or single quotes

Putting the installing between backticks utilizes one character not exactly the dollar and bracket combo. Be cautious on the other hand, backticks are frequently mistaken for single quotes.

The specialized contrast in the middle of `”` and ``` is critical!

```
[paul@RHELv4u3 gen]$ reverberation `var1=5;echo $var1`
```

```
[paul@RHELv4u3 gen]$ reverberation `var1=5;echo $var1`
var1=5;echo $var1
[paul@RHELv4u3 gen]$
```

## Shell Alternatives

Both `set` and `unset` are built-in shell commands, which are used to set alternatives to the bash shell. The following case will clear up this. The shell will treat unset variables as an unimportant variable. By setting the `-u` alternative, the shell will treat any reference to unset variables as an error.

```
[paul@RHEL4b ~]$ reverberation $var123
[paul@RHEL4b ~]$ set -u
[paul@RHEL4b ~]$ reverberation $var123
- bash: var123: unbound variable
[paul@RHEL4b ~]$ set +u
[paul@RHEL4b ~]$ reverberation $var123
[paul@RHEL4b ~]$
```

To rundown all the set choices for your shell, use `reverberation $-`. The noclobber (or `-C`).

```
[paul@RHEL4b ~]$ reverberation $-
himBH
[paul@RHEL4b ~]$ set -C ; set -u
[paul@RHEL4b ~]$ reverberation $-
himuBCH
[paul@RHEL4b ~]$ set +C ; set +u
[paul@RHEL4b ~]$ reverberation $-
himBH
[paul@RHEL4b ~]$
```

When writing a set without alternatives, you get a rundown of all variables without capacity, when the shell is in posix mode, you can set bash in posix mode writing `set -o posix`.

## Practice: Shell Installing

1. Discover the rundown of shell choices in the man page of bash. What is the contrast between situated `-u` and `set -o nounset`?
2. Actuate `nounset` in your shell. Test if it demonstrates an error message when using non-existing variables.

3. Deactivate nounset.
4. Execute `disc/var` and `ls` in an installed shell.

The reverberation command is expected to demonstrate the after effects of the `ls` command. Precluding will cause the shell to execute the first document as a command.

5. Add the variable `embvar` into an inserted shell and reverberation it. Does the variable exist in your present shell now?
6. Clarify what “`set - x`” does. Can this be useful?

(Optional)

7. Given the accompanying screenshot, add four characters to the command so that the aggregate yield is FirstMiddleLast.  
`[paul@RHEL4b ~]$ reverberate First; resound Middle; reverberate Last.`
8. Display a long listing (`ls - l`) of the `passwd` command utilizing the `which` command inside an installed shell.

#### 15.4. Arrangement: shell inserting

1. Discover the rundown of shell alternatives in the man page of `bash`. What is the distinction between situated

`u` and `set - o nounset`?

Check the man of `bash` and look for `nounset` - both mean the same thing.

2. Add `nounset` to your shell. Test that it demonstrates a lapse message when utilizing nonexistent variables.

```
set - u
```

Or

```
set - o nounset
```

Both lines have the same impact.

3. Deactivate `nounset`.

```
set +u
```

Or

```
set +o nounset
```

4. Execute `album/var` and `ls` in an implanted shell.

```
reverberation $(cd/var ; ls)
```

The reverberation command is used to demonstrate the consequence of the `ls` command. Precluding will make the shell to execute the first document as a command.

5. Make the variable `embvar` in an implanted shell and reverberation it.  
Does the variable exist in your present shell?

```
reverberation $(embvar=emb;echo $embvar) ; reverberation $embvar #the
last resound fizzles
```

`$embvar` does not exist in your present shell

6. Clarify what “`set - x`” does. Is this useful?

It shows shell development for investigating your command.

(Optional)

7. Given the accompanying screenshot, add four characters to the command line so that the aggregate yield is `FirstMiddleLast`.

```
[paul@RHEL4b ~]$ reverberate First; resound Middle; reverberate Last
```

```
reverberation - n First; resound - n Middle; resound Last
```

8. Show a long posting (`ls - l`) of the `passwd` command using the “`which` command” inside an implanted shell.

```
ls -l $(which passwd
```

Almost all-modern shell allows you to search command history if enabled by the user. History command can display the history list with line numbers, listed with a \* have been modified by user.

# Chapter Thirteen: SHELL HISTORY SEARCH COMMAND

Type history at a shell prompt:

```
$ history
```

Output:

```
6 du -c
7 du -ch
8 ls [01-15]*-2008
9 ls -ld [01-15]*-2008
10 ls -ld [1-15]*-2008
11 ls -ld [0]*-2008
12 ls -ld [01]*-2008
13 rm -vrf [01]*-2008
14 du -ch
15 ls
16 cd
17 umount /mnt
18 df -H
19 vnstat
20 yum update
21 vnstat -m
22 vnstat -m -i eth0
...
...
996 ping router.nixcraft.in
997 ssh vivek@p1.vpn.nixcraft.in
998 alias
999 ~/scripts/clean.rss --fetch
1000 vnstat
1001 ~/scripts/clean.rss --update
```

To search for a particular command, enter:

```
$ history | grep command-name
```

```
$ history | egrep -i 'scp|ssh|ftp'
```

## Emacs Line-Edit Mode Command History Searching

To find the command containing string, hit [CTRL]+[r] followed by search string:

(reverse-i-search):



To show the previous command, hit [CTRL] + [p]. You can also use up arrow key.

```
CTRL-p
```

To show the next command, hit [CTRL] + [n]. You can also use down arrow key.

```
CTRL-n
```

## fc command

fc stands for either the `find` command or the `fix` command. For example to list the last 10 commands, enter:

```
$ fc -l 10
```

to list commands 130 through 150, enter:

```
$ fc -l 130 150
```

to list all commands since the last command beginning with ssh, enter:

```
$ fc -l ssh
```

You can **edit commands** 1 through 5 using vi text editor, enter:

```
$ fc -e vi 1 5
```

## Delete command history

The `-c` option causes the history list to be cleared by deleting all of the entries:

```
$ history -c
```

## FILE NAME GLOBBING WITH \*, ?, [ ]

Sometimes you want to manipulate a group of files, e.g., delete all of them, without having to perform the command on each file separately. For example, suppose we want to delete all of the `.c` files in a directory. A **wildcard** is a pattern, which **matches** something else. Two commonly used \*nix wildcards are `*` (star) and `?` (question mark).

- Star (\*) means zero or more characters
- Question Mark (?) means exactly one character
- Brackets ([ ]) represent a set of characters

Commands involving filenames specified with wildcards are expanded by the shell (this is called **globbing** after the name of a former program called **glob** which used to do this outside the shell).

## The ‘\*’

‘**file\***’ will match any filename which starts with the characters “file”, and then is followed by zero or more occurrences of any character.

## Examples

Suppose **Fred's** home directory contains the files,

file01.cpp

file02.cpp

file03.cpp

file1.cpp

file01.o

file02.o

file03.o

file1.o

To delete all of the **.c** files, type,

```
$ rm *.c
```

To delete file01.cpp and file01.o,

```
$ rm file01.*
```

## The '?'

'?' Represents a single character.

### *Examples*

Consider again Fred's home directory from the previous example.

Delete file01.o, file02.o and file03.o, but not file1.o,

```
$ rm file???.o
```

Delete file01.o, but not file01.cpp,

```
$ rm file01.?
```

## The '[' Glob

A set of characters can be specified with brackets [ ]. '[ab]' means the **single** character can be 'a' OR 'b'. Ranges can also be specified (ex: '[1-57-9]' represents 1-5 OR 7-9).

### *Examples*

Delete file02.cpp and file03.cpp from Fred's directory,

```
$ rm file0[23].cpp
```

This will delete any files that start with f or F (remember linux is case sensitive),

```
$rm [fF]*
```

To delete all files that start with the string “**file**” followed by a single letter type,

```
$ rm file[a-zA-Z]
```

The a-z and A-Z in the last example means all the letters in the range lowercase a-z or uppercase A-Z.

There is much more to wildcard matching, but this is enough to get you started.

### ***More Examples***

Remove all files that are exactly 1 character,

```
$rm ?
```

Let’s say you have a directory named ‘9-15-2007-Backup-Really-Long-Name-blah...’ Rather than typing the whole name, you could just type a subset of the string and use it with the `cd` command.

```
$ cd 9-15-2007*
```

If you have multiple folders that start with 9-15-2007, your directory will be changed to the first one alphabetically.

You can use file name globbing on most commands that accept files as arguments.

# Chapter fourteen: UNIX - SHELL INPUT/OUTPUT REDIRECTIONS

Time is precious. It does not make sense to waste time typing a frequently used sequence of commands at a command prompt, more especially if they are abnormally long or complex. Scripting is a way by which one can alleviate this problem by automating these command sequences in order to make life at the shell easier and more productive. Scripting is all about making the computer, the tool, do the work. By the end of this tutorial you will have a good understanding of the kind of scripting languages available for Unix and how to apply them to your problems. UNIX contains many wonderful and strange commands that can be very useful in the world of scripting, the more tools you know and the better you know them, the more use you will find for them. Most of the Unix commands and many of the built-in commands have `man` pages; `man` pages contain the usage instructions pertaining to the parent tool. They are not always very clear and may require reading several times. In order to access a man page in Unix the following command sequence is applied

# Chapter Fifteen: UNIX SHELL FUNCTION

Functions enable you to break down the overall functionality of a script into smaller, logical subsections, which are then called upon to perform their individual tasks when needed. Using functions to perform repetitive tasks is an excellent way to create code reuse. Code reuse is an important part of modern object-oriented programming principles. Shell functions are similar to subroutines, procedures, and functions in other programming languages.

## Creating Functions

To declare a function, simply use the following syntax –

```
function_name () {
 list of commands
}
```

The name of your function is `function_name`, and that is what you will use to call it from elsewhere in your scripts. The function name must be followed by parentheses, which are followed by a list of commands enclosed within braces.

## Example

Example using function –

```
#!/bin/sh

Define your function here
Hello () {
 echo "Hello World"
}
```

```
Invoke your function
Hello
```

## Output

```
$. ./test.sh
Hello World
$
```

## Pass Parameters to a Function

You can define a function, which would accept parameters while calling those functions. These parameters would be represented by `$1`, `$2` and so on. The following is an example where we pass two parameters *Zara* and *Ali* and then we capture and print these parameters in the function.

```
#!/bin/sh
```

```
Define your function here
```

```
Hello () {
```

```
 echo "Hello World $1 $2"
```

```
}
```

```
Invoke your function
```

```
Hello Zara Ali
```

## Output

```
$/test.sh
```

```
Hello World Zara Ali
```

```
$
```

## Returning Values from Functions

If you execute an exit command from inside a function, its effect is not only to terminate execution of the function but also of the shell program that called the function. If you instead want to just terminate execution of the function, then there is a way to come out of a defined function. Based on the situation you can return any value from your function using the `return` command whose syntax is as follows –

```
return code
```

Here *code* can be anything you choose, but obviously you should choose something that is meaningful or useful in the context of your script as a whole.

## Example

Following function returns a value 1 –

```
#!/bin/sh
```

```
Define your function here
```

```
Hello () {
```

```
 echo "Hello World $1 $2"
```

```
 return 10
```

```
}
```

```
Invoke your function
```

```
Hello Zara Ali
```

```
Capture value returned by last command
```

```
ret=$?
```

```
echo "Return value is $ret"
```

## Output

```
$/test.sh
```

```
Hello World Zara Ali
```

```
Return value is 10
```

```
$
```

## Nested Functions

One of the more interesting features of functions is that they can call themselves as well as call other functions. A function that calls itself is known as a *recursive function*.

The following example demonstrates a nesting of two functions –

```
#!/bin/sh
```

```
Calling one function from another
```

```
number_one () {
```

```
 echo "This is the first function speaking..."
```

```
 number_two
```

```
}
```

```
number_two () {
```

```
 echo "This is now the second function speaking..."
```

```
}
```

```
Calling function one.
```

```
number_one
```

## Output

```
This is the first function speaking...
```

```
This is now the second function speaking...
```

## Function Call from Prompt

You can put definitions for commonly used functions inside your *.profile* so they will be available whenever you log in. Alternatively, you can group the definitions in a file, say *test.sh*, and then execute the file in the current shell by typing –

```
$. test.sh
```

This has the effect of causing any functions defined inside *test.sh* to be read in and defined to the current shell as follows –

```
$ number_one
```

```
This is the first function speaking..
This is now the second function speaking..
$
```

To remove the definition of a function from the shell, you use the `unset` command with the `.f` option. This is the same command you use to remove the definition of a variable to the shell.

```
$unset .f function_name
```

## Unix - Pipes and Filters

You can connect two commands together so that the output from one program becomes the input of the next program. Two or more commands connected in this way form a pipe. To make a pipe, put a vertical bar (`|`) on the command line between two commands. When a program takes its input from another program, performs some operation on that input, and writes the result to the standard output, it is referred to as a *filter*.

### The grep Command

The `grep` program searches a file or files for lines that have a certain pattern. The syntax is –

```
$grep pattern file(s)
```

The name “`grep`” derives from the `ed` (a UNIX line editor) command `g/re/p`, which means, “globally search for a regular expression and print all lines containing it.” A regular expression is either plain text (a word, for example) and/or special characters used for pattern matching. The simplest use of `grep` is to look for a pattern consisting of a single word. It can be used in a pipe so that only those lines of the input files containing a given string are sent to the standard output. If you do not give `grep` a filename to read, it reads its standard input; that is the way all filter programs work –

```
$ls -l | grep "Aug"
-rw-rw-rw- 1 john doc 11008 Aug 6 14:10 ch02
-rw-rw-rw- 1 john doc 8515 Aug 6 15:30 ch07
-rw-rw-r-- 1 john doc 2488 Aug 15 10:51 intro
-rw-rw-r-- 1 carol doc 1605 Aug 23 07:35 macros
$
```

Option	Description
--------	-------------

<b>-v</b>	Print all lines that do not match pattern.
-----------	--------------------------------------------

<b>-n</b>	Print the matched line and its line number.
-----------	---------------------------------------------

<b>-l</b>	Print only the names of files with matching lines (letter “l”)
-----------	----------------------------------------------------------------

There are various options, which you can use along with `grep` command –

Next, let’s use a regular expression that tells `grep` to find lines with “carol”, followed by zero or more



**-c** Print only the count of matching lines.

**-i** Match either upper- or lowercase.

other characters abbreviated in a regular expression as “.\*”), then followed by “Aug”. Here we are using *-i* option to

have case insensitive search –

```
$ls -l | grep -i "carol.*aug"
```

```
-rw-rw-r-- 1 carol doc 1605 Aug 23 07:35 macros
```

```
$
```

## The sort Command

The `sort` command arranges lines of text alphabetically or numerically. The example below sorts the lines in the `food` file –

```
$sort food
```

```
Afghani Cuisine
```

```
Bangkok Wok
```

```
Big Apple Deli
```

```
Isle of Java
```

```
Mandalay
```

```
Sushi and Sashimi
```

```
Sweet Tooth
```

```
Tio Pepe's Peppers
```

```
$
```

Option	Description
--------	-------------

<b>-n</b>	Sort numerically (example: 10 will sort after 2), ignore blanks and tabs.
-----------	---------------------------------------------------------------------------

<b>-r</b>	Reverse the order of sort.
-----------	----------------------------

<b>-f</b>	Sort upper- and lowercase together.
-----------	-------------------------------------

<b>+x</b>	Ignore first x fields when sorting.
-----------	-------------------------------------

The `sort` command arranges lines of text alphabetically by default. There are many options that control the sorting –

More than two commands may be linked by a pipe. Taking a previous pipe example using `grep`, we can further sort the files modified in August by

order of size.

The following pipe consists of the commands `ls`, `grep`, and `sort` –

```
$ls -l | grep "Aug" | sort +4n
```

```
-rw-rw-r-- 1 carol doc 1605 Aug 23 07:35 macros
```

```
-rw-rw-r-- 1 john doc 2488 Aug 15 10:51 intro
```

```
-rw-rw-rw- 1 john doc 8515 Aug 6 15:30 ch07
```

```
-rw-rw-rw- 1 john doc 11008 Aug 6 14:10 ch02
```

```
$
```

This pipe sorts all files in your directory modified in August by order of size, and

prints them to the terminal screen. The sort option `+4n` skips four fields (fields are separated by blanks) then sort the lines in numeric order.

## The `pg` and `more` Commands

A long output would normally zip by you on the screen, but if you run text through `more` or `pg` as a filter, the display stops after each screen of text. Let us assume that you have a long directory listing. To make it easier to read the sorted listing, pipe the output through `more` as follows –

```
$ls -l | grep "Aug" | sort +4n | more
-rw-rw-r-- 1 carol doc 1605 Aug 23 07:35 macros
-rw-rw-r-- 1 john doc 2488 Aug 15 10:51 intro
-rw-rw-rw- 1 john doc 8515 Aug 6 15:30 ch07
-rw-rw-r-- 1 john doc 14827 Aug 9 12:40 ch03
.
.
.
-rw-rw-rw- 1 john doc 16867 Aug 6 15:56 ch05
--More--(74%)
```

The screen will fill up with one screen of text consisting of lines sorted by order of file size. At the bottom of the screen is the `more` prompt where you can type a command to move through the sorted text. When you are done with this screen, you can use any of the commands listed in the discussion of the `more` program.

# Chapter Sixteen: UNIX USEFUL COMMAND

This quick guide lists commands, including a syntax and brief description. For more detailed information check out [commands man page](#).

```
$man command
```

## Files and Directories

These commands allow you to create directories and handle files.

Command	Description
<b>cat</b>	Display File Contents
<b>cd</b>	Changes Directory to dirname
<b>chgrp</b>	Change file group
<b>chmod</b>	Changing Permissions
<b>cp</b>	Copy source file into destination
<b>file</b>	Determine file type
<b>find</b>	Find files
<b>grep</b>	Search files for regular expressions.
<b>head</b>	Display first few lines of a file
<b>ln</b>	Create softlink on oldname
<b>ls</b>	Display information about file type.
<b>mkdir</b>	Create a new directory dirname
<b>more</b>	Display data in paginated form.
<b>mv</b>	Move (Rename) a oldname to newname.
<b>pwd</b>	Print current working directory.
<b>rm</b>	Remove (Delete) filename

<b>rmdir</b>	Delete an existing directory provided it is empty.
--------------	----------------------------------------------------

<b>tail</b>	Prints last few lines in a file.
-------------	----------------------------------

<b>touch</b>	Update access and modification time of a file.
--------------	------------------------------------------------

Command	Description
---------	-------------

<b>awk</b>	Pattern scanning and processing language
------------	------------------------------------------

<b>cmp</b>	Compare the contents of two files
------------	-----------------------------------

<b>comm.</b>	Compare sorted data
--------------	---------------------

<b>cut</b>	Cut out selected fields of each line of a file
------------	------------------------------------------------

<b>diff</b>	Differential file comparator
-------------	------------------------------

<b>expand</b>	Expand tabs to spaces
---------------	-----------------------

<b>join</b>	Join files on some common field
-------------	---------------------------------

<b>Perl</b>	Data manipulation language
-------------	----------------------------

<b>sed</b>	Stream text editor
------------	--------------------

<b>sort</b>	Sort file data
-------------	----------------

<b>split</b>	Split file into smaller files
--------------	-------------------------------

<b>tr</b>	Translate characters
-----------	----------------------

<b>uniq</b>	Report repeated lines in a file
-------------	---------------------------------

<b>wc</b>	Count words, lines, and characters
-----------	------------------------------------

<b>vi</b>	Opens vi text editor
-----------	----------------------

<b>vim</b>	Opens vim text editor
------------	-----------------------

<b>fmt</b>	Simple text formatter
------------	-----------------------

<b>spell</b>	Check text for spelling error
--------------	-------------------------------

## Manipulating data

The contents of files can be compared and altered with the following commands.

## Compressed Files

Files may be compressed to save space. Compressed files can be created and examined -

<b>ispell</b>	Check text for spelling error
---------------	-------------------------------

<b>Emacs</b>	GNU project Emacs
--------------	-------------------

<b>ex, edit</b>	Line editor
-----------------	-------------

<b>Emacs</b>	GNU project Emacs
--------------	-------------------

---

Command	Description
---------	-------------

<b>gzip</b>	Compress files
-------------	----------------

<b>gunzip</b>	Uncompress gzipped files
---------------	--------------------------

<b>gzip</b>	GNU alternative compression method
-------------	------------------------------------

<b>uncompress</b>	Uncompress files
-------------------	------------------

<b>unzip</b>	List, test and extract compressed files in a ZIP archive
--------------	----------------------------------------------------------

<b>zcat</b>	Cat a compressed file
-------------	-----------------------

<b>zcmp</b>	Compare compressed files
-------------	--------------------------

<b>zdiff</b>	Compare compressed files
--------------	--------------------------

<b>zmore</b>	File perusal filter for crt viewing of compressed text
--------------	--------------------------------------------------------

## Getting Information

Various Unix manuals and documentation are available on-line. The following Shell commands give information –

---

Command	Description
---------	-------------

<b>apropos</b>	Locate commands by keyword lookup
----------------	-----------------------------------

<b>info</b>	Displays command information pages online
-------------	-------------------------------------------

---

<b>man</b>	Displays manual pages online
<b>whatis</b>	Search the whatis database for complete words.
<b>yelp</b>	GNOME help viewer

## Network Communication

The following commands are used to send and receive files from a local UNIX hosts to the remote hosts around the world.

Command	Description
<b>ftp</b>	File transfer program
<b>rcp</b>	Remote file copy
<b>rlogin</b>	Remote login to a UNIX host
<b>rsh</b>	Remote shell
<b>tftp</b>	Trivial file transfer program
<b>telnet</b>	Make terminal connection to another host
<b>ssh</b>	Secure shell terminal or command connection
<b>scp</b>	Secure shell remote file copy
<b>sftp</b>	secure shell file transfer program

Some of these commands may be restricted on your computer for security reasons.

## Messages between Users

The UNIX system supports on-screen messaging to other users and worldwide electronic mail –

Command	Description
<b>Evolution</b>	GUI mail handling tool on Linux
<b>mail</b>	Simple send or read mail program
<b>mesg</b>	Permit or deny messages
<b>parcel</b>	Send files to another user
<b>Pine</b>	Vdu-based mail utility

<b>Talk</b>	Talk to another user
-------------	----------------------

<b>write</b>	Write message to another user
--------------	-------------------------------

## Programming Utilities

The following programming tools and languages are available based on which version of Unix you are using

Command	Description
<b>dbx</b>	Sun debugger
<b>gdb</b>	GNU debugger
<b>make</b>	Maintain program groups and compile programs.
<b>nm</b>	Print program's name list
<b>size</b>	Print program's sizes
<b>strip</b>	Remove symbol table and relocation bits
<b>cb</b>	C program beautifier
<b>cc</b>	ANSI C compiler for Suns SPARC systems
<b>Ctrace</b>	C program debugger
<b>Gcc</b>	GNU ANSI C Compiler
<b>Indent</b>	Indent and format C program source
<b>Bc</b>	Interactive arithmetic language processor
<b>Gcl</b>	GNU Common Lisp
<b>Perl</b>	General purpose language
<b>Php</b>	Web page embedded language

<b>Py</b>	Python language interpreter
<b>Asp</b>	Web page embedded language
<b>CC</b>	C++ compiler for Suns SPARC systems
<b>g++</b>	GNU C++ Compiler
<b>Javac</b>	JAVA compiler
<b>appletvieweir</b>	JAVA applet viewer
<b>Netbeans</b>	Java integrated development environment on Linux
<b>Sqlplus</b>	Run the Oracle SQL interpreter
<b>Sqlldr</b>	Run the Oracle SQL data loader
<b>MySql</b>	Run the mysql SQL interpreter

## Misc Commands

These commands list or alter system information –

td>groups

Command	Description
<b>chfn</b>	Change your finger information
<b>chgrp</b>	Change the group ownership of a file
<b>chown</b>	Change owner
<b>date</b>	Print the date
<b>determin</b>	Automatically find terminal type
<b>du</b>	Print amount of disk usage
<b>echo</b>	Echo arguments to the standard options
<b>exit</b>	Quit the system



<b>finger</b>	Print information about logged-in users
<b>groupadd</b>	Create a user group
<b>Show group memberships</b>	
<b>homequota</b>	Show quota and file usage
<b>iostat</b>	Report I/O statistics
<b>kill</b>	Send a signal to a process
<b>last</b>	Show last logins of users
<b>logout</b>	log off UNIX
<b>lun</b>	List user names or login ID
<b>netstat</b>	Show network status
<b>passwd</b>	Change user password
<b>passwd</b>	Change your login password
<b>printenv</b>	Display value of a shell variable
<b>ps</b>	Display the status of current processes
<b>ps</b>	Print process status statistics
<b>quota -v</b>	Display disk usage and limits
<b>reset</b>	Reset terminal mode
<b>script</b>	Keep script of terminal session
<b>script</b>	Save the output of a command or process
<b>setenv</b>	Set environment variables
<b>sty</b>	Set terminal options

<b>time</b>	Time a command
<b>top</b>	Display all system processes
<b>tset</b>	Set terminal mode
<b>tty</b>	Print current terminal name
<b>umask</b>	Show the permissions that are given to view files by default
<b>uname</b>	Display name of the current system
<b>uptime</b>	Get the system up time
<b>useradd</b>	Create a user account
<b>users</b>	Print names of logged in users
<b>vmstat</b>	Report virtual memory statistics
<b>w</b>	Show what logged in users are doing
<b>who</b>	List logged in users

# Chapter Seventeen: REGULAR EXPRESSION

A regular expression is a string that can be used to describe several sequences of characters. Regular expressions are used by several different Unix commands, including `ed`, `sed`, `awk`, `grep`, and, to a more limited extent, `vi`. Here `sed` stands for **s**tream **e**ditor a stream oriented editor, which was created exclusively for executing scripts. Thus, all the input you feed into it passes through and goes to `STDOUT` and it does not change the input file.

## Invoking sed

Before we start, let us take make sure you have a local copy of `/etc/passwd` text file to work with `sed`. As mentioned previously, `sed` can be invoked by sending data through a pipe to it as follows –

```
$ cat /etc/passwd | sed
```

```
Usage: sed [OPTION]... {script-other-script} [input-file]...
```

```
-n, --quiet, --silent
```

```
suppress automatic printing of pattern space
```

```
-e script, --expression=script
```

The `cat` command dumps the contents of `/etc/passwd` to `sed` through the pipe into `sed`'s pattern space. The pattern space is the internal work buffer that `sed` uses to do its work.

## The sed General Syntax

General syntax for `sed`

```
/pattern/action
```

Here, **pattern** is a regular expression, and **action** is one of the commands given in the following table. If **pattern** is omitted, **action** is performed for every line as we have seen above.

The slash characters ( `/` ) that surround the pattern are required because they are used as delimiters.

Range	Description
<b>P</b>	Prints the line
<b>D</b>	Deletes the line
<b>s/pattern1/pattern2/</b>	Substitutes the first occurrence of

## Deleting All Lines with sed

Invoke `sed` again, but this time tell `sed` to use the editing command delete line, denoted by the single letter `d` –

```
$ cat /etc/passwd | sed 'd'
$
```

Instead of invoking `sed` by sending a file to it through a pipe, you can instruct `sed` to read the data from a file, as in the following example. The following command does exactly the same thing as the previous example, without the `cat` command –

```
$ sed -e 'd' /etc/passwd
$
```

## The sed Addresses

`sed` also understands something called addresses. Addresses are either particular locations in a file or a range where a particular editing command should be applied. When `sed` encounters no addresses, it performs its operations on every line in the file.

The following command adds a basic address to the `sed` command you have been using –

```
$ cat /etc/passwd | sed '1d' | more
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
$
```

Notice that the number `1` is added before the delete edit command. This tells `sed` to perform the editing command on the first line of the file. In this example, `sed` will delete the first line of `/etc/passwd` and print the rest of the file.

## The sed Address Ranges

So, what if you want to remove more than one line from a file? You can specify an

address range with `sed` as follows –

```
$ cat /etc/passwd | sed '1, 5d' | more
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
$
```

The above command will be applied on all the lines starting from 1 through 5. Therefore, it deletes the first five lines.

Try out the following address ranges –

Range	Description
<b>'4,10d'</b>	Lines starting from 4th till 10th are deleted
<b>'10,4d'</b>	Only 10th line is deleted, because <code>sed</code> does not work in reverse direction.
<b>'4,+5d'</b>	This will match line 4 in the file, delete that line, continue to delete the next five lines, and then cease its deletion and print the rest
<b>'2,5!d'</b>	This will delete everything except starting from 2nd till 5th line.
<b>'1~3d'</b>	This deletes the first line, steps over the next three lines, and then deletes the fourth line. <code>Sed</code> continues applying this pattern until the end of the file.
<b>'2~2d'</b>	This tells <code>sed</code> to delete the second line, step over the next line, delete the next line, and repeat until the end of the file is reached.
<b>'4,10p'</b>	Lines starting from 4th till 10th are printed
<b>'4,d'</b>	This would generate syntax error.
<b>'10d'</b>	This would also generate syntax error.

**Note:** While using `p` action, you should use `-n` option to avoid repetition of line printing. Check the difference between these two commands –

```
$ cat /etc/passwd | sed -n '1,3p'
```

Check the above command without `-n` as follows –

```
$ cat /etc/passwd | sed '1,3p'
```

## The Substitution Command

The substitution command, denoted by `s`, will substitute any string that you specify with any other string that you specify. To substitute one string with another, you need to have a way of telling `sed` where your first string ends and the substitution string begins. This is traditionally done by bookending the two strings with the forward slash (`/`) character. The following command substitutes the first occurrence on a line of the string **root** with the string **amrood**.

```
$ cat /etc/passwd | sed 's/root/amrood/'
```

```
amrood:x:0:0:root user:/root:/bin/sh
```

```
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
```

It is very important to note that `sed` substitutes only the first occurrence on a line. If the string `root` occurs more than once on a line, only the first match will be replaced. To tell `sed` to do a global substitution, add the letter `g` to the end of the command as follows –

```
$ cat /etc/passwd | sed 's/root/amrood/g'
```

```
amrood:x:0:0:amrood user:/amrood:/bin/sh
```

```
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
```

```
bin:x:2:2:bin:/bin:/bin/sh
```

```
sys:x:3:3:sys:/dev:/bin/sh
```

## Substitution Flags

Flag	Description
<code>g</code>	Replace all matches, not just the first match.
<code>NUMBER</code>	Replace only <code>NUMBER</code> th match.
<code>p</code>	If substitution was made, print pattern space.
<code>w</code> <code>FILENAME</code>	If substitution was made, write result to <code>FILENAME</code> .

There are a number of other useful flags that can be passed in addition to the `g` flag, and you can specify more than one at a time.

### Using an Alternative String Separator

You may find yourself having to do a substitution on a string that includes the forward slash character. In this case, you can specify a different separator by providing the designated character after

I or i	Match in a case-insensitive manner.	the s. \$ cat /etc/passwd   sed 's:/root:/amrood:g'
M or m	In addition to the normal behavior of the special regular expression characters ^ and \$, this flag causes ^ to match the empty string after a newline and \$ to match the empty string before a newline.	amrood:x:0:0:amrood user:/amrood:/bin/sh
		daemon:x:1:1:daemon:/usr/sbin:/bin/sh

In the above example we used: as delimiter instead of slash / because we were trying to search /root instead of simple root.

## Replacing with Empty Space

Use an empty substitution string to delete the root string from the /etc/passwd file entirely –

```
$ cat /etc/passwd | sed 's/root//g'
:x:0:0:::/bin/sh
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
```

## Address Substitution

If you want to substitute the string 'sh' with the string 'quiet' on line 10, you can specify it as follows –

```
$ cat /etc/passwd | sed '10s/sh/quiet/g'
root:x:0:0:root user:/root:/bin/sh
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/quiet
```

Similarly, to do an address range substitution, you could do something like this –

```
$ cat /etc/passwd | sed '1,5s/sh/quiet/g'
root:x:0:0:root user:/root:/bin/quiet
```

```
daemon:x:1:1:daemon:/usr/sbin:/bin/quiet
bin:x:2:2:bin:/bin:/bin/quiet
sys:x:3:3:sys:/dev:/bin/quiet
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
```

As you can see from the output, the first five lines had the string 'sh' changed to 'quiet,' but the rest of the lines were left untouched.

## The Matching Command

You would use `p` option along with `-n` option to print all the matching lines as follows –

```
$ cat testing | sed -n '/root/p'
root:x:0:0:root user:/root:/bin/sh
[root@ip-72-167-112-17 amrood]# vi testing
root:x:0:0:root user:/root:/bin/sh
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
```

## Using Regular Expression

While matching patterns, you can use regular expressions, which provide greater flexibility. Check the following example, which matches all the lines starting with *daemon* and then deletes them –

```
$ cat testing | sed '/^daemon/d'
root:x:0:0:root user:/root:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
```



```
backup:x:34:34:backup:/var/backups:/bin/sh
```

Example that deletes all lines ending with **sh** –

```
$ cat testing | sed '/sh$/d'
```

```
sync:x:4:65534:sync:/bin:/bin/sync
```

The following table lists four special characters that are very useful in regular expressions.

Character	Description
<b>^</b>	Matches the beginning of lines.
<b>\$</b>	Matches the end of lines.
<b>.</b>	Matches any single character.
<b>*</b>	Matches zero or more occurrences of the previous character
<b>[chars]</b>	Matches any one of the characters given in chars, where chars is a sequence of characters. You can use the - character to indicate a range of characters.

## Matching Characters

Expression	Description
<b>/a.c/</b>	Matches lines that contain strings such as a+c, a-c, abc, match, and a3c, whereas the pattern
<b>/a*c/</b>	Matches the same strings along with strings such as ace, yacc, and arctic.
<b>/[tT]he/</b>	Matches the string The and the:
<b>/^\$/</b>	Matches Blank lines
<b>/^.*\$/</b>	Matches an entire line whatever it is.
<b>/ */</b>	Matches one or more spaces
<b>/^\$/</b>	Matches Blank lines

Expressions to demonstrate the use of metacharacters.

The following table shows some frequently used sets of characters –

## Character Class Keywords

Some special keywords are commonly available to regexps, especially GNU utilities that employ regexps. These are very useful for sed regular expressions as they simplify things and enhance readability.

For example, the characters a

Set	Description
[a-z]	Matches a single lowercase letter
[A-Z]	Matches a single uppercase letter
[a-zA-Z]	Matches a single letter
[0-9]	Matches a single number
[a-zA-Z0-9]	Matches a single letter or number

through z as well as the characters A through Z constitute one such class of characters that has the keyword `[:alpha:]` Using the alphabet character class keyword, this command prints only those lines in the `/etc/syslog.conf` file that start with a letter of the alphabet –

```
$ cat /etc/syslog.conf | sed -n '/^[:alpha:]/p'
authpriv.* /var/log/secure
mail.* -/var/log/maillog
cron.* /var/log/cron
uucp,news.crit /var/log/spooler
local7.* /var/log/boot.log
```

Character Class	Description
[:alnum:]	Alphanumeric [a-z A-Z 0-9]
[:alpha:]	Alphabetic [a-z A-Z]
[:blank:]	Blank characters (spaces or tabs)
[:cntrl:]	Control characters
[:digit:]	Numbers [0-9]
[:graph:]	Any visible characters (excludes whitespace)
[:lower:]	Lowercase letters [a-z]
[:print:]	Printable characters (noncontrol characters)
[:punct:]	Punctuation characters
[:space:]	Whitespace
[:upper:]	Uppercase letters [A-Z]

Complete list of available character class keywords in GNU sed.

## Ampersand Referencing

The sed metacharacter `&` represents the contents of the pattern that was matched. For instance, you have a file called `phone.txt` full of phone numbers –

```
5555551212
5555551213
5555551214
6665551215
6665551216
7775551217
```

You want to make the area code (the first three digits) surrounded by parentheses for easier reading. To do this, you can use the ampersand replacement character, like so

**[[:xdigit:]]** Hex digits [0-9 a-f A-F]

—

```
$ sed -e 's/^[[[:digit:]]][[:digit:]]][[:digit:]]/(&)/g' phone.txt
(555)5551212
(555)5551213
(555)5551214
(666)5551215
(666)5551216
(777)5551217
```

Here you are matching the first 3 digits and then using & replacing those 3 digits with surrounding parentheses.

## Using Multiple sed Commands

You can use multiple `sed` commands in a single `sed` command as follows –

```
$ sed -e 'command1' -e 'command2' ... -e 'commandN' files
```

Here `command1` through `commandN` are `sed` commands of the type discussed previously. These commands are applied to each line in the list of files given by `files`. Using the same mechanism, we can write above phone number example as follows –

```
$ sed -e 's/^[[[:digit:]]]\{3\}/(&)/g' \
 -e 's/)[[:digit:]]\{3\}/&-/g' phone.txt
(555)555-1212
(555)555-1213
(555)555-1214
(666)555-1215
(666)555-1216
(777)555-1217
```

**Note** – In the above example, instead of repeating the character class keyword `[[[:digit:]]]` three times, you replaced it with `\{3\}`, which means to match the preceding regular expression three times. Here I used `\` to give line break you should remove this before running this command.

## Back References

The ampersand metacharacter is useful, but even more useful is the ability to define specific regions in a regular expression so you can reference them in your replacement strings. By defining specific parts of a regular expression, you can then refer back to those parts with a special reference character. To do back references, you have to first define a region and then refer back to that region. To define a region you insert backslash parentheses around each region of interest. The first region that you surround with backslashes is then referenced by `\1`, the second region by `\2`, and so on.

Assuming phone.txt has the following text –

```
(555)555-1212
```

```
(555)555-1213
```

```
(555)555-1214
```

```
(666)555-1215
```

```
(666)555-1216
```

```
(777)555-1217
```

Now try the following command –

```
$ cat phone.txt | sed 's/\(.*\)\\\(.*-\\\)\\(.*$\\)/Area \
 code: \1 Second: \2 Third: \3/'
```

```
Area code: (555) Second: 555- Third: 1212
```

```
Area code: (555) Second: 555- Third: 1213
```

```
Area code: (555) Second: 555- Third: 1214
```

```
Area code: (666) Second: 555- Third: 1215
```

```
Area code: (666) Second: 555- Third: 1216
```

```
Area code: (777) Second: 555- Third: 1217
```

**Note:** In the above example each regular expression inside the parenthesis would be back referenced by \1, \2 and so on. Here I used \ to give line break you should remove this before running this command.

# Chapter Eighteen: FILE SYSTEM BASICS

A file system is a logical collection of files on a partition or disk. A partition is a container for information and can span an entire hard drive if desired. Your hard drive can have various partitions which usually contain only one file system, such as one file system housing the `/` file system or another containing the `/home` file system. One file system per partition allows for the logical maintenance and management of differing file systems. Everything in Unix is considered to be a file, including physical devices such as DVD-ROMs, USB devices, floppy drives, and so forth.

## Directory Structure

Unix uses a hierarchical file system structure, much like an upside-down tree, with root (`/`) at the base of the file system and all other directories spreading from there. A UNIX file system is a collection of files and directories that have the following properties –

- It has a root directory (`/`) that contains other files and directories.
- Each file or directory is uniquely identified by its name, the directory in which it resides, and a unique identifier, typically called an inode.
- By convention, the root directory has an inode number of 2 and the lost+found directory has an inode number of 3. Inode numbers 0 and 1 are not used. File inode numbers can be seen by specifying the `-i` option to `ls` command.
- It is self-contained. There are no dependencies between one file system and any other.

The directories have specific purposes and generally hold the same types of information for easily locating files. The following are the directories that exist on the major versions of Unix –

Directory	Description
<code>/</code>	This is the root directory, which should contain only the directories needed at the top level of the file structure.
<code>/bin</code>	This is where the executable files are located. They are available to all users.
<code>/dev</code>	These are device drivers.

<b>/etc</b>	Supervisor directory commands, configuration files, disk configuration files, valid user lists, groups, ethernet, hosts, where to send critical messages.
<b>/lib</b>	Contains shared library files and sometimes other kernel-related files.
<b>/boot</b>	Contains files for booting the system.
<b>/home</b>	Contains the home directory for users and other accounts.
<b>/mnt</b>	Used to mount other temporary file systems, such as cdrom and floppy for the CD-ROM drive and floppy diskette drive, respectively.
<b>/proc</b>	Contains all processes marked as a file by process number or other information that is dynamic to the system.
<b>/tmp</b>	Holds temporary files used between system boots.
<b>/usr</b>	Used for miscellaneous purposes, can be used by many users. Includes administrative commands, shared files, library files, and others.
<b>/var</b>	Typically contains variable-length files such as log and print files and other file types that may contain a variable amount of data.
<b>/sbin</b>	Contains binary (executable) files, usually for system administration. For example <i>fdisk</i> and <i>ifconfig</i> utilities.
<b>/kernel</b>	Contains kernel files

---

## Navigating the File System

Now that you understand the basics of the file system, you can begin navigating to the files you need. The following are commands you will use to navigate the

Command	Description
<b>cat filename</b>	Displays a filename.
<b>cd dirname</b>	Moves you to the directory identified.
<b>cp file1 file2</b>	Copies one file/directory to specified location.
<b>file filename</b>	Identifies the file type (binary, text, etc).
<b>find filename dir</b>	Finds a file/directory.
<b>head filename</b>	Shows the beginning of a file.
<b>less filename</b>	Browses through a file from end or beginning.
<b>ls dirname</b>	Shows the contents of the directory specified.
<b>mkdir dirname</b>	Creates the specified directory.
<b>more filename</b>	Browses through a file from beginning to end.
<b>mv file1 file2</b>	Moves the location of or renames a file/directory.
<b>pwd</b>	Shows the current directory the user is in.
<b>rm filename</b>	Removes a file.
<b>rmdir dirname</b>	Removes a directory.

<b>tail filename</b>	Shows the end of a file.
<b>touch filename</b>	Creates a blank file or modifies an existing file.s attributes.
<b>whereis filename</b>	Shows the location of a file.
<b>which filename</b>	Shows the location of a file if it is in your PATH.

---

You can use [Manpage Help](#) to check complete syntax for each of these commands.

## The df Command

The first way to manage your partition space is with the `df` (disk free) command. The command `df -k` (disk free) displays the disk space usage in kilobytes, as shown below –

```
$df -k
```

Filesystem	1K-blocks	Used	Available	Use%	Mounted on
/dev/vzfs	10485760	7836644	2649116	75%	/
/devices	0	0	0	0%	/devices

```
$
```

Some of the directories, such as `/devices`, shows `0` in the `kbytes`, `used`, and `avail` columns as well as `0%` for capacity. These are special (or virtual) file systems, and although they reside on the disk under `/`, by themselves they do not take up disk space.

The `df -k` output is generally the same on all Unix systems. Here's what it usually includes –

---

Column	Description
<b>Filesystem</b>	The physical file system name.
<b>Kbytes</b>	Total kilobytes of space available on the storage medium.

---



<b>Used</b>	Total kilobytes of space used (by files).
<b>Avail</b>	Total kilobytes available for use.
<b>Capacity</b>	Percentage of total space used by files.
<b>Mounted on</b>	What the file system is mounted on.

---

You can use the `-h` (human readable) option to display the output in a format that shows the size in easier-to-understand notation.

## The du Command

The `du` (disk usage) command enables you to specify directories to show disk space usage on a particular directory. This command is helpful if you want to determine how much space a particular directory is using. The following command will display the number of blocks used by each directory. A single block may take either 512 Bytes or 1 Kilo Byte depending on your system.

```
$du /etc
```

```
10 /etc/cron.d
```

```
126 /etc/default
```

```
6 /etc/dfs
```

```
...
```

```
$
```

The `-h` option makes the output easier to comprehend –

```
$du -h /etc
```

```
5k /etc/cron.d
```

```
63k /etc/default
```

```
3k /etc/dfs
```

```
...
```

```
$
```

## Mounting the File System

A file system must be mounted in order to be usable by the system. To see what is currently mounted (available for use) on your system, use this command –

```
$ mount
```

```
/dev/vzfs on / type reiserfs (rw,usrquota,grpquota)
```

```
proc on /proc type proc (rw,nodiratime)
```

```
devpts on /dev/pts type devpts (rw)
```

```
$
```

The `/mnt` directory, by Unix convention, is where temporary mounts (such as CD-ROM drives, remote network drives, and floppy drives) are located. If you need to mount a file system, you can use the `mount` command with the following syntax –

```
mount -t file_system_type device_to_mount directory_to_mount_to
```

For example, if you want to mount a CD-ROM to the directory `/mnt/cdrom`, for example, you can type –

```
$ mount -t iso9660 /dev/cdrom /mnt/cdrom
```

This assumes that your CD-ROM device is called `/dev/cdrom` and that you want to mount it to `/mnt/cdrom`. Refer to the `mount` man page for more specific information or type `mount -h` at the command line for help information. After mounting, you can use the `cd` command to navigate the newly available file system through the mountpoint you just created.

## Unmounting the File System

To unmount (remove) the file system from your system, use the `umount` command by identifying the mountpoint or device. For example, to unmount `cdrom`, use the following command –

```
$ umount /dev/cdrom
```

The `mount` command enables you to access your file systems, but on most modern Unix systems, the automount function makes this process invisible to the user and requires no intervention.

## User and Group Quotas

User and group quotas provide the mechanisms by which the amount of space used by a single user or all users within a specific group can be limited to a value defined by the administrator. Quotas operate around two limits that allow the user to take some action if the amount of space, or number of disk blocks start to reach the administrator defined limits –

- **Soft Limit** – If the user exceeds the limit defined, there is a grace period that allows the user to free up some space.
- **Hard Limit** – When the hard limit is reached, regardless of the grace period, no further files or blocks can be allocated.

There are a number of commands to administer quotas –

Command	Description
<b>quota</b>	Displays disk usage and limits for a user of group.

**edquota** This is a quota editor. Users or Groups quota can be edited using this command.

**quotacheck** Scan a file system for disk usage, create, check and repair quota files

**setquota** This is also a command line quota editor.

**quotaon** This announces to the system that disk quotas should be enabled on one or more file systems.

**quotaoff** This announces to the system that disk quotas should be disabled off one or more file systems.

**repquota** This prints a summary of the disc usage and quotas for the specified file systems

# Chapter Nineteen: UNIX-USER ADMINISTRATION

There are three types of accounts on Unix system –

- **Root account** – also called superuser and has complete and unfettered control of the system. A superuser can run any commands without any restriction. This user should be assumed as a system administrator.
- **System accounts** – System accounts are those needed for the operation of system-specific components for example mail accounts and the sshd accounts. These accounts are usually needed for some specific function on your system, and any modifications to them could adversely affect the system.
- **User accounts** – User accounts provide interactive access to the system for users and groups of users. General users are typically assigned to these accounts and usually have limited access to critical system files and directories.

UNIX supports a concept of *Group Account* , which logically groups a number of accounts. Every account would be a part of any group account. Unix groups plays important role in handling file permissions and process management.

## Managing Users and Groups

There are three main user administration files –

- **/etc/passwd:** – Keeps user account and password information. This file holds the majority of information about accounts on the Unix system.
- **/etc/shadow:** – Holds the encrypted password of the corresponding account. Not all systems support this file.
- **/etc/group:** – Contains group information for each account.
- **/etc/gshadow:** – Contains secure group account information.

Check all the above files using `cat` command.

Commands available on the majority of Unix systems to create and manage accounts and groups –

Command	Description
<b>useradd</b>	Adds accounts to the system.
<b>usermod</b>	Modifies account attributes.
<b>userdel</b>	Deletes accounts from the system.
<b>groupadd</b>	Adds groups to the system.
<b>groupmod</b>	Modifies group attributes.
<b>groupdel</b>	Removes groups from the system.

### Create a Group

You need to create groups before creating any account otherwise you would have to use existing groups on your system. You have all the groups listed in `/etc/groups` file. All the default groups would be system account specific groups and it is not recommended to use them for ordinary accounts. Use the following syntax to create a new group account –

```
groupadd [-g gid [-o]] [-r] [-f] groupname
```

Here are the details of the parameters:

Option	Description
<b>-g GID</b>	The numerical value of the group's ID.
<b>-o</b>	This option permits to add group with non-unique GID
<b>-r</b>	This flag instructs groupadd to add a system account
<b>-f</b>	This option causes to just exit with success status if the specified group already exists. With -g, if specified GID already exists, other (unique) GID is chosen

<b>groupname</b>	Actual group name to be created.
------------------	----------------------------------

If you do not specify any parameters then the system will use the default values.

The following example will create a *developers* group with default values, which is acceptable for most administrators.

```
$ groupadd developers
```

## Modify a Group

To modify a group, use the `groupmod` syntax –

```
$ groupmod -n new_modified_group_name old_group_name
```

To change the `developers_2` group name to `developer`, type –

```
$ groupmod -n developer developer_2
```

Here is how you would change the financial GID to 545 –

```
$ groupmod -g 545 developer
```

## Delete a Group:

To delete an existing group, all you need are the `groupdel` command and the group name. To delete the `developer` group, the command is –

```
$ groupdel developer
```

This removes only the group, not any files associated with that group. The files are still accessible by their owners.

## Create an Account

Let us see how to create a new account on your Unix system. Use the syntax to create a user's account –

```
useradd -d homedir -g groupname -m -s shell -u userid accountname
```

Available Parameters –

Option	Description
-d homedir	Specifies home directory for the account.

-g groupname	Specifies a group account for this account.
-m	Creates the home directory if it doesn't exist.
-s shell	Specifies the default shell for this account.
-u userid	You can specify a user id for this account.
accountname	Actual account name to be created

If you do not specify a parameter then system will use the default values. The `useradd` command modifies the `/etc/passwd`, `/etc/shadow`, and `/etc/group` files and creates a home directory. The following is an example which will create an account *mcmohd* setting its home directory to */home/mcmohd* and group as *developers*. This user would have Korn Shell assigned to it.

```
$ useradd -d /home/mcmohd -g developers -s /bin/ksh mcmohd
```

Before issuing the above command, make sure you already have a *developers* group created using *groupadd* command. Once an account is created you can set its password using the **passwd** command as follows –

```
$ passwd mcmohd20
```

```
Changing password for user mcmohd20.
```

```
New UNIX password:
```

```
Retype new UNIX password:
```

```
passwd: all authentication tokens updated successfully.
```

When you type *passwd accountname*, it gives you option to change the password provided you are superuser otherwise you would be able to change just your password using the same command but without specifying your account name.

## Modify an Account

The `usermod` command enables you to make changes to an existing account

from the command line. It uses the same arguments as the `useradd` command, plus the `-l` argument, which allows you to change the account name. For example, to change the account name *mcmohd* to *mcmohd20* and to change home directory accordingly, you would need to issue following command –

```
$ usermod -d /home/mcmohd20 -m -l mcmohd mcmohd20
```

## Delete an Account

The `userdel` command can be used to delete an existing user. This is a very dangerous command if not used with caution. There is only one argument or option available for the command: `-r`, for removing the account's home directory and mail file. For example, to remove account *mcmohd20*, you would need to issue following command –

```
$ userdel -r mcmohd20
```

If you want to keep the home directory for backup purposes, omit the `-r` option. You can remove the home directory as needed at a later time.



# Chapter Twenty: SYSTEM PERFORMANCE

The purpose of this tutorial is to introduce the performance analyst to some of the free tools available to monitor and manage performance on UNIX systems, and to provide a guideline on how to diagnose and fix performance problems in a Unix environment. UNIX has following major resource types that need to be monitored and tuned –

- CPU
- Memory
- Disk space
- Communications lines
- I/O Time
- Network Time
- Applications programs

## Performance Components

There are five major components where total system time goes –

Component	Description
<b>User state CPU</b>	The actual amount of time the CPU spends running the users program in the user state. It includes time spent executing library calls, but does not include time spent in the kernel on its behalf.
<b>System state CPU</b>	This is the amount of time the CPU spends in the system state on behalf of this program. All I/O routines require kernel services. The programmer can affect this value by the use of blocking for I/O transfers.
<b>I/O Time and Network Time</b>	These are the amount of time spent moving data and servicing I/O requests

## Virtual Memory Performance

This includes context switching and swapping.

## Application Program

Time spent running other programs - when the system is not servicing this application because another application currently has the CPU.

Command	Description
---------	-------------

<b>nice/renice</b>	Run a program with modified scheduling priority
--------------------	-------------------------------------------------

<b>netstat</b>	Print network connections, routing tables, interface statistics, masquerade connections, and multicast memberships
----------------	--------------------------------------------------------------------------------------------------------------------

<b>time</b>	Time a simple command or give resource usage
-------------	----------------------------------------------

<b>uptime</b>	System Load Average
---------------	---------------------

<b>ps</b>	Report a snapshot of the current processes.
-----------	---------------------------------------------

<b>vmstat</b>	Report virtual memory statistics
---------------	----------------------------------

<b>gprof</b>	Display call graph profile data
--------------	---------------------------------

<b>prof</b>	Process Profiling
-------------	-------------------

<b>top</b>	Display system tasks
------------	----------------------

## Performance Tools

Unix provides the following important tools to measure and fine-tune Unix system performance –

## Unix - System Logging

UNIX systems have a very flexible and powerful logging system, which enables you to record almost anything and then manipulate the logs to retrieve the information you require. Many versions of UNIX provide a general-purpose logging facility called syslog. Individual programs that need to have information logged send the information to syslog.

Unix syslog is a host-configurable, uniform system logging facility. The system uses a centralized system logging process that runs the program **/etc/syslogd** or **/etc/syslog**.

The operation of the system logger is quite straightforward. Programs send their log entries to syslogd, which consults the configuration file **/etc/syslogd.conf** or **/etc/syslog** and, when a match is found, writes the log message to the desired log file.

Term	Description
------	-------------

There are four basic syslog

<b>Facility</b>	The identifier used to describe the application or process that submitted the log message. Examples are mail, kernel, and ftp.
<b>Priority</b>	An indicator of the importance of the message. Levels are defined within syslog as guidelines, from debugging information to critical events.
<b>Selector</b>	A combination of one or more facilities and levels. When an incoming event matches a selector, an action is performed.
<b>Action</b>	What happens to an incoming message that matches a selector. Actions can write the message to a log file, echo the message to a console or other device, write the message to a logged in user, or send the message along to another syslog server.

terms that you should understand –

## Syslog Facilities

Here are the available facilities for the selector. Not all facilities are present on all versions of UNIX.

## Syslog Priorities

The syslog priorities are summarized in the following table –

Facility	Description
<b>auth</b>	Activity related to requesting name and password (getty, su, login).
<b>authpriv</b>	Same as auth but logged to a file that can only be read by selected users.
<b>console</b>	Used to capture messages that would generally be directed to the system console.
<b>cron</b>	Messages from the cron system scheduler.
<b>daemon</b>	System daemon catchall.
<b>ftp</b>	Messages relating to the ftp daemon.
<b>kern</b>	Kernel messages.
<b>local0.local7</b>	Local facilities defined per site.
<b>lpr</b>	Messages from the line printing system.
<b>mail</b>	Messages relating to the mail system.
<b>mark</b>	Pseudo-event used to generate timestamps

The combination of facilities and levels enables you to be discerning about what is logged and where that information goes. As each program sends its messages dutifully to the system logger, the logger makes decisions on what to keep track of and what to discard based on the levels defined in the selector. When you specify a level, the system will keep track of everything at that level and higher.

## The /etc/syslog.conf file

The /etc/syslog.conf file controls where messages are logged. A typical syslog.conf file might look like this –

	in log files.	<code>*.err;kern.debug;auth.notice /dev/console</code>
<b>news</b>	Messages relating to network news protocol (nntp).	<code>daemon,auth.notice /var/log/messages</code>
<b>ntp</b>	Messages relating to network time protocol.	<code>lpr.info /var/log/lpr.log</code> <code>mail.* /var/log/mail.log</code>
<b>user</b>	Regular user processes.	<code>ftp.* /var/log/ftp.log</code>
<b>uucp</b>	UUCP subsystem.	<code>auth.* @prep.ai.mit.edu</code>
Priority	Description	
<b>emerg</b>	Emergency condition, such as an imminent system crash, usually broadcast to all users.	<code>netinfo.err /var/log/netinfo.log</code> <code>install.* /var/log/install.log</code>
<b>alert</b>	Condition that should be corrected immediately, such as a corrupted system database.	<code>*.emerg *</code> <code>*.alert  program_name</code>
<b>crit</b>	Critical condition, such as a hardware error.	<code>mark.* /dev/console</code>
<b>err</b>	Ordinary error.	Each line of the file contains two parts –       • A message selector that specifies which kind of messages to log. For example, all error messages or all debugging messages from the kernel.     • An action field that says what should be done with the message. For example, put it in a file or send the message to a user's terminal.
<b>warning</b>	Warning.	
<b>notice</b>	Condition that is not an error, but possibly should be handled in a special way.	
<b>info</b>	Informational message.	
<b>debug</b>	Messages that are used when debugging programs.	
<b>none</b>	Pseudo level used to specify not to log messages.	

Following are the notable points for the above configuration –

- Message selectors have two parts: a facility and a priority. For example, *kern.debug* selects all debug messages (the priority) generated by the kernel (the facility).
- Message selector *kern.debug* selects all priorities that are greater than debug.

- An asterisk in place of either the facility or the priority indicates “all.” For example, \*.debug means all debug messages, while kern.\* means all messages generated by the kernel.
- You can also use commas to specify multiple facilities. Two or more selectors can be grouped together by using a semicolon.

## Logging Actions

The action field specifies one of five actions –

- Log messages to a file or a device. For example, /var/log/lpr.log or /dev/console.
- Send a message to a user. You can specify multiple usernames by separating them with commas (e.g., root, amrood).
- Send a message to all users. In this case, the action field consists of an asterisk (e.g., \*).
- Pipe the message to a program. In this case, the program is specified after the UNIX pipe symbol (|).
- Send the message to the syslog on another host. In this case, the action field consists of a hostname, preceded by an at sign (e.g., @tutorialspoint.com)

## The logger Command

UNIX provides the `logger` command, which is extremely useful for system logging. The `logger` command sends logging messages to the `syslogd` daemon, and consequently provokes system logging. This means we can check from the command line at any time using the `syslogd` daemon and its configuration. The `logger` command provides a method for adding one-line entries to the system log file from the command line.

The format of the command is –

```
logger [-i] [-f file] [-p priority] [-t tag] [message]...
```

Here is the detail of the parameters –

Option	Description
<b>-f filename</b>	Use the contents of file filename as the message to log.
<b>-i</b>	Log the process ID of the logger process with each

line.

**-p priority** Enter the message with the specified priority (specified selector entry); the message priority can be specified numerically, or as a facility.priority pair. The default priority is user.notice.

**-t tag** Mark each line added to the log with the specified tag.

**message** The string arguments whose contents are concatenated together in the specified order, separated by the space

You can use [Manpage Help](#) to check complete syntax for this command.

## Log Rotation

Log files have the propensity to grow very fast and consume large amounts of disk space. To enable log rotations, most distributions use tools such as *newsyslog* or *logrotate*. These tools should be called at frequent intervals using the cron daemon. Check the man pages for *newsyslog* or *logrotate* for more details.

## Important Log Locations

Application	Directory
httpd	/var/log/httpd
samba	/var/log/samba
cron	/var/log/
mail	/var/log/
mysql	/var/log/

All the system applications create their log files in */var/log* and its sub-directories. Here are few important applications and their corresponding log directories –

## Chapter Twenty-one: UNIX SIGNALS

## AND TRAPS

Signals are software interrupts sent to a program to indicate that an important event has occurred. The events can vary from user requests to illegal memory access errors. Some signals, such as the interrupt signal, indicate that a user has asked the program to do something that is not in the usual flow of

control.

The following are some of the more common signals you might encounter and want to use in your programs –

Signal Name	Signal Number	Description
<b>SIGHUP</b>	1	Hang up detected on controlling terminal or death of controlling process
<b>SIGINT</b>	2	Issued if the user sends an interrupt signal (Ctrl + C).
<b>SIGQUIT</b>	3	Issued if the user sends a quit signal (Ctrl + D).
<b>SIGFPE</b>	8	Issued if an illegal mathematical operation is attempted
<b>SIGKILL</b>	9	If a process gets this signal it must quit immediately and will not perform any cleanup operations
<b>SIGALRM</b>	14	Alarm Clock signal (used for timers)
<b>SIGTERM</b>	15	Software termination signal (sent by kill by default).

### LIST of Signals

There is an easy way to list all the signals supported by your system. Just issue the `kill -l` command and it will display all the supported signals –

\$ kill -l			
1) SIGHUP	2) SIGINT	3) SIGQUIT	4) SIGILL
5) SIGTRAP	6) SIGABRT	7) SIGBUS	8) SIGFPE
9) SIGKILL	10) SIGUSR1	11) SIGSEGV	12) SIGUSR2
13) SIGPIPE	14) SIGALRM	15) SIGTERM	16) SIGSTKFLT
17) SIGCHLD	18) SIGCONT	19) SIGSTOP	20) SIGTSTP
21) SIGTTIN	22) SIGTTOU	23) SIGURG	24) SIGXCPU
25) SIGXFSZ	26) SIGVTALRM	27) SIGPROF	28) SIGWINCH
29) SIGIO	30) SIGPWR	31) SIGSYS	34) SIGRTMIN
35) SIGRTMIN+1	36) SIGRTMIN+2	37) SIGRTMIN+3	38) SIGRTMIN+4
39) SIGRTMIN+5	40) SIGRTMIN+6	41) SIGRTMIN+7	42) SIGRTMIN+8
43) SIGRTMIN+9	44) SIGRTMIN+10	45) SIGRTMIN+11	46) SIGRTMIN+12
47) SIGRTMIN+13	48) SIGRTMIN+14	49) SIGRTMIN+15	50) SIGRTMAX-14

51) SIGRTMAX-13	52) SIGRTMAX-12	53) SIGRTMAX-11	54) SIGRTMAX-10
55) SIGRTMAX-9	56) SIGRTMAX-8	57) SIGRTMAX-7	58) SIGRTMAX-6
59) SIGRTMAX-5	60) SIGRTMAX-4	61) SIGRTMAX-3	62) SIGRTMAX-2
63) SIGRTMAX-1	64) SIGRTMAX		

The actual list of signals varies between Solaris, HP-UX, and Linux.

## Default Actions

Every signal has a default action associated with it. The default action for a signal is the action that a script or program performs when it receives a signal.

Some of the possible default actions are –

- Terminate the process.
- Ignore the signal.
- Dump core. This creates a file called core containing the memory image of the process when it received the signal.
- Stop the process.
- Continue a stopped process.

## Sending Signals

There are several methods for delivering signals to a program or script. One of the most common is for a user to type CONTROL-C or the INTERRUPT key while a script is executing.

When you press the *Ctrl+C* key a SIGINT is sent to the script and as per the defined default action script terminates.

The other common method for delivering signals is to use the kill command whose syntax is as follows –

```
$ kill -signal pid
```

Here **signal** is either the number or name of the signal to deliver and **pid** is the process ID that the signal should be sent to. For Example –

```
$ kill -1 1001
```

Sends the HUP or hang-up signal to the program that is running with process ID 1001. To send a kill signal to the same process use the following command –

```
$ kill -9 1001
```

This would kill the process running with process ID 1001.

## Trapping Signals

When you press the *Ctrl+C* or Break key at your terminal during execution of a shell program, normally that program is immediately terminated, and your



command prompt returned. This may not always be desirable. For instance, you may end up leaving a bunch of temporary files that will not get cleaned up.

Trapping these signals is quite easy, the `trap` command has the following syntax –

```
$ trap commands signals
```

Here *command* can be any valid Unix command, or even a user-defined function, and *signal* can be a list of any number of signals you want to trap.

There are three common uses for `trap` in shell scripts –

- Clean up temporary files
- Ignore signals

### Cleaning Up Temporary Files

As an example of the `trap` command, the following shows how you can remove files and then exit if someone tries to abort the program from the terminal –

```
$ trap "rm -f $WORKDIR/work1$$ $WORKDIR/dataout$$; exit" 2
```

From the point in the shell program that this `trap` is executed, the two files *work1\$\$* and *dataout\$\$* will be automatically removed if signal number 2 is received by the program. Therefore, if the user interrupts execution of the program after this `trap` is executed, you can be assured that these two files will be cleaned up. The **exit** command that follows the `rm` is necessary because without it execution would continue in the program at the point that it left off when the signal was received.

Signal number 1 is generated for hangup: Either someone intentionally hangs up the line or the line gets accidentally disconnected. You can modify the preceding `trap` to also remove the two specified files in this case by adding signal number 1 to the list of signals –`$ trap "rm $WORKDIR/work1$$ $WORKDIR/dataout$$; exit" 1 2`. Now these files will be removed if the line gets hung up or if the *Ctrl+C* key gets pressed. The commands specified to `trap` must be enclosed in quotes if they contain more than one command. Also, note that the shell scans the command line at the time that the `trap` command gets executed and also when one of the listed signals is received.

So in the preceding example, the value of `WORKDIR` and `$$` will be substituted at the time that the `trap` command is executed. If you wanted this substitution to occur at the time that either signal 1 or 2 was received you can put the commands

inside single quotes –

```
$ trap 'rm $WORKDIR/work1$$ $WORKDIR/dataout$$; exit' 1 2
```

## Ignoring Signals

If the command listed for trap is null, the specified signal will be ignored when received. For example, the command –

```
$ trap " 2
```

Specifies that the interrupt signal be ignored. You might want to ignore certain signals when performing some operations that you do not want interrupted. You can specify multiple signals to be ignored as follows –

```
$ trap " 1 2 3 15
```

Note that the first argument must be specified for a signal to be ignored and is not equivalent to writing the following, which has a separate meaning of its own –

```
$ trap 2
```

If you ignore a signal, all subshells also ignore that signal. However, if you specify an action to be taken on receipt of a signal, all subshells will still take the default action on receipt of that signal.

## Resetting Traps

After you have changed the default action to be taken on receipt of a signal, you can change it back again with trap if you simply omit the first argument; so.

```
$ trap 1 2
```

Resets the action to be taken on receipt of signals 1 or 2 back to the default.

## **DEDICATION**

**This book is dedicated to.....My Wife and Son**

## **SPECIAL THANKS**

To my Wife Katherine and my son Adam